LÖVE WIKI · PART 5 OF 9

Other pages

234 pages

wiki pages 1290–1523

Contents of this part

.lazr.
0.1.1
0.10.0
0.10.1
0.10.2
0.2.0
0.2.1
0.3.0
0.3.1
0.3.2
0.4.0
0.5.0
0.6.0
0.6.1
0.6.2
0.7.0
0.7.1
0.7.2
0.8.0
0.8.1
0.9.0
0.9.1
0.9.2
11.0
11.1
11.2
11.3
11.4
11.5
12.0
2dgraph
30log
32 lines of goodness
7-Zip
8-Bit Commando
A Guide to Getting Started w ith Love2D
Adult Lib
Advanced Tiled Loader
Aeternum Blammo
anim8
any
arcLine
Arne's Breakout
Arne's SpaceTaxi
array
Audio Formats
babel
Batch Start
Beetle Debug
binser
bitser
BlendMode Formulas
board game support
boipushy
Bomb Dodgers
boolean
breezefield
BSD License
built-in level editor
cdata
Chromatic Paths
cimgui-love
clasp
Class Commons
Code Style
Commies!
Common Organization of Controls Kit
Common Organization of Controls Kit Callbacks
Common Organization of Controls Kit Fields
Common Organization of Controls Kit Hat modes
Common Organization of Controls Kit Input devices
Common Organization of Controls Kit Input keys
Common Organization of Controls Kit Manual
Common Organization of Controls Kit Modes
Common Organization of Controls Kit Setup data
Comparing LÖVE Versions
cron
Dave Gone Apeshit
debugging
deleteme2
Desert Loot
Doctor Cat: The Game
DRAKON Editor
dt
Easy GUI System
Effect Constants Name Candidates
examples
fadeColor
fakecanvas
findProjectileTrajectory
findRotation
Fistul of Beef
Fizz X
flat table
fLUIds
Free Game Resources
Freeing Quency
freeing quency
function
Game Distribution/APKTool
gamera
General math
Getting Started w ith PÄSSION
Giga Pudding: The Game
GLES Shader Testing in Desktop
GNU General Public License
Graphical User Interface
Gspöt
GunFu Deadlands
Highscöre
HSL color
HSV color
hxdx
Image Formats
In Your Face City Trains
In-Memory Audio Streaming
Journey To The Stars
Layering Tiles
layouter
libSaveTableToFile
libxml
light userdata
likeliHUD
Live Simulator: 2
local
love dependencies versions in ubuntu repositories
Love-AIchemy
love-imgui
love-loader
love-nuklear
Lovely Mario Bros
Lovely Tactics Hex
Lovely Tiles
lovesize
love残片
LPC License
Lua FOV
lua-enet
lua-https
luaFortune
luasort
Luiz Renato
lure
Löve Bone
LÖVE CÖNNECTION
Löve Frames
LÖVE Game File
Löve Logo Graphics
LÖVE PhysicsEditor Loader
LövelyMoon
mat4
Message in a Bottle
Metanet Hunter
Metanet Hunter REMIX
Minimalist Sound Manager
Möan.lua
Nano Space
nearest
new ton
nil
Noahs Ark
Notepad++
NULLSoft Scriptable Installer System (NSIS)
number
Oh Snap, Aliens!
On The Roadside
Panel Attack
Paper Evolution
Pirate Bay
Plant game
PO2 Syndrome
Polygon splitting
polypoint
profile
Pröxima
psi2love
PÄSSION
Pölygamy
Remove Workaround
require
RL-Dice
Roleplaying Game
rstar
Scäle
SELÖVE
sequence
Shader Variables
Shädow s
Simple Educative Class Library
Skip list
Skip list:Draw ing Order
Skull Basher
Snipes Reloaded
socket
Source Obfuscation
Splash Screens
string
String exploding
String extensions
strong
subclass
Sublime Text
Super Chains
Super Sun Show dow n
svglover
table
Ten Second War
The King and the Crow n
Tile Studio
tile-collider
tiny-ecs
Totems Sound
trAInsported
Transform (Library)
TsT Module definition
TsT w ay to learn to Beginners
Tuesday Convention
Turres Monacorum
tw een
TÖVE
uLove
uLove Compliance conf.lua
utf8
variable
vec2
vec4
Virtualization Solutions Provider
Volley Braw l
vudu
Warlocks Tow er
Warp Run

yaoui zip ziplib

.lazr.

.lazr. · revision 14886 · last edited 2021-03-14

.lazr. is a local multiplayer cyberneon glitch-punk arena shooter for up to four datathieves.

You can get it right here: http://gaeel.itch.io/lazr ↗

0.1.1

0.1.1 · revision 21102 · last edited 2021-03-14

The codename of LÖVE 0.1.1 is "**Santa-Power**". It is currently **released**.

- Image loading and rendering.
- Sound loading and playing.
- Font loading and rendering.
- Lua-scriptable games.
- Config files.
- Stuff is loadable from archive files.
- Keyboard, mouse, display, timer, etc. (Basic devices).

LÖVE 0.1.1 does not have a "no-game screen" as all later releases do. When run without a game, it outputs the following text on the console:

```
LOVE 0.1.1a - Now with more Santa-power.

Usage: love [item]
 - [item] may be an archive file or directory ^.^
```

0.10.0

0.10.0 · revision 22056 · last edited 2021-03-14

On this page

- Changes from 0.9.2
  - Additions
  - Changed APIs
  - Renamed APIs
  - Removals
  - Bugfixes
  - Performance improvements
  - Other changes

The codename for LÖVE 0.10.0 is **Super Toast**. It is currently **released**.



Changes from 0.9.2

Additions

- Added an iOS port.
- Added an Android port.
- Added support for reading accelerometer values on mobile devices.
  Accelerometers are exposed as 3-axis Joysticks.
- Added the flag t.accelerometerjoystick to love.conf. Disables accelerometer-as-joystick functionality on mobile devices when false.
- Added the flag t.gammacorrect to love.conf (replaces t.window.srgb.)
  Enabling it globally enables gamma-correct rendering, when supported.
- Added video playback support for Ogg Theora videos, via love.graphics.newVideo.
- Added love.video module. It is not used for displaying videos on-screen (see Video objects for that.)
- Added love.touch module. Note that it has important differences from the touch implementation in the LÖVE 0.9.2 Android and iOS ports.
  - Added love.touch.getTouches
  - Added love.touch.getPosition
  - Added love.touch.getPressure
- Added love.touchpressed, love.touchreleased, and love.touchmoved.
- Added love.system.vibrate.
- Added love.filesystem.setRequirePath and love.filesystem.getRequirePath.
- Added an optional program exit status number argument to love.event.quit.
- Added love.filedropped and love.directorydropped event callback functions.
- Added love.lowmemory event callback function, called when the app is running out of memory on mobile operating systems.
- Added love.textedited event callback function, called when the user is compositing text (e.g. via an IME.)
- Added love.wheelmoved event callback function (replaces "wu" and "wd" constants for love.mousepressed.)
- Added love.mouse.hasCursor.
- Added a boolean argument to love.mousepressed and love.mousereleased indicating whether the button event originated from a touch press.
- Added love.keyboard.isScancodeDown.
- Added optional x/y/width/height arguments to love.keyboard.setTextInput. They tell the system where text will show up so on-screen keyboards can avoid that area.
- Added Source:getType (replaces Source:isStatic.)
- Added Source:getDuration and Decoder:getDuration.
- Added an optional string argument containing raw pixel data to the width/height variant of love.image.newImageData.
- Added love.graphics.ellipse.
- Added rounded-rectangle support to love.graphics.rectangle.
- Added love.graphics.points (replaces love.graphics.point.)
- Added love.graphics.intersectScissor.
- Added an optional argument to love.graphics.setBlendMode which indicates whether to treat the colors of drawn objects as having pre-multiplied alpha.
- Added a new built-in shader variable: "mat3 NormalMatrix".
- Added love.graphics.getSupported (replaces love.graphics.isSupported.)
- Added love.graphics.getSystemLimits (replaces love.graphics.getSystemLimit.)
- Added love.graphics.stencil, love.graphics.setStencilTest, and love.graphics.getStencilTest. (replaces love.graphics.setStencil.)
- Added love.graphics.isActive.
- Added color arguments to love.graphics.clear. It no longer always uses the background color value.

no-game screen

- Added love.graphics.discard.
- Added love.graphics.isGammaCorrect.
- Added the "clampzero" WrapMode.
- Added the ability to specify custom mipmaps when creating an image, via love.graphics.newImage(filename, {mipmaps={mip1, mip2, ...}}).
- Added optional x/y/width/height arguments to Image:refresh and Canvas:newImageData.
- Added Image:getFlags.
- Added one- and two-channel Canvas formats: r8, rg8, r16f, rg16f, r32f, and rg32f.
- Added support for different formats in each Canvas when using multi-canvas rendering.
- Added the "multicanvasformats" GraphicsFeature constant.
- Added support for OpenGL ES 2 and 3.
- Added support for loading ETC, EAC, PVRTC, and ASTC compressed textures on systems that support them.
- Added custom vertex attribute support for Meshes via new variants of love.graphics.newMesh.
- Added Mesh:setVertexAttribute and Mesh:getVertexAttribute.
- Added Mesh:getVertexFormat.
- Added an optional 'startvertex' argument to Mesh:setVertices.
- Added the ability for love.graphics.newMesh and Mesh:setVertices to accept a Data object.
- Added Mesh:flush.
- Added Mesh:setAttributeEnabled and Mesh:isAttributeEnabled.
- Added Mesh:attachAttribute.
- Added SpriteBatch:attachAttribute.
- Added Shader:sendColor.
- Added new shader functions: gammaCorrectColor, gammaToLinear, and linearToGamma. The functions also have 'precise' and 'fast' variants.
- Added Text objects and love.graphics.newText.
- Added per-character color support to love.graphics.print, love.graphics.printf, and Text objects.
- Added BMFont bitmap font file support to love.graphics.newFont and love.font.
- Added kerning support for TrueType/OpenType and BMFont Fonts.
- Added an optional font hinting argument to love.graphics.newFont when loading TrueType fonts.
- Added an optional spacing argument to love.graphics.newImageFont, which applies additional spacing to all rendered glyphs.
- Added Font:setFallbacks.
- Added love.window.maximize.
- Added love.window.close.
- Added love.window.requestAttention.
- Added love.window.setDisplaySleepEnabled and love.window.isDisplaySleepEnabled.
- Added BezierCurve:renderSegment and BezierCurve:removeControlPoint.
- Added BezierCurve:getSegment.
- Added love.math.compress and love.math.decompress.
- Added Channel:performAtomic.

Changed APIs

- Changed love.mousepressed, love.mousereleased, and love.mouse.isDown to use button numbers instead of named button constants.
- Changed love.keypressed to be love.keypressed(key, scancode, isrepeat).
- Changed love.keyreleased to be love.keyreleased(key, scancode).
- Changed Font:getWrap's second return value to be a table containing the text split into lines.
- Changed love.graphics.newImage's optional second argument to be a table of flags (flags are "mipmaps" and "linear".)
- Changed the arguments for the standard variants of love.graphics.newMesh. They're now love.graphics.newMesh(vertices [, drawmode, usage]) and love.graphics.newMesh(vertexcount [, drawmode, usage]).
- Changed ImageData:encode to return a FileData object. ImageData:encode's first parameter is now the format to encode to, and the second parameter is an optional filename to write to.

Renamed APIs

- Renamed the fullscreen type "normal" to "exclusive".

- Renamed the DistanceModel constants "inverse clamped", "linear clamped", and "exponent clamped" to "inverseclamped", "linearclamped", and "exponentclamped".
- Renamed blend modes "additive", "subtractive", and "multiplicative" to "add", "subtract", and "multiply".
- Renamed the key constant and Scancode representing the spacebar from " " to "space".
- Renamed File:eof to File:isEOF.
- Renamed Canvas:getImageData to Canvas:newImageData.
- Renamed love.image's CompressedData type to CompressedImageData.

Removals

- Removed callback variant of love.filesystem.getDirectoryItems.
- Removed the "wu" and "wd" constants for love.mousepressed (replaced by love.wheelmoved.)
- Removed the named mouse button constants (replaced by button numbers.)
- Removed Source:isStatic (replaced by Source:getType.)
- Removed image loading support for all (non-compressed texture) file formats except for PNG, JPEG, TGA, and BMP.
- Removed JPEG encoding support from ImageData:encode.
- Removed love.graphics.point (replaced by love.graphics.points.)
- Removed love.graphics.setPointStyle and love.graphics.getPointStyle.
- Removed love.graphics.isSupported (replaced by love.graphics.getSupported.)
- Removed love.graphics.getSystemLimit (replaced by love.graphics.getSystemLimits.)
- Removed love.graphics.setStencil (replaced by love.graphics.stencil and love.graphics.setStencilTest.)
- Removed the "canvas", "shader", "npot", "subtractive", and "mipmap" GraphicsFeature constants (the features always have guaranteed support now.)
- Removed the "multicanvas" GraphicsFeature constant (use love.graphics.getSystemLimits instead.)
- Removed the "srgb" GraphicsFeature constant (use love.graphics.isGammaCorrect or love.graphics.getCanvasFormats().srgb instead.)
- Removed the "srgb" flag in love.window.setMode and in the t.window table in love.conf (Replaced by t.gammacorrect.)
- Removed the "premultiplied" BlendMode (love.graphics.setBlendMode("alpha", "premultiplied") now does the same thing.)
- Removed Canvas:getPixel (use Canvas:newImageData instead.)
- Removed Canvas:clear (use love.graphics.clear instead.)
- Removed Mesh:getVertices.
- Removed Mesh:setVertexColors and Mesh:hasVertexColors (use Mesh:setAttributeEnabled("VertexColor", enable) instead.)
- Removed functions deprecated in LOVE 0.9.1 and 0.9.2:
    - Removed love.graphics.getMaxImageSize and love.graphics.getMaxPointSize (replaced by love.graphics.getSystemLimits.)
    - Removed Mesh:setImage, Mesh:getImage, SpriteBatch:setImage, SpriteBatch:getImage, ParticleSystem:setImage, and ParticleSystem:getImage (replaced by set/getTexture.)
    - Removed SpriteBatch:bind and SpriteBatch:unbind.
    - Removed Canvas:getFSAA and the "fsaa" flag in love.conf and love.window.setMode (replaced by Canvas:getMSAA and the "msaa" flag.)
    - Removed the "dxt" and "bc5" GraphicsFeature constants (replaced by love.graphics.getCompressedImageFormats.)
    - Removed the "hdrcanvas" GraphicsFeature constant (replaced by love.graphics.getCanvasFormats.)
    - Removed love.window.getWidth, love.window.getHeight and love.window.getDimensions (use love.graphics.getWidth, love.graphics.getHeight, love.graphics.getDimensions or love.window.getMode instead.)

Bugfixes

- Fixed utf8.char.
- Fixed detection of fused executables.
- Fixed crashes when objects which store Lua callback functions are garbage collected after being used in coroutines.
- Fixed World:getCallbacks and World:getContactFilter when used in coroutines.
- Fixed memory leaks in love.physics if World:destroy is never called. When a World is garbage collected it now destroys all associated bodies, fixtures, and joints.
- Fixed love.keyboard.getKeyFromScancode crashing when an invalid scancode is given.
- Fixed decoding of 8-bit WAV files.
- Fixed a crash issue when rewinding streaming ogg Sources, when certain versions of libvorbis are used.
- Fixed love.audio.stop() not rewinding streaming Sources.

- Fixed the stencil buffer in Canvases when an unsupported MSAA value is used to create the Canvas.
- Fixed Canvas:renderTo to restore the previous canvases if an error occurs in the passed function.
- Fixed love.graphics.draw(canvas) to cause a Lua error if that canvas is the active one.
- Fixed Mesh:getVertexMap to return nil, rather than an empty table, if no vertex map has been set.
- Fixed love.graphics.getColorMask.
- Fixed the default offset for particles when ParticleSystem:setQuads or ParticleSystem:setTexture is used.
- Fixed love.graphics.shear resetting all love.graphics transformations.
- Fixed the "add" and "subtract" BlendModes to no longer modify the alpha of the Canvas / screen.

Performance improvements

- Improved the performance of World:rayCast and World:queryBoundingBox.
- Improved the performance of love.graphics.line and other line drawing functions, when the "smooth" LineStyle is used.
- Improved the performance of Shader:send when matrices are used.
- Improved the performance of ImageData and SoundData methods when LuaJIT's JIT compiler is enabled, by using efficient FFI code.
- Improved the performance of love.math.noise, love.math.gammaToLinear, love.math.linearToGamma, love.math.random, and RandomGenerator:random when LuaJIT's JIT compiler is enabled.

Other changes

- Updated the compatibility warning notice to use a message box and to show the version specified in love.conf.
- Updated the compatibility warning notice to display before main.lua is loaded.
- Updated the __tostring metamethod of love objects to output the pointer value, similar to tostring(table).
- Updated World:setCallbacks, World:setContactFilter, World:queryBoundingBox, and World:rayCast to have improved argument type checking.
- Updated threads to load love.filesystem automatically.
- Updated love.filesystem to enable symlinks by default.
- Updated love.math.setRandomSeed and RandomGenerator:setSeed to produce better results for the first few random() calls.
- Updated love.math.random and RandomGenerator:random to produce slightly better results in general.
- Updated Source methods that deal with spatial audio to error rather than failing silently if the Source isn't mono.
- Updated the 3D and 4D variants of love.math.noise to use Perlin noise rather than Simplex noise, to avoid patent issues.
- Updated love.graphics.newImageFont to no longer treat separator pixels as spacing.
- Updated the default font to use less memory.
- Updated the behavior of text wrapping with love.graphics.printf and Font:getWrap to work better.
- Updated love.graphics.print and love.graphics.printf to no longer automatically round the x and y position arguments.
- Updated some of the error messages for love.graphics.newImage to be more descriptive.
- Updated love.graphics color functions to automatically apply love.math.gammaToLinear to color values when gamma-correct rendering is enabled.
- Updated the "normal" CanvasFormat to internally use "srgb" rather than "rgba8" when gamma-correct rendering is enabled.
- Updated love.graphics.setColor to affect all drawn objects, including ParticleSystems, SpriteBatches, and Meshes, even when per-vertex or per-sprite colors are used.
- Updated the default fullscreen type to be "desktop" rather than "exclusive".
- Updated the minimum runtime system requirements of LOVE to require OpenGL 2.1 or OpenGL ES 2 support.
- Updated the pixel shader effect function so screen_coords.y is 0 at the top of the screen instead of the bottom.
- Updated Images to require setting the mipmaps flag to true on creation in order to use mipmaps.
- Updated Images to allow mipmaps for non-power-of-two sizes.

0.10.1

0.10.1 · revision 22055 · last edited 2021-03-14

The codename for LÖVE 0.10.1 is **Super Toast**. It is currently **released**.



Changes from 0.10.0

Additions

- Added a new love.conf flag `t.externalstorage`, which determines whether files are saved in internal or external storage on Android devices.
- Added an optional 'collideconnected' boolean argument to love.physics.newMotorJoint.
- Added a new variant of love.graphics.arc which can draw different types of arcs ("pie", "open", or "closed").
- Added "lighten" and "darken" BlendModes. They can only be used with the "premultiplied" BlendAlphaMode.
- Added the "lighten" GraphicsFeature constant.
- Added the ability to avoid clearing specific Canvases when calling love.graphics.clear, if multiple Canvases are active at once via love.graphics.setCanvas.
- Added Text:getDimensions.

no-game screen

Bugfixes

- Fixed a Lua error in the no-game screen if the window's height is too small.
- Fixed the default error handler to reset the mouse Cursor.
- Fixed love.filesystem functions crashing when called if liblove is used directly without calling love.filesystem.init.
- Fixed audio on Android to pause when the app is inactive, and resume when the app becomes active again.
- Fixed Source:setRelative to set the relative state of playing Sources to the passed value rather than an old cached one.
- Fixed the Video decoding thread hanging after Video:seek or when a Video finishes.
- Fixed Video:isPlaying to always return false after it finishes playing.
- Fixed RandomGenerator:random crashing if a nil 'self' is used.
- Fixed loading BMFont files which have characters with 0 width or height (a space character, for example).
- Fixed love.graphics.newFont causing crashes if FileData is passed in.
- Fixed love.graphics.clear(colortable) causing crashes on OpenGL ES 2 systems when a Canvas is active.
- Fixed love.graphics object creation functions (e.g. love.graphics.newImage) to error instead of crash, if called before a window has been created.
- Fixed a driver bug on some Android devices which caused all objects to show up as black.
- Fixed a driver bug on Windows with AMD graphics cards where love.graphics.clear would not always work.
- Fixed Shader:sendColor incorrectly converting alpha values from sRGB to linear RGB when gamma-correct rendering is enabled.
- Fixed love.graphics.newMesh(vertices) double-converting colors from sRGB to linear RGB when gamma-correct rendering is enabled.

Other changes

- Updated the Windows executable to prefer the high-powered AMD graphics card on systems which have switchable Intel+AMD GPUs.
- Updated love.touch.getTouches to return the list of IDs in the relative order that the touches initially happened (i.e. the first ID in the list is the least recently pressed touch that is currently active), instead of being in a random order.
- Updated the error messages caused by invalid or bad arguments to ImageData and SoundData methods to be more descriptive.

0.10.2

0.10.2 · revision 22009 · last edited 2021-07-30

The codename for LÖVE 0.10.2 is **Super Toast**. It is currently **released**.



Changes from 0.10.1

Additions

- Added lovec.exe in Windows. It is the same as love.exe but built with the Console subsystem, so it always uses or provides a console.
- Added the ability to restart the game via love.event.quit("restart").
- Added support for passing a table to love.mouse.isDown, love.keyboard.isDown, love.keyboard.isScancodeDown, Joystick:isDown, and Joystick:isGamepadDown.
- Added love.window.isMaximized.
- Added 'ellipse' area distribution to ParticleSystems.
- Added support for BC4-7 compressed texture formats in KTX files.
- Added 'shaderswitches' field to the table returned by love.graphics.getStats.
- Added Quad:getTextureDimensions.
- Added PrismaticJoint:getAxis and WheelJoint:getAxis.
- Added 2-point version of love.physics.newRevoluteJoint.
- Added table variants of Fixture:setCategory and Fixture:setMask.
- Added EdgeShape:setNextVertex, EdgeShape:setPreviousVertex, EdgeShape:getNextVertex, and EdgeShape:getPreviousVertex.
- Added ChainShape:getNextVertex and ChainShape:getPreviousVertex.
- Added optional reference angle arguments to love.physics.newRevoluteJoint, love.physics.newPrismaticJoint, and love.physics.newWeldJoint.
- Added RevoluteJoint:getReferenceAngle, PrismaticJoint:getReferenceAngle, and WeldJoint:getReferenceAngle.

no-game screen

Deprecations

- Deprecated undocumented Shader:sendTexture, Shader:sendMatrix, Shader:sendInt, Shader:sendFloat, and Shader:sendBoolean methods.

Fixes

- Fixed love on iOS 6.
- Fixed os.execute always returning -1 on Linux.
- Fixed the love.lowmemory callback to call collectgarbage() after the callback has fired, instead of before.
- Fixed love.math.noise(nil) to error instead of returning nil.
- Fixed an occasional crash when a Thread ends.
- Fixed a hang at the end of video playback with some video files.
- Fixed the video decoding thread to not do any work when there are no videos to decode.
- Fixed love.graphics.newVideo(file) to no longer error if love.audio is disabled.
- Fixed a rare bug in Source:play for streaming Sources if the associated OpenAL source object was previously used for a static Source.
- Fixed corrupted Font glyphs in rare cases.
- Fixed stencils inside Canvases on some OpenGL ES 2 devices.
- Fixed an OpenGL error in OpenGL ES 3 when multiple render targets are used.
- Fixed love.window.setMode crashing when called with a Canvas active.
- Fixed love.window.maximize to update the reported window dimensions immediately.
- Fixed gamma correction of ImageFonts and BMFonts with colored images.
- Fixed the default shader improperly applying gamma correction to per-vertex colors when gamma correction is requested but not supported on OpenGL ES.
- Fixed text coloring breaking because of an empty string.
- Fixed large burst of particles when dramatically increasing the emission rate of a ParticleSystem.

- Fixed SpriteBatch:setBufferSize to keep old sprite data if it can fit.
- Fixed Joint:getBodies unconditionally erroring.
- Fixed memory leak in Text:set.
- Fixed incorrect kerning caused by using kerning information for the wrong character in some fonts.
- Fixed ImageData:setPixel, ImageData:getPixel, ImageData:mapPixel, SoundData:setSample, and SoundData:getSample to properly handle non-integer coordinates.

Other changes

- Improved performance of Channel methods by roughly 2x in many cases.
- Improved performance of Shader:send when small numbers of arguments are given.

- Updated love.filesystem.mount to accept a DroppedFile as the first parameter.
- Updated Shader:send to do type and argument checking based on the specified uniform variable's information instead of the arguments to the function.
- Updated Shader:send to accept a flat table for matrix uniforms.

0.2.0

0.2.0 · revision 14957 · last edited 2021-03-14

The codename of LÖVE 0.2.0 is "**Mini-Moose**". It is currently **released**.



Changes since 0.1.1

- Added ImageFont
- Added Animation (*since removed*)
- Added text formatting functions
- Added setCenter for Image and Animation. (*since removed*)
- Added methods for rendering of scaled/rotated sprites.
- Added the drawing of basic shapes.
- Added default font and embedded resources.
- Added Ctrl+R for reload. (*since removed*)
- Added blending and color modes.
- Fixed memory usage of Graphics.
- Fixed a bug where the set text color would change the color of any images rendered.
- Fixed CWD bug.
- Fixed titlebar. Game title is now displayed.

nogame screen

0.2.1

0.2.1 · revision 21096 · last edited 2021-03-14

The codename of LÖVE 0.2.1 is "**Impending Doom**". It is currently **released**.



nogame screen

Changes since 0.2.0

Additions

- Added many functions in love.filesystem.
- Added a dedicated save-folder for each game.
- Added love.timer.sleep.
- Added line heights to font objects.
- Added love.graphics.getWidth/getHeight.
- Added scaling and rotation for text.
- Added variable spacing to ImageFont.
- Added support for variable line quality when drawing primitives.
- Added several functions for drawing sections of images. (love.graphics.draws)
- Added image optimization function and padding function.

Changes

- Split devices up into actual SWIG-modules. This means that:
  - Functions are used like this: love.graphics.draw, not love.graphics:draw
  - love.objects is no more. Objects are created by an appropriate device.
- How you draw primitives has been altered.
  - draw(string, x, y, wrap, align) has become drawf(string, x, y, wrap, align)
- Changed getFps to getFPS.
- Escape is no more ... enter: Alt+F4.
- love.filesystem.include has been renamed to love.filesystem.require.
- ImageFonts now consider the spacing as well as the glyph size.
- Fixed a massive ImageFont bug which resulted in float-positioning failure.
- Fixed a bug when loading fonts where the specified size doesn't represent the true size of the font.

Others

- Updated DevIL to version 1.6.8-rc2 (Windows)
- Updated FreeType to freetype-2.3.5-1 (Windows)
- Updated Lua to 5.1.3 (Windows)
- Updated SDL to 1.2.13 (Windows)
- Removed boost::filesystem.

0.3.0

0.3.0 · revision 21101 · last edited 2021-03-14

The codename for LÖVE 0.3.0 is **Mutant Vermin**. It is currently **released**.



The no-game screen of LÖVE 0.3.0.

Changes from 0.2.1

Additions

- Added ParticleSystem.
- Added visual error reporting.
- Added love.system for game control needs.
- Added input grabbing.
- Added functions in love.graphics for display management.
- Added love.graphics.point.
- Added functions in love.graphics for getting current color, font, etc.
- Added love.filesystem.enumerate for getting folder contents.
- Added functions for setting the window caption.
- Added version checking. An error occurs if the game is incompatible.

Removals

- Removed all keyboard shortcuts.

Fixes

- Fixed print() :)

Other Changes

- Save folders are now created only if required.
- On Windows, the new save location is %APPDATA%\LOVE\game

0.3.1

0.3.1 · revision 21099 · last edited 2021-03-14

The codename for LÖVE 0.3.1 is **Space Meat**. It is currently **released**.



Changes from 0.3.0

Fixes

- Fixed segfault related to graphics.
- Fixed wait-forever bug related to audio.
- Fixed error reporting not working across modules.
- Fixed bug where games with a trailing "/" would not start.
- Fixed bug which caused love.timer.sleep to delay for (way) too long.

The no-game screen of LÖVE 0.3.1.

0.3.2

0.3.2 · revision 24318 · last edited 2021-03-14

The codename for LÖVE 0.3.2 is **Lemony Fresh**. It is currently **released**.



The no-game screen of LÖVE 0.3.2.

Changes from 0.3.1

Additions

- Added love.graphics.rectangle
- Added love.graphics.setLineWidth
- Added love.graphics.setLineStyle
- Added love.graphics.getLineWidth
- Added love.graphics.getLineStyle
- Added love.mouse.getPosition
- Added love.audio_loop
- Added love.timer.getTime

Fixes

- Fixed default color mode bug.
- Fixed line width being applied unnecessarily.
- Fixed line width bug related to fullscreen toggle.
- Fixed music not looping.

Other Changes

- Changed love.graphics.quad to accept CCW and CW ordering.

0.4.0

0.4.0 · revision 21094 · last edited 2021-03-14

The codename for LÖVE 0.4.0 is **Taco Beam**. It is currently **released**.



The no-game screen of LÖVE 0.4.0.

Changes from 0.3.2

Additions

- Added love.physics. (YES!)
- Added love.audio.setMode.
- Added love.audio.setChannels.
- Added love.graphics.polygon.
- Added love.graphics.setScissor and love.graphics.getScissor to handle scissoring the graphical area.

Fixes

- Fixed missing constants related to image optimization.
- Fixed memory leak related to love::File (thanks amnesiasoft!).

0.5.0

0.5.0 · revision 24738 · last edited 2021-03-14

The codename of LÖVE 0.5.0 is "**Salted Nuts**". It is currently **released**.



Download 0.5.0 from SourceForge (http://sourceforge.net/projects/love/files/love/love-0.5-0/)

Changes from 0.4.0

- Added love.joystick.
- Added network support via LuaSocket.
- Added support for loading of appended .love-file.

- Added love.filesystem.lines().
- Added a loader function to enable use of normal require().
- Added love.filesystem.load().
- Added love.filesystem.getSaveDirectory()
- Added love.filesystem.getWorkingDirectory()

nogame screen

- Added optional explicit destruction of Box2D objects.
- Added Shape:testSegment().
- Added love.graphics.newScreenshot() (.bmp only).
- Added default size (12) to font-related functions.
- Added love.graphics.setFont(filename, size)
- Added love.graphics.setLineStipple and related functions.
- Added love.graphics.setPointSize and related functions.

- Changed love.filesystem.read() to accept file name.
- Changed love.filesystem.write() to accept file name.
- Changed love.graphics.triangle() to accept CCW and CW ordering.

- Fixed love.filesystem.read adding bogus characters at the end of string.
- Fixed epic swigfusion bug.
- Fixed love.graphics.getFont so it returns nil if no font is present.
- Fixed bug where love.graphics.getBlendMode() always returns blend_normal.
- Fixed bug which caused error screen to be scissored (when enabled).
- Fixed Body:setAngle to accept degrees like everything else.

- Cleaned up love::File and love_physfs.
- Cleaned up love::Reference so it stores its reference in _G.

0.6.0

0.6.0 · revision 22566 · last edited 2021-03-14

The codename of LÖVE 0.6.0 is "**Jiggly Juice**". It is currently **released**.



Download 0.6.0 from SourceForge (http://sourceforge.net/projects/love/files/love/love-0.6.0/)

Changes from 0.5.0

- Lost track of 0.6.0 changes a long while ago. Don't trust the list below.

- Added love.graphics.print()/printf().
- Added unicode-translated parameter to love.keypressed().
- Added love.event.
- Added love.filesystem.setIdentity().
- Added OpenAL dependency.

nogame screen

- Fixed love.filesystem problems with internal \0 in strings.
- Fixed love.filesystem.mkdir/remove not working when write directory not set.
- Fixed position of Window.

- Changed parameter order of draws().
- Changed origin for images to top-left.
- Changed love.filesystem.open to accept mode (removed from love.filesystem.newFile).
- Changed love.filesystem.read() which now returns two parameters (data, length).
- Changed love.filesystem.write() which now takes up to four parameters (file, data, length, mode).
- Changed default color mode to "modulate".
- Changed name of love.color_normal to "replace".
- Changed name of love.blend_normal to "alpha".
- Changed the conf file format.

- Removed Color object.
- Removed Animation.
- Removed several constants.
- Removed love.graphics.draw() for strings.
- Removed love.system.
- Removed SWIG.
- Removed boost.
- Removed SDL_mixer.

0.6.1

0.6.1 · revision 22583 · last edited 2021-03-14

The codename of LÖVE 0.6.1 is "**Jiggly Juice**". It is currently **released**.



Download 0.6.1 from SourceForge (http://sourceforge.net/projects/love/files/love/love-0.6.1/)

Changes from 0.6.0

- Added Shape:setGroupIndex and getGroupIndex.
- Added Body:setFixedRotation and Body:getFixedRotation.
- Added Body:setInertia.
- Added CircleShape:getLocalCenter and CircleShape:getWorldCenter.
- Added icons and file associations for the debs.
- Added the demos folder to the Mac OS X DMG.
- It's now possible to run a .love from Resources in Mac OS X, thanks to Steve Johnson.
- Fixed a bug with multiple Sources on the same Music.
- Fixed a bug so the mouse doesn't get crippled when the keyboard is disabled.
- Fixed a bug where love.graphics.rectangle drew a too large rectangle.
- Fixed a bug where memory wouldn't be released correctly.
- Fixed epic physics typo (getRestituion->getRestitution).
- Fixed crash on opening non-existent image.
- The error screen redraws when an event occurs.
- The default love.run() now gracefully handles disabled modules.
- The debian packages should now successfully include icons, file associations, etc, and should give the correct architecture.
- Added support for drawing polylines to love.graphics.line - the syntax is the same as love.graphics.polygon.
- Removed Music and Sound. There are now only sources.
- Improved the stability of love.audio/love.sound.

nogame screen

0.6.2

0.6.2 · revision 22585 · last edited 2021-03-14

The codename of LÖVE 0.6.2 is "**Jiggly Juice**". It is currently **released**.

Download 0.6.2 from SourceForge (http://sourceforge.net/projects/love/files/love/love-0.6.2/)



Changes from 0.6.1

- Fixed a bug causing ImageFonts to cut off some pixels.
- Fixed a bug where filled rectangles were too small.
- Fixed a bug in Image:setFilter where it would switch the parameters.
- Fixed a bug in ImageRasterizer where it wasn't using the data.
- Image filter and wrap modes now use string constants as well.
- Fixed double-transform bug in SpriteBatch.
- Errors are reported on stdout again.
- Another fix for the icons on Ubuntu.

nogame screen

0.7.0

0.7.0 · revision 15616 · last edited 2021-03-14

The codename of LÖVE 0.7.0 is "**Game Slave**". It is currently **released**.



no-game screen

Changes from 0.6.2

Additions

- Added love.thread.
- Added love.font.
- Added love.graphics.Framebuffer.
- Added Source:play, Source:pause, etc.
- Added Source:isStatic.
- Added Source:getPosition/setPosition, Source:getVelocity/setVelocity, and Source:getDirection/setDirection to Source.
- Added CircleShape:getGroupIndex/setGroupIndex
- Added PolygonShape:getGroupIndex/setGroupIndex
- Added Font:getWrap.
- Added identity field to love.conf.
- Added love.quit callback.
- Added love.focus callback.
- Added extra meter parameter to love.physics.newWorld.
- Added love.graphics.setIcon.
- Added way to make the window desktop resolution.
- Added subtractive and multiplicative blend modes.
- Added Body:getAllowSleeping.
- Added Shape:getBody.
- Added love.filesystem.FileData for public usage.
- Added base64 support for love.filesystem.FileData.

Bugfixes

- Fixed the debug module not being an upvalue of the error handlers. (you can now override debug)
- Fixed some cases when love.audio.pause and friends were acting on everything, not just the passed Source.
- Fixed setFixedRotation enabling other flags.
- Fixed a bug in the loader (for require).
- Fixed ParticleSystem::setSprite not retaining the new image.
- Fixed setMode removing images settings (wrapping, filters).
- Fixed Shape:getBody, it's now exposed for LÖVE usage.
- Fixed DistanceJoint:getType() returning "circle" - it now returns "distance".
- Fixed SpriteBatches being unaffected by setColor
- Fixed the audio bug.
- Fixed invalid FSAA values crashing LÖVE.
- Fixed a bunch of compiler warnings.
- Fixed OS X not properly using UTIs for .love files.
- Fixed the modplug decoder not properly handeling files that fail to load.
- Fixed a memory leak in setFont.
- Fixed bug where errors in threads wouldn't get picked up by demand.
- Fixed part of the bug with newlines when scaling text (rotating still messes up the lines).
- Fixed the bug where newImageFont would try to create ImageData out of ImageData.
- Fixed error handler not resetting the blend mode.

Others

- Changed fonts, they're now po2 safe.
- Changed the traceback in the error screen.
- Changed font origin to top-left.

- Changed behavior of love.physics.newWorld(w, h), it now creates a w*h world, not a 4*w*h world.
- Removed font functions from love.graphics.

0.7.1

0.7.1 · revision 15615 · last edited 2021-03-14

The codename of LÖVE 0.7.1 is "**Game Slave**". It is currently **released**.



Changes from 0.7.0

Additions

- Added Source:isPaused()
- Added error when initial window can't be created.
- Added framebuffer filter modes.
- Added love.filesystem.getLastModified.
- Added filter modes for ImageFonts.
- Added dead key support by using "unknown" key with correct unicode value.
- Added 0 width and height in love.conf. (for current desktop resolution)
- Added alpha support when encoding TGA images.

no-game screen

Bugfixes

- Fixed a lot of bugs regarding zero characters in threads.
- Fixed handling of a directory named "love" in current directory.
- Fixed a few unhandled errors in setScissor.
- Fixed a bug where old physics callbacks were never dereferenced.
- Fixed loss of mouse visibility settings on setMode.
- Fixed creation of a framebuffer unbinding current framebuffer.
- Fixed several race conditions in love.thread.
- Fixed 'love .', so it won't use lovedir/. as save dir.
- Fixed setLineHeight.
- Fixed extended ascii and ImageFonts.
- Fixed printf's line wrapping.
- Fixed crash when playing sounds.
- Fixed playback of mp3s with arbitrary sample rates.
- Fixed handling of negative indices in love.joystick.
- Fixed toggleFullscreen.
- Fixed unexpected behaviour with hash tables to love.graphics.line.
- Fixed mouse coordinates being capped after setMode.
- Fixed setFont's error handling on a non-existant file.
- Fixed issue where Windows builds would hard crash on Lua errors

Other

- Removed custom sample rates for Decoders.

0.7.2

0.7.2 · revision 14969 · last edited 2021-03-14

The codename of LÖVE 0.7.2 is "**Game Slave**". It is currently **released**.



Changes from 0.7.1

Additions

- Added Framebuffer:get/setWrap.
- Added love.event.clear.
- Added support for any number of arguments to love.keyboard.isDown, love.mouse.isDown and love.joystick.isDown.
- Added SpriteBatch:setImage.

Bugfixes

no-game screen

- Fixed fused games not working.
- Fixed ParticleSystem:setSize ignoring the variation argument.
- Fixed some file-opening exceptions not being caught.
- Fixed files loaded by libmodplug being too loud.
- Fixed paths with periods in them not working.
- Fixed love.graphics.getBlendMode not detecting subtractive and multiplicative blend modes.
- Fixed crash when there was no memory available for newImageData(w, h).

Other

- Updated PhysicsFS version to 2.0.2 on Windows
- Updated OpenAL Soft version to 1.13 on Windows
- Updated libmodplug version to 0.8.8.1 on Windows
- Updated FreeType version to 2.4.4 on Windows
- Updated libmpg123 version to 1.13.2 on Windows
- Windows binary no longer depends on VC2005 runtime.
- Windows binary no longer depends on SSE2 support.

0.8.0

0.8.0 · revision 14970 · last edited 2021-03-14

The codename for LÖVE 0.8.0 is **Rubber Piggy**. It is currently **released**.



no-game screen

Changes from 0.7.2

Additions

- Added release error screen.
- Added alpha to love.graphics.setBackgroundColor.
- Added Canvas:clear(r, g, b, a).
- Added Canvas support to love.graphics.drawq.
- Added Canvas:getWidth and Canvas:getHeight.
- Added love.graphics.arc.
- Added Source:seek and Source:tell.
- Added color interpolation to ParticleSystems.
- Added automatic PO2 padding for systems not supporting the OpenGL extension.
- Added UTF-8 support for fonts.
- Added Box2D error handling for some commonly failing functions.
- Added ability for fused release games to have their write dir in appdata.
- Added love.graphics.shear.
- Added origin to font printing.
- Added love.graphics.getMode.
- Added per-sprite colors on SpriteBatches.
- Added PixelEffects.
- Added love.graphics.isSupported.
- Added love.graphics.getCanvas.
- Added love.event.quit.
- Added stencil masks.
- Added alternative SpriteBatch provider, it should work everywhere now.
- Added a loader for binary modules.
- Added Thread:getKeys().
- Added option of fractions for quads.
- Added PNG, JPEG and GIF support to ImageData:encode.
- Added 64-bit support for Mac OS X.
- Added premultiplied blending mode.
- Added love.graphics.setDefaultImageFilter/getDefaultImageFilter.
- Added SpriteBatch:set.
- Added new events system, with support for custom events and long event names.
- Added sound attenuation by distance.
- Added SpriteBatch:getImage.
- Added combine color mode.
- Added automatic random seeding to love.run.
- Added support for the subtract BlendMode on older graphics cards.
- Added love._os field, which contains the OS the game is running on.

Bugfixes

- Fixed wrapping for single words.
- Fixed tracebacks not showing filenames.
- Fixed love.graphics.push/pop capable of causing overflows/underflows.
- Fixed setScissor on Canvases.
- Fixed several issues with audio, e.g. clicks and pops in mp3s.
- Fixed crashes when bodies were destroyed during collisions.
- Fixed bound SpriteBatches corrupting when drawing.
- Fixed thread-safety issues with ImageData.
- Fixed memory leaks in audio sources.

- Fixed Thread:set (previously send) accidentally changing the type.
- Fixed SoundData allocating the wrong number of samples.
- Fixed SpriteBatch support on Intel cards.
- Fixed love.filesystem.lines leaking.
- Fixed most leaking on unclosed File objects.
- Fixed crashes when operating on non-existent files.
- Fixed a bug where empty files on Windows would never reach EOF.
- Fixed crash when SoundData runs out of memory.
- Fixed ordering of loaders, love should have priority over lua.
- Fixed several miscellaneous memory leaks.
- Fixed a few cases where strings with \0 in them would not be stored correctly.
- Fixed LÖVE's startup time being in the first dt.
- Fixed internal string conversions, they are faster now.
- Fixed (bad) performance of ImageData:paste.
- Fixed love.graphics.toggleFullscreen not maintaining graphics state.

Other Changes

- Renamed SpriteBatch:lock/unlock to bind/unbind.
- Renamed Framebuffer to Canvas.
- Renamed Thread:send/receive to set/get.
- Renamed love.graphics.setRenderTarget to setCanvas.
- Updated love.joystick to be 1-indexed.
- Updated Sources to update more cleanly and control more intuitively.
- Updated font engine.
- Updated line drawing to a custom system.
- Updated love.timer.sleep to use seconds, like the rest of love.
- Updated love.timer to be more accurate.
- Updated love.graphics.circle to have max(10, r) as default for segments.
- Updated ImageData:encode to write to files directly.
- Updated version compatibility system to actually do something.
- Updated love.run's order, events are checked just before update.
- Updated Box2D to version 2.2.1.
- Updated sdl to 1.2.15 on Windows
- Updated freetype to 2.4.9 on Windows
- Updated libjpeg to 8d on Windows
- Updated libmodplug to 0.8.8.4 on Windows
- Updated libogg to 1.3.0 on Windows
- Updated libpng to 1.5.10 on Windows
- Updated mpg123 to 1.13.7 on Windows
- Updated openal-soft to 1.14 on Windows
- Updated zlib to 1.2.6 on Windows

Removals

- Removed Canvas auto-clearing.
- Removed EncodedImageData.
- Removed old syntax for require (with extension and slashes).
- Removed love.graphics.setFont([file,][size]).
- Removed Thread:kill.

0.8.1

0.8.1 · revision 10325 · last edited 2021-03-14


LÖVE 0.8.1 was never released and development continued into 0.9.0.

All changes in 0.8.1 are transferred to 0.9.0.

0.9.0

0.9.0 · revision 14971 · last edited 2021-03-14

On this page

- Major changes from 0.8.0
- All changes from 0.8.0
  - Additions
  - Removals
- Renamed APIs
- Fixes
- Other Changes

The codename for LÖVE 0.9.0 is **Baby Inspector**. It is currently **released**.



Major changes from 0.8.0

- Added three new modules: love.window, love.math, and love.system.
- Reworked the love.joystick and love.thread modules.
- Added new features to love.graphics, including polygon meshes and vertex shaders.
- Integrated the ENet (http://enet.bespin.org) networking library via lua-enet (http://leafo.net/lua-enet/).
- Renamed many functions to be more consistent with the rest of the API.
- Improved the performance of several APIs in love.graphics.
- And a **lot** more!

no-game screen

All changes from 0.8.0

Additions

- Added better multiplayer networking support with ENet (http://enet.bespin.org) (via the lua-enet (http://leafo.net/lua-enet/) library.)
- Added --fused command line argument, to simulate fusing.
- Added liblove.
- Added the ability to have exit values.
- Added exit value of 1 in case of error by default.
- Added basic support for the file:// URI scheme.
- Added love.timer.getAverageDelta.
- Added Data:getString.
- Added Contact:getChildren.

- Added love.filesystem.isFused.
- Added love.filesystem.getIdentity.
- Added love.filesystem.append.
- Added love.filesystem.getSize.
- Added love.filesystem.mount and love.filesystem.unmount.
- Added an optional file search order parameter to love.filesystem.setIdentity.
- Added File:isOpen and File:getMode.
- Added File:setBuffer, File:getBuffer, and File:flush.

- Added textinput event for unicode text input.
- Added love.keyboard.setTextInput and love.keyboard.hasTextInput.

- Added previously internal Rasterizer and GlyphData methods.
- Added support for UTF-8 Image Fonts.
- Added Font:getAscent, Font:getDescent, and Font:getBaseline.
- Added Font:setFilter and Font:getFilter.
- Added Font:hasGlyphs.

- Added angle, scale, and shear parameters to love.graphics.printf.
- Added support for high dynamic range (HDR) Canvases.
- Added mipmapping support to Images (has love.graphics.isSupported test).
- Added vertex shader support.
- Added boolean support to Shader:send.

- Added Canvas:getPixel.
- Added BlendMode "replace".
- Added LineJoin modes.
- Added Mesh objects, allowing for arbitrary textured polygons.
- Added multiple render target support to love.graphics.setCanvas.
- Added love.graphics.setColorMask.
- Added love.graphics.origin.
- Added love.graphics.getRendererInfo.
- Added love.graphics.getMaxImageSize.
- Added SpriteBatch:getCount and SpriteBatch:getBufferSize.
- Added SpriteBatch:getColor.
- Added ParticleSystem:emit.
- Added ParticleSystem:setInsertMode.
- Added ParticleSystem:setAreaSpread.
- Added ParticleSystem:isPaused and ParticleSystem:isStopped.
- Added many ParticleSystem getter methods.
- Added DXT compressed texture support to Images via love.image.newCompressedData.
- Added love.image.isCompressed and Image:isCompressed.
- Added Image:getDimensions, Canvas:getDimensions, and ImageData:getDimensions.
- Added anisotropic filtering support for Images, Canvases, and Fonts.
- Added Image:refresh.
- Added Image:getData.

- Added love.math module.
- Added a platform-independent (good) random implementation to love.math.
- Added RandomGenerator objects.
- Added BezierCurve objects.
- Added love.math.triangulate and love.math.isConvex.
- Added love.math.noise.

- Added love.system module.
- Added love.system.getClipboardText and love.system.setClipboardText.
- Added love.system.getOS and love.system.getProcessorCount.

- Added love.window module.
- Added love.window.isVisible.
- Added flags to love.window.setMode.
- Added monitor choosing support to love.window.setMode.
- Added support for resizable, borderless, and non-centered windows.
- Added support for "fullscreen-desktop" mode.
- Added window resize and visible events.
- Added love.window.getIcon.
- Added t.window.icon to love.conf.

- Added SoundData:getDuration and SoundData:getSampleCount.
- Added Source:isPlaying.
- Added Source:setRelative and Source:isRelative.
- Added Source:setCone and Source:getCone.
- Added Source:getChannels.

- Added new Channels API for love.thread.
- Added limited table support to Channel:push.
- Added Thread:getError.
- Added Thread:isRunning.
- Added threaderror event.

- Added love.mousefocus and love.window.hasMouseFocus.
- Added custom hardware cursor support via love.mouse.newCursor.
- Added love.mouse.setX and love.mouse.setY.

- Added Joystick objects.

- Added joystick add and remove events.
- Added joystick axis and hat move events.
- Added unified Gamepad API for joysticks which have a similar layout to the Xbox controller.
- Added joystick vibration support.

Removals

- Removed release mode (but Fused mode is still there.)
- Removed love.keyboard.getKeyRepeat (see love.keyboard.hasKeyRepeat).
- Removed the unicode argument from love.keypressed (see love.textinput).

- Removed love.graphics.drawTest.
- Removed love.graphics.quad and love.graphics.triangle.
- Removed ColorModes and love.graphics.setColorMode.
- Removed love.graphics.newStencil.
- Removed love.graphics.setLine and love.graphics.setPoint.
- Removed love.graphics.drawq (functionality is merged into love.graphics.draw).
- Removed SpriteBatch:addq and SpriteBatch:setq (functionality is merged into SpriteBatch:add and SpriteBatch:set).
- Removed Quad:flip.
- Removed ParticleSystem:isFull and ParticleSystem:isEmpty.
- Removed ParticleSystem:getX and ParticleSystem:getY.
- Removed love.graphics.checkMode.

- Removed love.joystick module functions which operated on individual joysticks (see Joystick objects).
- Removed joystick ball support.
- Removed thread names.
- Removed old thread messaging API (see Channels).
- Removed love.timer.getMicroTime.

Renamed APIs

- Renamed love's boot script to 'love.boot', which can be required.
- Renamed love.filesystem.mkdir to love.filesystem.createDirectory.
- Renamed love.filesystem.enumerate to love.filesystem.getDirectoryItems.
- Renamed World:setAllowSleeping and World:getAllowSleeping to World:setSleepingAllowed and World:isSleepingAllowed.
- Renamed ChainShape:setPrevVertex to ChainShape:setPreviousVertex.
- Renamed Joint:enableMotor to Joint:setMotorEnabled.
- Renamed Joint:enableLimit and Joint:isLimitEnabled to Joint:setLimitsEnabled and Joint:hasLimitsEnabled.
- Renamed t.screen to t.window in love.conf.
- Renamed love.graphics.setCaption to love.window.setTitle.
- Renamed PixelEffect to Shader (but now with vertex shaders).
- Renamed love.graphics.setDefaultImageFilter to love.graphics.setDefaultFilter.
- Renamed ParticleSystem:setSprite to ParticleSystem:setImage.
- Renamed ParticleSystem:setGravity to ParticleSystem:setLinearAcceleration.
- Renamed ParticleSystem:setLifetime and ParticleSystem:setParticleLife to ParticleSystem:setEmitterLifetime and ParticleSystem:setParticleLifetime.
- Renamed ParticleSystem:count and all getNum* functions to get*Count.
- Renamed SoundData:getBits and Decoder:getBits to SoundData:getBitDepth and Decoder:getBitDepth.
- Renamed Source:setDistance and Source:getDistance to Source:setAttenuationDistances and Source:getAttenuationDistances.
- Renamed love.mouse.setGrab to love.mouse.setGrabbed.

Fixes

- Fixed fused mode in OS X.
- Fixed printing to the console in Windows before love.load is called.
- Fixed the default love.run to not include the time taken by love.load in the first frame's dt calculation.
- Fixed the error screen not always appearing until the next input event.
- Fixed love.event.clear.
- Fixed love.mouse.setPosition when called in love.load.

- Fixed scaling in several love.physics functions.

- Fixed Box2D exception in World:update.
- Fixed many uncaught Box2D / love.physics exceptions for Bodies and Joints.
- Fixed ChainShape:getPoints running out of Lua stack space and crashing.

- Fixed File:read reading past end of file.
- Fixed love.filesystem.setIdentity not removing read access from old directories.

- Fixed possible memory leak in UTF-8 decoder.
- Fixed spacing for the last character in an Image Font.
- Fixed line wrapping in love.graphics.printf.
- Fixed love.graphics.printf to error if the wrap limit is negative.
- Fixed love.graphics.print truncating strings with embedded zeros.
- Fixed crashes with font drawing on some ATI cards.
- Fixed artifacts when drawing lines at huge scale.
- Fixed Fonts and Canvases ignoring default image filter.
- Fixed love.graphics.getLineWidth returning incorrect values.
- Fixed love.graphics.getColor on some Windows systems.
- Fixed alpha BlendMode.
- Fixed multiplicative BlendMode.
- Fixed love.graphics.getPointStyle.
- Fixed line numbers in Shader errors.
- Fixed Shader:send with Images and Canvases failing sometimes.
- Fixed Shader:send to keep a reference to sent Images and Canvases.
- Fixed crash when binding SpriteBatches multiple times.
- Fixed SpriteBatches with more than 16,384 sprites.
- Fixed draw order for ParticleSystems.
- Fixed ParticleSystem:setSizes resetting the size variation.

- Fixed love.window.getMode and friends returning wrong values when using desktop size.
- Fixed keyrepeat settings being lost after (indirect) love.window.setMode.
- Fixed the icon being reset after love.window.setMode.

- Fixed memory leak in the mp3 decoder.
- Fixed sound issues with some versions of OpenAL soft, by enabling direct channels.
- Fixed 'random' hangs in audio.
- Fixed case (in)sensitivity of sound file extension parsing.
- Fixed looping support in tracker music formats.
- Fixed skipping/looping issues when playing streaming audio Sources.
- Fixed a race condition in Source:play.
- Fixed WAVE sound playback.

Other Changes

- Moved love's startup to modules/love.
- Moved window-related functions from love.graphics to love.window.

- Updated functions which return LOVE objects to re-use the Lua-side object instead of always recreating it.

- Updated the windows console, it now tries to re-use an active one first.
- Updated error handling, error handlers now get resolved when the error occurs.
- Updated order of sleep/present in love.run (now draws, *then* sleeps).

- Updated love.keyboard.setKeyRepeat to take a boolean argument instead of numbers.
- Updated love.keypressed's second argument to indicate whether the keypress is a repeat.
- Updated keyboard key constants with some modern keyboard keys.

- Updated love.filesystem to try to create the appdata directory if it doesn't exist yet.
- Updated the default filesystem identity to omit file extension.
- Updated love.filesystem.newFile to optionally open the file.
- Updated most love.filesystem functions to return nil, error on internal failure.

- Updated love.graphics.print's x and y arguments to default to 0.

- Updated the setFilter and setWrap methods, the second argument is now optional.
- Updated Font and ParticleSystem rendering code, now more performant.
- Updated SpriteBatch code, now more performant when adding/setting and (un)binding.
- Updated Canvas code to support more systems.
- Updated Canvas:getImageData and love.graphics.newScreenshot to be more efficient.
- Updated love.graphics.newScreenshot to create a fully opaque image by default.
- Updated error messages when sending bad values to Shaders.
- Updated love.graphics.newParticleSystem to have a default buffer size of 1000.

- Updated love.image.newImageData and love.graphics.newImage to accept FileData.
- Updated ImageData:setPixel to accept a table and default to 255 alpha.
- Updated ImageData:mapPixel, is now more efficient and now accepts optional x,y,w,h arguments.
- Updated love.image memory handling, improves errors and thread-safety.

- Updated love.audio.newSource, love.sound.newDecoder, and love.sound.newSoundData to accept FileData.
- Updated allocation for SoundData, it's more efficient and less wasteful.
- Updated SoundData:setSample and SoundData:getSample to error for invalid samples.
- Updated Source:set* functions to default z to 0.
- Updated Source:seek to error for negative offsets.

- Updated Thread:start to accept arguments. The arguments are available in the thread via **...** (the vararg expression.)

- Updated love.timer.getFPS to be microsecond-accurate.
- Updated love.timer.getTime to be microsecond-accurate and monotonic.

- Updated Box2D to version 2.3.

0.9.1

0.9.1 · revision 22608 · last edited 2021-03-14

The codename for LÖVE 0.9.1 is **Baby Inspector**. It is currently **released**.



Changes from 0.9.0

Additions

- Added love.getVersion.
- Added love.system.openURL.
- Added "screen" BlendMode.
- Added Source:clone.
- Added ParticleSystem:clone.
- Added ParticleSystem:moveTo, has smoother emitter movement compared to ParticleSystem:setPosition.

no-game screen

- Added ParticleSystem:setRelativeRotation.
- Added love.graphics.setWireframe and love.graphics.isWireframe for debugging.
- Added Mesh:setDrawRange and Mesh:getDrawRange.
- Added Mesh:setTexture, SpriteBatch:setTexture, and ParticleSystem:setTexture. Accepts Canvases and Images.
- Added high-dpi window support for Retina displays in OS X, via the 'highdpi' window flag.
- Added love.window.getPixelScale.
- Added love.graphics.getSystemLimit.
- Added antialiasing support to Canvases.
- Added Canvas:getFSAA.
- Added `love_ScreenSize` built-in shader variable.
- Added support for gamma-correct rendering.
  - Added a boolean `srgb` window setting to love.conf and love.window.setMode.
  - Added variant of love.graphics.newImage which interprets the image in the sRGB colorspace.
  - Added a new TextureFormat "srgb" for use with Canvases.
  - Added love.graphics.isSupported("srgb")
  - Added love.math.gammaToLinear and love.math.linearToGamma.
- Added RandomGenerator:getState and RandomGenerator:setState.
- Added love.math.setRandomState and love.math.getRandomState.
- Added CircleShape:getPoint and CircleShape:setPoint.
- Added Body:setUserData and Body:getUserData.
- Added some missing obscure KeyConstants.
- Added optional callback argument to love.filesystem.getDirectoryItems.

Deprecations

- Deprecated Mesh:setImage, SpriteBatch:setImage, and ParticleSystem:setImage.
- Deprecated love.graphics.getMaxImageSize and love.graphics.getMaxPointSize.

Fixes

- Fixed love.graphics.scale with negative values causing incorrect line widths.
- Fixed Joystick:isDown using 0-based button index arguments.
- Fixed love.graphics.setCanvas() to restore the proper viewport and scissor rectangles.
- Fixed TrueType font glyphs which request a monochrome bitmap pixel mode.
- Fixed love.graphics.reset causing crashes when called in between love.graphics.push/pop.
- Fixed tab characters ("\t") to display properly with love.graphics.print.
- Fixed love.graphics.isCreated to return false if love.window.setMode fails.
- Fixed love.window.setMode to only destroy OpenGL resources after checking if a fullscreen size is supported.
- Fixed loading BC4 compressed textures.
- Fixed World:getBodyList and World:getJointList causing hard crashes.
- Fixed Fixture:setUserData and Fixture:getUserData to work in coroutines.
- Fixed Source:setPitch to error when infinity or NaN is given.

- Fixed SoundData objects being initialized with garbage sample values.
- Fixed 8-bit SoundData samples when used with love.audio Sources.

Other Changes

- Updated t.console in love.conf to create the console before modules are loaded in Windows.
- Updated the default love.run code to make initial love.math.random calls more random.
- Updated the error text for love.filesystem's module searchers when require fails.
- Updated the love.filesystem module searchers to be tried after package.preload instead of before.
- Updated love.filesystem's C library loader to look in paths added via love.filesystem.mount, in Fused mode.
- Updated love.graphics.newParticleSystem, love.graphics.newSpriteBatch, and love.graphics.newMesh to accept Canvases.
- Updated Canvas drawing code, texture coordinates are no longer flipped vertically.
- Updated Canvas:renderTo to work properly if a Canvas is currently active.
- Updated ParticleSystem:setEmissionRate to accept non-integer numbers.
- Updated Mesh vertex maps (index buffers) to use less VRAM space.
- Updated love.graphics.newMesh and Mesh:setVertices to default the UV parameters to 0,0.
- Updated fullscreen-desktop and resizable window modes in OS X to use Mac OS 10.7's fullscreen Spaces.
- Updated Source:play to return a boolean indicating success.

0.9.2

0.9.2 · revision 14973 · last edited 2021-03-14

On this page

- Changes from 0.9.1
  - Additions
  - Deprecations
  - Renamed APIs
  - Fixes
  - Other Changes

The codename for LÖVE 0.9.2 is **Baby Inspector**. It is currently **released**.



Changes from 0.9.1

Additions

- Added Lua 5.3's UTF-8 module (http://www.lua.org/manual/5.3/manual.html#6.5), via `utf8 = require("utf8")`.
- Added Shader:getExternVariable.
- Added several new Canvas texture formats.
- Added love.graphics.getCanvasFormats.
- Added love.graphics.getCompressedImageFormats.
- Added ParticleSystem:setQuads.
- Added ParticleSystem:setLinearDamping.
- Added ParticleSystem:getLinearDamping.
- Added SpriteBatch:flush.
- Added love.graphics.getStats.
- Added `mirroredrepeat` WrapMode.
- Added love.audio.setDopplerScale and love.audio.getDopplerScale.
- Added optional duration argument to Joystick:setVibration.
- Added love.joystick.loadGamepadMappings and love.joystick.saveGamepadMappings.
- Added Joint:setUserData and Joint:getUserData.
- Added Joint:getBodies.
- Added GearJoint:getJoints.
- Added Body:getWorld.
- Added Body:getJointList.
- Added Contact:getFixtures and Body:getContactList.
- Added Body:isDestroyed, Fixture:isDestroyed, Joint:isDestroyed, and World:isDestroyed.
- Added love.mousemoved event callback.
- Added love.mouse.setRelativeMode and love.mouse.getRelativeMode.
- Added love.keyboard.getKeyFromScancode and love.keyboard.getScancodeFromKey.
- Added love.window.getDisplayName.
- Added love.window.minimize.
- Added love.window.showMessageBox.
- Added love.window.toPixels and love.window.fromPixels.
- Added love.window.setPosition and love.window.getPosition.
- Added `x` and `y` fields to the `flags` table in love.window.setMode and love.window.getMode, and to the `t.window` table in love.conf.
- Added `refreshrate` field to the table returned by love.window.getMode.
- Added love.filesystem.isSymlink, love.filesystem.setSymlinksEnabled, and love.filesystem.areSymlinksEnabled.
- Added love.filesystem.getRealDirectory.

no-game screen

Deprecations

- Deprecated SpriteBatch:bind and SpriteBatch:unbind.
- Deprecated all uses of the name 'FSAA' in favor of 'MSAA'.
- Deprecated the `hdrcanvas` GraphicsFeature constant in favor of love.graphics.getCanvasFormats.
- Deprecated the `dxt` and `bc5` GraphicsFeature constants in favor of love.graphics.getCompressedImageFormats.

- Deprecated love.window.getWidth, love.window.getHeight, and love.window.getDimensions in favor of the love.graphics equivalents or love.window.getMode.

Renamed APIs

- Renamed all cases of FSAA to MSAA. The FSAA names still exist for backward-compatibility.

Fixes

- Fixed crashes when LÖVE objects are garbage-collected in multiple threads simultaneously.
- Fixed love.filesystem.setIdentity breaking in some situations when called multiple times.
- Fixed the default love.filesystem identity when in Fused mode in Windows.
- Fixed love.system.openURL sometimes blocking indefinitely on Linux.
- Fixed love.joystick.setGamepadMapping.
- Fixed the order of vertices in ChainShapes.
- Fixed love.mouse.getPosition returning outdated values if love.mouse.setPosition was used in the same frame.
- Fixed love.graphics.newFont to error when given an invalid size argument.
- Fixed the file name, line number, and backtrace given if love.graphics.print errors.
- Fixed a small memory leak if love.graphics.newCanvas errors.
- Fixed Shader:getWarnings returning unnecessary information.
- Fixed a potential crash when Shader objects are garbage collected.
- Fixed a potential small memory leak triggered when love.graphics.newShader errors.
- Fixed love.graphics.newMesh(vertexcount, ...) causing the Mesh to do instanced rendering.
- Fixed Mesh:getVertexMap.
- Fixed Image:refresh generating mipmaps multiple times if mipmap filtering is enabled.
- Fixed Image:setMipmapFilter to not keep bad state around if it errors.
- Fixed Mesh:setDrawRange when the Mesh has a vertex map set.
- Fixed internal detection of the 'position' and 'effect' shader functions.
- Fixed some cases of noncompliant shader code not properly erroring on some nvidia drivers.
- Fixed Texture memory leak when Meshes are garbage collected.
- Fixed the default line join mode to be 'miter' instead of an undefined value.
- Fixed Canvas:getPixel to error if the coordinates are not within the Canvas' bounds.
- Fixed detection of compressed textures to work regardless of the file's extension.
- Fixed the default error handler screen's text size when highdpi mode is enabled on a Retina monitor in OS X.
- Fixed the default error handler background color when sRGB mode is enabled for the window.
- Fixed the state of wireframe mode when love.window.setMode is called.
- Fixed love.window.setMode to fall back to the largest available mode if a width or height greater than the largest supported is specified and exclusive-fullscreen mode is used.

Other Changes

- Updated the Windows executable to always prefer the higher performance GPU on nvidia Optimus systems.
- Updated the --console command-line argument in Windows to open the console before conf.lua is loaded.
- Updated t.console and the --console command-line argument in Windows to use the existing console window, if LÖVE was launched from one.
- Updated the love executable to verify that the love library version matches before using it.
- Updated the Lua wrapper code for modules to avoid crashes when the module's instance is created, deleted, and recreated.
- Updated internal code for handling garbage collection of love objects to be more efficient.
- Updated love's initialization code to trigger a Lua error if love.conf has an error in it.
- Updated love.filesystem.getSaveDirectory and friends to strip double-slashes ("//") from their returned paths.
- Updated the error message when love.filesystem.write or File:open fails because the directory doesn't exist.
- Updated error message when love.math.setRandomSeed(0) is attempted.
- Updated the error message when invalid UTF-8 strings are used in love functions that expect UTF-8.
- Updated love.physics.newChainShape to error if the number of arguments is invalid.
- Updated love.physics.newPolygonShape and love.physics.newChainShape to accept a table of vertices.
- Updated love.thread.newThread to accept a literal string of code.
- Updated love-created threads to use names visible in external debuggers.
- Updated SpriteBatch:unbind / SpriteBatch:flush to use less VRAM if the SpriteBatch has the static usage hint.
- Updated love.graphics.newImage, love.image.newImageData, etc. to leave less temporary Lua-owned memory around.
- Updated love.graphics.push to accept different stack types to push. Current types are "transform" and "all".

- Updated love shaders to accept GLSL ES precision qualifiers (`lowp`, `mediump`, and `highp`) on variables, although they do nothing.
- Updated the error message for love.graphics.newShader to be less cryptic if an invalid filename is given.
- Updated compressed texture loading code to support BC6 and BC7 compressed textures (if the graphics driver supports them.)

11.0

11.0 · revision 24629 · last edited 2021-03-14

On this page

- Changes from 0.10.2
  - Additions
  - Renamed APIs
  - Other Deprecations
  - Removals
  - Other changes
  - Performance Improvements
  - Fixes

The codename for LÖVE 11.0 is **Mysterious Mysteries**. It is currently **released**.



Changes from 0.10.2

Additions

- Added formal deprecation warnings, on by default for non-fused games.
- Added love.setDeprecationOutput and love.hasDeprecationOutput.
- Added Object:release.
- Added Data:clone.
- Added queueable audio Sources.
- Added microphone recording support.
- Added low pass, high pass, and band pass filters to love.audio Sources.
- Added audio effect APIs (reverb, echo, etc.)
- Added variants of SoundData:getSample and SoundData:setSample which take a channel index parameter.
- Added variants of love.audio.play, love.audio.pause, and love.audio.stop which take a list of Sources.
- Added love.system.hasBackgroundMusic.
- Added variants of all table-with-fields-returning get* functions, the new variants take an existing table to fill in.
- Added a variant to World:update for controlling the number of internal iterations to perform.
- Added Body:isTouching.
- Added RopeJoint:setMaxLength.
- Added a click count argument to love.mousepressed and love.mousereleased.
- Added variants of love.filesystem.mount which accept a Data or FileData object containing zipped data.
- Added love.filesystem.getInfo (replaces love.filesystem.exists / isFile / isDirectory / isSymlink / getLastModified / getSize).
- Added love.filesystem.setCRequirePath and love.filesystem.getCRequirePath, and use that to find C libraries for require.
- Added variants of File:read and love.filesystem.read which take an enum to determine whether they return a FileData or a string.
- Added Channel:hasRead, which checks if a message has been read. Takes an id, which Channel:push will now return.
- Added variants of Channel:demand and Channel:supply which take a timeout argument.
- Added a default love.threaderror callback, which raises the error in the main thread.
- Added love.data module. It includes hex/base64 encoding functions, MD5 and SHA hashing, string packing, compression APIs, and more.
- Added Transform objects to love.math.
- Added support for different ImageData PixelFormats, including RGBA8 (the default), RGBA16, RGBA16F, and RGBA32F.
- Added the ability to load Radiance HDR, OpenEXR, and 16 bit PNG images.
- Added love.graphics.getImageFormats (replaces love.graphics.getCompressedImageFormats).
- Added the ability to specify a per-object pixel density scale factor when creating Images, Canvases, Fonts, and Videos. Affects drawing.
- Added Texture:getPixelWidth, Texture:getPixelHeight, love.graphics.getPixelWidth, and love.graphics.getPixelHeight.
- Added Texture:getDPIScale, love.graphics.getDPIScale, and Font:getDPIScale.
- Added Texture:getMipmapCount, getFormat, getLayerCount, getDepth, and getTextureType.
- Added Array, Cubemap, and Volume TextureTypes and corresponding Texture APIs.
- Added love.graphics.newArrayImage, newVolumeImage, newCubeImage, and variants to love.graphics.newCanvas for creating different texture types.
- Added love.graphics.getTextureTypes, which returns a table with fields indicating support for each TextureType.
- Added Image:replacePixels (replaces Image:refresh).
- Added love.graphics.drawLayer, SpriteBatch:addLayer, and SpriteBatch:setLayer for easily drawing layers of Array Textures
- Added variants of love.graphics.print and love.graphics.printf which take a Font argument.
- Added mipmapping support to Canvases, including both auto-generated mipmaps and manually rendering to a specific mipmap level.
- Added 'stencil8', 'depth24stencil8', 'depth32fstencil8', 'depth16', 'depth24', and 'depth32f' PixelFormats for Canvases.

no-game screen

- Added variant of love.graphics.newCanvas which accepts a table of settings.
- Added optional 'readable' boolean field to the table passed into love.graphics.newCanvas.
- Added variant of love.graphics.getCanvasFormats which takes a 'readable' boolean.
- Added optional 'depthstencil' field to the table passed into love.graphics.setCanvas, for using a depth/stencil formatted Canvas when rendering.
- Added optional 'depth' and 'stencil' boolean fields to the table passed into love.graphics.setCanvas, for enabling internal depth and stencil buffers if 'depthstencil' isn't used.
- Added shadow sampler support for Canvases.
- Added love.graphics.setDepthMode for using the depth buffer for depth testing/writes. Depth values of rendered objects can currently only be set via shaders.
- Added love.graphics.setMeshCullMode, for culling back- or front-facing triangles when drawing a Mesh.
- Added love.graphics.setFrontFaceWinding.
- Added variants of love.graphics.clear to control how the active depth and stencil buffers are cleared.
- Added love.graphics.applyTransform and love.graphics.replaceTransform.
- Added love.graphics.transformPoint and love.graphics.inverseTransformPoint.
- Added love.graphics.getStackDepth.
- Added love.graphics.flushBatch for manually flushing automatically batched draws.
- Added SpriteBatch:setDrawRange.
- Added File and FileData variants of love.graphics.newShader.
- Added per-Shader opt in support for the GLSL 3.30 and GLSL ES 3.00 shading languages.
- Added 'void effect()' pixel shader entry point.
- Added love.graphics.validateShader.
- Added Shader:hasUniform (replaces Shader:getExternVariable).
- Added support for non-square Shader uniform matrices on desktop platforms and on mobile GLSL 3.
- Added Shader:send(matrixname, matrixlayout, matrix, ...) which specifies how to interpret the matrix table arguments.
- Added Shader:send variants which accept a Data object.
- Added 'borderellipse' and 'borderrectangle' ParticleSystem distributions.
- Added variant of ParticleSystem:setEmissionArea which accepts an area angle and a flag for whether particle directions are relative to the center of the area.
- Added love.graphics.captureScreenshot (replaces love.graphics.newScreenshot).
- Added 'glsl3', 'instancing', 'fullnpot','pixelshaderhighp', and 'shaderderivatives' GraphicsFeatures.
- Added 'anisotropy' graphics GraphicsLimit.
- Added 'drawcallsbatched' field to love.graphics.getStats.
- Added hardware Mesh instancing support via love.graphics.drawInstanced and a new Mesh:attachAttribute variant.
- Added a Mesh:attachAttribute variant that takes a different target attribute name.
- Added Mesh:detachAttribute.
- Added a variant of Mesh:setVertexMap which accepts a Data object.
- Added the ability to prevent love from creating a stencil buffer for the window in love.conf and love.window.setMode.
- Added love.window.updateMode.
- Added love.window.isMinimized.
- Added love.window.restore.
- Added support for header-less DEFLATE to love.data.compress/decompress.

Renamed APIs

All renamed APIs in 11.0 are deprecated rather than removed, until the next major release.

- Renamed love.window.getPixelScale to love.window.getDPIScale.
- Renamed love.mouse.hasCursor to love.mouse.isCursorSupported.
- Renamed ParticleSystem:setAreaSpread to ParticleSystem:setEmissionArea.
- Renamed love.errhand to love.errorhandler. If love.errhand is defined and love.errorhandler isn't, it will continue to work.
- Renamed Source/SoundData/Decoder:getChannels to Source/SoundData/Decoder:getChannelCount.
- Renamed PrismaticJoint:hasLimitsEnabled and RevoluteJoint:hasLimitsEnabled to PrismaticJoint:areLimitsEnabled and RevoluteJoint:areLimitsEnabled.
- Renamed love.audio.getSourceCount to love.audio.getActiveSourceCount.
- Renamed all get[Object]List functions to get[Object]s.

Other Deprecations

- Deprecated love.filesystem.exists / isFile / isDirectory / isSymlink / getLastModified / getSize (use love.filesystem.getInfo instead).
- Deprecated love.math.compress / decompress (use love.data.compress / decompress instead).

Removals

- Removed the default source type for love.audio.newSource.
- Removed Source:isStopped and Source:isPaused. Use Source:isPlaying instead.
- Removed Source:rewind. Use Source:stop or Source:seek(0) instead.
- Removed Source:resume and love.audio.resume. Use Source:play and love.audio.play instead.
- Removed variant of love.filesystem.newFileData which takes base64 data, use love.data.decode instead.
- Removed the no-argument variant of Text:set, use Text:clear instead.
- Removed Image:getData and Image:refresh, use Image:replacePixels instead.
- Removed love.graphics.getCompressedImageFormats, use love.graphics.getImageFormats instead.
- Removed the 'void effects(...)' pixel shader entry point. Use the new 'void effect()' instead.
- Removed Shader:getExternVariable, use Shader:hasUniform instead.
- Removed love.graphics.newScreenshot, use love.graphics.captureScreenshot instead.
- Removed SpriteBatch:setBufferSize, spritebatches now automatically grow when they run out of space.
- Removed deprecated enet function host:socket_get_address.
- Removed functions deprecated in LÖVE 0.10.2:
    - Removed undocumented Shader:sendInt, Shader:sendBoolean, Shader:sentFloat, Shader:sendMatrix, and Shader:sendTexture methods (use Shader:send instead).
    - Removed love.window.isCreated (use love.window.isOpen instead).

Other changes

- Windows XP is no longer officially supported.
- Changed all color values to be in the range 0-1, rather than 0-255. This affects the following functions:
    - love.graphics.setColor, love.graphics.getColor, love.graphics.setBackgroundColor, and love.graphics.getBackgroundColor.
    - SpriteBatch:setColor and SpriteBatch:getColor.
    - ParticleSystem:setColors and ParticleSystem:getColors.
    - love.graphics.clear.
    - love.graphics.newMesh, Mesh:setVertex, and Mesh:getVertex.
    - The colored text variants of love.graphics.print, love.graphics.printf, love.graphics.newText, Text:set, Text:setf, Text:add, and Text:addf.
    - Shader:sendColor.
    - love.graphics.points.
    - ImageData:setPixel, ImageData:getPixel, and ImageData:mapPixel.
    - love.math.gammaToLinear and love.math.linearToGamma.
- Changed high-dpi functionality to require much less code (often none at all) for graphics to appear at the correct sizes and positions.
- Changed love.graphics.print and friends to ignore carriage returns.
- Changed the 'multiply' BlendMode to error if not used with the 'premultiplied' BlendAlphaMode, since the formula only works with that anyway.
- Changed the matrix variant of Shader:send to interpret the matrix as row-major by default, instead of column-major.
- Changed the variant of Canvas:newImageData which accepts x/y/width/height to also require slice and mipmap index parameters, which are used when the Canvas isn't a 2D type or has mipmaps.
- Changed some love.graphics, love.window, and love.event APIs to cause an error if a Canvas is active.
- Changed stenciling functionality with a Canvas active to require stencil=true (or a custom stencil-formatted Canvas) to be set in love.graphics.setCanvas.
- Changed Mesh:setDrawRange to take 'start' and 'count' parameters instead of 'min' and 'max'.
- Changed the variant of love.image.newImageData which takes dimension arguments to take an optional PixelFormat argument as well.

- Changed the 'vsync' field of love.window.setMode and t.window in love.conf. It's now an integer with 0 disabling vsync.
- Changed Channel and love.event.push table serialization to accept non-flat tables and detect cycles, cycles now cause an error rather than a stack overflow.
- Changed Sources to either be playing or not playing, rather than playing, paused, or stopped. Source:stop now pauses and rewinds the Source.
- Changed love.audio.pause() to return a list of Sources that have been paused by that call.
- Changed Source seeking behaviour, all kinds of Sources now behave similarly when seeking past the boundaries.
- Changed love.timer.step to return the calculated delta time.
- Changed love.run and love.errorhandler to return a function for their main loop, which gets called until love quits.
- Changed enet to no longer set the 'enet' global.
- Changed love.keyboard.isDown and love.keyboard.isScancodeDown to error if an invalid enum value is given.

- Updated and improved command line argument handling.
- Updated the boot sequence to show an error instead of the no-game screen, if a nonexistant folder is passed in as the source game directory.
- Updated 'love.exe --version' on Windows to print to the parent console.
- Updated Android print rerouting and JIT compilation disabling to apply inside threads.
- Updated invalid enum value error messages to show a list of the valid enum values.
- Updated Source:seek to work if the Source isn't playing.
- Updated love.math.random to have improved numeric distribution.
- Updated love.graphics to support Core Profile OpenGL 3.3+ when available.
- Updated SpriteBatches to automatically grow their buffer when they run out of space.
- Updated Shaders to always expose derivative functions (dFdx, dFdy, fwidth) when available in OpenGL ES.
- Updated Shaders to allow using VERTEX and PIXEL as variable names in shader code.
- Updated love.graphics.circle, love.graphics.ellipse, love.graphics.arc, and love.graphics.rectangle to take transformation scale into account when determining the number of segments to use.
- Updated the error message when bad values are given to love.graphics.line.
- Updated Font glyph generation to improve antialiasing.
- Updated Canvas:newImageData to return an ImageData with a format that matches the Canvas' as closely as possible.
- Updated love.graphics.newImage to treat file names ending with "@2x", "@3x", etc. as a pixel density scale factor if none is explicitly supplied.
- Updated the maximum love.graphics transformation/state stack depth from 64 to 128.
- Updated the default error handler to allow copying the error to the clipboard when the user decides to do so.
- Updated love.filesystem.setRequirePath to support multiple template '?' characters in each path.
- Updated luasocket to version 3.0rc1.
- Updated love.joystick.loadGamepadMappings so it doesn't error when given an empty string.
- Updated love.joystick.setGamepadMapping to use the controller's name for the new mapping when possible.

Performance Improvements

- Improved performance when drawing Textures, shapes, lines, and points by automatically batching their draw calls together when possible.
- Improved performance of Shader:send when the Shader is not active.
- Improved performance of love.math.randomNormal when LuaJIT's JIT compiler is enabled.
- Improved performance of love.filesystem.lines and File:lines, especially when reading from a file inside a zip/.love.

Fixes

- Fixed an error in the default error handler when the error message contains non UTF-8 bytes.
- Fixed a memory leak when sending love Objects to threads which never load that object's module.
- Fixed a memory leak in enet when peer:send fails.
- Fixed os.execute always returning -1 in Linux.
- Fixed the default reference angle for WeldJoint, PrismaticJoint, and RevoluteJoint.
- Fixed Fixture:getShape to reuse the existing internal Fixture-owned Shape.
- Fixed MouseJoint:setFrequency to error instead of crashing if a frequency of 0 is set.
- Fixed love.system.setClipboardText and love.system.getClipboardText to error instead of crashing, when a window hasn't been created.

- Fixed Joystick:getGamepadMapping to work with xinput controllers.
- Fixed love.joystick.setGamepadMapping's replacement code.
- Fixed baseline calculation when rendering text.
- Fixed Shader:send and Shader:sendColor ignoring the last argument for an array.
- Fixed VaryingTexCoords and love_ScreenSize in shaders to be 'highp' in OpenGL ES, when supported.
- Fixed a crash when love.graphics.pop is called after love.window.setMode while the transformation stack was not empty.
- Fixed ParticleSystem:setParticleLifetime to error if a negative value is given.
- Fixed love.window.isMaximized.
- Fixed Video playback to work with a wider range of Ogg Theora files.
- Fixed Video seeking to be faster.
- Fixed BezierCurves to error instead of hanging in some situations.
- Fixed compilation of luasocket with newer luajit 2.1.0 beta versions.

11.1

11.1 · revision 23492 · last edited 2021-03-14

The codename for LÖVE 11.1 is **Mysterious Mysteries**. It is currently **released**.



Changes from 11.0

Fixes

- Fixed love.graphics.setCanvas failing randomly.
- Fixed love.graphics.clear(colortable) when no Canvas is active.
- Fixed stencil and depth support on older phones.
- Fixed love.event.quit causing crashes and other unexpected behaviour if a Canvas is active.
- Fixed Fixture:getShape crashing when returning a ChainShape.
- Fixed love.joystick.loadGamepadMappings outputting a deprecation warning about love.filesystem.isFile.
- Fixed Source:queue to show the correct argument name in the error message when an invalid data parameter is given.
- Fixed t.console=true causing an error on Windows if lovec.exe is used.

no-game screen

11.2

11.2 · revision 22066 · last edited 2021-03-14

The codename for LÖVE 11.2 is **Mysterious Mysteries**. It is currently **released**.



Changes from 11.1

Additions

- Added Source:setAirAbsorption and Source:getAirAbsorption.
- Added Body:setTransform and Body:getTransform.

Performance Improvements

no-game screen

- Improved performance of love.graphics.draw slightly on iOS and Android.

Fixes

- Fixed love.filesystem initialization on Windows 10 update 1809.
- Fixed various issues with running on macOS 10.14.
- Fixed compatibility with Lua 5.2 and 5.3.
- Fixed love.system.openURL crashing on Android in some cases.
- Fixed the explicit format + Data argument variant of love.data.decompress.
- Fixed love.joystick.setGamepadMapping not being able to change existing mappings.
- Fixed a crash on quit on Linux if a custom Cursor is active when quitting.
- Fixed a crash in the Data variant of Shader:send when it's called after love.window.setMode.
- Fixed a love.graphics.setCanvas error message to be less confusing.

11.3

11.3 · revision 23881 · last edited 2021-03-14

The codename for LÖVE 11.3 is **Mysterious Mysteries**. It is currently **released**.



no-game screen

Changes from 11.2

Additions

- Added support for FLAC audio files.
- Added support for microphone recording on Android.
- Added t.audio.mic (false by default). On Android, setting it to true requests microphone recording permission from the user.
- Added Decoder:clone.
- Added Data:getFFIPointer.
- Added Joystick:getDeviceInfo.
- Added Joystick:getGamepadMappingString and love.joystick.getGamepadMappingString.
- Added love.math.colorToBytes and love.math.colorFromBytes.
- Added 'usedpiscale' boolean (true by default) to love.window.setMode and love.conf. Disables automatic DPI scaling when false.
- Added love.window.getDisplayOrientation and a love.displayrotated callback.
- Added love.window.get/setVSync, to allow setting vsync without recreating the window.
- Added love.window.getSafeArea.
- Added an optional vertex count parameter to Mesh:setVertices.
- Added support for rgba4, rgb5a1, rgb565, rgb10a2, rg11b10f, r8, rg8, r16, rg16, r16f, rg16f, r32f, and rg32f formats in ImageData and Images.
- Added support for loading .dds files that contain uncompressed pixel data.

Other Changes

- Changed audio file type detection, so it probes all supported backends for unrecognized extensions.

Fixes

- Fixed "bad lightuserdata" errors when running love on some arm64 devices.
- Fixed boot.lua's line numbers in stack traces to match its source code.
- Fixed the deprecation system not fully restarting when love.event.quit("restart") is used.
- Fixed love.isVersionCompatible.
- Fixed named Channels persisting across love.event.quit("restart") occurrences.
- Fixed race conditions when different love.physics Worlds are used in different threads.
- Fixed World:getJoints to return the fully resolved type of the Joint, instead of the base type.
- Fixed love.timer.sleep(0) to return control to the OS scheduler instead of being a no-op.
- Fixed love.math.randomNormal incorrectly using cached state after love.math.setRandomSeed or setRandomState.
- Fixed love.data.hash returning an incorrect hash for certain input sizes.
- Fixed love.data.newByteData to cause a Lua error instead of crashing when invalid arguments are used.
- Fixed the Data-returning variant of love.filesystem.read and File:read to return the number of bytes that were read.
- Fixed love.filesystem's require loaders to error instead of crashing when no argument is given.
- Fixed love.filesystem.mount(Data).
- Fixed a memory leak when loading files in some situations.
- Fixed t.audio.mixwithsystem.
- Fixed audio clicks immediately after playing a Source on iOS.
- Fixed Source:play + Source:stop + Source:play looping the first few ms of sound for streaming Sources on iOS.
- Fixed Source:play + Source:seek looping the first few ms of sound for streaming Sources on iOS.
- Fixed occasional pops in streaming sources on iOS.
- Fixed love.audio.play(sources) to use previously set playback positions on stopped Sources.
- Fixed Source:setEffect(name, true) and Source:getEffect(name) when the effect has no associated Filter.

- Fixed love.audio.newSource(filename, "queue") to cause a Lua error.
- Fixed Source:setPitch to error if the given pitch is <= 0, NaN, or infinity.
- Fixed video seeking and pausing in various scenarios.
- Fixed an audio Source memory leak when a Video gets garbage collected after playing it.
- Fixed video playback support on some Adreno-based Android devices.
- Fixed black fringes around text in some situations.
- Fixed extreme flickering when text moves along non-integer coordinates.
- Fixed the first character in a string sometimes not being processed during text vertex generation.
- Fixed Text:set(" ") not clearing any previously set text in a Text object.
- Fixed love.graphics.getTextureTypes to return a table with boolean values in its fields instead of number values.
- Fixed lines not rendering properly if a single line has more than 65,000 vertices.
- Fixed a pixel shader performance regression on some graphics drivers when OpenGL 3 or OpenGL ES 3 is used.
- Fixed text not showing up on Radeon HD 3000-series graphics cards on Windows.
- Fixed non-integer DPI scale values being truncated to integers in love.graphics.newCanvas.
- Fixed creating depth canvases on Windows systems when using an Intel HD 3000 GPU.
- Fixed automatic batching performance to be more consistent on all operating systems.
- Fixed gammaToLinearPrecise in shaders not being as precise as it should be.
- Fixed ImageData:paste and ImageData:setPixel to have more consistent clamping and rounding of color values when different formats are used.

11.4

11.4 · revision 27345 · last edited 2022-01-02

The codename for LÖVE 11.4 is **Mysterious Mysteries**. It is currently **released**.



Changes from 11.3

Additions

- Added native Apple Silicon (arm64) support on macOS.
- Added a variant of love.filesystem.newFileData which accepts a Data object.
- Added Body:getLocalPoints.
- Added Font:getKerning.
- Added support for r16, rg16, and rgba16 pixel formats in Canvases.
- Added Shader:send(name, matrixlayout, data, ...) variant, whose argument order is more consistent than `Shader:send(name, data, matrixlayout, ...)`.

no-game screen

Other Changes

- Changed all builds and platforms where LOVE provides LuaJIT to use LuaJIT 2.1 instead of 2.0.
- Changed love.timer.getTime to start at 0 when the module is first loaded.
- Changed certain out-of-Lua-memory situations to show a message box instead of instantly crashing.
- Changed the naming scheme of LOVE's embedded Lua files for improved integration with Lua chunkname APIs.

Fixes

- Fixed build-time compatibility with Lua 5.4.
- Fixed code compatibility with math.mod and string.gfind when LuaJIT 2.1 is used.
- Fixed errors on some systems related to > 53 bit pointer addresses, when recent versions of LuaJIT 2.1 are used.
- Fixed the default error handler showing a blank screen on some mobile devices.
- Fixed drag-and-drop to open a love game on macOS causing love.event.quit("restart") to fail.
- Fixed fused macOS apps opening other love games when drag-and-drop is used (if the fused app hasn't already removed .love files from recognized document types).
- Fixed File:isEOF when called on a dropped file.
- Fixed support for > 2GB dropped files on desktops.
- Fixed ByteData missing a Data:clone implementation.
- Fixed love.physics meter scale value persisting after love.event.quit("restart").
- Fixed audio to resume properly after interruption on iOS.
- Fixed love.graphics.newVideo to error instead of crash when an invalid video file is given.
- Fixed initial window creation to set the window's title during creation instead of after.
- Fixed the window's screen position when exiting fullscreen via love.window.setFullscreen.
- Fixed love.displayrotated being given a boolean instead of an enum string.
- Fixed memory corruption and a crash when drawing smooth lines.
- Fixed a crash in Canvas:newImageData when the pixel format's pixel byte size multiplied by its width isn't a multiple of 4.
- Fixed love.graphics.newVolumeImage when explicit mipmaps are provided.
- Fixed freezes and crashes in automatic batching when an AMD GPU is used.
- Fixed love.graphics.print and Image:replacePixels on more AMD/ATI GPUs.
- Fixed Font:setFallbacks to account for different DPI scales in each fallback font.
- Fixed Font:getWrap to not remove trailing newlines.
- Fixed Text:getWidth when the Text's string only contains spaces.
- Fixed a crash with some Intel graphics drivers on Linux.
- Fixed a hang with some Intel graphics drivers on Windows, by preventing gamma correct rendering on affected systems.
- Fixed a crash with some Intel graphics drivers on Windows when mipmapped Canvases are used.
- Fixed texture memory reported by love.graphics.getStats when a volume or array Canvas is created.
- Fixed DXT1 textures which use 1 bit alpha-cutout.
- Fixed rare issues where textures were not sent to shaders correctly.

- Fixed Shader:send(name, data, matrixlayout, ...).
- Fixed quad offsets in ParticleSystems when ParticleSystem:setOffset is not used.
- Fixed a performance issue with setting a small subrange of data in non-stream Meshes and SpriteBatches.
- Fixed rounded rectangles breaking if the rx or ry parameters are negative.
- Fixed rounded rectangle automatic points calculation when rx or ry are more than half the rectangle's size.
- Fixed source code compilation on Xcode 12+.
- Fixed source code compilation on Linux systems that don't provide posix_spawn APIs.

11.5

11.5 · revision 29021 · last edited 2023-12-03

The codename for LÖVE 11.5 is **Mysterious Mysteries**. It is currently **released**.



Changes from 11.4

Additions

- Added "LÖVE Loader" launcher on Android for easier loading of .love files.

Other Changes

- Changed iOS game selector to alphabetically sort the list of .love files.
- Changed JIT compilation on macOS arm64 (Apple Silicon) to be off by default, since performance and available JIT memory isn't reliable.

no-game screen

Fixes

- Fixed inconsistent and buggy behaviour of pairs (https://www.lua.org/manual/5.1/manual.html#pdf-pairs) by updating LuaJIT.
- Fixed "unexpected alignment" errors when running love on some 32 bit Linux systems.
- Fixed running fused games on Windows when the executable has been code-signed.
- Fixed undefined behaviour in love.data.hash's implementation.
- Fixed writing files when a symlink exists in the save directory's path.
- Fixed love.threaderror not being called if the error message is an empty string.
- Fixed a race condition when a Thread is destroyed immediately after Thread:start.
- Fixed unexpectedly slow first frames on macOS.
- Fixed love.joystick.setGamepadMapping when replacing an existing mapping.
- Fixed love.joystick.getGamepadMappingString.
- Fixed duplicate platform fields in love.joystick.saveGamepadMappings.
- Fixed DistanceJoint type information.
- Fixed time drift in Source:tell after a Source loops.
- Fixed audio not always pausing when the app is minimized on Android.
- Fixed RecordingDevice:start to return false instead of hard-crashing on iOS.
- Fixed identical frames in Ogg Theora videos being skipped.
- Fixed love.font.newBMFontRasterizer's single file parameter variant.
- Fixed the original window size not always being restored when exiting fullscreen on Linux.
- Fixed some cases of framerate hitches in Windows when vsync is enabled in windowed mode.
- Fixed colors appearing over-saturated on P3 displays in macOS.
- Fixed textures looking washed out when gamma-correct rendering is used on some Android devices.
- Fixed images with mipmaps when ANGLE is used with an AMD GPU.
- Fixed line rendering when duplicate points are used in the line.
- Fixed line rendering with miter and bevel line join modes when antiparallel lines are formed.
- Fixed a crash when calling Text:add with an empty string parameter.

12.0

12.0 · revision 31653 · last edited 2026-08-15

On this page

- Changes from 11.5
  - Additions
  - Changes
- Deprecations
- Removals
- Fixes


The information on this page may not be accurate or up to date due to the information pertaining to a version that is currently in development.

The codename for LÖVE 12.0 is **Bestest Friend**. It is currently **not released**.

Work-in-progress GitHub action builds (https://github.com/love2d/love/actions?query=branch%3Amain+event%3Apush) or (unofficial) nightly builds (https://nightly.link/love2d/love/workflows/main/main) are available for download.

Changes from 11.5

Additions

General

- Added support for launching a specific .lua file as the main file.
- Added basic usage instructions printed to the console when '--help' is used as a command-line parameter, and when the no-game screen is run.
- Added an option to add LÖVE to the user's command-line path in the Windows installer.
- Added love.parsedGameArguments and love.rawGameArguments tables, in the main thread.
- Added love.markDeprecated.
- Added love.event.restart(optionalvalue). A new love.restart field will contain the value after restarting.
- Added an optional wait timeout parameter to love.event.pump.
- Added love.system.getPreferredLocales.
- Added love.localechanged callback.
- Added love.dropbegan, love.dropmoved, and love.dropcompleted callbacks.
- Added drop position parameters to love.filedropped and love.directorydropped.
- Added HTTPS Lua module.
- Added variants of enet's peer:send and host:broadcast methods that take light userdata and size parameters.
- Added support in love Objects for Lua 5.4's '<toclose>' syntax, when love is compiled with Lua 5.4.

Data

- Added Data methods for getting number values.
- Added ByteData methods for setting number values.
- Added optional byte offset and size parameters to Data:getString.
- Added ByteData:setString.
- Added a variant of love.data.pack which takes an existing ByteData object.

Filesystem

- Added love.filesystem.mountFullPath and love.filesystem.unmountFullPath, including opt-in mount-for-write support.
- Added love.filesystem.mountCommonPath, unmountCommonPath, and getFullCommonPath.
- Added 'readonly' field to love.filesystem.getInfo's returned table.
- Added an optional load mode parameter to love.filesystem.load to only allow binary chunks, text chunks, or both.
- Added love.filesystem.openFile (replaces love.filesystem.newFile).
- Added love.filesystem.openNativeFile.

Audio

- Added optional stream type parameter to love.audio.newSource streaming sources ("file" or "memory"). It defaults to "file".
- Added love.audio.getPlaybackDevice, love.audio.getPlaybackDevices, and love.audio.setPlaybackDevice.

- Added SoundData:copyFrom.
- Added SoundData:slice.
- Added love.audiodisconnected callback.

Input

- Added a love.sensor module, for getting data from device sensors such as an accelerometer or gyroscope.
- Added love.sensorupdated callback.
- Added love.joysticksensorupdated callback.
- Added Joystick:hasSensor.
- Added Joystick:setSensorEnabled and Joystick:isSensorEnabled.
- Added Joystick:getSensorData.
- Added Joystick:setPlayerIndex and Joystick:getPlayerIndex.
- Added Joystick:getJoystickType.
- Added Joystick:getGamepadType.
- Added new Gamepad API buttons: "misc1", "paddle1", "paddle2", "paddle3", "paddle4". and "touchpad".
- Added love.keyboard.isModifierActive.
- Added direction enum parameter to love.wheelmoved.
- Added love.mouse.getGlobalPosition.
- Added variant of love.mouse.newCursor which takes an array of image paths, for handling multiple DPI scales.
- Added TouchDeviceType enum and 'ismouse' boolean parameter to love.touchpressed, love.touchmoved, and love.touchreleased.
- Added love.touch.getDeviceType.
- Added love.touch.isMouse.
- Added optional touch device type filter parameter to love.touch.getTouches.
- Added 't.trackpadtouch' boolean to love.conf. Trackpad touch events will not be generated unless it's enabled.

Math

- Added love.math.perlinNoise and love.math.simplexNoise (replaces love.math.noise).

Physics

- Added World:rayCastAny and World:rayCastClosest.
- Added World:getShapesInArea.
- Added World:queryShapesInArea (replaces World:queryBoundingBox).
- Added Body:getShapes (replaces Body:getFixtures).
- Added Body:getShape convenience method.
- Added Contact:getShapes (replaces Contact:getFixtures).
- Added Body:hasCustomMassData.
- Added convenience functions for creating a Body and Shape together:
  - Added love.physics.newCircleBody.
  - Added love.physics.newRectangleBody.
  - Added love.physics.newPolygonBody.
  - Added love.physics.newEdgeBody.
  - Added love.physics.newChainBody.
- Any joint that used to use spring-dampers now use stiffness and damping. The following functions have been added to support the old behaviour.
  - Added love.physics.computeLinearStiffness.
  - Added love.physics.computeLinearFrequency.
  - Added love.physics.computeAngularStiffness.
  - Added love.physics.computeAngularFrequency.

Window

- Added t.highdpi startup flag in love.conf, replacing t.window.highdpi and the highdpi flag of love.window.setMode.
- Added support for highdpi mode in Windows.
- Added love.window.showFileDialog.
- Added love.window.focus.
- Added love.window.getPointer.

- Added love.exposed and love.occluded callbacks.
- Added love.window.getSystemTheme and love.themechanged.

Graphics

- Added a Metal backend to love.graphics, available on macOS and iOS.
- Added a Vulkan backend to love.graphics, available on Windows, Linux, and Android 10+.
- Added '--renderers a,b,c' and '--excluderenderers a,b,c' command line arguments.
- Added t.graphics.renderers and t.graphics.excluderenderers love.conf options.
- Added t.graphics.lowpower love.conf option.

- Added per-shader opt in support for the GLSL 4.30 (desktop) and GLSL ES 3.20 (mobile) shading languages, via `#pragma language glsl4`.
- Added new lower level 'vertexmain' and 'pixelmain' shader entry points.
- Added ability to set point size within a vertex shader by setting the 'love_PointSize' variable.
- Added Compute Shader support via love.graphics.newComputeShader and a new 'computemain' shader entry point.
- Added love.graphics.dispatchThreadgroups and love.graphics.dispatchIndirect for running compute shaders.
- Added Shader:hasStage.
- Added love.graphics.drawFromShader.
- Added love.graphics.drawFromShaderIndirect.
- Added love.graphics.drawIndirect.

- Added love.graphics.newTexture. newImage and newCanvas still exist as convenience constructor functions.
- Added optional 'mipmapcount' field to the settings table in love.graphics.newTexture/newImage/newCanvas.
- Added optional 'computewrite' boolean field to the settings table in love.graphics.newTexture/newImage/newCanvas.
- Added love.graphics.getTextureFormats, which replaces getImageFormats and getCanvasFormats.
- Added Texture:isCanvas and Texture:isComputeWritable.
- Added Texture:isFormatLinear, Texture:getMSAA, Texture:generateMipmaps, Texture:replacePixels, and Texture:renderTo (moved from old Canvas and Image subclasses).
- Added support for CompressedImageData to Texture:replacePixels.
- Added love.graphics.newTextureView.
- Added optional 'viewformats' field to the settings table in Texture constructors.
- Added Texture:getViewFormats.
- Added integer pixel formats for Textures and ImageData.
- Added explicit sRGB format enums for compressed texture pixel formats.
- Added ImageData:setLinear, CompressedImageData:setLinear, ImageData:isLinear, and CompressedImageData:isLinear.

- Added Graphics Buffer objects, including vertex, index, texel, shader storage, and indirect draw/dispatch argument buffers.
- Added love.graphics.copyBuffer, copyBufferToTexture, and copyTextureToBuffer.
- Added APIs for interacting with the Buffer objects owned by Meshes.
- Added variants of love.graphics.newMesh to construct a Mesh from existing Buffers.
- Added Mesh:getAttachedAttributes.
- Added an optional start array index parameter to Mesh:attachAttribute.
- Added variants of Mesh:attachAttribute, Mesh:setAttributeEnabled, and Mesh:isAttributeEnabled which take vertex attribute location numbers instead of names.
- Added integer buffer data formats.
- Added love.graphics.getQuadIndexBuffer.
- Added Shader:getBufferFormat.

- Added love.graphics.readbackTexture and love.graphics.readbackTextureAsync (replaces Canvas:newImageData).
- Added love.graphics.readbackBuffer and love.graphics.readbackBufferAsync.
- Added 'readback' buffer data usage enum, useful for advanced memory optimization when using love.graphics.readbackBuffer.
- Added debugname fields in the settings tables for Textures, Buffers, and Shaders.
- Added Texture:getDebugName and Shader:getDebugName.

- Added variants of love.graphics.applyTransform and replaceTransform which accept x,y,angle,sx,sy,ox,oy parameters.
- Added APIs to override the default orthographic projection: love.graphics.setProjection and resetProjection.
- Added love.graphics.setBlendState, which gives lower level control over blend operations than love.graphics.setBlendMode.
- Added high level love.graphics.setStencilMode and getStencilMode functions. Replaces love.graphics.stencil and love.graphics.setStencilTest.
- Added lower level love.graphics.setStencilState and love.graphics.getStencilState functions.
- Added a variant of love.graphics.setColorMask which accepts a single boolean.
- Added new 'clampone' WrapMode.
- Added 'clampone', 'texelbuffer', 'indexbuffer32bit', 'mipmaprange', and 'indirectdraw' GraphicsFeature enums.
- Added 'copybuffer', 'copybuffertotexture', 'copytexturetobuffer', and 'copyrendertargettobuffer' GraphicsFeature enums.
- Added 'buffers' and 'buffermemory' fields to the table returned by love.graphics.getStats.
- Added initial support for right-to-left (RTL) text when using TrueType and OpenType fonts.
- Added a variant of Font:getWidth which takes a codepoint number argument.
- Added optional settings table variants for love.graphics.newFont, love.font.newRasterizer, and love.font.newTrueTypeRasterizer.
- Added love.graphics.newTextBatch (renamed from love.graphics.newText).
- Added support for saving .exr image files via ImageData:encode.

Changes

- Changed the primary build system for Linux to CMake instead of autotools.
- Changed the minimum runtime macOS version to 10.15.
- Changed the minimum runtime iOS version to 13.0.
- Changed the minimum runtime Windows version to Windows 7 SP1 (64 bit).
- Changed the minimum runtime graphics requirement to OpenGL 3.3+, OpenGL ES 3.0+, Vulkan 1.0+, or Metal.

- Changed love.physics Shapes and Fixtures to be combined together, in Shape objects. Separate Fixtures no longer exist.
- Changed love.physics.newCircleShape, newRectangleShape, newPolygonShape, newEdgeShape, and newChainShape to take a Body as their first parameter.
- Changed love.physics callbacks that used to take Fixture parameters to now take Shape parameters.
- Changed Shape:setDensity to no longer need Body:resetMassData afterward unless the Body already has custom mass data.
- Changed the behaviour of attaching a Shape to a Body to no longer reset mass data if it's been customized.
- Changed RevoluteJoint:getMotorTorque and WheelJoint:getMotorTorque to take 'dt' as a parameter instead of 'inverse_dt'.

- Changed love.filesystem.exists to no longer be deprecated.
- Changed love.data.hash to have a container type parameter.
- Changed the size parameter of love.data.newDataView to be optional.
- Changed the default font from Vera size 12 to Noto Sans size 13.
- Changed TrueType and OpenType font handling to have improved kerning and character combining support.
- Changed love.timer.sleep to allow for more precise sleep durations than 1ms on platforms which support that.

- Changed love.textinput to be disabled by default on all platforms. Use love.keyboard.setTextInput to enable it only when needed.
- Changed love.keypressed, love.keyreleased, and love.keyboard.isDown to accept arbitrary non-ASCII keys.
- Changed love.wheelmoved to use precise fractions instead of integer values for its x and y wheel movement parameters, when a mouse wheel supports precise scrolling.

- Changed the Texture class and implementation to no longer have separate Canvas and Image subclasses.
- Changed Textures created from image files and from ImageData to no longer hold onto a CPU copy of their pixel data after creation.
- Changed love.graphics.newImage to error instead of loading a placeholder texture, when the image dimensions are too large for the system.
- Changed love.graphics.newImage to allow creating a mipmapped texture with less than the full mipmap range, instead of erroring.

- Changed vertex attribute declarations in Shader code to use location indices via the `layout (location = #)` qualifier, instead of using names to match with Mesh vertex attributes.
    - LÖVE's default vertex attributes use location 0 for position, location 1 for texture coordinates, and location 2 for vertex colors.
- Changed the vertex format used with love.graphics.newMesh to have format and 'location' named fields.
- Changed love.graphics.newMesh to no longer default to the "fan" Mesh draw mode.
- Changed the behaviour of Meshes to no longer allow a vertex map or index buffer when the "fan" mesh draw mode is used.
- Changed love.graphics.draw(Mesh) to no longer error if the Mesh has no 'VertexPosition' attribute but does have custom vertex attributes.
- Changed Mesh:getVertexFormat to use named fields, to match the new vertex format parameter setup in love.graphics.newMesh.
- Changed love.window.setMode to no longer clear the contents of Canvases or otherwise recreate graphics resources.
- Changed love.graphics.rectangle, love.graphics.polygon, and other shape drawing APIs to provide texture coordinates to shaders, for filled shapes.
- Changed love.graphics.points to require 'love_PointSize' to be set in the vertex shader, if a custom shader is used.
- Changed love.graphics.setCanvas to always clear auto-generated temporary depth and stencil buffers when they're used.
- Changed love.graphics.clear to no longer be affected by love.graphics.setScissor or love.graphics.setColorMask.
- Changed love.graphics.setDepthMode with the depth write parameter set to true to error if the active Canvases or window don't have a depth buffer.
- Changed text layout and rendering in high-dpi situations to be more consistent across DPI scales.
- Changed Shader code parsing to ignore shader entry point functions inside comments.
- Changed Shader code to use GLSL3 (GLSL 3.30 or GLSL ES 3.00) by default.
- Changed position output coordinate space in vertex shaders to be consistent across all graphics APIs and regardless of whether a Canvas is active.
    - Clip-space output is y-up. Texture contents and pixel shader coordinates are y-down.

- Changed audio file decoding to choose the most appropriate decoder based on file contents instead of the file extension.
- Changed audio initialization to give a more descriptive error if it fails.
- Changed Videos to stream audio from the file instead of loading all the video file into memory for use with audio decoding.
- Changed love.math.perlinNoise and love.math.simplexNoise to use higher precision numbers for its internal calculations.
- Changed the window 'depth' field in love.conf and love.window.setMode/updateMode/getMode to be a boolean instead of a bit count number.
- Changed love.window.isVisible to return false if the window is minimized.
- Changed love.visible to be called when minimizing and unminimizing the window.

- Renamed 'display' field to 'displayindex' in love.window.setMode/updateMode/getMode and love.conf.
- Renamed 'gammacorrect' field to 'graphics.gammacorrect' in love.conf.
- Renamed love.graphics Text objects to TextBatch.

- Updated SDL from 2.0 to 3.0.
- Updated Box2D from 2.3 to 2.4.1 (https://github.com/erincatto/box2d/releases/tag/v2.4.1)

Deprecations

- Deprecated usage of slashes instead of dots for module separators in require.
- Deprecated love.filesystem.newFile (replaced by love.filesystem.openFile).
- Deprecated love.math.noise (replaced by love.math.perlinNoise and love.math.simplexNoise).
- Deprecated love.event.wait (replaced by wait timeout parameter in love.event.pump).
- Deprecated the variants of love.physics.newCircleShape, newRectangleShape, newPolygonShape, newEdgeShape, and newChainShape that don't take a Body parameter.
- Deprecated love.physics.newFixture (replaced by the new Shape constructor variants).
- Deprecated Body:getFixtures (replaced by Body:getShapes).
- Deprecated Contact:getFixtures (replaced by Contact:getShapes).
- Deprecated World:queryBoundingBox (replaced by World:queryShapesInArea).
- Deprecated ChainShape:getChildEdge.

- Deprecated the variants of love.data.hash which do not have a container type parameter.
- Deprecated love.graphics.setNewFont (use love.graphics.newFont and love.graphics.setFont instead).
- Deprecated love.graphics.newText (renamed to love.graphics.newTextBatch).
- Deprecated love.graphics.getImageFormats and love.graphics.getCanvasFormats (replaced by love.graphics.getTextureFormats).
- Deprecated love.graphics.stencil (replaced by love.graphics.setStencilMode or love.graphics.setStencilState).
- Deprecated love.graphics.setStencilTest and love.graphics.getStencilTest (replaced by love.graphics.setStencilMode or love.graphics.setStencilState).
- Deprecated the variants of Mesh:attachAttribute, Mesh:setAttributeEnabled, and Mesh:isAttributeEnabled which take names instead of location numbers.
- Deprecated usage of `texture` as a uniform variable in Shader code, since it's a built-in function name in GLSL3.
- Deprecated usage of `texture2D` function calls in Shader code. Use `Texel` or the built-in function `texture` instead.
- Deprecated vertex attributes in Shader code when they aren't tagged with a `layout (location = #)` qualifier.
- Deprecated love.window.close.
- Deprecated t.window.highdpi in love.conf and the highdpi flag of love.window.setMode (replaced by t.highdpi in love.conf).
- Deprecated the variants of Mesh:attachAttribute and SpriteBatch:attachAttribute which accept a Mesh (replaced by variants which accept a GraphicsBuffer).
- Deprecated Canvas:newImageData (replaced by love.graphics.readbackTexture).

Removals

- Removed 32 bit Windows support for official builds.
- Removed the `t.accelerometerjoystick` love.conf flag (replaced by the love.sensor module).
- Removed Fixture objects. All methods previously in Fixtures are now available in Shapes.
- Removed distinct DroppedFile subtype.
- Removed the variant of SpriteBatch:setColor() which turns off all previously set colors.
- Removed the `NormalMatrix` Shader uniform variable.
- Removed support for the GLSL 1.20 and GLSL ES 1.00 Shader languages. GLSL 3.30 and GLSL ES 3.00 are used by default instead.
- Removed functions deprecated in LÖVE 11:
  - Removed love.audio.getSourceCount (renamed to love.audio.getActiveSourceCount).
  - Removed Source:getChannels (renamed to Source:getChannelCount).
  - Removed Decoder:getChannels (renamed to Decoder:getChannelCount).
  - Removed love.filesystem.isDirectory (replaced by love.filesystem.getInfo).
  - Removed love.filesystem.isFile (replaced by love.filesystem.getInfo).
  - Removed love.filesystem.isSymlink (replaced by love.filesystem.getInfo).
  - Removed love.filesystem.getLastModified (replaced by love.filesystem.getInfo).
  - Removed love.filesystem.getSize (replaced by love.filesystem.getInfo).
  - Removed ParticleSystem:setAreaSpread and ParticleSystem:getAreaSpread (renamed to ParticleSystem:setEmissionArea and ParticleSystem:getEmissionArea).
  - Removed love.math.compress and love.math.decompress (replaced by love.data.compress and love.data.decompress).
  - Removed World:getBodyList, World:getJointList, and World:getContactList (renamed to World:getBodies, World:getJoints, and World:getContacts).
  - Removed Body:getFixtureList, Body:getJointList, and Body:getContactList (renamed to Body:getFixtures, Body:getJoints, and Body:getContacts).
  - Removed PrismaticJoint:hasLimitsEnabled (renamed to PrismaticJoint:areLimitsEnabled).
  - Removed RevoluteJoint:hasLimitsEnabled (renamed to RevoluteJoint:areLimitsEnabled).

Fixes

- Fixed misleading print when there's an error in the error handler.
- Fixed BezierCurve:render adding collinear points in some situations.
- Fixed scissor state getting out of sync when switching between a Canvas and the screen while a scissor is active.
- Fixed a memory leak in TextBatch:add when the internal buffer is resized.
- Fixed justify text align mode not always aligning to both edges.
- Fixed sound Decoders to cause a Lua error instead of hard-crashing when memory for the decoding buffer can't be allocated.
- Fixed full platform-specific paths in love.filesystem APIs to use a consistent canonical representation for the platform.

2dgraph

2dgraph · revision 16107 · last edited 2021-03-14

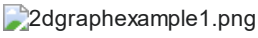
Both simple and customizable.

Download with an simple example.

Note that the example is for 0.9s, but 2dgraph will work on 10.0.

From anonfiles (https://anonfiles.com/file/bd7fb20daaea59ddc538defea4bf1340)

From 百度云 (http://pan.baidu.com/s/1bnsg9BL)



30log

30log · revision 14683 · last edited 2023-07-05

30log, in extenso *30 Lines Of Goodness* is a minified object orientation framework for Lua.
It features **class creation**, **instantiation**, **inheritance** and much more.
And it makes *30 lines*. No less, no more.

A very simple sample:

```
class = require '30log'
Window = class { width = 100, height = 100, x = 10, y = 10}

appFrame = Window()
print(appFrame.x,appFrame.y) --> 10, 10
print(appFrame.width,appFrame.height) --> 100, 100
```

Grab it on Github (https://github.com/Yonaba/30log).

- Source (https://github.com/Yonaba/30log/blob/master/30log.lua)
- Docs and samples (https://github.com/Yonaba/30log)
- Specification tests (https://github.com/Yonaba/30log/tree/master/specs)

32 lines of goodness

32 lines of goodness · revision 24444 · last edited 2023-07-05


This library has problems when a two classes inherit from the same base class. Check the thread for more details.

The library

```
local mt_class = {}

function mt_class:extends(parent)
   self.super = parent
   setmetatable(mt_class, {__index = parent})
   parent.__members__ = parent.__members__ or {}
   return self
end

local function define(class, members)
   class.__members__ = class.__members__ or {}
   for k, v in pairs(members) do
      class.__members__[k] = v
   end
   function class:new(...)
      local newvalue = {}
      for k, v in pairs(class.__members__) do
         newvalue[k] = v
      end
      setmetatable(newvalue, {__index = class})
      if newvalue.__init then
         newvalue:__init(...)
      end
      return newvalue
   end
end

function class(name)
    local newclass = {}
   _G[name] = newclass
   return setmetatable(newclass, {__index = mt_class, __call = define})
end
```

That's all there is to it! Just 32 lines!

Usage

Slap the above library in a file of your choose (maybe "32log.lua") and include it in your code using the `require` function.

The basic syntax is as follows:

```
class "ClassName" : extends(BaseClassName) {
    memberName = nonNilValue;
}
```

Once a class has been created you can create new instances of the class as follows:

```
local myInstance = ClassName:new()
```

If you create a method named `__init` it can be used to as a constructor with new.

```
class "Vector" {
    x = 0;
    y = 0;
    z = 0;
}
function Vector:__init(x, y, z)
    self.x = x
```

```
    self.y = y
    self.z = z
end
function Vector:print()
    print(self.x, self.y, self.z)
end
local vec = Vector:new(1, 0.5, 0.25)
vec:print()
```

Whatever value you set a member to in the definition, it will act as a default value for the member. This works well with values like numbers and strings where they are always copied by value but tables can get a little tricky:

```
class "Foo" {
   bar = {};
}
local foo = Foo:new()
foo.bar["foobar"] = 10;
local foo2 = Foo:new()
print(foo2.bar["foobar"])
```

Classes inherit their parent's default member values and meta-methods. You can also call a parent's method that was overloaded in the derived class using the `super` member:

```
class "Base" {}
function Base:foobar()
    print("foo")
end
class "Derived" : extends(Base) {}
function Derived:foobar()
   self.super.foobar(self)
   print("bar")
end
```

See Also

Post any questions you might have in the original post (http://love2d.org/forums/viewtopic.php?f=5&t=3344).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

7-Zip

7-Zip · revision 6270 · last edited 2021-03-14

7-Zip is an open-source file archiver with a high compression ratio. It supports a variety of archive formats, is small in size, and unlike many other all-in-one archive programs, its free to use.

Love should be able to understand .loves in all of 7-Zip's formats...but you should stick to basic ZIP for compatibility.

7-Zip website (http://w w w.7-zip.org/)

8-Bit Commando

8-Bit Commando · revision 29608 · last edited 2025-04-28

8-Bit Commando is a run-and-gun platformer with fast-paced action, a rocking soundtrack, cooperative multiplayer and explosions. The graphics will take you back to the classic age of gaming and the gameplay is bound to challenge even the most seasoned vets.



Features

- 7 stages of intense action
- Coop, versus and arcade modes
- Beautiful retro graphics
- Hiscores and replays
- Rocking chiptunes soundtrack
- Unlockable hard mode
- Local multiplayer

Play 8-Bit Commando (https://2dengine.com/game/commando?ref=love2d.org)

A Guide to Getting Started with Love2D

A Guide to Getting Started with Love2D · revision 14494 · last edited 2021-03-14

BlackBulletIV has written a guide on getting started with LÖVE. Topics covered include:

- What LÖVE is
- Getting setup
- Brief directions on learning Lua
- The structure of games, include .love files
- How to approach the API, including an overview of its structure, and the most important modules

You can find it here:

A Guide to Getting Started with LÖVE (http://nova-fusion.com/2011/06/14/a-guide-to-getting-started-with-love2d/)

Adult Lib

Adult Lib · revision 24465 · last edited 2023-07-05

A small lib to run old löve code with the current version. Currently it only supports 0.5.0 -> 0.6.2.

You can find it here (http://ghoulsblade.schattenkind.net/files/adult.lua)

How to use:

- Take your 0.5 code and include adult in the first line of your main.lua
- Then call adult.grow ("0.5")
- If you are lucky enough you will see your game :)

See also

- Original Thread (http://love2d.org/forums/viewtopic.php?f=5&t=1608)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Advanced Tiled Loader

Advanced Tiled Loader · revision 15131 · last edited 2021-03-14

Advanced Tiled Loader loads and displays Tiled (http://www.mapeditor.org/) maps in LÖVE. It is no longer being developed and is considered unstable.

It did support these features:

- Loading, drawing, and re-saving of Tiled maps
- Multiple layers
- Objects (regular and tile)
- Properties
- Transparent colors
- Margins and spacing
- External tilesets
- zlib/gzip compression
- Isometric maps
- Flipped tiles

For more information please visit the forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=2567) or the github repository (https://github.com/Kadoba/Advanced-Tiled-Loader). Tutorials and documentation can be found on the Advanced Tiled Loader Wiki (https://github.com/Kadoba/Advanced-Tiled-Loader/wiki).

Aeternum Blammo

Aeternum Blammo · revision 16740 · last edited 2021-03-14

Forum Thread: https://love2d.org/forums/viewtopic.php?f=5&t=3792&hilit=blammo

More download options on the official website: http://missingsentinelsoftware.com/game/aeternum-blammo ↗

anim8

anim8 · revision 24424 · last edited 2023-07-05

anim8 helps you create animations for LÖVE quickly and easily.

The latest version of anim8 can always be found on its github page: https://github.com/kikito/anim8 ↗

Please refer to the forum post (https://love2d.org/forums/viewtopic.php?f=5&t=8281) or the latest version link for examples of use.

Compatibility

Since anim8 uses LÖVE's graphical functions, and they change from version to version, you must choose the version of anim8 which is compatible with your LÖVE.

- The current version of anim8 is v2.1 (as of March 2015). It is compatible with LÖVE 0.9.x
- The last version of anim8 compatible with LÖVE 0.8.x was anim8 v2.0 (https://github.com/kikito/anim8/tree/v2.0.0).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

any

any · revision 27947 · last edited 2025-11-22

Any kind of Lua value, for example a number or table.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

arcLine

arcLine · revision 28415 · last edited 2023-06-18

This function takes XY coordinates as middle point of arc, starting and ending angles (clockwise), radius and amount of arc segments (optional). It returns the line with given prperties.

Source

```
local function getArcLine (x, y, radius, angle1, angle2, segments)
    local dAngle = (angle2-angle1)%(2*math.pi)
    segments = segments or math.floor(dAngle*radius/16)
    print ('segments', segments)
    dAngle = dAngle / segments
    local line = {}
    for i = 0, segments do
        local x1 = x + radius*math.cos (angle1 + i*dAngle)
        local y1 = y + radius*math.sin (angle1 + i*dAngle)
        table.insert (line, x1)
        table.insert (line, y1)
    end
    return line
end
```

Arne's Breakout

Arne's Breakout · revision 19519 · last edited 2021-03-14

This is a version of the classic brick-breaking game written by Arne Schwettmann.

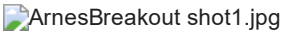
It features some hand-coded physics such as gravity, (in-)elastic collisions and friction.

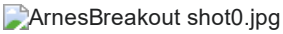
The player(s) can move freely across the whole screen but motion is limited by the existing bricks.

Try the two player simultaneous mode!

The game can be downloaded here: http://www.github.com/arneschwettmann/arnesbreakout/releases ↗

Screenshots:





Arne's SpaceTaxi

Arne's SpaceTaxi · revision 19515 · last edited 2021-03-14

This is a version of the classic space taxi game written by Arne Schwettmann.

It features pixel-perfect collisions.

Try the 5 player mode.

The game can be downloaded here: http://www.github.com/arneschwettmann/arnesspacetaxi/releases ↗

Screenshots:





array

array · revision 27076 · last edited 2023-07-05

A useful collection of methods to work with Lua tables in an array fashion.

Links

- Official forum topic (https://love2d.org/forums/viewtopic.php?f=5&t=92011)
- Repository (https://github.com/rick4stley/array)

Audio Formats

Audio Formats · revision 23490 · last edited 2026-05-17

As of LÖVE 0.9.1, the following audio formats are supported:

- Waveform Audio File Format (.wav)
- MPEG-1 or MPEG-2 Audio Layer III (.mp3)
- Ogg Vorbis (.ogg, .oga, .ogv)
- Tracker module formats (.699, .amf, .ams, .dbm, .dmf, .dsm, .far, .it, .j2b, .mdl, .med, .mod, .mt2, .mtm, .okt, .psm, .s3m, .stm, .ult, .umx, .xm)
- Other formats supported by libmodplug, without instrument data (.abc, .mid, .pat)

Available since LÖVE 11.3

- Free Lossless Audio Codec (.flac)

See Also

- love.audio
- love.sound

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

babel

babel · revision 24452 · last edited 2023-07-05

Description

BABEL is a very simple library to easily make multilingual games with LÖVE. It allow you to load multiple locales files and to switch beteween locales with one function call.

Links

Documentation : http://martin-damien.github.com/babel (http://martin-damien.github.com/babel)

Source code : https://github.com/martin-damien/babel (https://github.com/martin-damien/babel)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Batch Start

Batch Start · revision 28071 · last edited 2022-10-15

You can use a Windows Batch File (A .bat/.cmd that uses Command Promt) to test a LOVE game without having to zip it up and change the file type to love.

NOTE

This will only work for Windows Users, sorry Mac and Linux!

The Batch File

Open up Notepad or any other text/code editor.

Type in following code:

```
@ECHO OFF

start " " "%PROGRAMFILES(x86)%\LOVE\love.exe"
```

or

```
@ECHO OFF

start " " "Path\To\love.exe"
```

Depending on where you have LOVE installed.

Be sure to save the file as a .bat or .cmd or it won't work

Using the Batch File

Place your new Batch File into the same directory as your "*main.lua*" and double click it to run it.

i.e *"Desktop\LOVE\Game\Directory"*

And BAM! No more zipping your files every time you make a change to your games code!

Beetle Debug

Beetle Debug · revision 13392 · last edited 2021-03-14

nothing

binser

binser · revision 24491 · last edited 2023-07-05

binser - Customizable Lua Serializer

Save and load games or any Lua data quickly and in a very compact format.

Can Serialize:

- Tables
- Numbers
- Strings
- Booleans
- Nan
- Nil
- Nested and Cyclic tables
- Userdata (with setup)
- Functions (pure functions with no upvalues)

Example:

```lua
local binser = require "binser"

local mydata = binser.serialize(45, {4, 8, 12, 16}, "Hello, World!")

print(binser.deserialize(mydata))
-- 45   table: 0x7fa60054bdb0   Hello, World!
```

- GitHub (https://github.com/bakpakin/binser)
- binser.lua (https://raw.githubusercontent.com/bakpakin/binser/master/binser.lua)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

bitser

bitser · revision 24495 · last edited 2023-07-05

bitser is a serialization library for LuaJIT. It's very fast, produces tiny output and safely handles untrusted input.

```
local bitser = require 'bitser'

bitser.register('someResource', someResource)
bitser.registerClass(SomeClass)

serializedString = bitser.dumps(someValue)
someValue = bitser.loads(serializedString)
love.filesystem.write("filename", serializedString)
serializedData = love.filesystem.newFileData("filename")
someValue = bitser.loadData(serializedData:getPointer(), serializedData:getSize())
```

Get bitser from its GitHub repositiory (http://github.com/gvx/bitser).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BlendMode Formulas

BlendMode Formulas · revision 29328 · last edited 2024-09-28

On this page

- alpha
  - "alphamultiply" alpha mode
  - "premultiplied" alpha mode
- add
  - "alphamultiply" alpha mode
  - "premultiplied" alpha mode
- subtract
  - "alphamultiply" alpha mode
  - "premultiplied" alpha mode
- replace
  - "alphamultiply" alpha mode
  - "premultiplied" alpha mode
- multiply
  - "premultiplied" alpha mode
- lighten
  - "premultiplied" alpha mode
- darken
  - "premultiplied" alpha mode
- screen
  - "alphamultiply" alpha mode
  - "premultiplied" alpha mode
- Older Versions
  - alpha (0.9.0, 0.9.1, and 0.9.2)
  - alpha (0.8.0 and older)
  - premultiplied (0.9.2 and older)
  - screen (0.9.2 and older)
  - additive (0.9.2 and older)
  - subtractive (0.9.2 and older)
  - multiplicative (0.9.0, 0.9.1, and 0.9.2)
  - multiplicative (0.8.0 and older)
  - replace (0.9.2 and older)
- See Also
- Other Languages

Equations used when blending drawn content with the screen or active Canvas.

Color components are in the range of [0, 1]. Results are clamped to [0, 1] except when a Canvas is active that has a floating-point / HDR format.

Description:

- **dst** - existing color in the screen.
- **src** - the color of the drawn object (the color output by the pixel shader, or the global color multiplied by the texture's color – if any, if no shader is used.)
- **res** - resulting color.

Here are the BlendMode formulas for all 0.10.x and later versions:

alpha

"alphamultiply" alpha mode

```
res.r = dst.r * (1 - src.a) + src.r * src.a
res.g = dst.g * (1 - src.a) + src.g * src.a
res.b = dst.b * (1 - src.a) + src.b * src.a
res.a = dst.a * (1 - src.a) + src.a
```

"premultiplied" alpha mode

```
res.r = dst.r * (1 - src.a) + src.r
res.g = dst.g * (1 - src.a) + src.g
res.b = dst.b * (1 - src.a) + src.b
res.a = dst.a * (1 - src.a) + src.a
```

add

"alphamultiply" alpha mode

```
res.r = dst.r + (src.r * src.a)
res.g = dst.g + (src.g * src.a)
```

```
res.b = dst.b + (src.b * src.a)
res.a = dst.a
```

"premultiplied" alpha mode

```
res.r = dst.r + src.r
res.g = dst.g + src.g
res.b = dst.b + src.b
res.a = dst.a
```

subtract

"alphamultiply" alpha mode

```
res.r = dst.r - (src.r * src.a)
res.g = dst.g - (src.g * src.a)
res.b = dst.b - (src.b * src.a)
res.a = dst.a
```

"premultiplied" alpha mode

```
res.r = dst.r - src.r
res.g = dst.g - src.g
res.b = dst.b - src.b
res.a = dst.a
```

replace

"alphamultiply" alpha mode

```
res.r = src.r * src.a
res.g = src.g * src.a
res.b = src.b * src.a
res.a = src.a
```

"premultiplied" alpha mode

```
res.r = src.r
res.g = src.g
res.b = src.b
res.a = src.a
```

multiply

"premultiplied" alpha mode

```
res.r = src.r * dst.r
res.g = src.g * dst.g
res.b = src.b * dst.b
res.a = src.a * dst.a
```

Note: In 0.10.x, *multiply* with *alphamultiply* uses the same equations as with *premultiplied*. In 11.0 and later versions, this variation is not supported.

lighten

"premultiplied" alpha mode

```
res.r = max(src.r, dst.r)
res.g = max(src.g, dst.g)
```

```
res.b = max(src.b, dst.b)
res.a = max(src.a, dst.a)
```

darken

"premultiplied" alpha mode

```
res.r = min(src.r, dst.r)
res.g = min(src.g, dst.g)
res.b = min(src.b, dst.b)
res.a = min(src.a, dst.a)
```

screen

Note: The math for this blend mode is not completely correct when using the "alphamultiply" alpha mode. Prefer the "premultiplied" variant (and be sure your content has its RGB multiplied by its alpha at some point before blending), when possible.

"alphamultiply" alpha mode

```
res.r = dst.r * (1 - src.r) + (src.r * src.a)
res.g = dst.g * (1 - src.g) + (src.g * src.a)
res.b = dst.b * (1 - src.b) + (src.b * src.a)
res.a = dst.a * (1 - src.a) + src.a
```

"premultiplied" alpha mode

```
res.r = dst.r * (1 - src.r) + src.r
res.g = dst.g * (1 - src.g) + src.g
res.b = dst.b * (1 - src.b) + src.b
res.a = dst.a * (1 - src.a) + src.a
```

Older Versions

alpha (0.9.0, 0.9.1, and 0.9.2)

```
res.r = dst.r * (1 - src.a) + src.r * src.a
res.g = dst.g * (1 - src.a) + src.g * src.a
res.b = dst.b * (1 - src.a) + src.b * src.a
res.a = dst.a * (1 - src.a) + src.a
```

alpha (0.8.0 and older)

```
res.r = dst.r * (1 - src.a) + src.r * src.a
res.g = dst.g * (1 - src.a) + src.g * src.a
res.b = dst.b * (1 - src.a) + src.b * src.a
res.a = dst.a * (1 - src.a) + src.a * src.a
```

premultiplied (0.9.2 and older)

```
res.r = dst.r * (1 - src.a) + src.r
res.g = dst.g * (1 - src.a) + src.g
res.b = dst.b * (1 - src.a) + src.b
res.a = dst.a * (1 - src.a) + src.a
```

screen (0.9.2 and older)

```
res.r = dst.r * (1 - src.r) + src.r
res.g = dst.g * (1 - src.g) + src.g
```

```
res.b = dst.b * (1 - src.b) + src.b
res.a = dst.a * (1 - src.a) + src.a
```

additive (0.9.2 and older)

```
res.r = dst.r + (src.r * src.a)
res.g = dst.g + (src.g * src.a)
res.b = dst.b + (src.b * src.a)
res.a = dst.a + (src.a * src.a)
```

subtractive (0.9.2 and older)

```
res.r = dst.r - src.r * src.a
res.g = dst.g - src.g * src.a
res.b = dst.b - src.b * src.a
res.a = dst.a - src.a * src.a
```

multiplicative (0.9.0, 0.9.1, and 0.9.2)

```
res.r = src.r * dst.r
res.g = src.g * dst.g
res.b = src.b * dst.b
res.a = src.a * dst.a
```

multiplicative (0.8.0 and older)

```
res.r = dst.r * (1 - src.a) + src.r * dst.r
res.g = dst.g * (1 - src.a) + src.g * dst.g
res.b = dst.b * (1 - src.a) + src.b * dst.b
res.a = dst.a * (1 - src.a) + src.a * dst.a
```

replace (0.9.2 and older)

```
res.r = src.r
res.g = src.g
res.b = src.b
res.a = src.a
```

See Also

- BlendMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

board game support

board game support · revision 11735 · last edited 2021-03-14

no

boipushy

boipushy · revision 28271 · last edited 2023-07-05

An input module for LÖVE. Simplifies input handling by abstracting them away to actions, enabling pressed/released checks outside of LÖVE's callbacks and taking care of gamepad input as well.

Download

github (https://github.com/adonaac/boipushy)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Bomb Dodgers

Bomb Dodgers · revision 14378 · last edited 2021-03-14

Bomb Dodgers is a maze-based bombing game inspired by the classic Bomberman games.

You will have to win two out of three rounds against one, two and finally three enemies to reach the endboss of each level. After you have beaten an endboss the next level will be unlocked.

Released on itch.io (http://rmcode.itch.io/bomb-dodgers)

boolean

boolean · revision 7941 · last edited 2025-09-30

From the Lua 5.1 reference manual §2.2 (http://w w w.lua.org/manual/5.1/manual.html#2.2):

Boolean is the type of the values false and true. Both nil and false make a condition false; *any* other value makes it true.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

breezefield

breezefield · revision 26721 · last edited 2023-07-05

Breezefield is a simple, transparent, and easy wrapper that speeds up working with love.physics.

See the github (https://github.com/HDictus/breezefield) page for explanation, tutorial and download/install instructions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BSD License

BSD License · revision 17226 · last edited 2021-03-14

There are two main versions of the BSD License:

- 3-clause 'Revised' BSD (http://opensource.org/licenses/BSD-3-Clause)
- 2-clause 'Simplified' BSD (http://opensource.org/licenses/BSD-2-Clause)

built-in level editor

built-in level editor · revision 10858 · last edited 2021-03-14

Hello I was wondering if anybody has a clue on how to make a ***built-in level editor*** ??? I have seen them in many things but never have actually found one!

cdata

cdata · revision 23934 · last edited 2023-07-05

A C data object.

See http://luajit.org/ext_ffi_api.html ↗ for more information.

Other Language

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Chromatic Paths

Chromatic Paths · revision 7854 · last edited 2021-03-14





The game

This is a very simple, small puzzle game. You must visit every location on the board, but cannot move against an arrow, so your path is tricky. Some spots may even trap you, watch out!

Features awesome music by Trash80 (http://trash80.com/)! (used without permission)

Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=3719)

Download

Direct from Dropbox (http://dl.dropbox.com/u/3713769/w eb/Love/Chromatic%20Paths/Chromatic%20Paths.love)

cimgui-love

cimgui-love · revision 29028 · last edited 2023-12-09

cimgui-love

LÖVE module for Dear ImGui (https://github.com/ocornut/imgui) obtained by wrapping cimgui (https://github.com/cimgui/cimgui) (programmatically generated C-api) using LuaJIT FFI.

The wrappers are generated automatically (like cimgui itself) and can be easily updated for new versions of Dear ImGui. Currently based on version 1.90 (docking branch) of Dear ImGui and LÖVE 11.3.

Links

- git repository (https://codeberg.org/apicici/cimgui-love)
- forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=91307)
- pre-built binaries (https://codeberg.org/apicici/cimgui-love/releases)

Example of usage

See README on the git repository (https://codeberg.org/apicici/cimgui-love/src/branch/master/README.md) for more detailed instructions.

```
-- Make sure the shared library can be found through package.cpath before loading the module.
-- For example, if you put it in the LÖVE save directory, you could do something like this:
local lib_path = love.filesystem.getSaveDirectory() .. "/libraries"
local extension = jit.os == "Windows" and "dll" or jit.os == "Linux" and "so" or jit.os == "OSX" and "dylib"
package.cpath = string.format("%s;%s/?.%s", package.cpath, lib_path, extension)

local imgui = require "cimgui" -- cimgui is the folder containing the Lua module (the "src" folder in the github repository)

love.load = function()
    imgui.love.Init() -- or imgui.love.Init("RGBA32") or imgui.love.Init("Alpha8")
end

love.draw = function()
    -- example window
    imgui.ShowDemoWindow()

    -- code to render imgui
    imgui.Render()
    imgui.love.RenderDrawLists()
end

love.update = function(dt)
    imgui.love.Update(dt)
    imgui.NewFrame()
end

love.mousemoved = function(x, y, ...)
    imgui.love.MouseMoved(x, y)
    if not imgui.love.GetWantCaptureMouse() then
        -- your code here
    end
end

love.mousepressed = function(x, y, button, ...)
    imgui.love.MousePressed(button)
    if not imgui.love.GetWantCaptureMouse() then
        -- your code here
    end
end

love.mousereleased = function(x, y, button, ...)
    imgui.love.MouseReleased(button)
    if not imgui.love.GetWantCaptureMouse() then
        -- your code here
```

```
    end
end

love.wheelmoved = function(x, y)
    imgui.love.WheelMoved(x, y)
    if not imgui.love.GetWantCaptureMouse() then
        -- your code here
    end
end

love.keypressed = function(key, ...)
    imgui.love.KeyPressed(key)
    if not imgui.love.GetWantCaptureKeyboard() then
        -- your code here
    end
end

love.keyreleased = function(key, ...)
    imgui.love.KeyReleased(key)
    if not imgui.love.GetWantCaptureKeyboard() then
        -- your code here
    end
end

love.textinput = function(t)
    imgui.love.TextInput(t)
    if imgui.love.GetWantCaptureKeyboard() then
        -- your code here
    end
end

love.quit = function()
    return imgui.love.Shutdown()
end

-- for gamepad support also add the following:

love.joystickadded = function(joystick)
    imgui.love.JoystickAdded(joystick)
    -- your code here
end

love.joystickremoved = function(joystick)
    imgui.love.JoystickRemoved()
    -- your code here
end

love.gamepadpressed = function(joystick, button)
    imgui.love.GamepadPressed(button)
    -- your code here
end

love.gamepadreleased = function(joystick, button)
    imgui.love.GamepadReleased(button)
    -- your code here
end

-- choose threshold for considering analog controllers active, defaults to 0 if unspecified
local threshold = 0.2

love.gamepadaxis = function(joystick, axis, value)
    imgui.love.GamepadAxis(axis, value, threshold)
    -- your code here
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

clasp

clasp · revision 24518 · last edited 2023-07-05

Minimal Lua class library in 4 lines of code: https://github.com/evolbug/lua-clasp ↗

```lua
class = require "clasp"


-- Basic class
Vector = class {
  isVector = true; -- static values

  init = function(self, x, y) -- initializer function
    self.x = x
    self.y = y
  end;
}

a = Vector(10, 10)
print('Vector:', a.x, a.y, a.isVector) -- "Vector: 10 10 true"


-- Inheritance
Vector3 = Vector:extend {
  init = function(self, x, y, z) -- function overriding
    Vector.init(self, x, y) -- superclass method call
    self.z = z
  end;
}

b = Vector3(1, 2, 3)
print('Vector3:', b.x, b.y, b.z, b.isVector) -- "Vector3: 1 2 3 true"


-- Metamethods
Point = class {
  init = function(self, x, y)
    self.x = x
    self.y = y
  end;
  __ = { -- metamethod table
    tostring = function(self)
      return 'Point('..tostring(self.x)..', '..tostring(self.y)..')'
    end;
  };
}

c = Point(15, 25)
print(c) -- "Point(15, 25)"
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Class Commons

Class Commons · revision 10911 · last edited 2021-03-14

Class Commons is a project to provide a common compatibility interface for class systems. The goal of this is to make libraries independent of class libraries, a library using the Class Commons API can then be used with any class system adhering to it (albeit via a compatibility layer).

Github Repo

Class Commons on github (https://github.com/bartbes/Class-Commons)

Code Style

Code Style · revision 28560 · last edited 2023-07-08

On this page

- Introduction
- Namespaces
- Class declarations
- Initializer lists
- Functions/methods
- Control blocks
- Switch-case
- Pointer affinity
- Spaces vs Tabs
- Trailing whitespace
- Documentation blocks
- #includes
- Casts
- Writing null
- Other Languages

Introduction

"Every second spent enforcing code style is a waste of time". -- Someone, at some point

Status: **FALSE**

When I encounter really pretty code in the open-source world (and the closed-source world for that matter), I'm a lot more confident that the software has real quality. Of course, from case to case, this may actually not be true, and the style might lie to me, but the important thing is that it *appears* to have high quality to people. On the other hand, if I encounter code which appears to be shit (although it might really be of excellent quality), it's tempting for me to try and find some alternative. So while enforcing code style might *technically* be a waste of time, it's psychologically. Also, if you ever intend to use LÖVE as a reference when applying for a job (I've done that), there is a slight chance they will take a ten second look on the code. Even if they are advanced enough to evaluate the code properly, they will not be immune to good (nor bad) style.

Therefore, because I'm such a fucking drone, I'm going to refactor some code here and there (or everywhere) to make things more beautiful. And since I'm potentially fucking up everything anyway, I thought we could decide on a coding style that the majority likes. It's unlikely that we all agree on everything ... but hopefully everyone will accept the decision of the majority.

Namespaces

Namespace bodies are not indented.

YES:

```
namespace love
{
namespace graphics
{

class Foo
{
public:
  /* ... */
};

} // graphics
} // love
```

NO:

```
namespace love
{
  namespace graphics
  {

      class Foo
      {
      public:
          /* ... */
      };
```

```
  } // graphics
} // love
```

Class declarations

Should have a capital first letter.

YES:

```
class Foo
{
};
```

NO:

```
class foo
{
};

class FOO
{
};
```

Visiblity blocks should be in decreasing order.

It makes sense to have the visible "API" first. This is what a reader will normally want to know first anyway.

YES:

```
class Foo
{
public:
protected:
private:
};
```

NO:

```
class Foo
{
private:
protected:
public:
};
```

For multiple inheritance, each class should appear on its own line.

Each line should *start* with the colon or comma. My experience is that this is the most flexible style with respect to diffs and #ifdeffery.

YES:

```
class Foo : public Bar
{
};

class Foo
  : public Bar
  , public Foz
{
};
```

NO:

```
class Foo : public Bar, public Foz
{
};
```

Initializer lists

Like inheritance lists, each initialization on its own line.

YES:

```
Foo::Foo()
  : bar(0)
  , foo(1)
{
}
```

NO:

```
Foo::Foo() : bar(0) , foo(1)
{
}
```

Functions/methods

Functions and methods should use camelCase.

YES:

```
int doomFist(int where);
```

NO:

```
int DoomFist(int where);
int doom_fist(int where);
```

Parentheses should appear directly after the name, with no space in between.

YES:

```
int doomFist(int where);
```

NO:

```
int doomFist (int where);
```

There must be no unnecessary space immediately after the open-paren and immediately before the close-paren.

YES:

```
int doomFist(int where);
```

NO:

```
int doomFist( int where );
```

One-liners like getters and setters can be inlined in the class declaration. (More-than-one-liners should however not be inlined).

RECOMMENDED:

```
class Foo
{
public:
  void setBar(int bar) { this->bar = bar; }
  int getBar() const { return bar; }
private:
```

```
  int bar;
};
```

DISCOURAGED:

```
class Foo
{
public:
  void setBar(int bar);
  int getBar() const;
private:
  int bar;
};

void Foo::setBar(int bar)
{
  this->bar = bar;
}

int Foo::getBar() const
{
  return bar;
}
```

Control blocks

Control blocks should have a space between the keyword the parenthesized expression. This differentiates control statements visually from function calls.

YES:

```
if (foo)
  /* ... */;
```

NO:

```
if(foo)
  /* ... */;
```

Braces should normally be omitted if they can. (Saves vertical space).

YES:

```
if (foo)
  bar();
```

NO:

```
if (foo)
{
  bar();
}
```

YES:

```
if (foo)
  bar();
else
{
  foz();
  baz();
}
```

NO:

```
if (foo)
{
  bar();
}
```

```
else
{
  foz();
  baz();
}
```

Braces should appear on the next line.

YES:

```
if (foo)
{
  bar1();
  bar2();
}
```

NO:

```
if (foo) {
  bar1();
  bar2();
}
```

Switch-case

case keyword should be on same level as the braces for the switch keyword.

YES:

```
switch (foo)
{
case 1:
    // do stuff
    break;
case 2:
    // do another stuff
    break;
default:
    break;
}
```

NO:

```
switch (foo)
{
    case 1:
        // do stuff
        break;
    case 2:
        // do another stuff
        break;
    default:
        break;
}
```

If case codeblock must be enclosed with braces, the braces goes on same level as the case keyword itself.

```
switch (foo)
{
case 1:
{
    // do
    // something
    // with
    // this big
    // code
    break;
}
case 2:
    // do simpler stuff
```

```
    break;
default:
    break;
}
```

Pointer affinity

For pointers and references, the asterisk/ampersand should appear next to the variable name.

This makes it consistent with dereferencing (*foo), and is preferred by most LÖVE devs.

YES:

```
int *foo;
int &bar;
```

NO:

```
int* foo;
int& bar;

int * foo;
int & bar;
```

YES:

```
int *get_int_ptr();
int &get_int_ref();
```

NO:

```
int* get_int_ptr();
int& get_int_ref();

int * get_int_ptr();
int & get_int_ref();
```

Spaces vs Tabs

I personally prefer tabs, but I know I'm the only person in the world. We will probably end up with spaces, in which case I suggest 4 spaces per level.

EDIT: I was wrong, everyone (who has so far given a damn) has preferred tabs. In that case, let me specify a more advanced indentation guideline for the truly l33t, hardcore, and handsome.

MIXED TABS AND SPACES (*monocle pops*)

Use tabs for normal indentation.

I.e. when indicating scope.

YES:

```
if (foo)
{
<tab>bar();
}
```

NO:

```
if (foo)
{
<space><space><space><space>bar();
}
```

Use spaces for alignment.

If you *really* want to align stuff, then the important thing to keep in mind is that the alignment should not break if the viewer is using a tab-size that's different from yours. So, in this example, we're aligning stuff:

```
if (foo)
{
    int a_var      = 1;
    int anoter_var = 2;
    int moar       = 3;
    int doom       = 4;
}
```

YES:

```
if (foo)
{
<tab>int a_var<space * 6>= 1;
<tab>int anoter_var<space>= 2;
<tab>int moar<space * 7>= 3;
<tab>int doom<space * 7>= 4;
}
```

NO:

```
if (foo)
{
<tab>int a_var<tab * 2>= 1;
<tab>int anoter_var<tab>= 2;
<tab>int moar<tab * 2>= 3;
<tab>int doom<tab * 2>= 4;
}
```

However, use alignment sparingly, because it makes a lot of diff noise when you add something that's longer than the currently longest thing, and you have to realign everything. So the best option is actually:

Trailing whitespace

Trim all trailing whitespace.

Tabs and spaces before a newline character increase the noise in commits and the source code.

YES:

```
<tab>foo();

<tab>bar();
```

NO:

```
<tab>foo();
<tab>
<tab>bar();
```

Documentation blocks

Asterisks should be aligned vertically.

YES:

```
/**
 * Short description.
 *
 * @param foo The foo that goes in the bar.
 * @param bar The bar the foo foes into.
 */
int foz(int foo, int &bar);
```

NO:

```
/**
* Short description.
*
```

```
 * @param foo The foo that goes in the bar.
 * @param bar The bar the foo foes into.
 */
int foz(int foo, int &bar);
```

If you agree to this, and you also agree that we're using tabs. You're agreeing that tabs and spaces be mixed (although in a very defined way). For a function declaration that's indented one block, you would type something like:

```
<tab>/**
<tab><space>* Short description.
<tab><space>*
<tab><space>* Long description.
<tab><space>*/
```

#includes

Non-system includes should use the double-quotes syntax.

YES:

```
#include "common/config.h"
```

NO:

```
#include <common/config.h>
```

Casts

Use C-style casts.

Remember to also Know What You Are Doing when using C-style casts.

YES:

```
Foo *foo = (Foo *) bar;
```

NO:

```
Foo *foo = (Foo*)bar;
Foo *foo = (Foo*) bar;
Foo *foo = (Foo *)bar;
```

Writing null

Prefer nullptr over NULL or 0.

YES:

```
foo = nullptr;
bar = nullptr;
```

NO:

```
foo = NULL;
bar = 0;
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Commies!

Commies! · revision 9871 · last edited 2021-03-14

This project is no longer in development.

Common Organization of Controls Kit

Common Organization of Controls Kit · revision 19267 · last edited 2023-07-05

Note:

Current version: 1.3

The library does not currently have examples.

Common Organization of Controls Kit is a tool for handling control over your LÖVE games.

This is an input handling library. It enables you to easily define a (several of) set of control bindings to be used in your game, and provides a very user-friendly methods for handling it, e.g. you can access all gathered data as a table member, and it has a function to grab a user input to store it into control map, and to unset it, which is really handy in controls setup screen etc. **Check out the demo!** http://love2d.org/forums/download/file.php?id=8398

This library have much detail to it, please **consutlt the manual**.

NOTE: the library only support latest released version of LÖVE, and the author only support latest released version of the library. There may be no backwards compatibility.

Features

- Multiple separate control objects
- Comprehensive controls setup
- Support for "defaults"
- All available input devices supported
- Capture and bind user input
- Joystick deadzones and hat remapping, hat modes, pick up previously used joysticks
- Delta modes and inversion modes
- Literal or numerical arguments to functions; number of helper functions

Scheduled:

- XBox360 controller support for all OSes (for now only works with Windows, partially)
- Functions to save/load configurations (for now simple serialization of the object will do)

Links

- Project page: http://raidho36.net/stuff/cock ↗
- Forum thread: http://love2d.org/forums/viewtopic.php?f=5&t=40089

Changelog

- 1.3:
  - fixed:
    - code refactoring; improved consistency
    - fixed broken event handler locks
    - generating random IDs now work properly
    - fixed issue with spacebar key and :explodeCapturedData/:bind (unexplodedData) functions
    - fixed all :*Joystick* functions.
    - :addJoystick will now create a dummy joystick if it's impossible to actually create the joystick
  - added:
    - .defaultOption and defaultConfig fields
    - :convetJoystickHat, :convertJoystickHatMode, :convertKey
    - :find returns control object with given ID

        - :updateAll updates all registered objects
        - :addJoystick and :removeJoystick (handled automatically)
        - .controlcaptured, .controlpeaked, .controlzeroed, .controlpressed, .controlreleased, .controlchanged callbacks
        - :setCallbacks for enabling/disabling above callbacks
        - addded 4-way diagonal hat mode (4-way input, diagonal input generates both conjuncted directions input)
    - changed:
        - control objects now have unique identifiers
        - control binds data will no longer accept one-default arranged data
        - event callbacks are now supplied with ID of the sender object
        - all joystick-related functions now have "device id" field which specifies the joystick used in terms of control object
        - :setDefaultXBox360 now accepts device ID to use
        - :capturedDataExplode renamed to :explodeCapturedData (lol)
        - conversion functions now have argument "literal" to force either literal or numerical output
        - :getBinded and pals are now return raw unconverted data
        - :setCapture function will now by default invoke the callback
        - renamed joystick hat modes
        - lookup tables will now grow further to avoid extra garbage/increase function performance
        - :setDefaultXBox360 now returns joystick ID (could change)
    - removed:
        - :setControlMode and pals
        - :setDefaultXBox360 for Linux and OS X
- 1.2:
    - fixed:
        - a bunch of potential pitfalls
        - joystick axes and hats are now correctly captured
    - added:
        - no-cutoff for negative values for inversion
        - multiple binds per action via "options" tables
        - :setJoystickDeadzone and getJoystickDeadzone gets and sets joystick deadzone per axis
        - :getJoysticksList returns list of all found joysticks names and numbers
        - :remapJoystickHat maps four joystick buttons to act like joystick hat
        - :bind directly binds mappings
        - :getBinded returns list of binds for map
        - :getEmptyOption returns next (in arbitrary order) unused option for map
        - :getControlModesList returns list of all control modess
        - :getConvertedDevice, :getConvertedAxis, :getConvertedInverse, :getConvertedDelta
        - cutoff delta modes and inversion modes (only pass positive/negative part)
        - :setDefaultXBox360 function (currently used XBox360 controller API in Windows version sucks pretty hard)
        - :capturedDataExplode helper function (can't pass table to LÖVE's events)
    - changed:
        - :setCapture now accepts "callback" argument to be used instead of instant automatical binding
        - :grab now would either call callback function or instantly bind captured input
        - :unset renamed to :unbind
        - controls data table for assignment now have different format
        - :getJoystickHatMode and :getControlMode now only return literal value
        - joystick axis delta mode no longer requires stick to be on the corresponding side either (that was silly, too)
    - removed:
        - :getDeviceName (replaced with :getBinded)
- 1.1:
    - fixed:
        - capturing an input now sets both "previous" and "current" tables to captured input value
        - mouse axis delta mode no longer requires mouse pointer to be on the corresponding side of the offset
    - added:
        - getDeviceName function returns literal name of a device assiged to specified map
    - changed:
        - reloadJoystick now returns true on success
- 1.0:

- first release version

See also

- TLbind
- Common Organization of Controls Kit Manual

Common Organization of Controls Kit Callbacks

Common Organization of Controls Kit Callbacks · revision 11513 · last edited 2021-03-14

The library provides the functionality of envoking callbacks when certain action happens, just like love.keypressed or love.mousepressed callbacks.

Basics

Following callbacks are defined:

```
cock.controlcaptured = function ( id, longdata )
cock.controlpressed = function ( id, map, value )
cock.controlreleased = function ( id, map, value )
cock.controlpeaked = function ( id, map, value )
cock.controlzeroed = function ( id, map, value )
cock.controlchanged = function ( id, map, value )
```

string id
: ID of the object that issued the callback.

string map
: Control map that triggered the callback.

number value
: Readings on the input.

string longdata
: Programmatically arranged data that sums up all info about captured user input. It is used because all the fields wouldn't fit into callbacks' 4 available slots. You must use cock.explodeCapturedData helper function to convert it to meaningful data. You can pass it "as is" to the cock.bind function.

- controlcaptued called when in input mode some user input was captured
- controlpressed called when readings went above 0.5
- controlreleased called when readings went below 0.5
- controlpeaked called when readings reached 1.0 (or above)
- controlzeroed called when readings dropped to 0.0
- controlchanged called whenever reading is changed

Details

Because this system generates vastly more calls than regular LÖVE callbacks system, all callbacks are disabled by default. You must enable them with cock.setCallbacks except "controlcaptured", which is set at each cock.setCapture call.

Every callback is supplied with ID of the object that issued the callback. You may not need it, but if you have multiple control objects, that will help you figure which one has the input activity. To get corresponding control object table, you can use cock.find function.

If your game uses user-defined controls via capturing them, then your most important callback is "controlcaptured". In it, you would decide whether or not you want to use the captured data, alter it, discard or actually bind, and finally perform other related actions. E.g. you may check whether or not supplied key is already binded elsewhere and prompt user for override or cancel capture.

All other callbacks are used the same way as LÖVE's callbacks.

See also

- cock.find
- cock.bind
- cock.getBinded
- cock.setCapture
- cock.cancelCapture
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

Common Organization of Controls Kit Fields

Common Organization of Controls Kit Fields · revision 11616 · last edited 2021-03-14

Control object have a variety of fields that you can directly access.

Basics

Control objects' structure is defined as following:

- defaultOption
- defaultLayout
- current
- previous
- joysticks
  - name
  - number
- etc
  - id
  - mouse
    - x
    - y
    - fx
    - fy

Details

`string defaultOption`
: Option to use in functions that require it by default. Default value is "default".

`string defaultLayout`
: Layout to use in cock.setDefault by default. Default value is "default".

`table current`
: Table that contains all readings for maps at current frame.

`table previous`
: Table that contains all readings for maps at previous frame.

`table joysticks`
: Table that contains all used joysticks.

`string name`
: Joystick name.

`number number`
: Joystick number.

`table etc`
: Table that contains miscellaneous data.

`string id`
: Object ID.

`table mouse`
: Table that contains mouse-related settings.

`number x`
: X-coordinate of mouse offset. Default is 0.

`number y`
: Y-coordinate of mouse offset. Default is 0.

`number fx`
: Horizontal scaling factor. Default is 1.

`number fy`
: Vertical scaling factor. Default is 1.

Accessing object.current.* is a suggested way of getting current state of the input.

You can access real joystick names and number by accessing object.joysticks[#].name and object.joysticks[#].number respectively.

An ID can be read from object.etc.id field.

All mouse settings can be done vla functions (cock.setMouseOffset and cock.setMouseFactor) or directly throguh object.etc.mouse.* fields.

See also

- Common Organization of Controls Kit Manual

Common Organization of Controls Kit Hat modes

Common Organization of Controls Kit Hat modes · revision 11560 · last edited 2021-03-14

Default mode for joystick hat is "8-way" diagonal input, just like default LÖVE's joystick hats, the other general mode is "4-way".

Basics

Following hat modes are defined:

- "8-way" (1)
- "4-way vertical" (2)
- "4-way horizontal" (3)
- "4-way clockwise" (4)
- "4-way counter-clockwise" (5)
- "4-way diagonal" (6)

Details

Since 4-way mode would discard all diagonal input, the update handler have 5 modes that specifically tell how to resolve diagonal input:

- Prefer vertical input ("4-way vertical")
  - convert diagonal up to up and diagonal down to down
- Prefer horizontal input ("4-way horizontal")
  - convert diagonal left to left and diagonal right to right
- Prefer clockwise input ("4-way clockwise")
  - "turn" diagonal input 45 degrees clockwise, e.g. down-left becomes left, etc.
- Prefer counter-clockwise input ("4-way counter-clockwise")
  - "turn" diagonal input 45 degrees counter-clockwise, e.g. down-left becomes down, etc.
- Convert diagonal to adjacent lateral ("4-way diagonal")
  - convert diagonal up and left to both lateral up and lateral left

As the name suggests, first two modes are axially biased and will more likely result in getting corresponding direction when actively working on D-pad, and this is more of a gameplay-related setting than actual adjustment, e.g. if you use left and right to move the block in a tetris game, having it accidntally dropped because player would hit the D-pad diagonally down is not desired, and this is when you enable horizontal bias.

The other two are radially biased (in some way they are *unbiased*) and although they don't make any direction pressed any more likely, they make changing D-pad direction more fluent in one way and more stiff in the other way. This modes give better feel of purely 4-directional D-pad on immediate hat presses, while maintaining only allowing one direction at a time. You may, in fact, just randomly use either of them, but you may also provide choosing between them as an option, since not all joysticks' hats are made so great, and specific mode enabled may actually improve players' accuracy.

The last mode doesn't actually adjusts anything and just passes by the input, "splitting" it in two.

See also

- cock.setJoystickHatMode
- cock.convertJoystickHatMode
- cock.remapJoystickHat
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

Common Organization of Controls Kit Input devices

Common Organization of Controls Kit Input devices · revision 11612 · last edited 2021-03-14

To simplify handling all the variety of input devices, and simplify binding them, every device is represented with a numerical and literal value.

Basics

Following input devices are defined:

- "keyboard" (1)
- "mouse button" (2)
- "mouse axis" (3)
- "joystick button" (4)
- "joystick axis" (5)
- "joystick hat" / "joystick hat 1" (6)
- "joystick hat 2" (7)
- "joystick hat 3" (8), etc.

Additionally, every joystick is represented with internal number that may differ from LÖVE number, and internal name that *will* differ from LÖVE name. Name modification is really simple though, just a joystick number appended to the end of the joystick name. But that already allows for distinguishing joysticks with same names.

Details

When using any function that accepts an input device, you can either provide a numerical value, which is useful for programmatical input, or literal value, which is useful for user interaction. You shouldn't use numerical values for hard-coding data, because 1) that will make your code messier with all of these magic numbers and 2) internal conversion is an effecient and optimized operation, and it won't save execution time. Even if the libary didn't employed internal conversion and required to use numerical values, any performance difference would be completely neglectible. For lovers of the "Big O" notation: search time is O(1) average case and O(1+K) worst case. Every failed lookup creates programmatically generated data and stores for later use, thus once failed, the lookup will always be successful.

See also

- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual

Common Organization of Controls Kit Input keys

Common Organization of Controls Kit Input keys · revision 11614 · last edited 2021-03-14

Input keys are essentially what LÖVE's .isDown and such report when called.

Basics

Input keys for keyboard is strings reported by love.keyboard.isDown, same for mouse and joystick buttons and hats. Joystick axis are represented as themselves. Mouse axis are mapped to numbers: X axis is 1, and Y axis is 2. Additionally, following axis names are defined:

- "x" (1)
- "y" (2)
- "z" (3)
- "r" (4)
- "u" (5)
- "v" (6)
- "7" (7)
- "8" (8), etc.

Details

Input keys are what they are, except for mouse axis, which are mapped to numbers. When using any function that accepts an input key, you can either provide a regular value, which is useful for programmatical input, or for axis-like devices you can provide literal value, which is useful for user interaction. You shouldn't use numerical values for hard-coding data, because 1) that will make your code messier with all of these magic numbers and 2) internal conversion is an effecient and optimized operation, and it won't save execution time.

See also

- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Modes
- Common Organization of Controls Kit Manual

Common Organization of Controls Kit Manual

Common Organization of Controls Kit Manual · revision 11617 · last edited 2021-03-14

On this page

- Basic use
  - Preparations
  - Controls data format
  - Obtaining the values
  - Updating controls
- Enabling user to edit controls
- Advanced features
  - Mouse offset and scaling factor
  - Delta and inversion modes, cutoff modes
  - Joystick hat modes
  - Deadzones
  - Joystick hat remapping
- Displaying setup
- Function index
- See also

Basic use

The library designed to be reasonably easy to use while maintain great flexibility and features.

Preparations

First thing you need to do is, obviously, require the library. Then, you create a new control object using cock.new. You can optionally fill it with data at instant, or may do it later. Then you optionally load saved settings, via serialization for an instance. After that (but not before!) you can reload joystick with cock.reloadJoysticks, that essentially will search for previously used joystick and set it up. If you don't have controls to load, you assign default controls, joystick modes and mouse values with cock.setControls (if you didn't do it upon creation).

Controls data format

Functions cock.new and cock.setControls are accepting data in specific format that needs to be maintained. It is defined as following:

```
{ default1 = {
  up = { primary = { "keyboard", "up" }, secondary = { "mouse axis", "y", "negative", "positive cutoff" },
  down = { primary = { "keyboard", "down" }, secondary = { "mouse axis", "y", "positive", "positive cutoff" } },
 default2 = { ... }
}
```

It will define you controls "up" and "down" to be accessed later, with two options to use ("primary" and "secondary"), one with keyboard only control and another with mouse only, and pushes it to "default1" controls set. You can then use defined defaults with cock.setDefault to e.g. restore controls to default, or set them up as such if there's no user-edited controls.

Obtaining the values

This is done really easy: you simply access your control class "current" or "previous" table's child named as your defined control.

```
if control.current.up > 0 then
    player.y = player.y - 100 * dt
elseif control.current.down > 0 then
    player.y = player.y + 100 * dt
end
```

These variables (except for mouse axis) contain decimal values ranged -1 to 1 (0 to 1 in "cutoff" and "whole" mode), which you can use for analog control. Mouse axis-bound variables would store pretty arbitrary numbers, with undefined range, only influenced by mouse factor and offset.

You can also use library callbacks just like normal input callbacks:

```
function cock.controlpressed ( id, key, val )
    if key == "up" then
        player.accel = 1
    end
end
```

By default, all library callbacks are disabled, so to use them you have to enable them via cock.setCallbacks.

The library also have effecient getters and setters for that, but you are *strongly* discouraged to use them.

Updating controls

You simply put update function (cock.updateAll) to your "love.update" callback, and that's it. It is suggested to put it to beginning of the function. Placing it to other callbacks, such as "love.keypressed" is not necessary at all due to LÖVE's way of handling them.

Enabling user to edit controls

The library have a special function to assume "input capture" mode (cock.setCapture), in which any incoming user input will be captured and automatically binded to provided key. You may specify whether or not you want to lock input callbacks and update function, and ignore mouse motion, during input capture mode. You can abort this mode using cock.cancelCapture. In input capture mode, if mouse motion capture is enabled, the library will temporairly center the mouse and then set it back once input is captured, but it will not hide it because, with all the possible user implementations of mouse cursor, it's simply meaningless, so make sure you hide your cursor during capture, otherwise it'll weirdly jump to the center of the screen and back. In this case you would normally want to use controlcaptured callback, with cock.bind at the end.

You can bind the device and key manually, using the cock.bind function.

You can implement "arbitrary binds per action" using search for unused options via cock.getEmptyOption. It will return next unused option, and since for gathering input, option name is irrelevant, you can simply bind keys to free options arbitrairly. If the function fails to find empty option, you'd have to create a new option with cock.addOption rather than binding to unexisted option. This is due to internal structure of the library.

NOTE: in input capture mode, the library would temporary replace your input callbacks and update function with it's own wrappers for them, until some input is provided, which could interfere with other libraries/modules that would do the same (or even with itself), so use it with care. Make sure you do not assume another input capture while being in input capture, this creates an infinite recursion and the game will instantly bluescreen with a stack overflow error. The rule of thumb is to only allow one callbacks replacement at a time, or only allow nested replacements, but never intersecting replacements. If none of your other code would do that in a separate thread, but would do that in main thread, enabling both locks ensures safe capture. If your code doesn't do that at all, having locks enabled may be not necessary.

Advanced features

This library was created with robust design in mind, so it supports a bit of handy advanced features that otherwisely would be needed to implemented by the programmer, which is not always convenient.

Mouse offset and scaling factor

For simplicity of aquiring values from control table, it handles this two math operations for you. It subtracts actual mouse position from an offset (cock.setMouseOffset), and then multiplies result by scaling factor(cock.setMouseFactor), per axis. This creates an easy way to set mouse centre point and mouse sensevity.

Delta and inversion modes, cutoff modes

These mode alter the behavior of input readings. Inversion modes are negating the input (reverse axis and swap "pressed" and "released" states for keys), and delta mode would report change in the value rather than actual value. Uses for inversion are obvious, and delta modes can report only keypresses/releases, and velocities of the axis-like input devices.

More read: Common Organization of Controls Kit Modes

Joystick hat modes

Default mode for joystick hat is "8-way" diagonal input, just like default LÖVE's joystick hats, the other are variants of "4-way". Using this feature you can make axis horizontally or vertically biased, or even radially biased. You can also "split" the axis into 4 directions and it would report values per each direciton, effectively treating dpad like 4 keys rather than directional pad.

More read: Common Organization of Controls Kit Hat modes

You can change the modes on fly as necessary with cock.setJoystickHatMode.

Deadzones

You can define deadzones per-axis with cock.setJoystickDeadzone. This function works pretty straightforward. You might want to know a little detail about it's implementaiton, that is it first subtracts threshold from the value and then amplifies it by another number that results in axis being "shrinked", starting at threshold point and ending at 1 like normal, rather than instantly jumping from 0 to threshold parameter and such.

Joystick hat remapping

Due to the way some joysticks' hats are handled, you may want to map some of joysticks' buttons to a virtual joystick hat with cock.remapJoystickHat. To do that, you simply use corresponding function and pass it exactly 4 joystick buttons to use. After that, the joystick will have specified hat "remapped", and it'll work like regular joystick hat. Note that once you started remapping, it will instantly switch to "remapped hat" mode, and incomplete remapping will result in incomplete operability.

Displaying setup

Library provides handful of feedback functions, such as conversion from literal to numerical values and vice versa, for returning binded

controls, for finding unused options. More on this you can find in the demos.

Function index

cock.addJoystick	Hooks up new joystick.
cock.addOption	Creates new input option.
cock.bind	Binds given input.
cock.cancelCapture	Cancels input capture.
cock.convertAxis	Converts axis.
cock.convertDelta	Converts delta.
cock.convertDevice	Converts device.
cock.convertInverse	Converts inverse.
cock.convertJoystick	Converts joystick.
cock.convertJoystickHat	Converts joystick hat.
cock.convertJoystickHatMode	Converts hat mode.
cock.convertKey	Converts key.
cock.delete	Deletes object.
cock.deleteJoystick	Deletes joystick.
cock.deleteOption	Deletes input option.
cock.explodeCapturedData	Explodes longdata.
cock.find	Find object.
cock.getBinded	Gets binded controls.
cock.getCapture	Gets capture state.
cock.getEmptyOption	Finds unused option.
cock.getJoystickDeadzone	Gets deadzones.
cock.getJoystickHatMode	Sets hat modes.
cock.getJoysticksList	Lists joysticks.
cock.getMouseFactor	Gets mouse factor.
cock.getMouseOffset	Gets mouse offset.
cock.new	Creates new object.
cock.reloadJoysticks	Reloads joysticks.
cock.remapJoystickHat	Maps buttons to hats.
cock.setCallbacks	Toggles callbacks.
cock.setCapture	User-edit controls.
cock.setControls	Creates control layouts.
cock.setDefault	Reverts to defaults.
cock.setDefaultXBox360	(BROKEN) Creates XBox360 layout.
cock.setJoystickDeadzone	Sets deadzones.
cock.setJoystickHatMode	Sets hat modes.
cock.setMouseFactor	Sets mouse factor.

cock.setMouseOffset	Sets mouse offset.
cock.unbind	Unbinds map.
cock.update	Updates object.
cock.updateAll	Updates all objects.

See also

- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Modes
- Common Organization of Controls Kit Callbacks
- Common Organization of Controls Kit Hat modes
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Setup data
- Common Organization of Controls Kit

Common Organization of Controls Kit Modes

Common Organization of Controls Kit Modes · revision 11559 · last edited 2021-03-14

On this page

- Basics
- Details
- Modes list
  - Inverse
  - Delta
- See also

Inversion and delta mode are used to alter input readings in easy fashion.

Basics

Inversion mode is pretty straightforward, it simply negates input. Delta mode is a little bit trickier. In this mode, instead of reporting absolute value, the control object reports delta value, that is how much input have changed since last update. You can think of it as a velocity at which the reading changes, that is mouse velocity or joystick thumstick velocity. Both delta and inversion modes have "cutoff" mode to prevent it from returning negative values. Delta mode and inversion mode are both having "negative" modes that work together: any negatives negates the output, and double negative makes positive. This may seem to be excessive, but actually it's a lot simplier to do it that way. There's also one special inversion mode for joysticks - "whole" - that converts -1..1 range to 0..1 range, which is particularry useful for HOTAS controllers and XBox360 triggers.

Details

Delta mode is particularry useful when used with joystick; this opens possibility to implement obscure and cunning ways of using analog sticks and XBox360 triggers, that aren't normally can be seen in the games. When used with mouse, this simply returns distance travelled from previous coordinate. Note that if you want FPS-like controls where you would constantly set mouse back to the center of the screen, you should instead use "bypass" mode, since in this state, delta mode would yield acceleration of the mouse rather than distance travelled. You can still use that nevertheless, if you may.

You can also use delta mode to register key presses or key releases, which is useful for "jump" keys etc. Delta with "cutoff" mode could report you if joystick have moved further, but not when it moved the opposite direction, which results in "dragging"-like input.

You set these modes with cock.bind function. If you only want to change one or few them, you'd have to first obtain original values with cock.getBinded, modify them, and bind them back. This is done that way for the sake of simplicity.

Modes list

Inverse

- "negative whole" / "-=" (-3)
- "negative cutoff" / "--" (-2)
- "negative" / "-" (-1)
- "positive" / "+" (1)
- "positive cutoff" / "++" (2)
- "positive whole" / "+=" (3)

Delta

- "negative cutoff" / "--" (-2)
- "negative" / "-" (-1)
- "bypass" / "=" (0)
- "positive" / "+" (1)
- "positive cutoff" / "++" (2)

See also

- cock.getBinded
- cock.convertInverse
- cock.convertDelta
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

Common Organization of Controls Kit Setup data

Common Organization of Controls Kit Setup data · revision 18433 · last edited 2021-03-14

Functions cock.new and cock.setControls would accept specifically formatted table to build layouts from. It follows a simple pattern:

```
default
  key
    option
      data
```

Defaults, keys and options are unordered and can go in arbitrary fashion. But the "data" table is ordered list, and must follow this pattern:

```
device, key, inverse, delta
```

Last two parameters are optional and set by default ("positive cutoff" and "bypass" respectively) if not specified.

Example:

```
local data = {
  default_keyboard = {
    up = { optionA = { "keyboard", "up" }, optionB = { "mouse axis", "y", "negative", "positive cutoff" } },
    down = { optionA = { "keyboard", "down" }, optionB = { "mouse axis", "y", "positive", "positive cutoff" } }
    }
  default_joystick = {
    up = { optionA = { "joystick hat", "u" }, optionB = { "joystick axis", "y", "negative cutoff" } },
    down = { optionA = { "joystick hat", "d" }, optionB = { "joystick axis", "y", "positive cutoff" } }
    }
  }

control:setControls ( data )
control:setDefault ( "default_keyboard" )
```

This creates two default configurations: default_keyboard and default_joystick. Each of them has "optionA" and "optionB", just two different keys would trigger the same action. Then, there's two controls: "up" and "down". Note that there should not necessairly be mathing options across all defaults and controls: if you don't specify an option for certain control, missing option simply won't trigger it. Moreover, different defaults don't even have to have similar sets of controls. It just there won't happen anything if something is missing.

NOTE: if you only have one option per control, then it will be set to whatever value the binded device have. If there's more than one, then the biggest value will be selected, which effectively makes any of the bindings trigger the control. For analog devices, the biggest value is selected. In case two options have different signs of reading values, then they would cancel each other out. If there's more than two options and bingings are simultaneously giving different signs, then outcome is pretty much undefined due to arbitrary nature of data traversal. It could be an exact value of some of the binded devices, or could be cancelled out value. This is not normally a concern since one wouldn't normally bind two different axis to the same control.

NOTE: if you setting up joystick controls, make sure there's at least one joystick defined. Even if there's no joystick is hooked up, you can create a dummy joystick with cock.addJoystick(1). If there won't be any attached joysticks, many joystick-related functions will result in bluescreening the game.

See also

- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Modes
- Common Organization of Controls Kit Manual
- cock.setControls
- cock.setDefault
- cock.new

Comparing LÖVE Versions

Comparing LÖVE Versions · revision 24623 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- Source
- Other Languages

It's sometimes useful to check for certain LÖVE versions, for example, do automatic color conversion to 0..1 range in version 11.0 and later, and keep using 0..255 range in 0.10.2 and earlier.

Function

Synopsis

```
compare = compareLOVEVersion(major, minor, revision)
```

Arguments

number major
: Major version

number minor (nil)
: Minor version

number revision (nil)
: Revision

Returns

number compare
: -1 if current LÖVE version is lower than specified, 0 if *exactly* equal, 1 if it's later version

Examples

```
-- LÖVE version: 0.9.2
compareLOVEVersion(0, 9, 2) -- returns 0

-- LÖVE version: 0.10.0
compareLOVEVersion(11, 1) -- returns -1

-- LÖVE version: 0.10.2
compareLOVEVersion(0, 10, 0) -- returns 1
```

Source

```
function compareLOVEVersion(maj, min, rev)
    if love._version_major > maj then
        return 1
    elseif love._version_major < maj then
        return -1
    elseif min then
        if love._version_minor > min then
            return 1
        elseif love._version_minor < min then
            return -1
        elseif rev then
            if love._version_revision > rev then
                return 1
            elseif love._version_revision < rev then
                return -1
            end
        end
    end
    -- equal
```

```
    return 0
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

cron

cron · revision 24459 · last edited 2023-07-05

A set of functions for executing actions at a certain time interval by Enrique García Cota aka kikito (https://github.com/kikito).

Download (https://github.com/kikito/cron.lua/blob/master/cron.lua)

```lua
local cron = require 'cron'

local function printMessage()
  print('Hello')
end

-- the following calls are equivalent:
local c1 = cron.after(5, printMessage)
local c2 = cron.after(5, print, 'Hello')

c1:update(2) -- will print nothing, the action is not done yet
c1:update(5) -- will print 'Hello' once

c1:reset() -- reset the counter to 0

-- prints 'hey' 5 times and then prints 'hello'
while not c1:update(1) do
  print('hey')
end

-- Create a periodical clock:
local c3 = cron.every(10, printMessage)

c3:update(5) -- nothing (total time: 5)
c3:update(4) -- nothing (total time: 9)
c3:update(12) -- prints 'Hello' twice (total time is now 21)
```

Links

- Github (https://github.com/kikito/cron.lua)
- Readme (https://github.com/kikito/cron.lua/blob/master/README.md)
- cron.lua (https://github.com/kikito/cron.lua/blob/master/cron.lua)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Dave Gone Apeshit

Dave Gone Apeshit · revision 4877 · last edited 2021-03-14

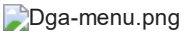
Dave Gone Apeshit is a run and gun game made by Lukasz Bruun and Matias Billeschou in one weekend.

Download

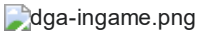
Download at Github (https://github.com/lukaszbruun/Dave-Gone-Apeshit/downloads)

More information in this forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=2379)

Screenshots







debugging

debugging · revision 10846 · last edited 2021-03-14

This page will help describe how you can debug the love binary on windows.

Download the Windows Debugging Tools, they are part of the Windows SDK;

http://msdn.microsoft.com/en-us/windows/hardware/gg463009.aspx ↗

We'll be using windbg.exe to find out where LÖVE crashes, or at least get pointed in the right direction. Also download this debug version of LÖVE. It will show more information about the crash than the release build.

http://dl.dropbox.com/u/14748254/love-0.8.0-win-x86-debug.zip ↗ (mirror this)

To gather information follow these steps:

1. Start windbg.exe.

2. Select "Open Executable" from the file menu.

3. Select the love.exe from the debug build above. To also make it load your game, enter the path to the .love file in the "Arguments" text box near the bottom of the dialog. Do avoid spaces and characters like ä, ö, and ü in this text box.

4. Confirm with "open". You don't have to save the workspace if it asks you.

5. It will now load the binary, all dependencies, and then show some info in a textbox. If it loaded correctly, you'll see something like:

```
ntdll!DbgBreakPoint:
76ec0004 cc              int     3
```

Don't worry about the "symbols not found" stuff.

6. To start the execution, enter the single character command "g" and hit return.

7. Try to make it crash again, then the execution of LÖVE should pause immediately. (Choose retry if a dialog box pops up with the options ignore, retry and cancel)

8. Go back to the windbg window.

9. To display more information about the current state, enter the commands "~* k", "u", "dv" and "r".

10. Save all the text displayed and post it on the issue tracker, a pastebin, or send a private message to me, whatever feels appropriate.

It's possible we won't be able to tell what goes wrong if the crash happens outside LÖVE, like in the graphics driver.

deleteme2

deleteme2 · revision 14482 · last edited 2021-03-14

(test page)

Desert Loot

Desert Loot · revision 6238 · last edited 2021-03-14

A man walks along the desert carrying a heavy burden. The sun beats down. Where is he going? Where is he coming from? He seems to be collecting a lot of stuff.

Desert Loot is a game that was created for the Game Prototype Challenge v4 (http://gameprototypechallenge.com/v4) event.

- Forum Thread (http://love2d.org/forums/viewtopic.php?f=5&t=2691)



Download

- .love (http://gloryfish.s3.amazonaws.com/games/desertloot/desertloot.love)
- Mac (http://gloryfish.s3.amazonaws.com/games/desertloot/DesertLoot.zip)
- Windows (http://gloryfish.s3.amazonaws.com/games/desertloot/desertloot_win.zip)
- Source (https://github.com/GloryFish/GPC-4-Desert)

Doctor Cat: The Game

Doctor Cat: The Game · revision 8149 · last edited 2021-03-14

GitHub Page: https://github.com/josefnpat/doctorcatthegame ↗

DRAKON Editor

DRAKON Editor · revision 20035 · last edited 2021-03-14

DRAKON Editor is flow-chart oriented visual language develop environment.

supported for Lua code generation.

- Website (http://drakon-editor.sourceforge.net/)

dt

dt · revision 19985 · last edited 2023-07-05

dt is the most common shorthand for delta-time, which is usually passed through love.update to represent the amount of time which has passed since it was last called. It is in seconds, but because of the speed of modern processors is usually smaller than 1 values like 0.01 are common.

Examples

Increase a variable x by 1 every second

```
x = 0
function love.update(dt)
   x = x + dt
end
```

Change a position x at a fixed speed while a key is held down

```
x = 0
speed = 32
function love.update(dt)
   if love.keyboard.isDown("right") then
      x = x + (speed * dt)  -- x will increase by 32 for every second right is held down
   elseif love.keyboard.isDown("left") then
      x = x - (speed * dt)
   end
end
```

See Also

- love.update
- love.timer.getDelta

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Easy GUI System

Easy GUI System · revision 16785 · last edited 2023-12-08

On this page

- Class.lua
  - Defining a class
  - Defining a class with multiple properties
  - Defining a class with multiple properties with default values
  - Instantiating a class
  - Creating multiple intances of a class
  - Inheriting properties
  - End note
- Using my GUI library
  - Events
  - What are Structural Classes?
  - What are Non-Structural Classes?
- Buttons
  - Making a TextLabel appear when a Button is hovered
  - Making a button respond to hovering
  - Making a TextLabel with a message when you hover

Easy GUI System is an easy-to-use GUI system made by Ashley Davies (https://github.com/ashleydavies). It is available on its Github page (https://github.com/ashleydavies/EGS/). EGS is not currently maintained, but functions on the current versions of LÖVE, thanks to contributions by buckle2000 on GitHub. Ashley is considering maintaining, tidying, and expanding on this library. If you would like for this to happen, please see further detail on the GitHub page for the repository.

General tutorial

Class.lua

My class system is simple to use. The following few tutorials will teach you the basics.

Defining a class

```
Class "ClassName" {Properties}
```

Here we have a basic class called 'ClassName'. It has a property called "Properties", with a default value of an empty string (We didn't give it a default value).

Defining a class with multiple properties

```
Class "ClassName" {
    Property1,
    Property2,
}
```

Here's another basic class. It just creates a class with two properties.

Defining a class with multiple properties with default values

```
Class "ClassName" {
    ["Property1"] = 7,
    Property2,
}
```

To give a property a default value, make it a key instead of a value, as above, and give it a value of the default value. Default values can be any type.

Instantiating a class

```
Class "ClassName" {
    ["Property1"] = 7,
    Property2,
}

ClassName:new("InstanceName")
```

```
print(InstanceName.Property1)
>7
```

As you can see, we pass in the names, rather than getting a return value, so we do the above, as opposed to:

```
ClassName = Class {
    ["Property1"] = 7,
    Property2,
}
InstanceName = ClassName:new()
```

The class function makes use of the `getfenv()` function, which allows it to define variables in the calling environment. Classes are not local variables, they are global. So are instances.

Creating multiple intances of a class

```
Class "ClassName" {
    ["Property1"] = 7,
    Property2,
}

ClassName:new("InstanceName")
InstanceName.Property2 = "Hello world"
print("InstanceName1 says: " .. InstanceName.Property2)
ClassName:new("InstanceName2")
print("InstanceName2 says: " .. InstanceName2.Property2)

>InstanceName1 says: Hello world
>InstanceName2 says:
```

If no default is set, a class property will get a default value of an empty string, instead of nil, so it won't error if you try to concatenate it.

Inheriting properties

```
Class "ClassName" {
    ["val"] = true,
}

Class "ClassName2" ({
    ["Property1"] = 7,
    Property2,
}, {ClassName})

ClassName2:new("InstanceName")
print(tostring(InstanceName.val))
>true
```

To inherit, instead of typing

```
Class "cName" {Properties}
```

We type

```
Class "cName" ({Properties}, {Which classes to inherit from})
```

As it's a table you can inherit from multiple classes.

If you have the same property in both classes, for example

```
Class "cName" {["Prop1"] = "val1"}
Class "cName2" ({["Prop1"] = "val2"}, {cName})
```

It inherits from an object with a property the same, the second class (cName2)'s Prop1 will become it's default, as it overrides the inherited value.

So if I instanced "cName", and printed it's Prop1, it'd print val1, but doing the same to cName2 would print val2. This mean you can make classes that inherit the majority of things from another, but change or completely destroy (just set it to nil) some, you can.

End note

You can get by without understanding this object system, so long as you understand the examples below.

Using my GUI library

By now you should be familiar with the class system.

My GUI library classes are organised into two sections: Structural and Non-Structural classes.

Events

Events behave identically to those found on [Roblox (http://roblox.com)].

```
Object.Event:connect(function)
```

For example:

```
CheckBox:new("Checkbox1")
Checkbox1.checkStateChanged:connect(function()
    print("Checkbox1's state was changed by the :changeState() function. It fires on mouse click automatically but can be fired manually.")
end)
```

What are Structural Classes?

In this library, structural classes are classes whose only purpose is for other classes to inherit from. They are not all entirely documented as they should not be instanced.

What are Non-Structural Classes?

Non-Structural classes are things like Checkboxes. You can instance them, and they are mainly GUI items.

Examples

Buttons

Making a TextLabel appear when a Button is hovered

Note that since TextLabel only loses the events 'mouseClicked', 'mousePressed', and 'mouseReleased', but not 'mouseEntered' and 'mouseExited', it's possible to do this with two TextLabels with identical results. However TextLabels were not made to be functional so it's not recommended.

First we need a TextLabel and a TextButton. Put these lines in love.load:

```
TextButton:new("HoverButton")
TextLabel:new("Display")
```

Now that we've got them we can define some variables. Also put these lines below the ones you just did in love.load:

```
HoverButton.text = "Hover me to see the TextLabel."
HoverButton.autoSize = false
HoverButton.size = {100,20}
HoverButton:centerText()--Aligns text to center on Y and X axis.
HoverButton.hasBorder = false
Display.text = ":o"
Display.visible = false
Display.size = {100,20}
Display:centerText()
Display.position = {0,20}
Display.hasBorder = false
```

Now we need #Events. These are also in love.load.

```
HoverButton.mouseEntered:connect(function()
    Display.visible = true
end)
HoverButton.mouseExited:connect(function()
    Display.visible = false
end)
```

Now if we run that, it's a functional example!

It is straightforward, as EGS was made to be easy, and achieves this while maintaining a good range of customisability. For example, the shading when you hover on the button can be turned off by just adding:

```
HoverButton.hasShading = false
```

Making a button respond to hovering

In this tutorial we'll make a button respond to hovering. The purpose is to explore more properties of GUI elements. For example, all three color properties (textColor, borderColor, backgroundColor) are just a table with three values in, {R, G, B}, for example, {255, 0, 0} represents pure red.

I recommend you set the window backgroundColor to white for this tutorial! (love.graphic.setBackgroundColor(255,255,255))

First we make the button and initialise some values:

```
TextButton:new("TB")
TB.text = "Hover me!"
TB.autoSize = false
TB.size = {100,20}
TB.position = {10,10}
TB.backgroundColor = {0,200,255}
--Aligns text to center on Y and X axis.
TB:centerText()
```

Let's mix it up a little.

```
TB.hasShading = false --Get rid of shading.
TB.borderWidth = 1 --Make the border thinner.
TB.mouseEntered:connect(function()
    TB.borderWidth = 2
end)
TB.mouseExited:connect(function()
    TB.borderWidth = 1
end)
```

It looks much nicer with the thinner border! We'll now try changing the color a little. Replace those two events you did with this:

```
TB.mouseEntered:connect(function()
    TB.borderWidth = 2
    TB.backgroundColor = {0,150,255} --Less green!
end)
TB.mouseExited:connect(function()
    TB.borderWidth = 1
    TB.backgroundColor = {0,200,255} --Back to skyblue.
end)
```

Getting better, but still not perfect. How about making the borderColor and textColors change too?

```
TB.mouseEntered:connect(function()
    TB.borderWidth = 2
    TB.backgroundColor = {0,150,255} --Less green!
    TB.textColor = {50,100,50}
    TB.borderColor = TB.textColor
end)
TB.mouseExited:connect(function()
    TB.borderWidth = 1
    TB.backgroundColor = {0,200,255} --Back to skyblue.
    TB.textColor = {0,0,0}
```

```
    TB.borderColor = TB.textColor
end)
```

Making a TextLabel with a message when you hover

It's simple to achieve this, as ALL GUIElements have this feature, called *Caption*.

```
TextLabel:new("TL")
TL.text = "Hover me!"
TL.autoSize = false
TL.size = {100,20}
TL.position = {10,10}
TL.backgroundColor = {0,200,255}
TL:centerText()
TL.hasShading = false
TL.borderWidth = 1
```

We're going to stick with the nice style we made in the last tutorial. But we need a few more variables for a caption. Add this:

```
TL.hasCaption = true
TL.caption = "This is a caption!"
```

Execute it, and huzzah!

If you want the caption to come up quicker or slower, there's a neat function for that, add this just below TL.caption = "This is a caption!":

```
--Will wait 3 seconds of mouse hovering to display caption.
TL:setSecondsForCaption(3)
```

Effect Constants Name Candidates

Effect Constants Name Candidates · revision 22628 · last edited 2021-03-14

On this page

- Filter and Effect "Objects"
- Filters
  - Synopsis
  - Filter Arguments
  - FilterType
  - Banter
- Effects
  - Synopsis
  - Effect Arguments applicable for all types
  - EffectType
  - EffectWaveform
  - Banter
- Reverb Arguments
  - Banter
- Chorus Arguments
  - Banter
- Distortion Arguments
  - Banter
- Echo ("Delay") Arguments
  - Banter
- Flanger Arguments
  - Banter
- Ring modulator Arguments
  - Banter
- Compressor Arguments
  - Banter
- Equalizer Arguments
  - Banter
- Unsupported Effects
- Queuable Sources
- Audio Recording / Input

The upcoming löve version (11.0) will include substantial amount of audio-related advancements. This page serves two functions:

- To decide on a neat and consistent terminology used for the to-be-added Filters and Effects (the other stuff has already been finalized),
- To have a compendium for people using the nightlies so they won't need to browse the source to find out what works how (and so that the wiki article creation will happen a bit more smoothly)

Alternative suggested names go into parentheses; Banter to be added to the respective sub-headers.

Refer to OpenAL EFX handbook for underlying system details: http://kcat.strangesoft.net/misc-downloads/Effects%20Extension%20Guide.pdf ↗

Filter and Effect "Objects"

Effects and Filters are generic Lua tables with named parameters, e.g.:

```
{ type = "reverb", volume = 1.0, density = 0.3 } -- This is a reverb Effect.
```

More precisely, they are *descriptions* of Effects and Filters. When appropriate function is called, Effect and Filter parameters are modified using data from that table.

Passing nil as parameter value, as well as omitting the parameter, will apply default value. The "type" and "volume" parameters are mandatory for all Effects and Filters, all other parameters are optional. Values will be restricted to valid range internally.

Filters

Filters are basic low-pass, high-pass and band-pass (a combination of both high- and low-pass) frequency filters. They can be applied to each sound Source individually, but only one Filter per Source at a time.

Synopsis

```
-- Apply Filter to Source.
success = Source:setFilter(Filter)
--              applyFilter(Filter)

-- Get the currently bound Filter of a Source.
Filter = Source:getFilter()
```

Filter Arguments

`FilterType type`
: The type of the Filter. See FilterType below.

`number volume`
: Gain of the whole spectrum. Range: [0,1]

number lowgain
: Gain on the low-end. Range: [0,1]

number highgain
: Gain on the high-end. Range: [0,1]

Note: Low-pass Filters don't have a lowgain parameter, High-pass Filters don't have a highgain parameter.

FilterType

string lowpass
: Used to remove high frequency content from a signal.

string highpass
: Used to remove low frequency content from a signal.

string bandpass
: Used to remove high and low frequency content from a signal.

Banter

I'd go with setFilter so it pairs up with getFilter, plain and simple; don't care about the implementation. Zorg (talk) 19:59, 1 February 2017 (CET)

> i dont think the argument name `volume` is smart as it might make people use the filter section in a wrong manner. a more suitable name would be `outgain` or `output`--Roccdcasbah (talk) 18:32, 13 March 2017 (CET)

Effects

Effects are advanced modules modifying the audio signals that can be used to recreate realistic acoustic properties of specific areas, as well as creating cool effects. This sound system is built on top of OpenAL, so it's only possible to set 4 Effects at a time.

Each Source can feed its output to any of these 4 Effects, and it can also feed to multiple Effects simultaneously. Each feed line can have a Filter attached to it, which will filter the sound between the Source and Effect.

Effects work continuously, even when no sound is fed, e.g. the reverb Effect can still output a decaying sound long after all input is cut.

Synopsis

```
-- Apply Effect to scene.
success = love.audio.setSceneEffect(slot, Effect)
--                   setEffect(slot, Effect)

-- Get current scene Effect.
Effect = love.audio.getSceneEffect()
--                  getEffect()

-- Use scene Effect with Source.
success = Source:setSceneEffect(line, slot, Filter)
--               useSceneEffect(line, slot, Filter)
--               setEffect(line, slot, Filter)

-- Get Source's currently used scene Effect.
slot, Filter = Source:getSceneEffect(line)
--                   getEffect(line)
```

Effect Arguments applicable for all types

EffectType type
: Type of the Effect. See EffectType below.

number volume
: Master volume of the Effect. Range: [0,1]

Note: Effect-specific parameters detailed below.

EffectType

string echo (delay)
: Decaying feedback based effect on the order of seconds.

string reverb
: Decaying feedback based effect on the order of milliseconds. Used to simulate location properities.

string chorus
: Overlays the signal on top of itself with pitch and time variation. Used to make sounds sound "fuller".

string flanger
: Overlays the signal on top of itself with phase variation.

string distortion
: Clips the sound artificially to make it sound distorted.

string ringmodulator (ringmod)
: Varies the volume over time; with a fast enough speed, it'll sound kinda like a bell.

string compressor
: Increases loudness over the whole spectrum.

string equalizer
: A 4-band equalizer used to modify the signal, boost or cut specific frequency ranges.

EffectWaveform

string sine
: A sine wave.

string triangle
: A triangle wave.

string sawtooth
: A ramp / sawtooth wave.

string square
: A symmetric pulse / square wave.

Banter

I'd again vote for keeping function names simple; with a disclaimer on the wiki page that modifying an effect's parameter won't apply it here until you call this again... at least that's how i understood it from the docs; i may be wrong. Also, the docs state that only sine and triangle waveforms are implemented, then again, i guess the other two might exist in AL-Soft. Zorg (talk) 19:59, 1 February 2017 (CET)

> The waveforms are implemented in ALsoft. And yes audio state only changes when you apply any settings. Tables are *descriptions* of effects, not Effect objects. Raidho36 (talk) 21:34, 1 February 2017 (CET)

I think I prefer `echo`, which is the OpenAL name anyway. The others are fine. -Bartbes (talk) 20:55, 1 February 2017 (CET)

Reverb Arguments

number density
: Reverb Modal Density controls the coloration of the late reverb. Lowering the value adds more coloration to the late reverb. Range: [0,1]

number diffusion
: The Reverb Diffusion property controls the echo density in the reverberation decay. It's set by default to 1.0, which provides the highest density. Reducing diffusion gives the reverberation a more "grainy" character that is especially noticeable with percussive sound sources. If you set a diffusion value of 0.0, the later reverberation sounds like a succession of distinct echoes. Range: [0,1]

number gain (reverbgain)
: The Reverb Gain property is the master volume control for the reflected sound (both early reflections and reverberation) that the reverb effect adds to all sound sources. It sets the maximum amount of reflections and reverberation added to the final sound mix. The value of the Reverb Gain property ranges from 1.0 (0db) (the maximum amount) to 0.0 (-100db) (no reflected sound at all).

number hfgain (reverbhighgain, highgain)
: The Reverb Gain HF property further tweaks reflected sound by attenuating it at high frequencies. It controls a low-pass filter that applies globally to the reflected sound of all sound sources feeding the particular instance of the reverb effect. The value of the Reverb Gain HF property ranges from 1.0 (0db) (no filter) to 0.0 (-100db) (virtually no reflected sound).

number decay (decaytime)
: The Decay Time property sets the reverberation decay time. It ranges from 0.1 (typically a small room with very dead surfaces) to 20.0 (typically a large room with very live surfaces).

number hfdecay (decayhighratio)

The Decay HF Ratio property adjusts the spectral quality of the Decay Time parameter. It is the ratio of high-frequency decay time relative to the time set by Decay Time. The Decay HF Ratio value 1.0 is neutral: the decay time is equal for all frequencies. As Decay HF Ratio increases above 1.0, the high-frequency decay time increases so it's longer than the decay time at mid frequencies. You hear a more brilliant reverberation with a longer decay at high frequencies. As the Decay HF Ratio value decreases below 1.0, the high-frequency decay time decreases so it's shorter than the decay time of the mid frequencies. You hear a more natural reverberation. Range: [0.1,2.0]

number earlygain

The Reflections Gain property controls the overall amount of initial reflections relative to the Gain property. (The Gain property sets the overall amount of reflected sound: both initial reflections and later reverberation.) The value of Reflections Gain ranges from a maximum of 3.16 (+10 dB) to a minimum of 0.0 (-100 dB) (no initial reflections at all), and is corrected by the value of the Gain property. The Reflections Gain property does not affect the subsequent reverberation decay. Range: [0,3.16]

number earlydelay

The Reflections Delay property is the amount of delay between the arrival time of the direct path

from the source to the first reflection from the source. It ranges from 0 to 300 milliseconds. You can reduce or increase Reflections Delay to simulate closer or more distant reflective surfaces— and therefore control the perceived size of the room. Range: [0,0.3]

number lategain

The Late Reverb Gain property controls the overall amount of later reverberation relative to the Gain property. (The Gain property sets the overall amount of both initial reflections and later reverberation.) The value of Late Reverb Gain ranges from a maximum of 10.0 (+20 dB) to a minimum of 0.0 (-100 dB) (no late reverberation at all). Range: [0,10]

number latedelay

The Late Reverb Delay property defines the begin time of the late reverberation relative to the time of the initial reflection (the first of the early reflections). It ranges from 0 to 100 milliseconds. Reducing or increasing Late Reverb Delay is useful for simulating a smaller or larger room. Range: [0,0.1]

number rolloff (roomrolloff)

The Room Rolloff Factor property is one of two methods available to attenuate the reflected

sound (containing both reflections and reverberation) according to source-listener distance. Range: [0,10]

number airhfgain (airabsorptionhfgain, highdamp, airabsorption)

The Air Absorption Gain HF property controls the distance-dependent attenuation at high frequencies caused by the propagation medium. It applies to reflected sound only. You can use Air Absorption Gain HF to simulate sound transmission through foggy air, dry air, smoky atmosphere, and so on. Range: [0.892,1]

boolean hflimiter (highlimit)

When this flag is set, the high-frequency decay time automatically stays below a limit value that's derived from the setting of the property Air Absorption HF. This limit applies regardless of the setting of the property Decay HF Ratio, and the limit doesn't affect the value of Decay HF Ratio.

Banter

I prefer the alternate names pretty much everywhere (and I suggested mine otherwise), for completeness sake, these are `gain`, `highgain`, `decaytime`, `decayhighratio`, `roomrolloff`, `airabsorption` and `highlimit`. An additional note on `airabsorption`: It's not like there's a parameter to set any other value, so I think adding "high" or "hf" is superfluous. -Bartbes (talk) 20:55, 1 February 2017 (CET)

Chorus Arguments

EffectWaveform waveform

This property sets the waveform shape of the LFO that controls the delay time of the delayed signals.

number phase

This property controls the phase difference between the left and right LFO's. At zero degrees the two LFOs are synchronized. Use this parameter to create the illusion of an expanded stereo field of the output signal. Range: [-180,+180]

number rate

This property sets the modulation rate of the LFO that controls the delay time of the delayed signals. Range: [0,10]

number depth

This property controls the amount by which the delay time is modulated by the LFO.

number delay

This property controls the average amount of time the sample is delayed before it is played back, and with feedback, the amount of time between iterations of the sample. Larger values lower the pitch. Smaller values make the chorus sound like a flanger, but

with different frequency characteristics. Range: [0,0.016]

number feedback
: This property controls the amount of processed signal that is fed back to the input of the chorus effect. Negative values will reverse the phase of the feedback signal. At full magnitude the identical sample will repeat endlessly. At lower magnitudes the sample will repeat and fade out over time. Use this parameter to create a “cascading” chorus effect. Range: [-1,1]

Banter

Phase offset extremes are represented in degrees, may alternatively be normalized to math.pi or 1. Raidho36 (talk) 21:40, 1 February 2017 (CET)

I'd be for math.pi, but if positional audio also uses 1 as units then that's fine too. Zorg (talk) 23:12, 1 February 2017 (CET)

Distortion Arguments

number gain (distortiongain)
: This property allows you to attenuate the distorted sound. Range: [0.01,1]

number edge
: This property controls the shape of the distortion. The higher the value for Edge, the ‘dirtier’ and ‘fuzzier’ the effect. Range: [0,1]

number lowcut
: Input signal can have a low pass filter applied, to limit the amount of high frequency signal feeding into the distortion effect. Range: [80,24000]

number eqcenter
: This property controls the frequency at which the post-distortion attenuation (Distortion Gain) is active. Range: [80,24000]

number eqbandwidth
: This property controls the bandwidth of the post-distortion attenuation. Range: [80,24000]

Banter

What does "eq" mean here? Equalizer? And how does that relate to distortion? As, admittedly, an audio noob these two param names confuse me the most. -Bartbes (talk) 20:55, 1 February 2017 (CET)

> Distortion effect has a 1-band post-distortion equalizer; gain, eqcenter and eqbandwidth control it. Raidho36 (talk) 21:41, 1 February 2017 (CET)

Echo ("Delay") Arguments

number delay
: This property controls the delay between the original sound and the first ‘tap’, or echo instance. Subsequently, the value for Echo Delay is used to determine the time delay between each ‘second tap’ and the next ‘first tap’. Range: [0,0.207]

number lrdelay
: This property controls the delay between the first ‘tap’ and the second ‘tap’. Subsequently, the value for Echo LR Delay is used to determine the time delay between each ‘first tap’ and the next ‘second tap’. Range: [0,0.404]

number damping
: This property controls the amount of high frequency damping applied to each echo. As the sound is subsequently fed back for further echoes, damping results in an echo which progressively gets softer in tone as well as intensity. Range: [0,0.99]

number feedback
: This property controls the amount of feedback the output signal fed back into the input. Use this parameter to create “cascading” echoes. At full magnitude, the identical sample will repeat endlessly. Below full magnitude, the sample will repeat and fade. Range: [0,1]

number spread
: This property controls how hard panned the individual echoes are. With a value of 1.0, the first ‘tap’ will be panned hard left, and the second tap hard right. A value of –1.0 gives the opposite result. Settings nearer to 0.0 result in less emphasized panning. Range: [-1,1]

Banter

As mentioned in IRC, I'm confused by `lrdelay`, as I'm not sure what LR means, if it's not Left/Right. And if it **is** Left/Right, what does that have to do with echo? -Bartbes (talk) 20:55, 1 February 2017 (CET)

Echo effect produces two taps (echoes) of the same sound, lrdelay defines the delay between first and second tap. The parameter that controls stereo separation of those two taps is spread.Raidho36 (talk) 21:44, 1 February 2017 (CET)

So then why is it called `lrdelay`? -Bartbes (talk) 22:47, 1 February 2017 (CET)

No clue. That's what it's called in OpenAL and it's a direct carry over.Raidho36 (talk) 23:06, 1 February 2017 (CET)

Flanger Arguments

`EffectWaveform waveform`

Selects the shape of the LFO waveform that controls the amount of the delay of the sampled signal.

`number phase`

This changes the phase difference between the left and right LFO's. At zero degrees the two LFOs are synchronized. Range: [-180,+180]

`number rate`

The number of times per second the LFO controlling the amount of delay repeats. Higher values increase the pitch modulation. Range: [0,10]

`number depth`

The ratio by which the delay time is modulated by the LFO. Use this parameter to increase the pitch modulation. Range: [0,1]

`number delay`

The average amount of time the sample is delayed before it is played back; with feedback, the amount of time between iterations of the sample. Range: [0,0.004]

`number feedback`

This is the amount of the output signal level fed back into the effect's input. A negative value will reverse the phase of the feedback signal. Use this parameter to create an "intense metallic" effect. At full magnitude, the identical sample will repeat endlessly. At less than full magnitude, the sample will repeat and fade out over time. Range: [-1,1]

Banter

Ring modulator Arguments

`EffectWaveform waveform`

This controls which waveform is used as the carrier signal. Traditional ring modulator and tremolo effects generally use a sinusoidal carrier. Sawtooth and square waveforms are may cause unpleasant aliasing.

`number frequency`

This is the frequency of the carrier signal. If the carrier signal is slowly varying (less than 20 Hz), the result is a tremolo (slow amplitude variation) effect. If the carrier signal is in the audio range, audible upper and lower sidebands begin to appear, causing an inharmonic effect. The carrier signal itself is not heard in the output. Range: [0,8000]

`number highcut`

This controls the cutoff frequency at which the input signal is high-pass filtered before being ring modulated. If the cutoff frequency is 0, the entire signal will be ring modulated. If the cutoff frequency is high, very little of the signal (only those parts above the cutoff) will be ring modulated. Range: [0,24000]

Banter

Compressor Arguments

None.

Banter

Technically it has an ON/OFF switch, dunno if you want to expose that or not. Zorg (talk) 19:59, 1 February 2017 (CET)

Equalizer Arguments

`number lowgain`

This property controls amount of cut or boost on the low frequency range.

`number lowcut`

This property controls the low frequency below which signal will be cut off.

`number mid1gain (lowmidgain)`
: This property allows you to cut / boost signal on the “mid1” range.

`number mid1frequency (lowmidfrequency)`
: This property sets the center frequency for the “mid1” range.

`number mid1bandwidth (lowmidbandwidth)`
: This property controls the width of the “mid1” range.

`number mid2gain ( highmidgain)`
: This property allows you to cut / boost signal on the “mid2” range.

`number mid2frequency (highmidfrequency)`
: This property sets the center frequency for the “mid2” range.

`number mid2bandwidth (highmidbandwidth)`
: This property controls the width of the “mid2” range.

`number highgain`
: This property allows you to cut / boost the signal at high frequencies.

`number highcut`
: This property controls the high frequency above which signal will be cut off.

Banter

I'm not sure how accurate `lowmid` and `highmid` are (it looks like openal doesn't care), but I definitely prefer that over `mid1` and `mid2`. -Bartbes (talk) 20:55, 1 February 2017 (CET)

> It does care, lower mid band has lower frequency range than higher mid band.Raidho36 (talk) 21:47, 1 February 2017 (CET)
>> In that case `lowmid`/`highmid` is definitely better than `mid1`/`mid2` -Bartbes (talk) 22:47, 1 February 2017 (CET)
>>> Took a look at openal.. the Low Mid is sweepable between 200-1000Hz and the High Mid is sweepable between 1000-8000Hz. therefore the codes `lowmid` and `highmid` would be correct.--Roccdcasbah (talk) 18:32, 13 March 2017 (CET)
>>>> And the argument "bandwidth" is not really common "soundlingo", the more correct name would simply be "Q". so `mid1bandwidth` would become `mid1Q` or if you use the suggestion in my previous comment `lowmidQ`.--Roccdcasbah (talk) 18:32, 13 March 2017 (CET)
>>>>> I tend to agree with using Q here, but only because it both makes the parameter names shorter, and that it is an accepted term for these kinds of parameters, as said above. ("In physics and engineering the quality factor or Q factor is a dimensionless parameter that describes how under-damped an oscillator or resonator is,[1] and characterizes a resonator's bandwidth relative to its center frequency." ~ Wikipedia (https://en.wikipedia.org/wiki/Q_factor)) That said, from a somewhat layman's perspective, explaining it as the bandwidth from the center frequency of the bands on the wiki should be good enough. --Zorg (talk) 18:41, 13 March 2017 (CET)

Unsupported Effects

Following effects and their constants are not supported in current version of AL-soft but as they get support in the future they may get enabled in LÖVE.

Effect types:

- frequencyshifter
- freqshifter
- vocalmorpher
- morpher
- pitchshifter
- autowah

Frequency shifter:

- frequency
- leftdirection
- rightdirection
- Frequency shifting directions:
  - up
  - down
  - none

Vocal morpher:

- waveform
- rate
- phonemea (formant1)
- phonemeb (formant2)
- tunea
- tuneb

Vocal morpher includes a substantial range of effect-specific phoneme constants, but they are no subject for debate, the only variable is letter casing: capital or lowercase, current being latter. Not that it matters, since it's unsupported for now, but i'd vote for uppercase for better legibility.

Pitch shifter:

- pitch

Autowah:

- attack
- release
- resonance
- peakgain

Queuable Sources

Audio Recording / Input

examples

examples · revision 8197 · last edited 2021-03-14

main.lua这个真是别人写的

main.lua

fadeColor

fadeColor · revision 19085 · last edited 2021-03-14

fadeColor is a helper function to make smooth color transitions suitable to fading effects.

```
function fadeColor(
  time, prologue, attack, sustain, decay, epilogue,
  fade_in_r, fade_in_g, fade_in_b,
  fade_out_r, fade_out_g, fade_out_b
)
  -- [0, prologue)
  if time < prologue then
    return
      fade_in_r,
      fade_in_g,
      fade_in_b,
      255
  end

  -- (prologue, prologue + attack]
  time = time - prologue
  if time < attack then
    return
      fade_in_r,
      fade_in_g,
      fade_in_b,
      ( math.cos( time / attack * math.pi ) + 1 ) / 2 * 255
  end

  -- (prologue + attack, prologue + attack + sustain]
  time = time - attack
  if time < sustain then
    return
      fade_in_r,
      fade_in_g,
      fade_in_b,
      0
  end

  -- (prologue + attack + sustain, prologue + attack + sustain + decay]
  time = time - sustain
  if time < decay then
    return
      fade_out_r,
      fade_out_g,
      fade_out_b,
      255 - ( ( math.cos( time / decay * math.pi ) + 1 ) / 2 * 255 )
  end

  -- (prologue + attack + sustain + decay, prologue + attack + sustain + decay + epilogue]
  time = time - decay
  if time < epilogue then
    return
      fade_out_r,
      fade_out_g,
      fade_out_b,
      255
  end

  -- End of fading, return all nils.
end
```

Arguments:

- time is the 0-based time to evaluate the color.
- prologue is how much time the resulting color will be equal to the fully opaque fade_in color.
- attack is how much time it takes to go from fully opaque fade_in to fully transparent fade_in.
- sustain is how much time the color resulting color will be the fully transparent fade_in.
- decay is how much time it takes to go from fully transparent fade_out color to fully opaque fade_out color.

- epilogue is how much time the resulting color will be equal to the fully opaque fade_out color.
- fade_in_[rgb] are the components of the fade_in color.
- fade_out_[rgb] are the components of the fade_out color.

The function returns nil when the transitions are over. The transitions from fully opaque to fully transparent are evaluated with the cosine function to make it smooth. Usage:

```
-- This is part of your update loop, draw whatever you need here.

-- Evaluate a color that makes the screen fade from white and then to black.
local r, g, b, a = fadeColor(
  dt,
  2,   -- Two seconds opaque white.
  2,   -- Two seconds to make it transparent.
  3,   -- Three seconds transparent.
  2,   -- Two seconds to go to opaque black.
  0,   -- Zero seconds opaque black.
  255, 255, 255, -- White.
  0, 0, 0 -- Black.
)

-- If the fading is done, better go do something else...
if not r then
  -- Change state to something else...
  return
end

-- Fill the entire screen with the resulting color.
love.graphics.setColor( r, g, b, a )
love.graphics.rectangle( 'fill', 0, 0, love.graphics.getWidth(), love.graphics.getHeight() )
```

fakecanvas

fakecanvas · revision 9050 · last edited 2023-07-05

fakecanvas aims to expose the awesome capabilities of canvases to those whose hardware/software are not able to use them. it is meant to be a drop-in library, only needing a `require 'fakecanvas'` call in order to be used.

fakecanvas automatically detects canvas capabilities and will use real canvases if supported, but it can be configured to always use fake canvases (useful for testing!).

canvas emulation is not bulletproof, and may have a fair bit of overhead, but it's a nice fallback for many cases.

External Links

- github link (https://github.com/fogfx/love2d-fakecanvas)
- forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=10324)

findProjectileTrajectory

findProjectileTrajectory · revision 19086 · last edited 2023-07-05

Finds a projectile's position at a certain time.

Variable Definitions : ri = initial position {x, y} ; vi = intial velocity {vx, vy} ; g = gravity {gx, gy} ; t = time

```
-- This assumes that ri, vi, and g are put in the {x, y} notation
function findProjectileTrajectory(ri, vi, g, t)
    local rfx = ri[1] + vi[1]*t + (g[1] * t^2)/2
    local rfy = ri[2] + vi[2]*t + (g[2] * t^2)/2
    return rfx, rfy
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

findRotation

findRotation · revision 19087 · last edited 2021-03-14

This function takes two sets of XY coordinates. It returns the direction from point A to point B.

Syntax

```
float findRotation(float Xa, float Ya, float Xb, float Yb)
```

Required Arguments

- **Xa**: The X coordinate of the starting point.
- **Ya**: The Y coordinate of the starting point.
- **Xb**: The X coordinate of the target point.
- **Yb**: The Y coordinate of the target point.

Source

```
function findRotation(x1,y1,x2,y2)

  local t = -math.deg(math.atan2(x2-x1,y2-y1))
  if t < 0 then t = t + 360 end;
  return t;

end
```

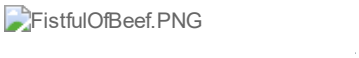
Fistul of Beef

Fistul of Beef · revision 6303 · last edited 2021-03-14

Once upon a time (actually, Wednesday the 18th of August 2010) the lover TheLinx stumbled upon the web site of Ludum Dare, home of the famous competition with the same name. He got inspired, and felt Ludum Dare could need some LÖVE, so he spawned the LÖVE-PARTY, the now infamous group that creates epic games.

The first game spawned by LÖVE-PARTY was **Fistful of Beef** (what a coincidence, you're looking at its wiki page right now!).

The game



Fistful of Beef is a game in which you kill bulls, because.. you are a cowboy.. in a bullfighting arena.. you know what I mean, right? Anyway, you kill bulls, but, you only have a lasso, and choking them isn't exactly an option either. So what do you do? Correct, you use the bulls to kill bulls!

Download

You can find it on github here (http://github.com/LOVE-Party/Love-LD18).

On the download page (here (http://github.com/LOVE-Party/Love-LD18/dow nloads)) are binary releases and 'source' releases.

Fizz X

Fizz X · revision 31347 · last edited 2026-04-14

Fizz is a lightweight collision library in Lua based on the earlier work of Taehl. This project is no longer available.

Links

- Original forum thread: https://love2d.org/forums/viewtopic.php?f=5&t=7844

flat table

flat table · revision 18622 · last edited 2023-07-05

We use the term flat table to denote a table which doesn't contain any other (nested) tables.

```
-- Flat table
flatTable = {
    'foo',
    'bar',
    'baz',
    'qux',
    -- ...
}

-- Nested tables
nestedTables = {
    foo = {
        'bar',
        'baz',
        'qux',
    },
    -- ...
}
```

In the context of love threads, a flat table is understood to not contain tables, functions, userData or a metatable.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

fLUIds

fLUIds · revision 24626 · last edited 2023-07-05

love-imgui API wrap for simplicity, inspired by LaTeX TikZ node positioning. (written in MoonScript!)

```
drawable = function() imgui.Text("Hello World!") end
fluids.Window("Titlebar", drawable, {10, 10, 100, 300}, {"NoTitleBar"})
```

GitHub: https://github.com/twentytwoo/fLUIds ↗

Forum thread: https://love2d.org/forums/viewtopic.php?f=5&t=84525

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Free Game Resources

Free Game Resources · revision 22571 · last edited 2021-03-14

On this page

- Game Design/Ideas
- Graphics/Art
  - 3D:
  - Software:
  - Procedural generation:
  - Misc:
- Sound
  - Music:
  - Software:
- Fonts
- Physics / Movement
  - Editors
- Maps
  - Software
  - Procedural generation
- General

Links to free stuff (Creative Commons-licensed and the like) that can be used in game creation.

Game Design/Ideas

- Lost Garden - Directory of Posts (http://www.lostgarden.com/2008/07/directory-of-posts.html)
- Three Hundred Mechanics (http://www.squidi.net/three/). Lot of game ideas, free to use

Graphics/Art

- FreeGameDev wiki: 2D, 3D and audio resources (http://wiki.freegamedev.net/index.php/Free_3D_and_2D_art_and_audio_resources) (it also has a search (http://search.freegamedev.net/) section)
- OpenGameArt (http://opengameart.org)
- Openclipart (http://openclipart.org/)
- Genetica Texture Packs (http://www.spiralgraphics.biz/packs/) High quality tiling textures.
- WayOfThePixel forum post (http://www.wayofthepixel.net/pixelation/index.php?topic=3467.0) with lots of links to utilities, pixel-art tutorials, resources, etc.
- http://reinerstileset.4players.de/englisch.html ↗ isometric sprites
- http://www.spiralgraphics.biz/packs/ ↗
- http://www.spriters-resource.com/ ↗
- Sprite Database (http://sdb.drshnaps.com/) (Careful! Most of the stuff there is copyrighted)
- http://www.burningwell.org/ ↗
- http://game-icons.net/ ↗ Many icons (and emoji) in SVG Licensed CC-BY with some even Public Domain.

3D:

- Yobi3D - 3D model search engine (http://www.yobi3d.com)

Software:

- Freegamedev's list of 2D (http://wiki.freegamedev.net/index.php/2D_software) and 3D (http://wiki.freegamedev.net/index.php/3D_software) software.
- graphicsgale (http://www.humanbalance.net/gale/us/)
- Pixothello (http://teknopants.com/pixothello/)
- ASEPrite (http://www.aseprite.org)
- Texturepacker (http://www.texturepacker.com/) (has free & paid versions)
- Slate (https://github.com/mitchcurtis/slate) A simple bitmap editor for 2D tilesets.

Procedural generation:

- A Bitwise Method For Applying Tilemaps (http://www.saltgames.com/?p=184) is an interesting technique related with tilemaps and graphics.

- Tile/Map-Based Game Techniques: Handling Terrain Transitions (http://www.gamedev.net/page/resources/_/technical/game-programming/tilemap-based-game-techniques-handling-terrai-r934) shows a more elaborate technique for handling transitions between tiles.

Misc:

- CG graphics tutorials. (http://lodev.org/cgtutor/) Written for C++ but the concepts are generic, of course.

Sound

- bfxr (http://www.bfxr.net/) is an online sound generator.
- Independent Ljudbank (http://www.johannespinter.com/inu/ljudbank.htm) (Swedish)
- Freesound (http://www.freesound.org/) (Requires registration for downloads, but otherwise bullshit-free)
- PDSounds (http://www.pdsounds.org/) (might be down, check out this (http://pdsounds.tuxfamily.org/) in that case)
- Soundsnap (http://www.soundsnap.com/) (make sure you learn the license (http://www.soundsnap.com/licence), it is not exactly CC)
- Media College's sound effects (http://www.mediacollege.com/downloads/sound-effects/)
- Newground's audio (http://www.newgrounds.com/audio/) contains free songs and loops.
- http://www.artistx.org/site2/media/free-resources/free-music.html ↗ another listing-type page. Licenses vary.

Music:

- Jamendo's free music (http://www.jamendo.com/es/albums?order=ratingweek_desc&tag_idstr=&location_country=all&license_minrights_c=on&license_minrights_d=on) (Various)
- Musopen (http://www.musopen.com/)
- Binärpilot (http://binaerpilot.no) (Evil chiptunes. Big plus for umlaut. Licensed CC-NC-ND. Use with caution.)
- 8bitcollective (http://8bitcollective.com/) (Chiptunes) (Licensed CC-NC-SA. Use with caution.)
- Fanatical Productions (http://www2.fanaticalproductions.net:8080/)
- Creative Commons' own list of music (http://creativecommons.org/legalmusicforvideos), containing lots of links. Two examples below:
  - ccmixter (http://ccmixter.org/) is a "a community music site featuring remixes licensed under Creative Commons where you can listen to, sample, mash-up, or interact with music in whatever way you want."
  - sampleswap (http://sampleswap.org/) is a "100% free AIFF/WAV collection contains 6.4 GB of sounds"
- abundant-music (http://www.abundant-music.com/) procedural music generator.
- Modarchive (http://modarchive.org/) Tracker music (MOD, XM, S3M, IT ...)

Software:

- Freegamedev's list of audio software (http://wiki.freegamedev.net/index.php/Audio_software)
- sfxr (http://www.cyd.liu.se/~tompe573/hp/project_sfxr.html) (Sound effect generator)
  - Flash version (http://www.superflashbros.net/as3sfxr/) of the same software (google project page (http://code.google.com/p/as3sfxr/))
  - Cocoa version (http://thirdcog.eu/apps/cfxr) for mac.
- Musagi (http://www.drpetter.se/project_musagi.html) Is a music editor, Windows only.
- pxtone (http://hp.vector.co.jp/authors/VA022293/pxtone/index.html) Comes with a composition program (ptCollage), instrument editor (ptVoice), and a sound editor (ptNoise). For an English user interface delete japanese.ico in the root directory :D

Fonts

- Google Web Fonts (http://www.google.com/webfonts) 915 font families (+ variants) which can be freely used for whatever you want (license (http://www.google.com/webfonts#AboutPlace:about))
- OpenFontLibrary (http://openfontlibrary.org/) (this (http://forum.freegamedev.net/viewtopic.php?f=6&t=176) helps you browse it)
- dafont (http://www.dafont.com/)
- acidfonts (http://www.acidfonts.com) has both free and non-free fonts.
- 40+ Excellent Free Fonts (http://www.smashingmagazine.com/2007/11/08/40-excellent-freefonts-for-professional-design/) A collection of selected fonts

Physics / Movement

- Collision Detection for Dummies (http://www.wildbunny.co.uk/blog/2011/04/20/collision-detection-for-dummies/)
- Sonic the Hedgehog's physics guide (http://info.sonicretro.org/Sonic_Physics_Guide) is a group of pages explaining in detail how physics are modeled in Sonic the Hedgehog.
- N tutorial A (http://www.metanetsoftware.com/technique/tutorialA.html) and N tutorial B (http://www.metanetsoftware.com/technique/tutorialB.html) explain how they did the physics in N: the way of the Ninja (http://www.thewayoftheninja.org/).

- Movement mechanics on "The Legend of Zelda" (http://troygilbert.com/the-legend-of-zelda/)
- Iron Coding's Excellent tutorial for tile-based movement (http://www.ironcoding.com/2011/02/flash-as3-tile-based-game-tutorial-pt-1-intro/) (It's flash, but very well explained. The ideas can be used for any language)
- The guide to implementing 2D-platformers (http://higherorderfun.com/blog/2012/05/20/the-guide-to-implementing-2d-platformers/) A good read about 2d platformers: main types, general implementations, and some gritty details
- Hardcoding & Slopes (http://concernedjoe.com/devblog/hardcoding-slopes/) An elegant method to implement slopes and rounded tiles in an orthogonal game.
- [1] (http://games.greggman.com/game/programming_m_c__kids/)(Programming M. C. Kids) Describes how they did platforms in the old days

Editors

- Physics Body Editor (previously known as box2d-editor) (http://code.google.com/p/box2d-editor/) Visual application to easily create collision complex polygons for in-game collision test.

Maps

Software

- Tiled map editor (http://mapeditor.org/) A generic map editor.
- Ogmo Editor (http://www.ogmoeditor.com/) A generic level editor, saving to XML files.
- DAME Editor (http://dambots.com/dame-editor/) Another generic level editor, done in flixel. You can create your own map exporter in Lua.

Procedural generation

- Dynamic Tile Blending Technique (http://www.gamedev.net/page/resources/_/technical/game-programming/tilemap-based-game-techniques-handling-terrai-r934)
- Procedural Content Generation wiki (http://pcg.wikidot.com)
- Think Labyrinth! (http://www.astrolog.org/labyrnth.htm) Software, algorithms and history for making mazes.
- Unangbard Dungeon generation (http://roguelikedeveloper.blogspot.com/2007/11/unangband-dungeon-generation-part-one.html)
- Jamis Buck's rubyconf2011 presentation (http://www.jamisbuck.org/presentations/rubyconf2011/) about maze generation algorithms
- Maze generation algorithm recap (http://weblog.jamisbuck.org/2011/2/7/maze-generation-algorithm-recap) Descriptions, demos and implementations of various maze-generating algorithms.

General

- The Ultimate Indie Game Resource List (http://www.mangatutorials.com/forum/showthread.php?742-The-Ultimate-Indie-Game-Developer-Resource-List)

Freeing Quency

Freeing Quency · revision 25416 · last edited 2021-03-14

Freeing Quency is a game about defeat onslaughts of vile monsters to free Quency from the dungeon's deepest depths.

"Into the dungeon Quency went and behind Quency I followed..."

You can get it here: https://v3x3d.itch.io/freeing-quency ↗

freeing quency

freeing quency · revision 25415 · last edited 2021-03-14

Freeing Quency is a game about defeat onslaughts of vile monsters to free Quency from the dungeon's deepest depths.

"Into the dungeon Quency went and behind Quency I followed..."

You can get it here: https://v3x3d.itch.io/freeing-quency ↗

function

function · revision 4685 · last edited 2023-07-05

Functions in lua (http://w w w.lua.org/manual/5.1/manual.html#2.5.8).

```
function foo()
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Game Distribution/APKTool

Game Distribution/APKTool · revision 29645 · last edited 2025-07-22

This method will assume the usage of APKTool (https://apktool.org/). To distribute your game for Android using APKTool:

> 
> **Note:** APK is the format used to distribute your games independently for Android outside of Google's Play Store. Google Play Store mandate Android Application Bundle since 2021 (https://android-developers.googleblog.com/2020/08/recent-android-app-bundle-improvements.html), thus this method is **not suitable** if you're looking to distribute your app into Play Store

> 
> Some notes before performing this method:
>
> 1. This method assume **you have PC or Mac available**.
> 2. Make sure to specify valid, acceptable package name and **sign the APK** afterwards! Often errors include "Problem parsing package" and "App not installed" error. If you sure it's already signed and has valid package name, try installing via ADB.

1. Download APKTool (https://apktool.org/) and follow the installation instructions.

2. Download latest LÖVE APK from here (https://github.com/love2d/love-android/releases/download/11.5a/love-11.5-android-embed.apk) and put it somewhere in folder.

3. In current folder (where you store the LÖVE APK), open Command Prompt/Terminal in the current directory and execute `apktool d -s -o love_decoded love-11.5-android-embed.apk` and new folder `love_decoded` will be created. You can use any folder name, just make sure to make it consistent in later operations.

4. In `love_decoded` folder, create new folder named `assets` and put your `game.love` inside that folder. Your packaged game must be named exactly `game.love`!

5. Change the APK icon by replacing the LÖVE icon in `res/drawable-{mdpi,{x,{x,{x,}}}hdpi}` (mdpi, hdpi, xhdpi, xxhdpi, and xxxhdpi). Replace `love.png` with your game icon in those folders. Make sure the image dimensions are same!

6. Modify `AndroidManifest.xml` and use this template:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest package="${GamePackageName}"
        android:versionCode="${GameVersionCode}"
        android:versionName="${GameVersionSemantic}"
        android:installLocation="auto"
        xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.VIBRATE" />
    <uses-permission android:name="android.permission.BLUETOOTH" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" android:maxSdkVersion="18" />
    <!--Uncomment line below if your game uses microphone capabilities. !-->
    <!--uses-permission android:name="android.permission.RECORD_AUDIO" /!-->

    <!-- OpenGL ES 2.0 -->
    <uses-feature android:glEsVersion="0x00020000" />
    <!-- Touchscreen support -->
    <uses-feature android:name="android.hardware.touchscreen" android:required="false" />
    <!-- Game controller support -->
    <uses-feature android:name="android.hardware.bluetooth" android:required="false" />
    <uses-feature android:name="android.hardware.gamepad" android:required="false" />
    <uses-feature android:name="android.hardware.usb.host" android:required="false" />
    <!-- External mouse input events -->
    <uses-feature android:name="android.hardware.type.pc" android:required="false" />
    <!-- Low latency audio -->
    <uses-feature android:name="android.hardware.audio.low_latency" android:required="false" />
    <uses-feature android:name="android.hardware.audio.pro" android:required="false" />

    <application
            android:allowBackup="true"
            android:icon="@drawable/love"
            android:label="${GameName}"
            android:usesCleartextTraffic="true" >
        <activity
                android:name="org.love2d.android.GameActivity"
                android:exported="true"
```

```
                android:configChanges="orientation|screenSize|smallestScreenSize|screenLayout|keyboard|keyboardHidden|navigation"
                android:label="${GameName}"
                android:launchMode="singleInstance"
                android:screenOrientation="${ORIENTATION}"
                android:resizeableActivity="false"
                android:theme="@android:style/Theme.NoTitleBar.Fullscreen" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
                <category android:name="tv.ouya.intent.category.GAME" />
            </intent-filter>
            <intent-filter>
                <action android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

```
Here are the explanation of the variables. You just have to substitute them:
* ${GamePackageName} is your game internal package name.
* ${GameVersionCode} is the internal APK version code.
* ${GameVersionSemantic} is the readable APK version string. Example: 2.1.0
* ${GameName} is your game name which is going to be displayed in home screen/launcher.
Make sure to substitute those variables.
```

7. Back to folder where you put the LÖVE APK then execute `apktool b -o your_game.apk love_decoded`. You can use any name, but again, make it consistent.

8. **Sign the resulting APK!!!** This page doesn't cover how to do that, because it can be too long. **You can't install the APK if it's not signed!!!**. Hint: https://github.com/patrickfav/uber-apk-signer ↗

If you find something is wrong or having some problems, don't hesitate to edit or mention me in LOVE Discord: "auahdark".

Termux


Using Termux is not supported. Proceed at your own risk (YMMV)!

You can also create APKs from your Android device directly with the help of Termux (https://termux.dev).

1. Install Termux from F-Droid (https://f-droid.org/en/packages/com.termux).

2. Install termux-apktool (https://github.com/rendiix/termux-apktool).

3. Follow the instructions above as usual.

See Also

- Game Distribution

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

gamera

gamera · revision 10848 · last edited 2023-07-05

Forum thread:

https://love2d.org/forums/viewtopic.php?f=5&t=12033

Latest version:

https://github.com/kikito/gamera ↗

gamera is a library for creating cameras in LÖVE. The cameras can move, rotate, zoom, follow a player, and be restricted to a window or be full screen.

Please refer to the forum post above or the latest version link for examples of use.

General math

General math · revision 30149 · last edited 2025-10-26

This page lists mathematical functions which are useful in game programming. They are generally short, simple, and optimal.

```
-- Averages an arbitrary number of angles (in radians).
function math.averageAngles(...)
    local x,y = 0,0
    for i=1,select('#',...) do local a= select(i,...) x, y = x+math.cos(a), y+math.sin(a) end
    return math.atan2(y, x)
end


-- Returns the distance between two points.
function math.dist(x1,y1, x2,y2) return ((x2-x1)^2+(y2-y1)^2)^0.5 end
-- Distance between two 3D points:
function math.dist(x1,y1,z1, x2,y2,z2) return ((x2-x1)^2+(y2-y1)^2+(z2-z1)^2)^0.5 end


-- Returns the angle between two vectors assuming the same origin.
function math.angle(x1,y1, x2,y2) return math.atan2(y2-y1, x2-x1) end


-- Returns the closest multiple of 'size' (defaulting to 10).
function math.multiple(n, size) size = size or 10 return math.round(n/size)*size end


-- Clamps a number to within a certain range.
function math.clamp(low, n, high) return math.min(math.max(low, n), high) end


-- Linear interpolation between two numbers.
function lerp(a,b,t) return (1-t)*a + t*b end
function lerp2(a,b,t) return a+(b-a)*t end

-- Cosine interpolation between two numbers.
function cerp(a,b,t) local f=(1-math.cos(t*math.pi))*.5 return a*(1-f)+b*f end


-- Normalize two numbers.
function math.normalize(x,y) local l=(x*x+y*y)^.5 if l==0 then return 0,0,0 else return x/l,y/l,l end end


-- Returns 'n' rounded to the nearest 'deci'th (defaulting whole numbers).
function math.round(n, deci) deci = 10^(deci or 0) return math.floor(n*deci+.5)/deci end


-- Randomly returns either -1 or 1.
function math.rsign() return love.math.random(2) == 2 and 1 or -1 end


-- Returns 1 if number is positive, -1 if it's negative, or 0 if it's 0.
function math.sign(n) return n>0 and 1 or n<0 and -1 or 0 end


-- Gives a precise random decimal number given a minimum and maximum
function math.prandom(min, max) return love.math.random() * (max - min) + min end


-- Checks if two line segments intersect. Line segments are given in form of ({x,y},{x,y}, {x,y},{x,y}).
function checkIntersect(l1p1, l1p2, l2p1, l2p2)
    local function checkDir(pt1, pt2, pt3) return math.sign(((pt2.x-pt1.x)*(pt3.y-pt1.y)) - ((pt3.x-pt1.x)*(pt2.y-pt1.y))) end
    return (checkDir(l1p1,l1p2,l2p1) ~= checkDir(l1p1,l1p2,l2p2)) and (checkDir(l2p1,l2p2,l1p1) ~= checkDir(l2p1,l2p2,l1p2))
end

-- Checks if two lines intersect. If seg is true, the line is treated as a line segment. If ray is true, the line is treated as a ray
originating at p1.
-- Lines are given as four numbers (two coordinates)
function findIntersect(l1p1x,l1p1y, l1p2x,l1p2y, l2p1x,l2p1y, l2p2x,l2p2y, seg1,seg2, ray1,ray2)
    ray1,ray2 = ray1 or seg1, ray2 or seg2
```

```
    local rx, ry = l1p2x - l1p1x, l1p2y - l1p1y
    local sx, sy = l2p2x - l2p1x, l2p2y - l2p1y
    local qpx, qpy = l2p1x - l1p1x, l2p1y - l1p1y

    local det = ( rx * sy - ry * sx )
    if det == 0 then return nil end -- The lines are parallel.

    local t = ( ( qpx ) * sy - ( qpy ) * sx ) / det
    local u = ( ( qpx ) * ry - ( qpy ) * rx ) / det
    if ( ray1 and t <= 0 ) or ( seg1 and t >= 1 ) or ( ray2 and u <= 0 ) or ( seg2 and u >= 1 ) then
        return nil -- The lines do not intersect
    end

    return l1p1x + t * rx, l1p1y + t * ry
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Getting Started with PÄSSION

Getting Started with PÄSSION · revision 2884 · last edited 2021-03-14

This page intends to help friendly souls with exact instructions on how to get the LÖVE library PÄSSION started without having to dwell through its source code.

Downloading and Installing PÄSSION

The latest version of PÄSSION can be found at GitHub (http://github.com/kikito/passion). You find the download button in the upper right. For more information on downloading PÄSSION, see the GitHub Wiki Instructions (http://w iki.github.com/kikito/passion/installation)

The files should be unpacked to a subdirectory of your game called "passion". Make sure that you do not get two levels of directories such as "passion/kikito-passion-0ae4fdc/...". Other directory names will not work.

When unextracting the files, you may be asked whether to overwrite "timer.lua" with "Timer.lua" or vice versa. These are two distinct files so answer "no".

Initializing PÄSSION

To get PÄSSION running,

1. Include PÄSSION into your project with

```
require("passion/init.lua")
```

.

PÄSSION can now be accessed through the module "passion".

2. Update your love callbacks to call the corresponding passion callbacks.

```
function love.draw()
  passion.draw()
end

function love.update(dt)
  passion.update(dt)
end

-- other callbacks your project uses
```

3. Done!

Now someone should write a small tutorial on creating actors.

Giga Pudding: The Game

Giga Pudding: The Game · revision 8147 · last edited 2021-03-14

Forum Thread: https://love2d.org/forums/viewtopic.php?f=5&t=8202

GLES Shader Testing in Desktop

GLES Shader Testing in Desktop · revision 29109 · last edited 2024-02-16

Writing shaders to support mobile devices might be not trivial. You may be wondering if there is a way to test shaders on mobile devices without really using mobile device.

The answer is yes, there's a way.

GPUs nowadays (even integrated ones) should support creating OpenGL ES context, which means all shaders must be GLSL ES/ESSL compatible. We can make LOVE use OpenGL ES context by using the environment variable `LOVE_GRAPHICS_USE_OPENGLES=1`.

Note that this guide is mostly Windows oriented. For other desktop platforms, please adapt accordingly!

love.graphics.validateShader

Since 11.0, there's love.graphics.validateShader which can be used to validate both GLSL and ESSL shader code. Note that it only validates, it doesn't run your shader code. If you really want to run your game under GLES context, proceed below!

Requirements

You need a GPU which supports OpenGL ES context or ANGLE. As previously written, today's GPUs should support this, thus ANGLE is normally unnecessary.

Tell LOVE to use OpenGL ES context

FIXME: AMD drivers expose OpenGL ES context under EGL (and it's very hidden) which is not compatible with LOVE SDL build. For AMD GPUs in Windows, proceed below on using ANGLE directly.

Now, we need to tell LOVE to use OpenGL ES context. You need to set this environment variable before starting LOVE. It can be set using Command Prompt/Terminal and launching LOVE from same Terminal, or from Windows, from Advanced System Settings.

- Windows: `set LOVE_GRAPHICS_USE_OPENGLES=1` then `lovec ...`. 0.10.2 or later is assumed.
- Linux/macOS: `LOVE_GRAPHICS_USE_OPENGLES=1 love ...`

Verify if LOVE uses OpenGL ES context by checking the 1st return value of love.graphics.getRendererInfo which should be "OpenGL ES".

Once you've got OpenGL ES context, you can test if your shader code works on mobile devices. Otherwise, if you're on Windows, proceed below.

Using ANGLE in Windows


Using ANGLE with AMD GPU in Windows requires LÖVE 11.5 or later.

ANGLE (https://chromium.googlesource.com/angle/angle) is an implementation of OpenGL ES 2.0 and 3.0 on top of other rendering backends. In Windows, it will run OpenGL ES using Direct3D 11 or Direct3D 9. If your GPU doesn't support OpenGL ES context or if you want to force ANGLE, you can copy **libEGL.dll** and **libGLESv2.dll** from here (https://github.com/MikuAuahDark/angle-w inbuild), or on one of these applications:

- Google Chrome
- Microsoft Edge Chromium
- Visual Studio Code
- osu! stable (32-bit DLL)

Note: Using Firefox's DLLs doesn't work because it links with Firefox-specific DLLs and copying all the dependent DLLs can be troublesome.

One thing for sure is **make sure the bit count of the DLL matches**, i.e. 64-bit Chrome has 64-bit DLLs and can only be used on 64-bit LOVE.

Next, set the environment `LOVE_GRAPHICS_USE_OPENGLES=1` and `SDL_OPENGL_ES_DRIVER=1` then launch LOVE. To confirm you're using ANGLE backend, check the 2nd return value of love.graphics.getRendererInfo which should contain "ANGLE" (all uppercase).

GNU General Public License

GNU General Public License · revision 11669 · last edited 2021-03-14

The GPL is a "viral" "Copyleft" licence, there are three versions:

- GPLv1 (Deprecated) (http://opensource.org/licenses/GPL-1.0)
- GPLv2 (http://opensource.org/licenses/GPL-2.0)
- GPLv2 (http://opensource.org/licenses/GPL-3.0)

Graphical User Interface

Graphical User Interface · revision 31655 · last edited 2026-07-21

GUI Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
LOVELi	Retained mode layout and GUI library with StackLayout, AbsoluteLayout, Grid, Label, Button, Image, CheckBox, RadioButton, Switch, ProgressBar, Slider, TextBox and Border		11.5	
likeliHUD	A simple UI library with a QML-like declarative approach.		11.5	11.5
layouter	Simple UI grid layout library.		11.4+	
cimgui-love	LÖVE module for Dear ImGui, obtained by wrapping cimgui with LuaJIT FFI.	No (LuaJIT FFI wrapper to cimgui shared library)	11.4	11.3
Thranduil	UI module for LÖVE **sources, website : 404**		0.9.x	0.9.x
Quickie	Simple(r), but powerful GUI. **(renamed to SUIT)**		0.8.x	0.7.x
Goo	GUI library for Love		0.7.x	
LoveUI	GUI library for Love		0.7.x	0.5.0
Message in a Bottle	A message library for LOVE.		0.7.0	
Gspöt	GUI library for Love		0.11.x	
LUIGI	Lovely User Interfaces for Game Inventors	Yes	0.11.x	0.9.x
Löve Frames	An advanced GUI library for LÖVE		0.11.x	11.0
Talkies.lua	A messagebox system with multiple-choices, typing effect + sounds and more.		0.11.x	0.11.x
love-nuklear	Lightweight immediate mode GUI for LÖVE games	No (C wrapper to Nuklear)	0.11.2	0.11.2
Easy GUI System	EGS is a simple GUI system with events and multiple controls, without over-complicating things.		0.10.x	
Möan.lua	A messagebox system with multiple-choices and more **(renamed to Talkies)**		0.10.x	
SUIT	Simple User Interface Toolkit	Yes	0.10.x	0.10.x
fLUIds	love-imgui API wrap for simplicity	No (uses love-imgui (a C wrapper to IMGUI))	0.10.x	
love-imgui	IMGUI module for LOVE	No (C wrapper to IMGUI)	0.10.x	0.10.x
ListBox	A dynamic ListBox for LÖVE 2D. Supports touches, mouses and keyboards		0.10.2	0.10.x
yaoui	UI Kit for LÖVE **(no longer maintained)**	Yes	0.10.0	

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Gspöt

Gspöt · revision 25333 · last edited 2023-07-05

Gspöt is a stately GUI library, originally made by trubblegum for LOVE 0.8.0. It has been ported to LOVE 0.9.0 and greater by pgimeno.

Links

- Original Thread (https://love2d.org/forums/viewtopic.php?f=5&t=8351)
- Source code for LOVE 0.8.0 (https://github.com/trubblegum/Gspot)
- Source code for LOVE 0.9.0 and greater (https://notabug.org/pgimeno/Gspot) (including 11.x)
- Documentation (https://notabug.org/pgimeno/Gspot/w iki)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GunFu Deadlands

GunFu Deadlands · revision 12350 · last edited 2021-03-14

GunFu Deadlands 

A Far West themed 2D shooter featuring bullet time.

Download: https://sourceforge.net/projects/gunfudeadlands/ ↗

Highscöre

Highscöre · revision 9636 · last edited 2021-03-14

Highscöre is a library which creates simple highscore tables.

Download it here (https://dl.dropbox.com/u/37768120/libraries/highscore.lua).

See also SICK (read: SICK is better).

Updates

v1.1 - 8/2/10 - Cleaned up some code a bit (replaced some while loops with for statements). **Compatible with v1.0.**

How to use

First use love.filesystem.setIdentity or set the `identity` field in conf.lua. Also be sure to use the following line of code before using the functions:

```
require "path/to/highscore.lua"
```

Then you can start using the functions seen below.

Functions

```
highscore_new(filename, places, name, score)
```

`filename` - Name of text file where highscore table is stored.

`places` - Number of places in highscore table.

`name` - Name for "Nobody."

`score` - Score for "Nobody."

Creates an empty highscore table. Use this if the highscore table doesn't exist yet.

```
highscore_load(filename)
```

`filename` - Name of text file to load highscore table from.

Loads a highscore table. Be sure to use this before using highscore_add or highscore_write!

```
highscore_write(filename)
```

`filename` - Name of text file to write highscore table to.

Writes the currently stored high scores. You'll probably want to use this after using highscore_add.

```
highscore_add(score, name)
```

`score` - Score to enter into highscore table.

`name` - Name that goes with score.

Enter a score into the highscore tale.

HSL color

HSL color · revision 27510 · last edited 2023-07-05

The HSL color space (https://secure.w ikimedia.org/w ikipedia/en/w iki/HSL_and_HSV) is based on how humans perceive color, and as such, makes various aesthetically-pleasing color transformations very simple. Instead of saying how much light a monitor will emit (not a very natural way to think about colors), colors are described in terms of Hue, Saturation, and Lightness.

- **Hue** describes where in the spectrum the color is. As Hue increases, a color will transition in the following order: Red, orange, yellow, green, cyan, blue, violet, purple, magenta, red.
- **Saturation** denotes how vibrant a color is. Ranges from grey to pure color.
- **Lightness** is how light or dark a color is. Ranges from black to white. In the middle is either grey (if low saturation) or a color (if high saturation).

Here is a function Taehl wrote to convert HSL colors to RGB:

```
-- Converts HSL to RGB. (input and output range: 0 - 1)
function HSL(h, s, l, a)
    if s<=0 then return l,l,l,a end
    h, s, l = h*6, s, l
    local c = (1-math.abs(2*l-1))*s
    local x = (1-math.abs(h%2-1))*c
    local m,r,g,b = (l-.5*c), 0,0,0
    if h < 1     then r,g,b = c,x,0
    elseif h < 2 then r,g,b = x,c,0
    elseif h < 3 then r,g,b = 0,c,x
    elseif h < 4 then r,g,b = 0,x,c
    elseif h < 5 then r,g,b = x,0,c
    else              r,g,b = c,0,x
    end return r+m, g+m, b+m, a
end
```

This could be used, for example, like this: `love.graphics.setColor(HSL(0.59,0.39,0.78))` (this would color things a greyish blue)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

HSV color

HSV color · revision 27509 · last edited 2023-07-05

The HSV color space (https://secure.w ikimedia.org/w ikipedia/en/w iki/HSL_and_HSV) is based on how humans perceive color, and as such, makes various aesthetically-pleasing color transformations very simple.

- **Hue** describes where in the spectrum the color is. As Hue increases, a color will transition in the following order: Red, orange, yellow, green, teal, blue, violet, purple, magenta, red.
- **Saturation** denotes how vibrant a color is. Ranges from grey to pure color.
- **Value** is how saturated an image is, the amount of each color.

This function converts HSV values to RGB:

```
-- Converts HSV to RGB. (input and output range: 0 - 1)

function HSV(h, s, v)
    if s <= 0 then return v,v,v end
    h = h*6
    local c = v*s
    local x = (1-math.abs((h%2)-1))*c
    local m,r,g,b = (v-c), 0, 0, 0
    if h < 1 then
        r, g, b = c, x, 0
    elseif h < 2 then
        r, g, b = x, c, 0
    elseif h < 3 then
        r, g, b = 0, c, x
    elseif h < 4 then
        r, g, b = 0, x, c
    elseif h < 5 then
        r, g, b = x, 0, c
    else
        r, g, b = c, 0, x
    end
    return r+m, g+m, b+m
end
```

and the same function implemented in GLSL, for those that need it:

```
    vec3 HSV(float h, float s, float v)
    {
        if (s <= 0 ) { return vec3 (v); }
        h = h * 6;
        float c = v*s;
        float x = (1-abs((mod(h,2)-1)))*c;
        float m = v-c;
        float r = 0.0;
        float g = 0.0;
        float b = 0.0;

        if (h < 1) { r = c; g = x;b = 0.0;}
        else if (h < 2) { r = x; g = c; b = 0.0; }
        else if (h < 3) { r = 0.0; g = c; b = x; }
        else if (h < 4) { r = 0.0; g = x; b = c; }
        else if (h < 5) { r = x; g = 0.0; b = c; }
        else  { r = c; g = 0.0; b = x; }

        return vec3(r+m,g+m,b+m);
    }
```

Or for those who want to save up on space (and achieve better performance, as branching is quite costly on GPUs), here's one-liner:

```
vec3 hsv(float h,float s,float v) { return mix(vec3(1.),clamp((abs(fract(h+vec3(3.,2.,1.)/3.)*6.-3.)-1.),0.,1.),s)*v; }
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

hxdx

hxdx · revision 19226 · last edited 2023-07-05

hxdx is a physics module for LÖVE. It wraps LÖVE's physics API so that using box2d becomes as simple as possible.

Download

github (https://github.com/adonaac/hxdx)

Image Formats

Image Formats · revision 25023 · last edited 2026-04-09

In addition to compressed image formats, the following formats are supported:

Format Name	Common Extension	Can Decode	Can Encode	Note(s)
Joint Photographic Experts Group / JPEG	.jpg, .jpeg	Yes		
Portable Network Graphics / PNG	.png	Yes	Yes	
Bitmap image file / BMP	.bmp	Yes		
Truevision TGA / TARGA	.tga	Yes	Yes	
Radiance HDR / RGBE	.hdr, .pic	Yes		
OpenEXR	.exr	Yes		

See Also

- love.image
- love.graphics
- ImageEncodeFormat

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

In Your Face City Trains

In Your Face City Trains · revision 6638 · last edited 2021-03-14

Description

IYFCT is a simple but fun one-button-game similar to Canabalt.

The goal of the game is to survive as long as you can. Jump over trains with closed doors and try (as much as possible) to run through trains with open doors to avoid birds and tunnels. Some open trains will give you coffee. When your coffee-meter is full, you have one extra life.

Controls

- Space: Jump
- R: Restart
- 1-4: Set zoom
- P: Pause game
- M: Mute sound

Media

Gameplay video (http://www.youtube.com/watch?v=KoBsTFz5xqU)



Downloads

- Windows binary (http://github.com/downloads/SimonLarsen/iyfct/iyfct-win32.zip)
- LÖVE package (http://github.com/downloads/SimonLarsen/iyfct/iyfct.love)
- Source code (http://github.com/SimonLarsen/iyfct) [Github] (GPLv3)

Licence/Credits

The source code for this game is released under GPLv3 licence. See: [1] (http://www.gnu.org/licenses/gpl.txt)

All assets including the background music is released under a CC BY 3.0 licence. See: [2] (http://creativecommons.org/licenses/by/3.0/)

The background music is 'The rb Sequence' by Fighter X from 'Little Fighter X', and can be downloaded for free here: [3] (http://www.crunchyco.com/wp-content/plugins/download-monitor/download.php?id=9)

In-Memory Audio Streaming

In-Memory Audio Streaming · revision 21394 · last edited 2023-07-05

This code snippet basically shows the possibly of loading audio FileData, then having a Decoder decode the audio from memory. This is different than loading the whole SoundData into memory, which would decode the whole file, which can be memory and time consuming.

Works in Super Toast and Mysterious Mysteries versions of LÖVE. It probably also work in Baby Inspector (needs confirmation).

Simple Snippet

A very simple snippet which shows how it works.

```
-- Load audio file data into memory
local audioFile = love.filesystem.newFileData("file/to/audio.ogg")
-- Create new "stream" source from FileData
local source = love.audio.newSource(audioFile, "stream")
-- Ensure it has "stream" as its type. This line is just for checking purposes
assert(source:getType() == "stream")
-- Play sound
source:play()
```

Full Snippet

This reflects what is actually happening inside the LÖVE framework.

```
-- Open audio file
local audioHandle = love.filesystem.newFile("file/to/audio.ogg", "r")
-- Read the audio file contents. It's still in encoded form.
local audioContents = audioHandle:read()
-- Create a new FileData based on the audio contents. Make sure to specify
-- a correct file extension in the second parameter! "Decoder"s determine
-- audio formats by their extension, and not by their contents!
local audioData = love.filesystem.newFileData(audioContents, "_.ogg"))
-- Create a new Decoder to decode the audio
local decoder = love.sound.newDecoder(audioData)
-- Create a new "stream" type source from the created Decoder
local source = love.audio.newSource(decoder, "stream")
--  Ensure it has "stream" as its type. This line is just for checking purposes
assert(source:getType() == "stream")
-- Play sound
source:play()
```

Journey To The Stars

Journey To The Stars · revision 13894 · last edited 2021-03-14

Journey To The Stars is a *simple* endless runner set in the debris-filled orbit of a future planet earth. Your goal is to avoid collision and to survive as long as you can.

On every flight you will earn experience points, which you can spend to further upgrade your ship.

Released on itch.io (http://rmcode.itch.io/journey-to-the-stars)



Layering Tiles

Layering Tiles · revision 19000 · last edited 2021-03-14

Hello there! Today I will be demonstrating how to **layer** your images using a tile map system I saw here.

This is what my **main.lua** looks like:

```
require "map"

function love.load() --Run only at start of game
    love.graphics.setBackgroundColor( 255, 255, 255 )
end
```

You have to require 'map' (the name of the file that does all the tile mapping) so that you can call all of the functions in 'main.lua'

```
function love.draw() --Run every time game can
    draw_map()
end

function love.update(dt) --Run every time the game can
end

function love.focus(bool) --Run if mouse goes in or out of window
end

function love.keypressed( key, unicode ) --Run if key is pressed
    move_map(key)
end

function love.keyreleased( key, unicode ) --Run if key is released
end

function love.mousepressed( x, y, button ) --Run if mouse is pressed
end

function love.mousereleased( x, y, button ) --Run if mouse is released
end

function love.quit() --Run right before game is closed
end
```

The rest of the code is self explanatory: you draw the tiles, and then check to see if a key is pressed to move the map.

Here is the 'map.lua' that I based off of the aforementioned code, broken down step by step.

```
layers = {}

tile = {}

for i = 0, 4 do -- change 3 to the number of tile images minus 1.
    tile[i] = love.graphics.newImage( "tiles/tile"..i..".png" )
end
```

What this does is makes two tables called 'layers' and 'tile'. It then uses a **for loop** to register each picture instead of doing it separately. Note that I have my pictures stored in a folder called 'tiles'.

```
layers[1] = {
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
```

```
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
}

layers[1].map_w = 20 --defines the width of the map. IMPORTANT: this (as well as the map_h) must match the vertical and horizontal
dimensions you have made
layers[1].map_h = 20 --defines the height of the map.
layers[1].map_x = 0 --where you start drawing the map horizontally.
layers[1].map_y = 0 --where you start drawing the map vertically.
layers[1].map_offset_x = -50 --size of tile width for best results
layers[1].map_offset_y = -50 --size of tile height for best results
layers[1].map_display_w = 800/50 --for best results divide the screen width by the width of the tile
layers[1].map_display_h = 700/50 --divide the screen height by the height of the tile
layers[1].tile_w = 50 --should correlate with your picture
layers[1].tile_h = 50 --should correlate with your picture
```

What this does is make a map using all '0' (a clear image for my purposes). If you were to run this right now (which you can't, since you haven't made the draw_map yet) it would look like white background. [1] (http://www.love2d.org/w/images/c/c4/pic_1.png)

```
layers[2] = {
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
    { 1, 0, 2, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 2, 0, 1 },
    { 1, 3, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 3, 1 },
    { 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 3, 1 },
    { 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
    { 1, 0, 2, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 2, 0, 1 },
    { 1, 3, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 3, 1 },
    { 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 3, 1 },
    { 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
    { 1, 0, 2, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 2, 0, 1 },
    { 1, 3, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 3, 1 },
    { 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 3, 1 },
    { 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
    { 1, 0, 2, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 2, 0, 1 },
    { 1, 3, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 3, 1 },
    { 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 3, 1 },
    { 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
    { 1, 0, 2, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 2, 0, 1 },
    { 1, 3, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 3, 1 },
    { 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 3, 1 },
    { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
}

layers[2].map_w = 20 --defines the width of the map. IMPORTANT: this (as well as the map_h) must match the vertical and horizontal
dimensions you have made
layers[2].map_h = 20 --defines the height of the map.
layers[2].map_x = 0 --where you start drawing the map horizontally.
layers[2].map_y = 0 --where you start drawing the map vertically.
layers[2].map_offset_x = -50 --negative size of tile width for best results
layers[2].map_offset_y = -50 --negative size of tile height for best results
layers[2].map_display_w = 800/50 --for best results divide the screen width by the width of the tile
layers[2].map_display_h = 700/50 --divide the screen height by the height of the tile
layers[2].tile_w = 50 --should correlate with your picture
layers[2].tile_h = 50 --should correlate with your picture
```

Same thing, but 1 is green, 2 is blue, 3 is red. It would look a bit like this: [2] (http://www.love2d.org/w/images/d/d8/pic_2.png)

```
ayers[3] = {
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 0 },
```

```
    { 0, 0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
}

layers[3].map_w = 20 --defines the width of the map. IMPORTANT: this (as well as the map_h) must match the vertical and horizontal
dimensions you have made
layers[3].map_h = 20 --defines the height of the map.
layers[3].map_x = 0 --where you start drawing the map horizontally.
layers[3].map_y = 0 --where you start drawing the map vertically.
layers[3].map_offset_x = -50 --negative size of tile width for best results
layers[3].map_offset_y = -50 --negative size of tile height for best results
layers[3].map_display_w = 800/50 --for best results divide the screen width by the width of the tile
layers[3].map_display_h = 700/50 --divide the screen height by the height of the tile
layers[3].tile_w = 50 --should correlate with your picture
layers[3].tile_h = 50 --should correlate with your picture
```

And finally the last one. 4 is orange. It should look like this: [3] (http://www.love2d.org/w/images/6/6f/pic_3.png)

```
function draw_map()
    for l = 1, #layers do
        for y = 1, layers[l].map_display_h do
            for x = 1, layers[l].map_display_w do
                love.graphics.draw(
                tile[layers[l][y+layers[l].map_y][x+layers[l].map_x]],
                (x*layers[l].tile_w)+layers[l].map_offset_x,
                (y*layers[l].tile_h)+layers[l].map_offset_y)
            end
        end
    end
end
```

This draws the map. See the article to understand how it works (just edited).

```
function move_map(key)
    for l = 1, #layers do
        if key == 'up' then
            layers[l].map_y = layers[l].map_y-1
            if layers[l].map_y < 0 then
                layers[l].map_y = 0;
            end
        end
        if key == 'down' then
            layers[l].map_y = layers[l].map_y+1
            if layers[l].map_y > layers[l].map_h - layers[l].map_display_h then
                layers[l].map_y = layers[l].map_h - layers[l].map_display_h;
            end
        end
        if key == 'left' then
            layers[l].map_x = math.max(layers[l].map_x-1, 0)
        end
        if key == 'right' then
            layers[l].map_x = math.min(layers[l].map_x+1, layers[l].map_w - layers[l].map_display_w)
        end
    end
end
```

Again, see previous article.

layouter

layouter · revision 29124 · last edited 2024-03-13

Layouter is a simple UI grid layout library for LÖVE 2D game engine.

It currently supports these element types:

- text (including spacer = blank text)
- text button
- image

Download and more documentation:

https://github.com/nekromoff/layouter ↗

libSaveTableToFile

libSaveTableToFile · revision 24493 · last edited 2023-07-05

On the Lua Users Wiki (http://lua-users.org/wiki) are a few solutions for serializing tables. This entry is about the one called Save table to file (http://lua-users.org/wiki/SaveTableToFile).

It provides two functions:

- **table.save(tbl, filename)** saves a table to a text file.
- **table.load(filename)** generates a table from the output file from `table.save()`.

They allow you to easily save and load complex data. It uses Lua syntax for the data files, so it is easy to edit by hand.

It is licensed under the same license as Lua and available here (http://lua-users.org/files/wiki_insecure/users/chill/table.save-0.94.lua).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

libxml

libxml · revision 24500 · last edited 2023-07-05

On this page

- Summary
- Articles / Tutorials
- Download
- Bugs / Feature Requests
- Documentation
  - Namespaces
- Other Languages

Summary

libxml is a minimal XML Dom library which enables lovers to load, save, and traverse xml documents within their love2d applications

Articles / Tutorials

- Forum Post (https://love2d.org/forums/viewtopic.php?f=5&t=10809)

Download

Download libxml at github (https://github.com/admin36/libxml)

Bugs / Feature Requests

For bugs or feature requests, please use the "Issues" section over at github (https://github.com/admin36/libxml/issues)

Documentation

Namespaces

Namespace	Description	Implemented
libxml	libxml Root Namespace	Yes
libxml.dom	libxml Document Object Model namespace	Yes

libxml DOM Object Reference

Object	Description	Implemented	W3C
DOM Node	The Node object represents a single node in the document tree	Yes	Yes
DOM NodeList	The NodeList object represents an ordered list of nodes	Yes	Yes
DOM NamedNodeMap	The NamedNodeMap object represents an unordered list of nodes.	Yes	Yes
DOM Document	The Document object represents the entire XML/HTML document.	Yes	Yes
DOM Element	The Element object represents an element in an XML/HTML document	Yes	Yes
DOM Attribute	The Attr object represents an attribute of an Element object.	Yes	Yes
DOM Text	The Text object represents the textual content of an element or attribute.	Yes	Yes
DOM CDATA	The CDATASection object represents a CDATA section in a document.	Yes	Yes
DOM Comment	The Comment object represents the content of comment nodes in a document.	Partial	Yes
DOM DOMParser	DOMParser object parses xml/html text and returns a DOM Object	Yes	No

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

light userdata

light userdata · revision 12758 · last edited 2023-07-05

From the Lua Reference Manual:

(Full) userdata represent C values in Lua.

A light userdata represents a pointer. It is a value (like a number): you do not create it, it has no individual metatable, and it is not garbage collected (as it was never created).

A light userdata is equal to "any" light userdata with the same C address.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

likeliHUD

likeliHUD · revision 31693 · last edited 2026-08-02

A simple (yet another) UI library with a QML-like declarative approach.

(will add more details later)

Disclaimer on the love2d version: I put 11.5 as the minimum required love2d version just because I haven't tried (nor investigated whether it will work for other versions) this lib on an earlier love2d.

Links

- Github page (https://github.com/LRDPRDX/LikeliHUD)
- Documentation (https://lrdprdx.github.io/LikeliHUD)
- Thread (https://love2d.org/forums/viewtopic.php?t=96850)
- Queuest (https://love2d.org/forums/viewtopic.php?t=96856)

Widgets

- TextField (quite raw)
- ImageButton
- Rectangle
- Layout
- Stack
- Image
- Label
- You can create your own

Features

- Event driven communication.
- Declarative approach. Similar to QML.
- Grid layout. Nested layouts are possible.
- Auto alignment. Alignment hints : center, top, bottom, left or a combination.
- Each element inside a layout can be set to fill x (width), y (height), both or none.

Dependencies

- subclass (https://love2d.org/wiki/subclass)
- lua-utf8 (https://github.com/starw ing/luautf8)

Screenshots

I am in the middle of development my medieval-flavoured TODO list with the usage of this lib:



Live Simulator: 2

Live Simulator: 2 · revision 22291 · last edited 2021-03-14

Clone of LL!SIF (https://play.google.com/store/apps/details?id=klb.android.lovelive_en) game, can be used for training purpose.



Information

Supports variety of different beatmap formats. Common used one is Custom Beatmap Festival (https://www.reddit.com/r/CustomBeatmapFestival/) and SIFTrain (https://www.reddit.com/r/SIFTrain/). Intended as replacement of SIFTrain.

Runs under LÖVE 0.10.0 or later (LÖVE 11.0 compatible)

Notes

Main menu

Since v3.0.0, Live Simulator: 2 uses this fork (https://github.com/MikuAuahDark/livesim3-love) of LÖVE for Windows and Android. Additionally, in Android, this fork (https://github.com/MikuAuahDark/livesim3-love-android) of love-android-sdl2 is used.

Links

- GitHub Repository (https://github.com/MikuAuahDark/livesim2)

local

local · revision 29299 · last edited 2024-08-17

On this page

- What?
- When should I use globals?
- TL;DR
- Example
- See Also
- Other Languages

In Lua, variables can be global or local. Unlike global variables, local variables have their scope limited to the block where they are declared.

What?

In many common programming languages, they infer the scope of your variable from where you put it in your code (ie. declaring a variable in a function means that you can only see said variable in that function), or require explicit modifiers put on declarations that specify public or private access (like in Java). By default in Lua, your variables are in the global scope, unless you define them with the `local` statement

A variable or function declared with the `local` statement restricts their scope to the block they're declared in. If you declare a `local` at the file level, then that value is only visible to that file, same for when you declare values within a function.

Using `local` is beneficial because:

- **globals are unknown**: globals are mutable, which means you don't know what has modified it, or when. If you're aware of it, then you'd have to perform many checks whenever you wish to use it or modify it, which just adds work, not reducing it
- **globals promote bad design**: Since they're so easy to access, globals generally get loaded with a bunch of stuff that is unrelated. globals generally indicate "spaghetti" code - code that seems to do everything, but none of it well, and whenever you go to edit it, it's a nightmare.
- **locals are fast**: Without going into too many details, *locals are faster in Lua*. Every global is stored in the `_G` table, and whenever you go to access one, Lua has to go hunting into this table to find it. Locals, however, are stored on 'registers', which makes them super fast to access. Adding two `local` numbers takes a single stack instruction, but adding two globals might take up to four! (https://www.lua.org/gems/sample.pdf) A program chock-full of globals could be markedly slower than one with locals.

When should I use globals?

All of this doesn't mean you shouldn't use Globals. In fact, if you're learning how to program, it's probably more than you have to worry about, however, if you're starting out in Lua, and you're already familiar with the idea of scopes from, say, C# or Java, then you're already halfway there.

Globals are useful in a handful of situations, however by introducing one, **you should be strict on what you do with it**. For example, only read from a global, don't modify any of its internal state. Don't overload your globals with functions and variables because it's "easy" - take the time to tie your program together properly (future you would love you for it).

TL;DR

In a nutshell, elect to use `local` over global declarations as it's:

- much safer (no other code is touching and changing the variable without your knowledge).
- promotes better program structure (by not cluttering up the global namespace).
- and above all else a very easy optimization (http://lua-users.org/wiki/OptimisingUsingLocalVariables) to make early on.

That doesn't mean you should forbid all use of globals, just when you use them, be conscious of what you're using them for.

Example

```lua
-- ============================================================
-- File: "locals.lua"
-- ============================================================

local LocalTest = {}

Global = "This is a global variable"
function GlobalFunction()
  print("This is a global function")
```

```lua
end

local localVariable = "This is a file-scoped local variable"

local function localFunction()
  local localFunctionVariable = "This is a function-scoped local variable"
  local localTable = { "This is a value in a table" }
  print("This is a local function")
  print(Global)                -- prints "This is a global variable
  print(localVariable)         -- prints "This is a file-scoped local variable"
  print(localFunctionVariable) -- prints "This is a function-scoped local variable"
  print(localTable[1])         -- prints "This is a value in a table"
end

LocalTest.exposedVariable = "This is an exposed, non-global variable"

function LocalTest.exposedFunction()
  print("This is an exposed function from Local Test")
  print(Global)                -- prints "This is a global variable
  print(localVariable)         -- prints "This is a file-scoped local variable"
  print(localFunctionVariable) -- prints "nil"
  print(localTable[1])         -- throws an error
end

return LocalTest

-- ================================================================
-- File: "main.lua"
-- ================================================================

local locals = require "locals"

print(Global)           -- prints "This is a global variable"
print(GlobalFunction()) -- prints "This is a global function"

-- Note how we have to say "look in the imported `locals` file for `exposed` variables
print(locals.exposedVariable) -- prints "This is an exposed, non-global variable"
print(exposedVariable)        -- prints "nil"

-- Both of these blow up, which means the second will never get called.
print(locals.exposedFunction())    -- throws and error with "attempt to index global 'localTable' (a nil value)"
print(locals.localFunction())      -- throws an error with "attempt to call field 'localFunction' (a nil value)"
```

See Also

Programming in Lua (http://w w w.lua.org/pil/4.2.html)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love dependencies versions in ubuntu repositories

love dependencies versions in ubuntu repositories · revision 7345 · last edited 2021-03-14

This data shows what versions of the love dependencies are on specific platforms and ubuntu distributions. ~josefnpat

---

PLATFORM amd64

	hardy	karmic	lucid	maverick	natty	oneiric	precise
mercurial	0.9.5-3	1.1.2-2ubuntu1	1.3.1-1	1.4.3-1	1.6.3-1	1.7.5-1ubuntu1	1.9.1-1
build-essential	11.3ubuntu1	11.4	11.4	11.4build1	11.5	11.5ubuntu1	11.5ubuntu1
automake	1:1.10.1-2	1:1.10.2-0ubuntu3	1:1.11-1	1:1.11.1-1	1:1.11.1-1	1:1.11.1-1ubuntu1	1:1.11.1-1ubuntu1
libmpg123-dev		1.4.3-4ubuntu1	1.7.3-0ubuntu1	1.12.1-0ubuntu1	1.12.1-3ubuntu1	1.12.1-3ubuntu1	1.12.1-3.1fakesync1
libmodplug-dev	1:0.7-7	1:0.8.4-3ubuntu1	1:0.8.7-1	1:0.8.7-1build1	1:0.8.8.1-1ubuntu1	1:0.8.8.1-2	1:0.8.8.2-3ubuntu1
libphysfs-dev	1.0.0-5	1.0.0-5	2.0.0-3	2.0.0-4	2.0.1-1	2.0.1-2	2.0.2-4
libfreetype6-dev	2.3.5-1ubuntu4	2.3.9-4build1	2.3.9-5	2.3.11-1ubuntu2	2.4.2-2	2.4.4-1ubuntu2	2.4.4-2ubuntu1
libdevil-dev	1.6.7-5.1	1.7.5-4	1.7.8-2	1.7.8-6	1.7.8-6	1.7.8-6build1	1.7.8-6build2
liblua5.1-0-dev	5.1.2-4	5.1.4-2	5.1.4-3	5.1.4-5	5.1.4-5	5.1.4-5	5.1.4-10
libopenal-dev	1:0.0.8-7	1:1.4.272-2	1:1.11.753-1	1:1.11.753-1	1:1.12.854-2	1:1.12.854-2	1:1.13-2
libsdl1.2-dev	1.2.13-1ubuntu1	1.2.13-4ubuntu3	1.2.13-4ubuntu4	1.2.14-4ubuntu1	1.2.14-6ubuntu3	1.2.14-6.1ubuntu3	1.2.14-6.1ubuntu4
libvorbis-dev	1.2.0.dfsg-2	1.2.0.dfsg-3.1	1.2.0.dfsg-6	1.2.3-3ubuntu1	1.3.1-1	1.3.2-1ubuntu1	1.3.2-1ubuntu2

PLATFORM armel

	hardy	karmic	lucid	maverick	natty	oneiric	precise
mercurial		1.1.2-2ubuntu1	1.3.1-1	1.4.3-1	1.6.3-1	1.7.5-1ubuntu1	1.9.1-1
build-essential		11.4	11.4	11.4build1	11.5	11.5ubuntu1	11.5ubuntu1
automake		1:1.10.2-0ubuntu3	1:1.11-1	1:1.11.1-1	1:1.11.1-1	1:1.11.1-1ubuntu1	1:1.11.1-1ubuntu1
libmpg123-dev		1.4.3-4ubuntu1	1.7.3-0ubuntu1	1.12.1-0ubuntu1	1.12.1-3ubuntu1	1.12.1-3ubuntu1	1.12.1-3.1fakesync1
libmodplug-dev		1:0.8.4-3ubuntu1	1:0.8.7-1	1:0.8.7-1build1	1:0.8.8.1-1ubuntu1	1:0.8.8.1-2	1:0.8.8.2-3ubuntu1
libphysfs-dev		1.0.0-5	2.0.0-3	2.0.0-4	2.0.1-1	2.0.1-2	2.0.2-4
libfreetype6-dev		2.3.9-4build1	2.3.9-5	2.3.11-1ubuntu2	2.4.2-2	2.4.4-1ubuntu2	2.4.4-2ubuntu1
libdevil-dev		1.7.5-4	1.7.8-2	1.7.8-6	1.7.8-6	1.7.8-6build1	1.7.8-6build2
liblua5.1-0-dev		5.1.4-2	5.1.4-3	5.1.4-5	5.1.4-5	5.1.4-5	5.1.4-10
libopenal-dev		1:1.4.272-2	1:1.11.753-1	1:1.11.753-1	1:1.12.854-2	1:1.12.854-2	1:1.13-2
libsdl1.2-dev		1.2.13-4ubuntu3	1.2.13-4ubuntu4	1.2.14-4ubuntu1	1.2.14-6ubuntu3	1.2.14-6.1ubuntu3	1.2.14-6.1ubuntu4
libvorbis-dev		1.2.0.dfsg-3.1	1.2.0.dfsg-6	1.2.3-3ubuntu1	1.3.1-1	1.3.2-1ubuntu1	1.3.2-1ubuntu2

PLATFORM i386

	hardy	karmic	lucid	maverick	natty	oneiric	precise
mercurial	0.9.5-3	1.1.2-2ubuntu1	1.3.1-1	1.4.3-1	1.6.3-1	1.7.5-1ubuntu1	1.9.1-1
build-essential	11.3ubuntu1	11.4	11.4	11.4build1	11.5	11.5ubuntu1	11.5ubuntu1
automake	1:1.10.1-2	1:1.10.2-0ubuntu3	1:1.11-1	1:1.11.1-1	1:1.11.1-1	1:1.11.1-1ubuntu1	1:1.11.1-1ubuntu1
libmpg123-dev		1.4.3-4ubuntu1	1.7.3-0ubuntu1	1.12.1-0ubuntu1	1.12.1-3ubuntu1	1.12.1-3ubuntu1	1.12.1-3.1fakesync1
libmodplug-dev	1:0.7-7	1:0.8.4-3ubuntu1	1:0.8.7-1	1:0.8.7-1build1	1:0.8.8.1-1ubuntu1	1:0.8.8.1-2	1:0.8.8.2-3ubuntu1

libphysfs-dev	1.0.0-5	1.0.0-5	2.0.0-3	2.0.0-4	2.0.1-1	2.0.1-2	2.0.2-4
libfreetype6-dev	2.3.5-1ubuntu4	2.3.9-4build1	2.3.9-5	2.3.11-1ubuntu2	2.4.2-2	2.4.4-1ubuntu2	2.4.4-2ubuntu1
libdevil-dev	1.6.7-5.1	1.7.5-4	1.7.8-2	1.7.8-6	1.7.8-6	1.7.8-6build1	1.7.8-6build2
liblua5.1-0-dev	5.1.2-4	5.1.4-2	5.1.4-3	5.1.4-5	5.1.4-5	5.1.4-5	5.1.4-10
libopenal-dev	1:0.0.8-7	1:1.4.272-2	1:1.11.753-1	1:1.11.753-1	1:1.12.854-2	1:1.12.854-2	1:1.13-2
libsdl1.2-dev	1.2.13-1ubuntu1	1.2.13-4ubuntu3	1.2.13-4ubuntu4	1.2.14-4ubuntu1	1.2.14-6ubuntu3	1.2.14-6.1ubuntu3	1.2.14-6.1ubuntu4
libvorbis-dev	1.2.0.dfsg-2	1.2.0.dfsg-3.1	1.2.0.dfsg-6	1.2.3-3ubuntu1	1.3.1-1	1.3.2-1ubuntu1	1.3.2-1ubuntu2

PLATFORM ia64

	hardy	karmic	lucid	maverick	natty	oneiric	precise
mercurial	0.9.5-3	1.1.2-2ubuntu1	1.3.1-1	1.4.3-1			
build-essential	11.3ubuntu1	11.4	11.4	11.4build1			
automake	1:1.10.1-2	1:1.10.2-0ubuntu3	1:1.11-1	1:1.11.1-1			
libmpg123-dev		1.4.3-4ubuntu1	1.7.3-0ubuntu1	1.12.1-0ubuntu1			
libmodplug-dev	1:0.7-7	1:0.8.4-3ubuntu1	1:0.8.7-1	1:0.8.7-1build1			
libphysfs-dev	1.0.0-5	1.0.0-5	2.0.0-3	2.0.0-4			
libfreetype6-dev	2.3.5-1ubuntu4	2.3.9-4build1	2.3.9-5	2.3.11-1ubuntu2			
libdevil-dev	1.6.7-5.1	1.7.5-4	1.7.8-2	1.7.8-6			
liblua5.1-0-dev	5.1.2-4	5.1.4-2	5.1.4-3	5.1.4-5			
libopenal-dev	1:0.0.8-7	1:1.4.272-2	1:1.11.753-1	1:1.11.753-1			
libsdl1.2-dev	1.2.13-1ubuntu1	1.2.13-4ubuntu3	1.2.13-4ubuntu4	1.2.14-4ubuntu1			
libvorbis-dev	1.2.0.dfsg-2	1.2.0.dfsg-3.1	1.2.0.dfsg-6	1.2.3-3ubuntu1			

PLATFORM powerpc

	hardy	karmic	lucid	maverick	natty	oneiric	precise
mercurial	0.9.5-3	1.1.2-2ubuntu1	1.3.1-1	1.4.3-1	1.6.3-1	1.7.5-1ubuntu1	1.9.1-1
build-essential	11.3ubuntu1	11.4	11.4	11.4build1	11.5	11.5ubuntu1	11.5ubuntu1
automake	1:1.10.1-2	1:1.10.2-0ubuntu3	1:1.11-1	1:1.11.1-1	1:1.11.1-1	1:1.11.1-1ubuntu1	1:1.11.1-1ubuntu1
libmpg123-dev		1.4.3-4ubuntu1	1.7.3-0ubuntu1	1.12.1-0ubuntu1	1.12.1-3ubuntu1	1.12.1-3ubuntu1	1.12.1-3.1fakesync1
libmodplug-dev	1:0.7-7	1:0.8.4-3ubuntu1	1:0.8.7-1	1:0.8.7-1build1	1:0.8.8.1-1ubuntu1	1:0.8.8.1-2	1:0.8.8.2-3ubuntu1
libphysfs-dev	1.0.0-5	1.0.0-5	2.0.0-3	2.0.0-4	2.0.1-1	2.0.1-2	2.0.2-4
libfreetype6-dev	2.3.5-1ubuntu4	2.3.9-4build1	2.3.9-5	2.3.11-1ubuntu2	2.4.2-2	2.4.4-1ubuntu2	2.4.4-2ubuntu1
libdevil-dev	1.6.7-5.1	1.7.5-4	1.7.8-2	1.7.8-6	1.7.8-6	1.7.8-6build1	1.7.8-6build2
liblua5.1-0-dev	5.1.2-4	5.1.4-2	5.1.4-3	5.1.4-5	5.1.4-5	5.1.4-5	5.1.4-10
libopenal-dev	1:0.0.8-7	1:1.4.272-2	1:1.11.753-1	1:1.11.753-1	1:1.12.854-2	1:1.12.854-2	1:1.13-2
libsdl1.2-dev	1.2.13-1ubuntu1	1.2.13-4ubuntu3	1.2.13-4ubuntu4	1.2.14-4ubuntu1	1.2.14-6ubuntu3	1.2.14-6.1ubuntu3	1.2.14-6.1ubuntu4
libvorbis-dev	1.2.0.dfsg-2	1.2.0.dfsg-3.1	1.2.0.dfsg-6	1.2.3-3ubuntu1	1.3.1-1	1.3.2-1ubuntu1	1.3.2-1ubuntu2

PLATFORM sparc

	hardy	karmic	lucid	maverick	natty	oneiric	precise
mercurial	0.9.5-3	1.1.2-2ubuntu1	1.3.1-1	1.4.3-1			
build-essential	11.3ubuntu1	11.4	11.4	11.4build1			
automake	1:1.10.1-2	1:1.10.2-0ubuntu3	1:1.11-1	1:1.11.1-1			
libmpg123-dev		1.4.3-4ubuntu1	1.7.3-0ubuntu1	1.12.1-0ubuntu1			
libmodplug-dev	1:0.7-7	1:0.8.4-3ubuntu1	1:0.8.7-1	1:0.8.7-1build1			
libphysfs-dev	1.0.0-5	1.0.0-5	2.0.0-3	2.0.0-4			
libfreetype6-dev	2.3.5-1ubuntu4	2.3.9-4build1	2.3.9-5	2.3.11-1ubuntu2			
libdevil-dev	1.6.7-5.1	1.7.5-4	1.7.8-2	1.7.8-6			
liblua5.1-0-dev	5.1.2-4	5.1.4-2	5.1.4-3	5.1.4-5			
libopenal-dev	1:0.0.8-7	1:1.4.272-2	1:1.11.753-1	1:1.11.753-1			
libsdl1.2-dev	1.2.13-1ubuntu1	1.2.13-4ubuntu3	1.2.13-4ubuntu4	1.2.14-4ubuntu1			
libvorbis-dev	1.2.0.dfsg-2	1.2.0.dfsg-3.1	1.2.0.dfsg-6	1.2.3-3ubuntu1			

The code to build the source file anew is:

```
<?php
$dists = array("hardy","karmic","lucid","maverick","natty","oneiric","precise");
$packages = array("mercurial","build-essential","automake","libmpg123-dev","libmodplug-dev","libphysfs-dev","libfreetype6-dev","libdevil-dev","liblua5.1-0-dev","libopenal-dev","libsdl1.2-dev","libvorbis-dev");
$comp = array("amd64","armel","i386","ia64","powerpc","sparc");
$total_dl = count($dists)*count($packages);
$current_dl = 0;
foreach($dists as $dist){
  foreach($packages as $package){
    $current_dl++;
    echo "Downloading page $current_dl/$total_dl. $dist\t$package \t";
    $raw = file_get_contents("https://launchpad.net/ubuntu/$dist/+package/$package ↗");
    foreach($comp as $c){
      $matches = array();
      preg_match('@<a href="https://love2d.org/ubuntu/'.$dist.'/'.$c.'/'.$package.'/(.+){1,}">@',$raw,$matches);
      $data[$dist][$package][$c] = @$matches[1];
      echo "$c ..";
    }
    echo "done.\n";
  }
}
file_put_contents('build.out',serialize($data));
```

And then the code to format the output, as seen above.

```
<?php
$platform = $argv[1];
$data = unserialize(file_get_contents('build.out'));
```

```
$width = 19;
$dists = str_pad("",$width);
$rows = array();
foreach($data as $dist => $packages){
  $dists .= str_pad("$dist",$width);
  if(!isset($disp)){
    foreach($packages as $package => $platforms){
      $row[$package] = str_pad($package,$width);
    }
    $disp = 1;
  }
  foreach($packages as $package => $platforms){
    $row[$package] .= str_pad(rtrim($platforms[$platform]),$width);
  }
}
echo $dists."\n";
foreach($row as $r){
  echo "$r\n";
}
```

And the source I used to concatonate them all:

```
<?php
$comp = array("amd64","armel","i386","ia64","powerpc","sparc");
foreach($comp as $c){
  echo "PLATFORM $c\n";
  echo `php ./show.php $c`;
  echo "\n";
}
?>
```

Love-Alchemy

Love-Alchemy · revision 28073 · last edited 2023-07-05

Love Alchemy is a Framework for writing SWF games in the Lua language and the Love API.

It is based on Lua Alchemy (http://code.google.com/p/lua-alchemy/) and Box2dFlash (http://Box2dflash.org)

Love Alchemy Forum Thread (http://love2d.org/forums/viewtopic.php?f=5&t=)

Love Alchemy is hosted on github (http://github.com/Snaker1/Love-Alchemy)



"Love Alchemy"

Try it in your Browser! (http://github.com/downloads/Snaker1/Love-Alchemy/LoveAlchemyDemoPhy.swf)

love-imgui

love-imgui · revision 28072 · last edited 2023-07-05

LOVE-IMGUI

Binary IMGUI module for LÖVE. It uses IMGUI 1.50 WIP and supports 258 functions (40 unsupported).

Examples

Simple integration

```
require "imgui"

local showTestWindow = false
local showAnotherWindow = false
local floatValue = 0;
local sliderFloat = { 0.1, 0.5 }
local clearColor = { 0.2, 0.2, 0.2 }
local comboSelection = 1
local textValue = "text"

--
-- LOVE callbacks
--
function love.load(arg)
end

function love.update(dt)
    imgui.NewFrame()
end

function love.draw()
    local status

    -- Menu
    if imgui.BeginMainMenuBar() then
        if imgui.BeginMenu("File") then
            imgui.MenuItem("Test")
            imgui.EndMenu()
        end
        imgui.EndMainMenuBar()
    end

    -- Debug window
    imgui.Text("Hello, world!");
    status, clearColor[1], clearColor[2], clearColor[3] = imgui.ColorEdit3("Clear color", clearColor[1], clearColor[2], clearColor[3]);

    -- Sliders
    status, floatValue = imgui.SliderFloat("SliderFloat", floatValue, 0.0, 1.0);
    status, sliderFloat[1], sliderFloat[2] = imgui.SliderFloat2("SliderFloat2", sliderFloat[1], sliderFloat[2], 0.0, 1.0);

    -- Combo
    status, comboSelection = imgui.Combo("Combo", comboSelection, { "combo1", "combo2", "combo3", "combo4" }, 4);

    -- Windows
    if imgui.Button("Test Window") then
        showTestWindow = not showTestWindow;
    end

    if imgui.Button("Another Window") then
        showAnotherWindow = not showAnotherWindow;
    end

    if showAnotherWindow then
        imgui.SetNextWindowPos(50, 50, "FirstUseEver")
        status, showAnotherWindow = imgui.Begin("Another Window", true, { "AlwaysAutoResize", "NoTitleBar" });
        imgui.Text("Hello");
        -- Input text
```

```
        status, textValue = imgui.InputTextMultiline("InputText", textValue, 200, 300, 200);
        imgui.End();
    end

    if showTestWindow then
        showTestWindow = imgui.ShowTestWindow(true)
    end

    love.graphics.clear(clearColor[1] * 255, clearColor[2] * 255, clearColor[3] * 255, 255)
    imgui.Render();
end

function love.quit()
    imgui.ShutDown();
end

--
-- User inputs
--
function love.textinput(t)
    imgui.TextInput(t)
    if not imgui.GetWantCaptureKeyboard() then
        -- Pass event to the game
    end
end

function love.keypressed(key)
    imgui.KeyPressed(key)
    if not imgui.GetWantCaptureKeyboard() then
        -- Pass event to the game
    end
end

function love.keyreleased(key)
    imgui.KeyReleased(key)
    if not imgui.GetWantCaptureKeyboard() then
        -- Pass event to the game
    end
end

function love.mousemoved(x, y)
    imgui.MouseMoved(x, y)
    if not imgui.GetWantCaptureMouse() then
        -- Pass event to the game
    end
end

function love.mousepressed(x, y, button)
    imgui.MousePressed(button)
    if not imgui.GetWantCaptureMouse() then
        -- Pass event to the game
    end
end

function love.mousereleased(x, y, button)
    imgui.MouseReleased(button)
    if not imgui.GetWantCaptureMouse() then
        -- Pass event to the game
    end
end

function love.wheelmoved(x, y)
    imgui.WheelMoved(y)
    if not imgui.GetWantCaptureMouse() then
        -- Pass event to the game
    end
end
```

Links

- GitHub page (https://github.com/slages/love-imgui)

- Pre-built binaries (https://github.com/slages/love-imgui/releases)
- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=82467)
- IMGUI GitHub page (https://github.com/ocornut/imgui)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love-loader

love-loader · revision 7826 · last edited 2023-07-05

Original forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=7721).

Latest version:

https://github.com/kikito/love-loader ↗

This lib will allow your game to "do stuff" (play animations, etc) while loading resources from the disk without getting "jerky". It moves the blocking calls to a separate thread, and invokes a callback when the loading is done.

Please refer to the forum post above or the latest version link for examples of use.

love-nuklear

love-nuklear · revision 25332 · last edited 2023-07-05

A LÖVE binding for Nuklear (https://github.com/vurtun/nuklear)

- Download : keharriso/love-nuklear: Lightweight immediate mode GUI for LÖVE games (https://github.com/keharriso/love-nuklear)
- Forum: LÖVE-Nuklear - a lightweight immediate mode GUI - LÖVE (https://love2d.org/forums/viewtopic.php?t=83120)

Other Languages

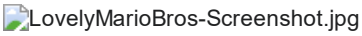
Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Lovely Mario Bros

Lovely Mario Bros · revision 7150 · last edited 2021-03-14

(September 2011) After deserting my old project for over a year, I decided I'm going to finish this.

Basically, it's Super Mario with Box2D and soon: a level editor.



Download the most recent version as of June 14, 2010: lovely mario bros forum post version ?.?** (http://love2d.org/forums/viewtopic.php?f=5&t=1619&p=15995&hilit=lovely+mario#p15881)

New updates will come soon

Lovely Tactics Hex

Lovely Tactics Hex · revision 29428 · last edited 2025-01-08

Framework for creation of tactical RPGs. View the project page on GitHub (https://github.com/gloamingcat/lovely-tactics-hex/).

Lovely Tiles

Lovely Tiles · revision 10435 · last edited 2023-07-05

LOVEly Tiles is a collection of tools to assist in loading and rendering tile maps. It supports orthogonal and isometric orientation.

Github Repo (https://github.com/markandgo/LOVEly-tiles) Thread (https://love2d.org/forums/viewtopic.php?f=5&t=29159)

lovesize

lovesize · revision 24617 · last edited 2023-07-05

Library that makes it possible to work with a fixed game resolution, while scaling the screen and clipping objects outside of it, making a letterbox effect.

To download it, learn how to use it and maybe help improving it, check out the github page:

https://github.com/RicardoBusta/lovesize ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love残片

love残片 · revision 8196 · last edited 2021-03-14

examples(不光是我写的大部分是下的)

examples

LPC License

LPC License · revision 17231 · last edited 2021-03-14

The LPCL, version 1.0:

```
             Lovely Public Community License (LPCL)
                Version 1.0, February 20, 2009
----  ------  ------  ------  ------  ------  ------  ------  --

Copyright (c) 2009 Bart Bes and Osuf Oboys
  This license, but not the licensed product, is licensed to
"the LCPL Development Team" under LPCL.

  The intention of this license is to provide a means to release
a product in a community without any particular user governing
the project. In particular, the license should encourage the
liberties taken in wiki systems, provide a means to settle
disputes, and allow new users to continue a project where every
developer has gone inactive. This license imposes NO
restrictions on work that is released under other names than the
name of this product. If such restrictions are desired, the work
may be licensed under more than one license.

Definitions.
  "This product" refers to all copyrightable work licensed under
this license. A "derivative work" is any copyrightable material
based on or using this product. "The community of this product"
is a body of entities, usually people, which may be defined in
an accompanying policy document or section as stated below.

  A separate document or section defining this product's
community and policy may accompany this license and is treated
as constituent of the referenced product as far as this license
is concerned. Such a document or section may only define "the
community" and how "the community" may exercise  clause 3.2 of
this license. In particular, changing such a document does not
demand a new release of this license but it does demand a new
release of the product. If no such document or clause
accompanies the product, then "the community" is the empty set.

Terms and Conditions.
  This product comes without any warranty, express or otherwise.
Under no circumstances will the creators of this product be held
responsible for any negative effects arising from the use of
this or derivative works.
  Permission is granted to use this product in any fashion and
for any purpose, including commercial, with or without this
license, provided the following conditions are met.
 1. Derivative works released under more than one license are
subject to the restrictions of each such license.
 2. Any derivative work advertised under the same name and the
same version as this product must constitute a complete
unaltered copy of this product without any additions and be
accompanied by an unaltered copy of this license.
 3. If a derivative work is advertised under the same name as
this product but under a different version, then the following
additional conditions apply.
  3.1. The derivative work must clearly state that they are a
derivative of this product and must be licensed under this
license or a newer version of this license as approved by the
community.
  3.2. The community of this product has the right to demand
that any derivative work, or any derivative work of a derivative
in any number of steps, changes the name of the derivative work
to a name other than the name of this product or else be subject
to copyright infringement. The owners of the derivative work
must in such an event be notified about the decision. The
community has no right in choosing the new name.
  3.3. Clause 3.2 may be invoked any number of times permitted
by the accompanying policy.
```

The LPCL Development Team.

  This section does not apply to work licensed under LPCL other than the license itself. The LPCL Development Team can NOT invoke clause 3.2 for any work other than the license and the license only.

  The LPCL Development Team consists of every real person with an unbanned account on any forum hosted by http://love2d.org with at least ten posts made a week or more ago.

  To invoke clause 3.2, a post must be made by a team member in a new thread hosted by the domain http://love2d.org. This post must clearly state the opening of a vote, the expected format of votes, and the effect of the respective answers.

  A valid vote is any post in this thread made by a member of the LPCL development team which

 * contains "yes." or "no.", but not both, as a subword of any capitalization,
 * does not have any other valid vote cast by the same member of the same or a newer time,
 * is stamped at most 604800 seconds (one week) older than the post that opened the vote.

  After one week, the votes are tallied with the winning being either 'yes.' or 'no.', whichever received the most answers. In the case of a draw, clause 3.2 is not invoked.

  At most three votes per can be made about the same product and at least one week apart. A vote may concern more than one product. No legal person may open more than four votes per month.

---- ------ ------ ------ ------ ------ ------ ------ --

The LÖVE Community

  This document defines the community which this product is licensed under by the Lovely Public Community License.

  The LÖVE Community consists of every real person with an unbanned account on any forum hosted by http://love2d.org with at least ten posts made a week or more ago.

  To invoke clause 3.2, a post must be made by a team member in a new thread hosted by the domain http://love2d.org. This post must clearly state the opening of a vote, the expected format of votes, and the effect of the respective answers.

  A valid vote is any post in this thread made by a member of the LPCL development team which

 * contains "yes." or "no.", but not both, as a subword of any capitalization,
 * does not have any other valid vote cast by the same member of the same or a newer time,
 * is stamped at most 604800 seconds (one week) older than the post that opened the vote.

  After one week, the votes are tallied with the winning being either 'yes.' or 'no.', whichever received the most answers. In the case of a draw, clause 3.2 is not invoked.

  At most three votes per can be made about the same product and at least one week apart. A vote may concern more than one product. No legal person may open more than four votes per month.

Lua FOV

Lua FOV · revision 13593 · last edited 2023-07-05

Field of view algorithms in Lua. Two flavors are available: recursive shadowcasting and precise permissive.

https://github.com/markandgo/Lua-FOV ↗

lua-enet

lua-enet · revision 29481 · last edited 2026-04-05

Available since LÖVE 0.9.0

This module is not supported in earlier versions.


Official documentation for lua-enet is available here (http://leafo.net/lua-enet/). ENet's features are listed on its homepage (http://enet.bespin.org/Features.html). The official documentation may have typos. The documentation on this wiki reflects Löve's implementation, meaning it should be safe to follow what's written here.

lua-enet is a binding to the ENet library for Lua. It comes included with LÖVE.

ENet's purpose is to provide a relatively thin, simple and robust network communication layer for games on top of UDP (User Datagram Protocol).The primary feature it provides is optional reliable, in-order delivery of packets.

ENet omits certain higher level networking features such as authentication, lobbying, server discovery, encryption, or other similar tasks that are particularly application specific so that the library remains flexible, portable, and easily embeddable.

Types

Type	Description
host	An ENet host for communicating with peers.
peer	An ENet peer which data packets may be sent or received from.
event	A simple table containing information on an event.

Functions

Function	Description
host_create	Returns a new host.
linked_version	Returns the included ENet's version string.

Examples

server.lua

```lua
-- server.lua
local enet = require "enet"
local host = enet.host_create("localhost:6789")
while true do
  local event = host:service(100)
  while event do
    if event.type == "receive" then
      print("Got message: ", event.data, event.peer)
      event.peer:send( "pong" )
    elseif event.type == "connect" then
      print(event.peer, "connected.")
    elseif event.type == "disconnect" then
      print(event.peer, "disconnected.")
    end
    event = host:service()
  end
end
```

client.lua

```lua
-- client.lua
local enet = require "enet"
```

```
local host = enet.host_create()
local server = host:connect("localhost:6789")
while true do
  local event = host:service(100)
  while event do
    if event.type == "receive" then
      print("Got message: ", event.data, event.peer)
      event.peer:send( "ping" )
    elseif event.type == "connect" then
      print(event.peer, "connected.")
      event.peer:send( "ping" )
    elseif event.type == "disconnect" then
      print(event.peer, "disconnected.")
    end
    event = host:service()
  end
end
```

This is another example that creates a server (receiver) and a client (sender) on the same computer with the same script. The client will say "Hi" to the server endlessly because ClientSend() is called inside love.update.

```
enet = require "enet"

enethost = nil
hostevent = nil
clientpeer = nil

function love.load(args)

    -- establish host for receiving msg
    enethost = enet.host_create("localhost:6750")

    -- establish a connection to host on same PC
    enetclient = enet.host_create()
        clientpeer = enetclient:connect("localhost:6750")

end

function love.update(dt)
    ServerListen()
    ClientSend()
end

function love.draw()
end

function ServerListen()

    hostevent = enethost:service(100)

    if hostevent then
        print("Server detected message type: " .. hostevent.type)
        if hostevent.type == "connect" then
            print(hostevent.peer, "connected.")
        end
        if hostevent.type == "receive" then
            print("Received message: ", hostevent.data, hostevent.peer)
        end
    end
end

function ClientSend()
    enetclient:service(100)
    clientpeer:send("Hi")
end
```

See Also

- love

- socket

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

lua-https

lua-https · revision 29585 · last edited 2025-11-25

On this page

- Reference
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Compile From Source
  - Windows
  - Linux
  - Android
- See Also
- Other Languages

Available since LÖVE 12.0

Users can compile from source for earlier versions of LÖVE.

lua-https is a simple Lua HTTPS module using native platform backends specifically written for LÖVE 12.0 and supports Windows, Linux, macOS, iOS, and Android.

lua-https is licensed under zLib license, same as LÖVE.

Reference

To use lua-https, load it with require like `local https = require("https")`. lua-https does **not** create global variables!

The `https` table expose single function: `https.request`

Function

Synopsis

```
code, body = https.request( url )
```

Arguments

`string url`
: HTTP or HTTPS URL to access.

Returns

`number code`
: HTTP status code, or 0 on failure.

`string body`
: HTTP response body or nil on failure.

Function

Synopsis

```
code, body, headers = https.request( url, options )
```

Arguments

`string url`
: HTTP or HTTPS URL to access.

`table options`
: Options for advanced mode.

  `string data`
  : Additional data to send as application/x-www-form-urlencoded (unless specified otherwise in Content-Type header).

  `string method`

HTTP method. If absent, it's either "GET" or "POST" depending on the data field above.

table headers

Additional headers to add to the request as key-value pairs.

Returns

number code

HTTP status code, or 0 on failure.

string body

HTTP response body or nil on failure.

table headers

HTTP response headers as key-value pairs or nil on failure.

Notes

To urlencode a Lua table suitable for application/x-www-form-urlencoded data, use a LuaSocket function.

```
local url = require("socket.url")

function urlencode(list)
    -- Since order of pairs is undefined, the key-value order is also undefined.
    local result = {}
    for k, v in pairs(list) do
        result[#result + 1] = url.escape(k).."="..url.escape(v)
    end
    return table.concat(result, "&")
end

-- Usage
code, body, headers = https.request("https://example.com", {data = urlencode({key = "value", foo = "bar"})
```

Compile From Source

While lua-https will be bundled in LÖVE 12.0 by default, it's possible to compile the module from source and use it on earlier version of LÖVE. Grab the source code at https://github.com/love2d/lua-https ↗. All compilation requires Lua 5.1 (or LuaJIT) headers and libraries available.

Windows

Compilation is done using CMake. This assume MSVC toolchain is used. Change "x64" to "Win32" to compile for x86 32-bit platform or "ARM64" for ARM64 platform.

```
cmake -Bbuild -S. -A x64 -DCMAKE_INSTALL_PREFIX=%CD%\install
cmake --build build --config Release --target install
```

https.dll can be found in the **install** folder.

Linux

Compilation is done using CMake. Ensure you have OpenSSL (https://www.openssl.org/) and cURL (https://curl.se/) development libraries installed to enable all supported Linux backends.

```
cmake -Bbuild -S. -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=$PWD\install
cmake --build build --target install
```

https.so can be found in the **install** folder.

Android

Available since LÖVE 11.4

Proper 3rd-party C module support requires this LÖVE version.

Compilation is done by placing lua-https source code in `<love-android>/love/src/jni/lua-modules`. The structure will look like this:

```
<love-android>/love/src/jni/lua-modules/lua-https
+ example
+ src
+ Android.mk
+ CMakeLists.txt
+ java.txt
+ license.txt
```

Afterwards compile love-android as usual. The modules will be automatically embedded to the APK. This can be verified by checking the APK with Zip viewer application and inspecting files in `lib/arm64-v8a` and `lib/armeabi-v7a`.

See Also

- source code (https://github.com/love2d/lua-https)
- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

luaFortune

luaFortune · revision 15540 · last edited 2023-07-05

A bunch of libraries created by Jmaa for procedural generation in lua.

Grab it on Bitbucket (https://bitbucket.org/Jmaa/luafortune), or discuss it in its forum thread (https://github.com/adonaac/).

luasort

luasort · revision 24556 · last edited 2023-07-05

Just a short implementation of three sorting algorithms

1. Quick-sort
2. Insertion-sort
3. Selection-sort

Repository Link (https://github.com/matiasah/luasort)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Luiz Renato

Luiz Renato · revision 19700 · last edited 2021-03-14

Programmer and digital game programmer

https://www.facebook.com/darkmetalic ↗

https://github.com/darkmetalic ↗

@darkmetalic

lure

lure · revision 9158 · last edited 2023-07-05

On this page

Summary
Download
Active Development
Bug Reports/Feedback
Articles/Tutorials
Documentation
Namespaces

Summary

LURE is an attempt to implement a html DOM parser and renderer for the Love2d game engine. The goal of LURE is to provide the following:

- A non-validating html parser which generates a (somewhat) W3C Standards complaint DOM Object for use with love.
- A html renderer which will display html markup in the love window.

Lure's purpose will NOT be to implement a web browser in Love2d, that is just ridiculous :D Instead, Lure's purpose will be to provide the core html DOM interfaces allowing Lovers the ability to to work with html data structures and to render basic html to the love window for simple UI.

Download

Github (https://github.com/admin36/LURE)

Active Development

Status: In development

Last Stable: N/A

Development Tracker: Trello Board (https://trello.com/board/lure/503c5baea077f04c54795d7d)

Bug Reports/Feedback

Please place bug reports and feedback on Github (https://github.com/admin36/LURE/issues)

Articles/Tutorials

- Parser Preview
- Using XMLHttpRequest
- Render Preview

Documentation

Namespaces

Namespace	Description	Implemented
lure	Lure Root Namespace	Yes
lure.core	Lure core namespace	Yes
lure.layers	Lure layers namespace	Yes
lure.dom	Lure Document Object Model namespace	Yes
lure.dom.css	Lure Document Object Model css namespace	Yes
lure.rom	Lure Render Object Model namespace	Yes
lure.rom.layout	Lure Render Object Model layout namespace	Yes

Lure Object Reference

Object	Description	Implimented
LURE Window		yes

LURE Layer		yes

Lure DOM Object Reference

Objects present in this table are DOM objects currently planned for implementation. Not all W3C DOM objects will be implemented.

Object	Description	Implemented
DOM Node	The Node object represents a single node in the document tree	Yes
DOM NodeList	The NodeList object represents an ordered list of nodes	Yes
DOM NamedNodeMap	The NamedNodeMap object represents an unordered list of nodes.	Yes
DOM document	The Document object represents the entire XML/HTML document.	Yes
DOM Element	The Element object represents an element in an XML/HTML document	Yes
DOM Attribute	The Attr object represents an attribute of an Element object.	Yes
DOM Text	The Text object represents the textual content of an element or attribute.	Yes
DOM CDATA	The CDATASection object represents a CDATA section in a document.	Yes
DOM Comment	The Comment object represents the content of comment nodes in a document.	Yes
DOM DOMParser	DOMParser object parses xml/html text and returns a DOM Object	Yes
DOM XMLHttpRequest	XMLHttpRequest object can make synchronous or synchronous http requests to external web resources	Yes
DOM HTMLEvent		No
DOM HTMLAnchor		No
DOM HTMLBody		No
DOM HTMLButton		No
DOM HTMLDiv		No
DOM HTMLForm		No
DOM HTMLImage		No
DOM HTMLInput Button		No
DOM HTMLInput Checkbox		No
DOM HTMLInput Hidden		No
DOM HTMLInput Password		No
DOM HTMLInput Radio		No
DOM HTMLInput Reset		No
DOM HTMLInput Submit		No
DOM HTMLInput Text		No
DOM HTMLLink		No
DOM HTMLOption		No
DOM HTMLScript	Represents an individual script element. asserts lua script directly from the document.	Yes
DOM HTMLSelect		No
DOM HTMLStyle	The Style object represents an individual style element.	Yes
DOM HTMLStylesheet	Stylesheet object represents an individual stylesheet.	Yes
DOM HTMLCssRule	HTMLCssRule object represents an individual Css Rule.	Yes
DOM HTMLTable		No
DOM HTMLtd/th		No
DOM HTMLtr		No

DOM HTMLTextArea		No

Lure ROM Object Reference

The Lure ROM (Render Object Model) is comprised of all core render objects and methods to render a DOM model to the screen

Object	Description	Implemented
ROM Viewport	The viewport object represents the root render object of the ROM model	Yes
ROM node	The ROM node object represents the base object most other ROM nodes inherit from	Yes
ROM nodelist	The ROM nodelist is used to maintain indexed lists of ROM Nodes	yes
ROM Nodeattribute	The Nodeattribute object maintains ROM Node attributes	Yes
ROM box	The ROM box object represents the base box render object	yes
ROM blockBox		yes
ROM inlineBox		yes
ROM inlineBlockBox		Partial
ROM lineBox		yes
ROM boxComputedStyle		yes
ROM boxRenderStyle		yes
ROM layoutResponse		yes
ROM namednodemap		yes
ROM romParser		yes
ROM textNode		yes

MORE TO COME!

Löve Bone

Löve Bone · revision 24433 · last edited 2023-07-05

Löve Bone is a robust object-oriented 2D Skeletal Animation framework for LÖVE.

Grab it on Github (https://github.com/GeekWithALife/love-bone).

- Docs and Tutorials (https://github.com/GeekWithALife/love-bone/wiki)
- Sample Projects (https://github.com/GeekWithALife/love-bone/tree/master/examples)
- Forum thread (https://www.love2d.org/forums/viewtopic.php?f=5&t=79607)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LÖVE CÖNNECTION

LÖVE CÖNNECTION · revision 24554 · last edited 2023-07-05

On this page

- Functions
- Events
  - Client Events
  - Server Events
- Variables/Tables
- Tutorial
  - Server Setup
  - Client Setup
- Other Languages

LÖVE CÖNNECTION is a networking module coded and maintained by Nicholas Scott [1] (https://love2d.org/forums/viewtopic.php?t=80330)

When the module is loaded with require() it will return a table of functions for the list I will be assuming that the require is set equal to Net:

Functions

- Net:connectedUsers() --*Returns a list of currently connected users*
- Net:kickUser(id, reason) --*Kicks the user with specified id and the reason is currently printed to console*
- Net:isServer() --*Returns true if currently running as a server*
- Net:isClient() --*Returns true if currently running as a client*
- Net:setMaxPing(ping) --*If called while running as server it will set the maximum ping(milliseconds) for clients before they timeout, if called as client it will set the ping time before it pings the server again*
- Net:init(state) --*Starts the networking, state needs to be either server, to run as a server, or client, to run as a client*
- Net:update(dt) --*Add this into any **ACTIVE** love.update() and pass the deltatime from the love.update!! IMPORTANT!!!!*
- Net:connect(ip, port) --*If called as server you must set ip to nil and port to the port you want to listen on, defaults to 20024, when called as client ip and port are required, and port must be the same as the servers, obviously :D*
- Net:disconnect() --*Disconnects from online, if it is ran from client it will nil out the socket entirely and send a disconnect packet to the server it was on*
- Net:registerCMD(cmd, function) --*Registers a command for networking, cmd is a string name of the command you want to register, and function is the function ran when a client(or server) receives the command specified. More on this below :D*
- Net:send(table, cmd, param, id) --*Id is only supplied if running as server, table is a table of data you want to send, cmd is the command your running on the other end, must be registered, param is parameters to the command, id is the id of the client you want to send to(only if server leave nil if client)*
- Net:encode(table) --*Encodes a table of data into a string to be sent across the network*
- Net:decode(string) --*Decodes a string into a table of data when received*

Events

These are called the same way you call love.update() function Net.event.server.userConnected(id) end

Client Events

- Net.event.client.connect(ip, port) --*Called when client connects to a server with net:connect() ip is ip of server and port is port of server*
- Net.event.client.receive(table, dt, cmd, param) --*Called when client receives a packet table is the table client sent, dt deltatime of net:update(), cmd is the command from client, param is parameters from the client*
- Net.event.client.disconnect() --*Called when client disconnects from server with net:disconnect()*
- Net.event.client.cmdRegistered(cmd, functionS) --*Called when a command is registered, cmd is the cmd and function is the function called by the cmd*
- Net.event.client.send(table, cmd, param) --*Called when a packet is sent to the server, table is the table being sent, cmd is the cmd, param is parameters*
- Net.event.client.kickedFromServer(reason) --*Called when we(the client) is kicked from the server, reason is the reason for the kick*

Server Events

- Net.event.server.userConnected(id) --*Called when a user connects to the server*
- Net.event.server.userDisconnected(id) --*Called when a user disconnects from the server*
- Net.event.server.userTimedOut(id) --*Called when a user times out from the server*

- Net.event.server.userKicked(id, reason) *--Called when a user is kicked, reason is the reason they were kicked*
- Net.event.server.receive(table, dt, id, cmd, param) *--Called when we receive a packet, table is the table sent by client, id is id of client, dt is deltatime of net:update(), cmd is the command sent by client, param is parameters of the client*
- Net.event.server.connect(port) *--Called when we connect to a port, port is the port connected to*
- Net.event.server.disconnect() *--Called when we disconnect from the port and stop listening to packets*
- Net.event.server.cmdRegistered(cmd, functionS) *--Called when we register a command, cmd is the command registered, function is the function called by the cmd*
- Net.event.server.send(table, cmd, param, id) *--Called when we send a packet to a client, table is the table sent, cmd is the cmd sent, param is the parameters sent, id is the id of the client we sent to*

Variables/Tables

- Net.client = table *--Holds Ip and port of currently connected server, if not connected ip and port are nil*
- Net.server = table *--Holds the port currently being listened on, nil if not listening*
- Net.commands = table *--Holds all currently registered commands in format CMD = FUNCTION, command editing function coming soon!*
- Net.users = table *--Holds all currently connected users in format ID = PLAYERTABLE, player table editing function coming soon!*
- Net.maxPing = integer *--Current max ping of server, or client*
- Net.connected = boolean *--True if connected to a server or listening to a port, false if not*

Tutorial

Server Setup

Step 1: Require the module as a variable

```
Net = require( "net" )
```

Step 2: Initialize the networking

```
Net:init( "Server" )
```

Step 3: Connect to a port to listen for packets on

```
Net:connect( nil, 25045 )
```

Step 4: Register some commands for received packets, the table is a table encoded and sent to us by the client :D useful for this

```
Net:registerCMD( "updateStatsOfPlayer", function( table, parameters, id, dt ) Net.users[id].name = table.name Net.users[id].color = table.color end )
```

Step 5: Send some packets to a client! :D This will print to every client on the server's console "Hello There Mr. Client!"

```
for id,playerdata in pairs( Net.users ) do
   Net:send( {}, "print", "Hello There Mr. Client!", id )
end
```

Client Setup

Step 1: Require the module as a variable

```
Net = require( "net" )
```

Step 2: Initialize the networking

```
Net:init( "Client" )
```

Step 3: Connect to a server, this will also initialize the handshake feature and allow us to use net:send()

```
Net:connect( "127.0.0.1", 25045 )
```

Step 4: Register some commands for received packets, the table is a table encoded and sent to us by the client :D useful for this

```
Net:registerCMD( "updateStatsOfBattlefield", function( table, parameters, id, dt )Tank.x = table.x Tank.y = table.y Tank.name = table.name
end )
```

Step 5: Send some packets to the server! :D This will print to the servers console "Hello there Mr. Server!", the {} is an empty table, THIS IS REQUIRE DON'T LEAVE IT NIL

```
Net:send( {}, "print", "Hello There Mr. Server!" )
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Löve Frames

Löve Frames · revision 25334 · last edited 2023-07-05

Löve Frames is an advanced GUI library created by Nikolai Resokav for LÖVE.

Links

- Release Thread (https://love2d.org/forums/viewtopic.php?f=5&t=9116)
- Repository (https://github.com/linux-man/LoveFrames)
- Wiki (https://github.com/linux-man/LoveFrames/wiki)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LÖVE Game File

LÖVE Game File · revision 22335 · last edited 2025-03-18

LÖVE game files or **.love files** are games, packaged for the LÖVE engine.

The file format is the same as a Zip file format with a ".love" file name ending instead of ".zip".

LÖVE game files contain the main.lua at the top-most directory level of the Zip archive script and other assets (e.g. fonts, images, music, scripts, sounds, etc.).

See Create a .love-file to learn how to get a .love file for distribution. See Running Games for information on how to start games in .love file.

Other Languages

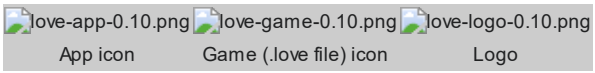
Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Löve Logo Graphics

Löve Logo Graphics · revision 28130 · last edited 2023-07-05

If you are looking to add an animated LÖVE-splashscreen to your game consider taking a look at the splashes library O.png (https://github.com/love2d-community/splashes).

Official LÖVE icon and logo graphics for use in splash, title, credits screens, website etc.



These graphics are officially used since LÖVE 0.10.1 (forum announcement (https://love2d.org/forums/viewtopic.php?f=3&t=81776)).

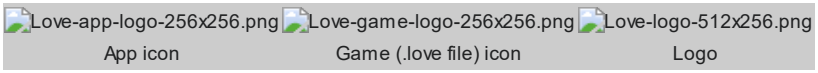
Download zip from https://love2d.org/logos.zip (https://love2d.org/logos.zip).

You can download the icons individually by pressing the "Raw" link/button on GitHub:

- App icon (https://github.com/love2d/love/blob/60278b0532036d404c0b7b011c7b63ab58a5ddaf/platform/unix/love.svg)
- Game (.love file) icon (https://github.com/love2d/love/blob/60278b0532036d404c0b7b011c7b63ab58a5ddaf/platform/unix/application-x-love-game.svg)
- iOS icons (https://github.com/love2d/love/commit/57697b372be8163613dc4315807a3e7a6785fc0e)

Old versions

Official graphics, until LÖVE 0.9.



License

All artwork in this section is licensed under the ZLIB license (https://www.zlib.net/zlib_license.html), like the rest of LÖVE.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LÖVE PhysicsEditor Loader

LÖVE PhysicsEditor Loader · revision 24443 · last edited 2023-07-05

LÖVE PhysicsEditor Loader is an loading libray for loading objects modeled with PhysicsEditor, library created by RamiLego4Game for LÖVE.

Note: The documentation is inside the code !

Links

- Library Repsoitory (https://github.com/RamiLego4Game/Love2D-PhysicsEditor-Library) Plz Star it :D
- PhysicsEditor Tool Website (https://w w w.codeandw eb.com/physicseditor)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LövelyMoon

LövelyMoon · revision 24467 · last edited 2023-07-05

Main forum topic: https://love2d.org/forums/viewtopic.php?f=5&t=38702

LövelyMoon is a library that adds a very neat way of handling gamestates, in fact, it is made to make your game focus on gamestates and such.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

mat4

mat4 · revision 27944 · last edited 2023-12-23

A matrix with 4x4 floating-point number components. These can be accessed with `[0/1/2/3][0/1/2/3]`.

```
mat4 example = mat4(1.0, 0.0, 0.0, 0.0,
                    0.0, 1.0, 0.0, 0.0,
                    0.0, 0.0, 1.0, 0.0,
                    0.0, 0.0, 0.0, 1.0);

float firstValue = example[0][0];
float lastValue  = example[3][3];
```

Read more:

- Matrices on the OpenGL Wiki (https://www.khronos.org/opengl/wiki/Data_Type_(GLSL)#Matrices)
- mat4 on The Book of Shaders (https://thebookofshaders.com/glossary/?search=mat4)

See Also

- vec4
- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Message in a Bottle

Message in a Bottle · revision 24474 · last edited 2023-07-05

On this page

- About
  - Download
  - Contact
- Features
  - Missing Features
- Configuring
- Usage
  - Setting up Ocean
  - Example
- Constructors
  - Ocean
  - MessageBottle
  - StayBottle
  - TimeBottle
- Setter Functions
  - Timeout
  - setX
  - setY
  - setPosition
  - setWidth
  - setHeight
  - setButton
  - setEase
  - setEaseTime
  - setFade
  - setFadeTime
  - setEnterCallback
  - setExitCallback
  - setSound
  - setVolume
  - setIndicator
  - setFgColor
  - setFont
  - setAlign
  - setBgColor
  - setBgRadius
  - setBgRadiusSegments
- Other Languages

About

This is a Message management library for LOVE 0.7.0. It aims to be unobtrusive and easy to use.



Download

Download the code or sample love from bitbucket (http://bitbucket.org/dannyfritz/message-in-a-bottle/downloads).

This library requires MiddleClass. It is included.

Contact

See the forum post (http://love2d.org/forums/viewtopic.php?f=5&t=2153#p22671) for public questions. Go to the Issue Tracker (http://bitbucket.org/dannyfritz/message-in-a-bottle/issues?status=new&status=open) for bugs. Look around for TechnoCat if you have any further questions or concerns.

It would be cool if you told me if you use it in your project. I would love to see what this is used for.

Features

- StayBottle, bottles that stay until a button is pressed.
- TimedBottle, bottles that stay until a button is pressed or until a duration of time has passed.
- Ocean, a management object for Bottles
- Very (and hopefully easily) Configurable Bottles
- Sleek and Stylish Bottle defaults

Missing Features

Not implemented yet, but plan to.

- Boolean Bottles

- Choice Bottles

Configuring

Much of the configuration you are going to want to do will take place at the top of MessageInABottle.lua.

Usage

Setting up Ocean

```
require 'MessageInABottle.lua'

function love.load()
  ocean = Ocean:new()
end

function love.update(dt)
  ocean:update(dt)
end

function love.draw()
  ocean:draw()
end
```

Example

```
require 'MessageInABottle.lua'

function love.load()
  love.graphics.setBackgroundColor(255,255,255)

  --Initialize the ocean to manage the bottles.
  --It is going to act as a bottle queue.
  ocean = Ocean:new()

  --Create a timed bottle and add it to the ocean
  ocean:addBottle(TimeBottle:new(nil,"...",nil))

  --Create a stay bottle and add it to the ocean
  local bottle = StayBottle:new(nil,"What is this?")
  --Change some bottle settings to non-default ones
  bottle:setX(310)
  bottle:setVolume(0)
  ocean:addBottle(bottle)
end

function love.update(dt)
  --Required to update the bottles
  ocean:update(dt)
end

function love.draw()
  love.graphics.setColor(0,0,0)
  --This is showing some useful variables ocean stores.
  love.graphics.print(
      'last bottle return was: '..ocean.response..
      ' at '..(math.floor(ocean.responseTime*10)/10)..
      's from '..ocean.responseID,12,12)
  --This is all that is needed to draw the current bottle
  ocean:draw()
end

function love.keypressed(k,u)
  --You can add a bottle on the fly.
  if k=="d" then
    local x = math.floor(math.random(400))
    local y = math.floor(math.random(150))
    ocean:addBottle(TimeBottle:new("PRESSED_D","Timed Box!"))
  elseif k=="escape" then
```

```
    love.event.push("q")
  end
end
```

Constructors

Ocean

```
Ocean:new()
```

MessageBottle

Do not use this. It is the parent class of all Bottles.

```
MessageBottle:new(id, text)
```

string id
: The unique identifier of the Bottle. e.g. "Open Door"

string text
: The text to display on the Bottle. e.g. "Will you open the door?"

StayBottle

```
StayBottle:new(id, text)
```

string id
: The unique identifier of the Bottle. e.g. "Open Door"

string text
: The text to display on the Bottle. e.g. "Will you open the door?"

TimeBottle

```
TimeBottle:new(id, text, timeout)
```

string id
: The unique identifier of the Bottle. e.g. "Open Door"

string text
: The text to display on the Bottle. e.g. "Will you open the door?"

number timeout
: The amount of time the bottle will display for.

Setter Functions

Timeout

```
TimeBottle:setTimeout(timeout)
```

number timeout
: The time left for a TimeBottle

setX

```
MessageBottle:setX(x)
```

number x
: The x position of the Bottle

setY

```
MessageBottle:setY(y)
```

number y
: The y position of the Bottle

setPosition

```
MessageBottle:setPosition(x, y)
```

number x
: The x position of the Bottle

number y
: The y position of the Bottle

setWidth

```
MessageBottle:setWidth(width)
```

number width
: The width of the Bottle

setHeight

```
MessageBottle:setHeight(height)
```

number height
: The height of the Bottle

setButton

```
MessageBottle:setButton(button)
```

KeyConstant button
: The button the Bottle will use.

setEase

```
MessageBottle:setEase(ease)
```

function ease
: The ease function to use for easing-in.

setEaseTime

```
MessageBottle:setEaseTime(easeTime)
```

number easeTime
: The time to spend easing-in.

setFade

```
MessageBottle:setFade(fade)
```

function fade
: The fade function to use for fading-in.

setFadeTime

```
MessageBottle:setFadeTime(fadeTime)
```

number fadeTime

The time to spend fading-in.

setEnterCallback

```
MessageBottle:setEnterCallback(enterCB)
```

function enterCB
The callback function to call on opening a bottle.

setExitCallback

```
MessageBottle:setEnterCallback(exitCB)
```

function exitCB
The callback function to call on exiting a bottle. The return of the bottle is passed to the exitCallback.

setSound

```
MessageBottle:setSound(sound)
```

string sound
The location of a sound file.

setVolume

```
MessageBottle:setVolume(volume)
```

number volume
The volume of the sound. Normal volume is 1.0.

setIndicator

```
MessageBottle:setIndicator(indicator)
```

string indicator
The location of the indicator image file. The indicator is the icon that bounces up and down on certain bottles.

setFgColor

```
MessageBottle:setFgColor(r, g, b, a)
```

number r
The red value from 0-255.
number g
The green value from 0-255.
number b
The blue value from 0-255.
number a
The alpha value from 0-255.

setFont

```
MessageBottle:setFont(font, size)
```

string font
The location of the font file.
number size

The point value size.

setAlign

```
MessageBottle:setAlign(align)
```

AlignMode align
The align mode.

setBgColor

```
MessageBottle:setFgColor(r, g, b, a)
```

number r
The red value from 0-255.

number g
The green value from 0-255.

number b
The blue value from 0-255.

number a
The alpha value from 0-255.

setBgRadius

```
MessageBottle:setBgRadius(r)
```

number r
The radius of the corners.

setBgRadiusSegments

```
MessageBottle:setBgRadiusSegments(n)
```

number n
The number of segments to draw on each corner.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Metanet Hunter

Metanet Hunter · revision 11679 · last edited 2021-03-14



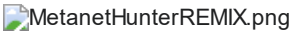
Metanet Hunter is a 2D platform game released by NEO NEKUZEN on August 4th. The game's primary objective is to destroy the Chaos Orb at the end of every level, and ultimately reach the Shadow Realm. The game features 35 main levels, with tons of bonuses, giving the game itself a total of 80 levels.

The game itself is free to download and featured on itch.io's homepage. (http://nightkaw ata.itch.io/metanet-hunter)

The man who created Metanet Hunter also has his own webpage, available at this link. (http://nekuzen.net)

Metanet Hunter REMIX

Metanet Hunter REMIX · revision 13606 · last edited 2021-03-14



Metanet Hunter: REMIX is the followup to Metanet Hunter, also released by NEO NEKUZEN. The game was given release on June 30th. The game also keeps the objective to destroy the Chaos Orb at the end of every level, but this time, the primary objective is to destroy the Metanet Terminal, located in Clockwork Chaos.

The second entry in the series features eight worlds, and two highly-powerful armors collected from finding the hidden shinies in the first four worlds. Presumably, the game is to receive a special game-mode called "Fantasy Game" should the game become popular.

The game itself costs $3.99 for PC, OSX, and Linux, and can be purchased at the itch.io page. (http://nightkaw ata.itch.io/metanet-hunter)

Program: NightKawata of NEO NEKUZEN (http://nightkaw ata.itch.io)

Graphics: SilverDeoxys563

Music: SDAce

Minimalist Sound Manager

Minimalist Sound Manager · revision 29210 · last edited 2025-09-30

A minimalist sound manager to make playing sounds easier without adding a whole library:

```lua
do
    -- will hold the currently playing sources
    local sources = {}

    -- check for sources that finished playing and remove them
    -- add to love.update
    function love.audio.update()
        local remove = {}
        for _,s in pairs(sources) do
            if s:isStopped() then
                remove[#remove + 1] = s
            end
        end

        for i,s in ipairs(remove) do
            sources[s] = nil
        end
    end

    -- overwrite love.audio.play to create and register source if needed
    local play = love.audio.play
    function love.audio.play(what, how, loop)
        local src = what
        if type(what) ~= "userdata" or not what:typeOf("Source") then
            src = love.audio.newSource(what, how)
            src:setLooping(loop or false)
        end

        play(src)
        sources[src] = src
        return src
    end

    -- stops a source
    local stop = love.audio.stop
    function love.audio.stop(src)
        if not src then return end
        stop(src)
        sources[src] = nil
    end
end
```

Now you can play sound effects and background music without worrying about the actual source objects:

```lua
function love.load()
    -- you will still get a source handle if you need it
    bgm = love.audio.play("background-music.ogg", "stream", true) -- stream and loop background music
end

function love.update(dt)
    love.audio.update()
end

function love.keypressed(key)
    if key == 's' then
        love.audio.stop(bgm)
    elseif key == 'p' then
        love.audio.play(bgm) -- still streaming and looping
    else
        love.audio.play("explosion.ogg") -- play explosion sound once
```

```
    end
end
```

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Möan.lua

Möan.lua · revision 25339 · last edited 2023-07-05

A messagebox system with multiple-choices and more.

```
Moan.speak("Title", {"Hello world!", "It's me;--Möan.lua!"})
```

GitHub: https://github.com/twentytwoo/Moan.lua ↗

Documentation: http://twentytwoo.github.io/Moan.lua/ ↗

Other Language

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Nano Space

Nano Space · revision 22312 · last edited 2021-03-14

Nano Space is a pixel space shooter with a lot of unique spaceships and missions.

You can get it on itch.io: https://luke100000.itch.io/nano-space ↗

nearest

nearest · revision 29386 · last edited 2024-12-03

Useful functions to find the nearest.

Nearest point in array of points

```lua
-- function to calculate the distance between two points
local function distance(x1, y1, x2, y2)
    return math.sqrt((x2 - x1)^2 + (y2 - y1)^2)
end

-- function to find the nearest point from a list of points
function findNearestPointInPoints (points, point)
-- points is array of pairs: {x1, y1, x2, y2, ...}
    local minDistance = math.huge
    local nearestX, nearestY = nil, nil
    for i = 1, #points, 2 do
        local x, y = points[i], points[i + 1]
        local dist = distance(x, y, point[1], point[2])
        if dist < minDistance then
            minDistance = dist
            nearestX, nearestY = x, y
        end
    end
    return nearestX, nearestY, minDistance
end
```

Nearest point on line segment

```lua
-- function to calculate the distance between two points
local function distance(x1, y1, x2, y2)
    return math.sqrt((x2 - x1)^2 + (y2 - y1)^2)
end

-- function to find the nearest point on a segment to the given point
function findNearestPointOnSegment(segment, point)
-- segment is array of pairs: {x1, y1, x2, y2}
    local x1, y1, x2, y2 = segment[1], segment[2], segment[3], segment[4]
    local px, py = point[1], point[2]

    -- vector from the first point of the segment to the second point
    local dx, dy = x2 - x1, y2 - y1
    -- vector from the first point of the segment to the point
    local dx1, dy1 = px - x1, py - y1
    -- segment length squared
    local segmentLengthSquared = dx * dx + dy * dy

    if segmentLengthSquared == 0 then
        -- degenerate segment (same point), return the start point
        return {x1, y1}, distance(px, py, x1, y1)
    end

    -- projection of the point onto the line (parameter t)
    local t = ((dx1 * dx + dy1 * dy) / segmentLengthSquared)

    -- clamp t to be between 0 and 1 to stay within the segment
    if t < 0 then
        t = 0
    elseif t > 1 then
        t = 1
    end

    -- calculate the nearest point on the segment
    local nearestX = x1 + t * dx
    local nearestY = y1 + t * dy

    -- return the nearest point and its distance from the given point
```

```
    return nearestX, nearestY, distance(px, py, nearestX, nearestY)
end
```

Nearest point on the polyline

A polyline is a broken line made up of a series of connected line segments.

```
function findNearestPointOnPolyline(polyline, point)
    -- polyline is array of pairs: {x1, y1, x2, y2, ...}
    local minDistance = math.huge
    local nearestX, nearestY = nil, nil

    for i = 1, #polyline - 2, 2 do
        -- get segment
        local segment = {polyline[i], polyline[i + 1], polyline[i + 2], polyline[i + 3]}
        -- find the nearest point on the current segment
-- function findNearestPointOnSegment(segment, point) is above
        local px, py, dist = nearestPointOnSegment(segment, point)

        -- update the minimum distance and nearest point
        if dist < minDistance then
            minDistance = dist
            nearestX, nearestY = px, py
        end
    end

    return nearestX, nearestY, minDistance
end
```

Nearest point on the circle

```
function nearestPointOnCircle(px, py, cx, cy, r)
    -- calculate the angle between the point and the center
    local theta = math.atan2(py - cy, px - cx)

    -- calculate the nearest point on the circle using the angle
    local nx = cx + r * math.cos(theta)
    local ny = cy + r * math.sin(theta)

    return nx, ny
end
```

newton

newton · revision 24603 · last edited 2023-07-05

Use the tools in this library to export Box2D bodies created using PhysicsEditor and load them into your LÖVE game.

Usage example:

```
local loader = require "loader"
local wdraw = require "wdraw"
local world = love.physics.newWorld()
local body
local x, y = 100, 100

function love.load()
    body = loader(world, "physics/exportedFile", "bodyName", x, y)
end

function love.update(dt)
    world:update(dt)
end

function love.draw()
    wdraw(world)
end
```

Links

- Bitbucket repo (https://bitbucket.org/tcadamson/new ton)
- PhysicsEditor (https://w w w.codeandw eb.com/physicseditor)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

nil

nil · revision 7940 · last edited 2025-09-30

From the Lua 5.1 reference manual §2.2 (http://www.lua.org/manual/5.1/manual.html#2.2):

Nil is the type of the value nil, whose main property is to be different from any other value; it usually represents the absence of a useful value.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Noahs Ark

Noahs Ark · revision 4500 · last edited 2021-03-14

Developed by TechnoCat
LÖVE: 0.7.0

Noahs's Ark is a Christian inspired game, but don't be fooled, I'm not a Christian. *Noahs's Ark* is an adventure platform game with a slight comical element.

Download: Forum post (http://love2d.org/forums/viewtopic.php?f=5&t=2341) HG
Repo: bitbucket.org (https://bitbucket.org/dannyfritz/noahs-ark/overview)



"Look at them animals."

Notepad++

Notepad++ · revision 15235 · last edited 2021-03-14

Notepad++ is a free source code editor and Notepad replacement that supports several languages. Based on the powerful editing component Scintilla and running in the MS Windows environment, its use is governed by GPL License.

Running love projects from Notepad++

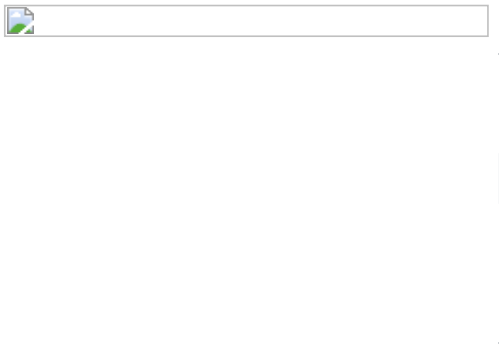
You can use Notepad++ to edit your löve code adding the following in the Run=>Run... command:

```
"c:\your\path\to\love.exe" "$(CURRENT_DIRECTORY)"
```

Just make sure the path and the variable are quoted.

Noteworthy Features

- Tabbed document interface (You can even have multiline tabs)
- Drag-and-drop
- Split screen editing and synchronized scrolling
- Find and replace over multiple documents, including regular expressions
- File comparison
- Zooming



Notepad++ with auto-complete plugin

Auto-completion for 0.7.2

With this xml your notepad++ can auto-complete löve functions.

UPDATED

Now supports up to 0.9.0 thanks to Nabakin

Instructions:

- Download the lua.xml and place it in your Notepad++/plugins/APIs/ directory.
- In Notepad++, go to Settings > Preferences in the menu.
- Select the Backup/Auto-Completion tab.
- Make sure "Enable auto-completion on each input" is checked.
- Set it to "Function completion."
- And check "Function parameters hint on input."

XML Download: [1] (https://mega.co.nz/#!w AoU2TRD!HSc-lHyygm6kNPG-qi2Nmgrl1bgdSX4Qql-0lLKrpCc)

See also

- AAuto-complete plugin with prototype for Notepad++ and LOVE 0.6.2 (http://love2d.org/forums/viewtopic.php?f=3&t=1575&start=0)
- Notepad++ Website (http://notepad-plus-plus.org/)

NULLSoft Scriptable Installer System (NSIS)

NULLSoft Scriptable Installer System (NSIS) · revision 11668 · last edited 2021-03-14

NSIS (Nullsoft Scriptable Install System) is a professional open source system to create Windows installers. It is designed to be as small and flexible as possible and is therefore very suitable for internet distribution.

Features:

- Low Size Overhead (only 34 KB)
- Compatible with Windows 95, 98, ME, NT, 2000, XP, Server 2003 Server 2008, Vista, and Windows 7
- Supports ZLib, BZip2, and LZMA compression
- Localisation support
- Creates Self-Contained and Self-Verifying Executable Installers
- Uninstall support
- Supports packages up to 2GB
- Optional silent mode
- Permissive licencing (mixed CPLv1, bZip2, zLib)

Links

NSIS (http://nsis.sourceforge.net/)

Download NSIS (http://nsis.sourceforge.net/Download)

number

number · revision 7939 · last edited 2025-09-30

From the Lua 5.1 reference manual §2.2 (http://w w w.lua.org/manual/5.1/manual.html#2.2):

Number represents real (double-precision floating-point) numbers.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Oh Snap, Aliens!

Oh Snap, Aliens! · revision 2794 · last edited 2021-03-14

Oh Snap, Aliens! is a space shooter with the highly unique and revolutionizing goal of defending earth from a seemingly unstoppable and neverending onslaught of alien attackers.

Unlike other games in the genre, *Oh Snap, Aliens!* doesn't give you unlimited ammunition. Instead, you have to conserve your missiles so that you don't run out right when there's a ufo in your sights. In return for this inconveniance, the ship has the ability to aim its weapons 180 degrees.

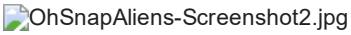
Development

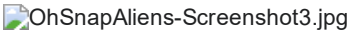
Version: 0.3

Status: Abandoned

LÖVE: 0.6.2

Screenshot





Download

The game can be downloaded from Github's Download section (http://github.com/Nevon/Oh-snap--Aliens-/dow nloads).

If you're looking for the latest development version, you can also grab that from Github (http://github.com/Nevon/Oh-snap--Aliens-).

See Also

- Developed by Nevon

On The Roadside

On The Roadside · revision 19440 · last edited 2021-03-14

On The Roadside is a turn-based strategy game in which you take control of a squad of mercenaries fighting for survival in a world shaped by unknown forces.



Released on itch.io (https://rmcode.itch.io/on-the-roadside)

Panel Attack

Panel Attack · revision 29463 · last edited 2025-02-21

Panel Attack is a free modern puzzle game inspired by popular games such as Tetris Attack and Pokemon Puzzle League while still maintaining authentic mechanics.

Arrange colored panels in rows and columns of three or more to match matches that clear. Panels then fall from gravity and can make chains that give bonuses or attack the other player.

Supports Windows/Mac/Linux and to some degree Android.

Website: Panel Attack Website (https://panelattack.com)

Discord: Panel Attack Discord (http://discord.panelattack.com)

GitHub: Panel Attack GitHub (https://github.com/panel-attack/panel-game)

Paper Evolution

Paper Evolution · revision 5228 · last edited 2021-03-14

Paper Evolution is puzzle game about erasing lines in a 2D platform environment so our hero is able to to advance through the ages and avoid extinction.

The game was made at Nordic Game Jam 2001 and it was 1 of 12 finalists chosen out of about 50 games.

Gameplay video (http://w w w.youtube.com/w atch?v=2FZ4U0goknQ)


PaperEvolution.png

More information in this forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=2410)

Download

Global Game Jam site (http://globalgamejam.org/2011/paper-evolution)

Pirate Bay

Pirate Bay · revision 12325 · last edited 2021-03-14

Pirate Bay

Pirate Bay is a physics based game done in 19 hours during the Devmania 2011 Overnight Contest. A Pirate airship is carrying a big barrel of rum on a rope and you have to use the barrel to smash cannons and airships shooting at it.

All code (~60k), graphics & voicesounds are created in only one Night!



Downloads for Windows/Linux/MacOSX

http://gamejams.schattenkind.net/2011/10/pirate-bay.html (http://gamejams.schattenkind.net/2011/10/pirate-bay.html)

Pirate Bay

Plant game

Plant game · revision 9902 · last edited 2021-03-14





(These screenshots are from a beta build - the graphics are unfinished)

The game

This (currently-unnamed) game is about crossbreeding plants with a simple genetics system - offspring will borrow and combine traits from both parents. The goal is to make a plant to certain specifications (such as, "breed a plant with red fruit), escape disease (this is a frantic, "survival" type of gameplay), or for the player's own enjoyment (sandbox mode). It's not complete, and is awaiting features which should be added in Love 0.8. A playable beta with several levels and game modes is available.

Forum thread (http://love2d.org/forums/viewtopic.php?t=2244)

Download

Direct from Dropbox (http://dl.dropbox.com/u/3713769/w eb/flow er/flow er.love)

Media

- An animation of the day/night cycle (greatly sped up) (http://dl.dropbox.com/u/3713769/w eb/flow er/love%202011-03-16%2010-11-30-43.gif)

PO2 Syndrome

PO2 Syndrome · revision 8138 · last edited 2023-07-05

PO2 Syndrome (power-of-two syndrome) refers to the problem older graphic cards/drivers have: they can't display graphics whose width and height are not a power of two, that is: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, …

There are multiple things lovers can do to support their poor afflicted users.

1. Only use PO2-sized image files. (One can add a transparent border for that end.)
2. Use an ImageData object to pad non-PO2-sized image files (see example).

Both have a downside: getting the width, height and center of the image does not work right, which can be especially wrong for rotation. The solution is to keep the *real* width and height around, and use them rather than getWidth() and getHeight().

Drop-in solution

Both for users and lovers, there exists a drop-in solution (for Linux only, at the moment), based on method #2. See http://love2d.org/forums/viewtopic.php?f=5&t=1433&p=16922#p16922.

It does not take the real-size problem, described above, into account.

RichText

Another solution, with limited scope, is the RichText library by Robin, only for 0.7.0. See GitHub (http://github.com/gvx/richtext) for more information

A cure has been found

Starting version 0.8.0, all images are auto-padded on systems that need it, this means we can forget about all this madness and celebrate.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Polygon splitting

Polygon splitting · revision 19071 · last edited 2021-03-14

This script is used for splitting **convex** polygons.

```
-- Usage:
--[[
table_of_polygons = polygoncut(cut_vertices,polygon_vertices)

polygons = polygoncut({x1,y1,x2,y2},{px,py,px1,py1,px2,py2,...})

for i,polygon in ipairs(polygons) do
   love.graphics.polygon('fill',unpack(polygon))
end
--]]
```

```
local insert= table.insert

local function cross(dx,dy,dx2,dy2)
    return dx*dy2-dy*dx2
end

local polygonCut = function(cvertices,pvertices)
    local plen     = #pvertices
    local x,y,x2,y2= unpack(cvertices)
    local dx,dy    = x2-x,y2-y
    local polygons = {{},{}}
    local set      = polygons[1]
    local intersections = 0
    for i = 1,plen,2 do
        local x3,y3,x4,y4= pvertices[i],pvertices[i+1],pvertices[i+2] or pvertices[1],pvertices[i+3] or pvertices[2]
        insert(set,x3) insert(set,y3)
        local dx2,dy2    = x4-x3,y4-y3
        local denom      = cross(dx,dy,dx2,dy2)
        if denom ~= 0 then
            local tnum = cross((x3-x),(y3-y),dx2,dy2)
            local unum = cross((x3-x),(y3-y),dx,dy)
            local ct,pt= tnum/denom,unum/denom
            if ct >= 0 and ct <= 1 and pt >= 0 and pt <= 1 then
                local ix,iy = x+ct*dx,y+ct*dy
                if pt ~= 1 then
                    if pt ~= 0 then
                        insert(set,ix) insert(set,iy)
                    end
                    set = set == polygons[1] and polygons[2] or polygons[1]
                    insert(set,ix) insert(set,iy)
                    intersections = intersections+1

                end
            end
        end
    end
    if intersections == 2 then return polygons end
end

return polygonCut
```

polypoint

polypoint · revision 29387 · last edited 2024-12-03

Polygon Point Collision Detection

here is the original love2d forum thread: [1] (https://love2d.org/forums/viewtopic.php?f=4&p=233792#p233792)

here is the original source website, beware its written in C++ [2] (http://w w w.jeffreythompson.org/collision-detection/poly-point.php)

The code below is a function that is called to return true when a point is within the vertices of a polygon and false otherwise. The math used is the Jordan Curve Theorem --> [3] (https://en.w ikipedia.org/w iki/Jordan_curve_theorem)

```
function polyPoint(vertices,px,py)
local collision = false
local next = 1
 for current = 1, #vertices do
   next = current + 1
   if (next > #vertices) then
     next = 1
   end
   local vc = vertices[current]
   local vn = vertices[next]
   if (((vc.y >= py and vn.y < py) or (vc.y < py and vn.y >= py)) and
   (px < (vn.x-vc.x)*(py-vc.y) / (vn.y-vc.y)+vc.x)) then
       collision = not(collision)
   end
 end
 return collision
end
```

Note: The code assumes your polygon vertices are stored in a table with tuples or pairs of x and y like this.

```
v = {}
v[1] = {x = 200,y = 100}
v[2] = {x = 400,y = 130}
v[3] = {x = 350,y = 300}
v[4] = {x = 250,y = 300}
```

You can find the .love here for an example. Love Version 11.3 [4] (https://love2d.org/forums/download/file.php?id=18245)

profile

profile · revision 29352 · last edited 2024-10-09

profile.lua

profile.lua is a small module used for finding bottlenecks in your Lua code. To use the profiler, you need to require the profile.lua file and specify when to start or stop collecting data. Once you are done profiling, a report is generated, describing:

- which functions were called most frequently and
- how much time was spent executing each function

Example

```
-- setup
function love.load()
  love.profiler = require('profile')
  love.profiler.start()
end

-- generates a report every 100 frames
love.frame = 0
function love.update(dt)
  love.frame = love.frame + 1
  if love.frame%100 == 0 then
    love.report = love.profiler.report(20)
    love.profiler.reset()
  end
end

-- prints the report
function love.draw()
  love.graphics.print(love.report or "Please wait...")
end
```

Documentation and source code

https://github.com/2dengine/profile.lua ↗

https://2dengine.com/doc/profile.html ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Pröxima

Pröxima · revision 11677 · last edited 2021-03-14

Proxima-Logo.jpg

Pröxima is a 2D space strategy game. It takes the player on an epic journey nearby the planet Pröxima. The epic theme/story of the Homeworld series (http://en.w ikipedia.org/w iki/Homew orld) (developed by Relic Entertainment)inspired the game.

Story

Shortly after the colonization of Mars, mankind discovered the knowledge to travel at light speed. A new era of exploration began...

'Pröxima' was quickly chosen as the first destination outside the Solar System. It was a small, uninhabited planet orbiting the star 'Proxima Centauri' in the 'Centauri 'constellation. The people of Earth combined their efforts and started the construction of the largest spacecraft ever built… 'The Galician'. It would take nearly a decade to complete it.

'The Galician', with more than thousand people on board, embarked and began his journey on the 2nd of April 2085. After 2 years and 276 days travelling in space, the first human recon shuttle lands on Pröxima…

A day that will change mankind forever…

Development

Version: Version 0.2 (unreleased)

Status: Work in Progress (Currently On Hold)

LÖVE: 0.6.x

Screenshot

Proxima-Screenshot.jpg

See Also

- Developed by Virox

psi2love

psi2love · revision 19088 · last edited 2021-03-14

psi2love.cpp is a small C++ program which takes a HGE psi file and outputs Lua source code to create an equivalent particle system in LÖVE.

```
#include <stdio.h>
#include <stdint.h>
#include <errno.h>
#include <string.h>
#include <math.h>

struct hgeColor
{
    float       r,g,b,a;
};

struct hgeParticleSystemInfo
{
    uint32_t sprite; // hgeSprite*  sprite;    // texture + blend mode
    int32_t nEmission; // int           nEmission; // particles per sec
    float       fLifetime;

    float       fParticleLifeMin;
    float       fParticleLifeMax;

    float       fDirection;
    float       fSpread;
    int32_t bRelative; // bool      bRelative;

    float       fSpeedMin;
    float       fSpeedMax;

    float       fGravityMin;
    float       fGravityMax;

    float       fRadialAccelMin;
    float       fRadialAccelMax;

    float       fTangentialAccelMin;
    float       fTangentialAccelMax;

    float       fSizeStart;
    float       fSizeEnd;
    float       fSizeVar;

    float       fSpinStart;
    float       fSpinEnd;
    float       fSpinVar;

    hgeColor    colColorStart; // + alpha
    hgeColor    colColorEnd;
    float       fColorVar;
    float       fAlphaVar;
};

int main( int argc, const char* argv[] )
{
    if ( argc != 5 )
    {
        fprintf( stderr, "USAGE: psi2love filename.psi func_name image_name max_particles\n" );
        return 1;
    }

    FILE* psi = fopen( argv[ 1 ], "rb" );
    if ( psi == NULL )
    {
        fprintf( stderr, "Error: %s\n", strerror( errno ) );
        return 1;
    }
```

```
    hgeParticleSystemInfo info;

    if ( fread( &info, 1, sizeof( info ), psi ) != sizeof( info ) )
    {
        fprintf( stderr, "Error: %s\n", strerror( errno ) );
        fclose( psi );
        return 1;
    }

    fclose( psi );

    printf( "local function %s()\n", argv[ 2 ] );
    printf( "  -- created from %s\n", argv[ 1 ] );
    printf( "\n" );
    printf( "  local ps = love.graphics.newParticleSystem( %s, %s )\n", argv[ 3 ], argv[ 4 ] );
    printf( "\n" );

    // nEmission
    printf( "  ps:setEmissionRate( %d )\n", info.nEmission );

    // fLifetime
    printf( "  ps:setLifetime( %g )%s\n", info.fLifetime, info.fLifetime == -1.0 ? " -- forever" : "" );

    // fParticleLifeMin, fParticleLifeMax
    printf( "  ps:setParticleLife( %g, %g )\n", info.fParticleLifeMin, info.fParticleLifeMax );

    // fDirection
    printf( "  ps:setDirection( %g )\n", info.fDirection - M_PI / 2 );

    // fSpread
    printf( "  ps:setSpread( %g )\n", info.fSpread );

    // bRelative
    printf( "  -- ps:setRelative( %s )\n", info.bRelative ? "true" : "false" );

    // fSpeedMin, fSpeedMax
    printf( "  ps:setSpeed( %g, %g )\n", info.fSpeedMin, info.fSpeedMax );

    // fGravityMin, fGravityMax
    printf( "  ps:setGravity( %g, %g )\n", info.fGravityMin, info.fGravityMax );

    // fRadialAccelMin, fRadialAccelMax
    printf( "  ps:setRadialAcceleration( %g, %g )\n", info.fRadialAccelMin, info.fRadialAccelMax );

    // fTangentialAccelMin, fTangentialAccelMax
    printf( "  ps:setTangentialAcceleration( %g, %g )\n", info.fTangentialAccelMin, info.fTangentialAccelMax );

    // fSizeStart, fSizeEnd, fSizeVar
    printf( "  ps:setSize( %g, %g ) -- there's a bug in 0.7.1 that forces us to set the size variation using its own function\n",
info.fSizeStart, info.fSizeEnd );
    printf( "  ps:setSizeVariation( %g )\n", info.fSizeVar );

    // fSpinStart, fSpinEnd, fSpinVar
    printf( "  ps:setSpin( %g, %g, %g )\n", info.fSpinStart, info.fSpinEnd, info.fSpinVar );

    // colColorStart, colColorEnd
    printf(
        "  ps:setColor( %d, %d, %d, %d, %d, %d, %d, %d )\n",
        (int)(info.colColorStart.r * 255), (int)(info.colColorStart.g * 255), (int)(info.colColorStart.b * 255), (int)(info.colColorStart.a
* 255),
        (int)(info.colColorEnd.r * 255), (int)(info.colColorEnd.g * 255), (int)(info.colColorEnd.b * 255), (int)(info.colColorEnd.a * 255)
    );

    // fColorVar
    printf( "  -- ps:setColorVariation( %g )\n", info.fColorVar );

    // fAlphaVar
    printf( "  -- ps:setAlphaVariation( %g )\n", info.fAlphaVar );

    printf( "\n" );
    printf( "  return ps\n" );
    printf( "end\n" );
```

```
    return 0;
}
```

To compile with MinGW, just type "g++ -o psi2love psi2love.cpp" on the prompt.

The bRelative, fColorVar and fAlphaVar members of HGE's particle system structure don't have counterparts in LÖVE so I generate them in comments.

Just make sure to change the parameters to love.graphics.newParticleSystem in the generated function. I've tested with a couple of emitters created with HGE's particle editor and the results were satisfactory.

Have fun!

PÄSSION

PÄSSION · revision 2728 · last edited 2021-03-14

Not much here yet.

Need Help Running PÄSSION?

Try Getting Started with PÄSSION.

See Also

- GitHub (http://w iki.github.com/kikito/passion)
- LÖVE Forum Thread (http://love2d.org/forums/viewtopic.php?f=5&t=1039)

Pölygamy

Pölygamy · revision 24487 · last edited 2023-07-05

Pölygamy is a small LÖVE framework.

Currently, a state and a keyboard modules are available, and a timer module is in the works. Each module can be used in isolation, but they are designed to be complementary.

Modules:

Polygamy.keyboard

- allows to bind keys to callbacks in a declarative fashion,
- supports multiple, switchable configurations
- supports configuration (single) inheritance.
- supports useful ranges ([f-j] is expanded to f, g, h, j) and other patterns inspired by the Lua patterns library.
- allows to treat the keyboard as a key matrix (full support for US qwerty keyboards only at the moment, but the common subset of qwerty, azerty and qwertz can be used universally (sorry, dvorak users))
- is freindly with both imperative and OO coding styles.

Polygamy.state

- allows to design each game state as if it were a full LÖVE app (custom update and draw callbacks),
- supports state transitions (through before(from, to, ...) and after(from, to, ...) callbacks)

See this forum post (http://love2d.org/forums/viewtopic.php?f=5&t=1240) for the latest version and the (extensive) documentation.

Here's an dummy app written with Pölygamy:

```
love.filesystem.load("library/Polygamy/init.lua")("library/Polygamy/") -- must be outside the load callback.
                                            -- since it creates a custom love.run loop

local state, goto, pkeyboard = Polygamy.state, Polygamy.state.goto, Polygamy.keyboard   -- for convenience
-- load your ressources, initialize your code... then :


--------------------------------------------------------------------------------

title = state( "Title Screen" )
              --============--

function title.before() pkeyboard.use( "title" ); print"menu" end
function title.update(dt)  --[[ your title update callback ]] end
function title.draw()      --[[ draw your fine title here  ]] end

pkeyboard( "title" ):setConfig( "pressed", {
    [{" ", "return"}] = function() goto("In Game")       end,
    escape            = function() love.event.push('q') end,
    [Polygamy.default] = print
})


--------------------------------------------------------------------------------

ingame = state( "In Game" )
               --========--

function ingame.before( from, to, ... )
    if from == "Title Screen" then --[[Game.state.reset()]] end
    Polygamy.timer:resume()
    pkeyboard.use( to ) -- to == "In game" in this case
    print "In Game"
end

function ingame.update( dt ) --[[ instill life in your game world ]] end

function ingame.draw()   --[[ and render it in its full glory ]] end

function ingame.after( from, to, ... )
```

```
    Polygamy.timer:pause()
    if to == "Pause Screen" then
        -- dim the screen, take a screenshot
    end
end



pkeyboard("In Game"):setConfig("pressed", {

    [" %arrows "] = { held = function(key,_,_,dt) Hero:move(key,dt); end },
    [" "]  = function() Hero:shoot()           end,
    escape = function() goto( "Title Screen" ) end,
    p      = function() goto( "Pause Screen" ) end
})


--------------------------------------------------------------------------------

pause = state( "Pause Screen" )
             --------------
function pause.before() pkeyboard.use( "pause" ); print"Pause" end
function pause.update(dt) end -- if you don't define it, the previous callback remains active.
function pause.draw() --[[ display "Paused" on top of the screenshot.]] end


pkeyboard( "pause" ):setConfig( "pressed", {
    escape = function() goto( "Title Screen" ) end,
    p      = function() goto( "In Game" )      end
})

-- then get the ball rolling :-)

function love.load()
    print"Start"
    goto("Title Screen")
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Remove Workaround

Remove Workaround · revision 4935 · last edited 2023-07-05

Workaround for a Physics module issue

There is an unresolved issue (http://love2d.org/forum/tracker.php?p=1&t=191) with the third callback of World:setCallbacks, the remove callback. If there are active contacts when a shape is destroyed, the next physics update will cause a crash when LÖVE attempts to call the callback. This issue appears at least in LÖVE version 0.6.2.

To work around this problem, a shape must first leave from all the active contacts before it can be destroyed safely. This can be done by moving the shape into a special area, or more simply by setting the shape to not collide with anything.

```
-- remove a shape from all contact categories
shape:setMask(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16)
-- add it into the pending destroy list
table.insert(removals, shape)
-- either remove the shape from the list of drawn objects, or mark
-- it with a flag so it won't be no longer drawn
```

After this is done, the shape will no longer have an affect on the world. After the next physics update is performed, it will also have lost all of its contacts, and can be safely destroyed.

```
-- world update happens usually in love.update callback
world:update(dt)
-- you can follow it by destroying any shaped listed to be removed
for _, shape in ipairs(removals) do
    shape:destroy()
end
```

Remember that updating the world before removing the shape makes sure it will no longer be in contact with anything.

See also

- Shape:setMask
- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

require

require · revision 31379 · last edited 2026-04-15

Opens and executes Lua modules.

Use periods to seperate folders. If the module is a lua file, don't use the .lua extension in the string passed to `require`.

```
require("foo")
require("subfolder.bar")
```

LÖVE adds two loaders (before the Lua loaders) that search for modules in the game and save directory. These loaders are not affected by Lua's `package.path`. The paths can be altered with love.filesystem.setRequirePath for Lua modules and love.filesystem.setCRequirePath for C modules, the latter only work on files located at save directory.

Notes

It's strongly recommended to use period as directory separator! Forward slashes *accidently* work, hence its usage is discouraged! One of the reasons is that loading modules that are not Lua code files is only possible with dots as separator, and using dots and slashes at the same time might lead to mixing separators, and due to mixing require might load the same code multiple times.

You may encounter problems with using functions from required modules. Such as some/all functions being nil. To fix this, make sure all function names are matched up correctly, and that the required file itself is not named any built-in Lua module (such as, "debug," "string," or "math"), calling require on any of these would load the built-in Lua module instead of the file. If that is the problem, simply rename the file to something else (e.g. "my_debug").

See Also

- require on the Lua manual. (http://w w w.lua.org/manual/5.1/manual.html#pdf-require)
- Programming in Lua. (http://w w w.lua.org/pil/8.1.html)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RL-Dice

RL-Dice · revision 24419 · last edited 2023-07-05

This is a general purpose dice library that was built for roguelikes. Of course it can be utilized by other games for dice rolls. The code is currently hosted on Github (https://github.com/Codex-Dev/RL-Dice)

Questions/Bugfixes/Suggestions

Please refer to the forum post (http://love2d.org/forums/viewtopic.php?f=5&t=42163) if you have any issues.

Documentation

The documentation is located here. (https://github.com/Codex-Dev/RL-Dice/blob/master/README.md)

Return to

- Libraries

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Roleplaying Game

Roleplaying Game · revision 4595 · last edited 2021-03-14

Role-playing games, or RPGs, tend to belong to one of three categories. For more information, see the Wikipedia (https://secure.wikimedia.org/wikipedia/en/wiki/Role-playing_video_game).

Western RPGs

Emphasize real-time combat. Characters have stats and skills which are improved by gaining experience through fighting or progressing through the story. Generally feature lots of useable weapons and other randomized loot. Common settings include medieval, futuristic, and cyberpunk.

Japanese RPGs (JRPGs)

Almost always feature turn-based combat. Graphics are in the Anime style, and the protagonists are usually teenagers out to save the world from an ancient evil. Their settings almost always either feudal Japan or super-high-tech future/space.

"True" RPGs

These games are about playing a character in a story - combat may not even be present. They generally have branching storylines, multiple endings, and lots of choices with later consequences. This is the least-common form of RPG.

rstar

rstar · revision 26826 · last edited 2023-07-05

rstar is a OOP implementation of the R*Tree, can be used to store rectangles in a very efficient way.

Features

- **insert** a box in the tree;
- **delete** a box from the tree;
- **search** all boxes in an area;
- **select** all boxes at a given position;
- **draw** the tree's structure for debug purposes (LOVE 0.7 is required)

Links

- Get the library here (https://github.com/rick4stley/rstar/)
- Read me (https://github.com/rick4stley/rstar/blob/main/README.md)
- Official forum topic (https://love2d.org/forums/viewtopic.php?f=5&t=91782)

Scäle

Scäle · revision 6857 · last edited 2021-03-14

Scäle is a library that scales graphics to look right at any resolution. Graphics are always the same distance from the center of the screen no matter what the horizontal resolution, and things zoom in or out based on the vertical resolution.

To use Scäle, replace love.graphics calls with Scäle's counterparts. Draw graphics where you would want them in an 800x600 window.

The functions currently included:

scale_circle(mode, x, y, radius, segments)

scale_rectangle(mode, x, y, width, height)

scale_draw(drawable, x, y, r, sx, sy, ox, oy)

scale_line(x1, y1, x2, y2, ...)

scale_point(x, y)

scale_polygon(mode, x1, y1, x2, y2, ...)

scale_triangle(mode, x1, y1, x2, y2, x3, y3)

All arguments are the same as their love.graphics counterparts.

Download the library here! http://tesselode.com/files/libraries/scale.lua ↗

SELÖVE

SELÖVE · revision 6250 · last edited 2021-03-14

A sandboxed version of LÖVE.

See also

- Home page (http://gvx.github.com/selove.html)
- Bitbucket (https://bitbucket.org/gvx/selove/overview)
- UbuntuSecurity

sequence

sequence · revision 14602 · last edited 2025-10-01

From the Lua 5.2 reference manual §2.1 (http://w w w.lua.org/manual/5.2/manual.html#2.1):

We use the term sequence to denote a table where the set of all positive numeric keys is equal to {1..n} for some integer n, which is called the length of the sequence (see §3.4.6 (http://w w w.lua.org/manual/5.2/manual.html#3.4.6)).

```
sequence = { 'foo', 'bar', 'baz', 'qux', --[[ ... ]] }

-- or

sequence = {
    [1] = 'foo',
    [2] = 'bar',
    [3] = 'baz',
    [4] = 'qux',
    -- [n] = ...,
}
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader Variables

Shader Variables · revision 28164 · last edited 2025-10-22

On this page

- Global built-in variables
- Vertex Shader built-in variables
- Pixel Shader built-in variables
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

These built-in shader variables are not supported in earlier versions.

There are several built-in variables LÖVE provides in vertex and pixel shaders. All built-in variables are read-only unless otherwise specified.

Global built-in variables

`mat4 TransformMatrix`

The transformation matrix affected by love.graphics.translate and friends. Note that automatically batched vertices are transformed on the CPU, and their TransformMatrix will be an identity matrix.

`mat4 ProjectionMatrix`

The orthographic projection matrix.

`mat4 TransformProjectionMatrix`

The combined transform and projection matrices. Used as the `transform_projection` argument to the vertex shader position function.

`vec4 VaryingTexCoord`

The interpolated per-vertex texture coordinate. Automatically set to the value of `VertexTexCoord` in the vertex shader before the position function is called. Used as the `texture_coords` argument to the pixel shader effect function. Writable in the vertex shader. Use this variable to change the texture coordinate in the vertex shader.

`vec4 VaryingColor`

The interpolated per-vertex color. Automatically set to the value of `ConstantColor * gammaCorrectColor(VertexColor)` in LÖVE 0.10.0 and newer, or `VertexColor` in 0.9.2 and older, in the vertex shader before the position function is called. Used as the `color` argument to the pixel shader effect function. Writable in the vertex shader. Use this variable to change the per-vertex or constant color in the vertex shader.

Available since LÖVE 0.9.1

This shader variable is not supported in earlier versions.

`vec4 love_ScreenSize`

The width and height of the screen (or canvas) currently being rendered to, stored in the x and y components of the variable. The z and w components are used internally by LÖVE. You can convert it to a vec2 with `love_ScreenSize.xy` or `vec2(love_ScreenSize)`.

Vertex Shader built-in variables

`vec4 VertexPosition`

The pre-transformed position of the vertex. Used as the `vertex_position` argument to the vertex shader position function. The third and fourth components of the vector are normally (0, 1).

`vec4 VertexTexCoord`

The texture coordinate of the vertex. The third and fourth components of the vector are normally (0, 1). Meshes allow for custom texture coordinates.

`vec4 VertexColor`

The color of the vertex, sprite, or text character if a Mesh, SpriteBatch, or Text object with per-vertex colors is drawn, or in LÖVE 0.9.2 and older the global color set with love.graphics.setColor. It does not have gamma-correction applied.

Available since LÖVE 0.10.0

This shader variable is not supported in earlier versions.

`vec4 ConstantColor`

The global color set with love.graphics.setColor. If global gamma-correction is enabled, it will already be gamma-corrected.

Pixel Shader built-in variables

`vec4 array love_Canvases[]`

Array used to set per-canvas pixel colors when multiple canvases are set with love.graphics.setCanvas and the `void effect` variant is used instead of the `vec4 effect` variant of the function. Note that arrays in shaders are 0-based. Writable in the pixel shader when the **void effect** variant is used.

`vec2 love_PixelCoord`

Coordinates of the pixel on screen. The same as `screen_coords` passed to the `vec4 effect` Shader function

Notes

If you wish to access the texture used for the drawing operation, you may define a uniform called `MainTex` of the appropriate type yourself, e.g. `uniform Image MainTex;`

See Also

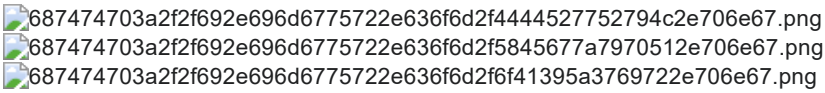
- Shader
- love.graphics.newShader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shädows

Shädows · revision 24413 · last edited 2023-07-05


687474703a2f2f692e696d6775722e636f6d2f4444527752794c2e706e67.png
687474703a2f2f692e696d6775722e636f6d2f5845677a7970512e706e67.png
687474703a2f2f692e696d6775722e636f6d2f6f41395a3769722e706e67.png

Shädows is a dynamic light engine for löve.
More information about this engine can be seen here.
https://github.com/Starkkz/shadows/wiki ↗

The code repository is here.
https://github.com/Starkkz/shadows ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Simple Educative Class Library

Simple Educative Class Library · revision 15223 · last edited 2023-07-05

Also known as SECL. This is a simple implementation of classes that works, it's not commented, so it's a nice test of your lua skills.

The current version can always be found in the love-misc-libs repository (https://github.com/bartbes/love-misc-libs)

Basic version example

```lua
require "class" --this assumes you've saved the code above in class.lua

myclass = class:new()
myclass.value = 13
function myclass:setvalue(v)
    self.value = v
end
object = myclass:new()
object:setvalue(128)
--myclass.value is still 13 and object.value is now 128
anotherObject = object:new()
--anotherObject.value is 128
```

Advanced version example

```lua
--NOTE: this example only contains features the basic version doesn't, if you want to see those functions see the basic example
--require it here

--create the tables/classes
cl1 = class:new()
cl2 = class:new()
cl3 = {}

--fill the tables/classes
cl1.val1 = 12
cl2.val2 = 24
cl3.val3 = 48

--create the new merged class
merged = cl1:new()
print(merged.val1, merged.val2, merged.val3) --> 12  nil  nil
merged:addparent(cl2)
print(merged.val1, merged.val2, merged.val3) --> 12  24  nil
--remember cl3 is a normal table, not a class?
class:convert(cl3) --converts the table in place and returns it, so you can use it in expressions
merged:addparent(cl3) --could also be merged:addparent(class:convert(cl3))
print(merged.val1, merged.val2, merged.val3) --> 12  24  48

--convert is part of any class, the parent of the converted class is the class where convert is called from
--NOTE: addparent supports multiple parents at once, the way that it's done here is to serve as an example
```

Full version example

```lua
--NOTE: This example only contains features the advanced version doesn't have
--require it here

cl1 = class() --new is implied
cl2 = class()

cl1.val1 = 12
cl2.val2 = 24

merged = cl1() + cl2 --yes, we can just add cl2 as if we were doing math
print(merged.val1, merged.val2) --> 12  24

if merged > cl1 then
    print("Merged is a derivative of cl1") --which it is, so this is printed
```

```
end
if merged > cl2 then
    print("Merged is a derivative of cl2") --it's that as well, so print
end

merged2 = cl1() + cl2  --create another derivative with the same parents
merged2.val1 = 36  --we change one value

if merged == merged2 then
    print("Merged2 is of the same type as merged") --they are the same type, note this only works when changing, when you add or remove values it is considered as a new class
end
```

Revised version example

Same as full.

See also

Bartbes, creator and maintainer of SECL.

Skip list

Skip list · revision 25575 · last edited 2021-03-14

The **skip list** (http://en.wikipedia.org/wiki/Skip_list) is a nifty tool to implement a time line, a Z-index for sprites or other tasks where a data structure with fast insert and delete operations is needed, rather than one with fast random access.

Warning, hardcore paragraph: an (indexable) skip list is an ordered, probabilistic data structure with O(log(n)) insertions, deletions and search, and O(n*log(n)) space usage. Their main advantage over binary trees is that they're very easy to implement ~= 150 LOC in Lua.

The only restriction is that, since it's an ordered data structure, its elements have to be comparable with < and <=.

Here's a Lua implementation based on this python version (http://code.activestate.com/recipes/576930/).

```
-- file: skiplist.lua

--[[------------------------------------------------------------------
The MIT License

Original Python version Copyright (c) 2009 Raymond Hettinger
see http://code.activestate.com/recipes/576930/

Lua conversion + extensions Copyright (c) 2010 Pierre-Yves Gérardy

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
THE SOFTWARE.
--]]------------------------------------------------------------------

local log, floor, ceil, min, random
= math.log, math.floor, math.ceil, math.min, math.random

local makeNode = function(value,size)
    return {
        value=value,
        next={},
        width={},
        size=size
    }
end

local End ={}
local NIL = makeNode(End,0)

local insert = function(self,value)
    local node, chain, stepsAtLevel = self.head, {}, {}
    for i=1, self.maxLevel do stepsAtLevel[i]=0 end
    for level = self.maxLevel, 1, -1 do
        while node.next[level] ~= NIL and node.next[level].value <= value do
            stepsAtLevel[level] = ( stepsAtLevel[level] or 0 ) + node.width[level]
            node = node.next[level]
            --print(level, stepsAtLevel[level],value)
        end
        chain[level]=node
    end

    local nodeLevel = min( self.maxLevel, - floor(log(random()) / log(2) ) )
    local newNode = makeNode( value,  nodeLevel)
    local steps, prevNode = 0
```

```
    for level= 1, nodeLevel do
        prevNode = chain[level]
        newNode.next[level] = prevNode.next[level]
        prevNode.next[level] = newNode
        newNode.width[level] = prevNode.width[level] - steps
        prevNode.width[level] = steps + 1
        steps = steps + stepsAtLevel[level]
    end
    for level = nodeLevel + 1, self.maxLevel do
        chain[level].width[level] = chain[level].width[level] +1
    end
    self.size = self.size + 1
end

local delete = function(self,value)
    -- find first node on each level where node.next[levels].value >= value

    node, chain = self.head, {}
    for level = self.maxLevel, 1, -1 do
        while node.next[level] ~= NIL and node.next[level].value < value do
            node = node.next[level]
        end
        chain[level] = node
    end
    if value ~= chain[1].next[1].value then
        return nil, "value not found: "..value
    end

    -- remove one link at each level
    nodeLevel = chain[1].next[1].size
    for level = 1, nodeLevel do
        prevnode = chain[level]
        prevnode.width[level] = prevnode.width[level] + prevnode.next[level].width[level] - 1
        prevnode.next[level] = prevnode.next[level].next[level]
    end
    for level = nodeLevel+1, self.maxLevel do
        chain[level].width[level] = chain[level].width[level] - 1
    end
    self.size = self.size - 1
    return true --success
end


local first = function(self)
    return self.head.next[1].value
end

local pop=function (self)
    if self.size == 0 then return nil, "Trying to pop an empty list" end

    local node, head = self.head.next[1], self.head
    for level = 1, node.size do
        head.next[level]=node.next[level]
        head.width[level]=node.width[level]
    end
    for level = node.size + 1, self.maxLevel do
        head.width[level] = head.width[level] -1
    end
    self.size = self.size - 1
    return node.value
end

-- get the value of the node at index i ( O( log( n ) ) )

local tostring = function (self)
    local t = {}
    for k,v in self:ipairs() do table.insert(t,v) end
    return "( "..table.concat(t,", ").. " )"
end


local islMT = {
    __index = function(self,i)
```

```
        if type(i) ~= "number" then return end
        if i > self.size then return end
        local node = self.head

        for level=self.maxLevel, 1, -1 do
            while node.width[level] <= i do
                i = i - node.width[level]
                node = node.next[level]
            end
        end
        return node.value
    end,
    __tostring=tostring
}


local ipairs = function (self)
    local node, size = self.head.next[1] , self.size
    count = 0
    return function()
        value=node.value
        node = node.next[1]
        count = count+1
        return count <= size and count or nil, value
    end
end

local function new (expected_size)
    local size = expected_size or 16
    if not expected_size then
        expected_size = 16
    end

    local maxLevel = floor( log(expected_size) / log(2) )
    local head = makeNode("HEAD",maxLevel)
    for i=1,maxLevel do
        head.next[i] = NIL
        head.width[i] = 1
    end

    return setmetatable( {
        size = 0,
        head = head,
        maxLevel = maxLevel,
        insert = insert,
        delete = delete,
        first = first,
        tostring = tostring,
        ipairs=ipairs,
        pop = pop
        }, islMT
    )
end

return {
    new=new
}
```

The API by example:

```
local skiplist = require"skiplist"

math.randomseed(os.time())

-- Creation:

insdel = skiplist.new(8) -- you must pass an estimation of the length of the list to get optimal performance.

-- insertions
s = "YeeHoyeE"
for i=1,#s do
```

```
     insdel:insert(s:sub(i,i))
    print(insdel)
end
print()

--  indexing
print(insdel[4])
print(insdel[8])
print(insdel[12]) -- out of bounds --> nil
print()

-- iterate over the list.
for k,v in insdel:ipairs() do
    print(k,v)
end
print()

-- remove elements
print(insdel:delete"Z") --> not found

for i=1,#s do
    insdel:delete(s:sub(i,i))
    print(insdel)
end

-- alternate insertion syntax, pop() and first()
insdel:insert("e")
insdel[0]=("g")

print( insdel:first() ) -- returns the first element.
print( insdel:pop() ) -- returns the first element and removes it from the list
print( insdel:first() )
print( insdel.size )
print( insdel:pop() )
print( insdel:pop() ) -- attempt to pop an empty list --> nil + error message.
print()

------------------------------
-- ASCII representation of the structure:

-- Uncomment the _End assignment at the penultimate line
-- of the previous code section for this to work.

-- This is required because the following code goes
-- under the hood to show the inner structure of the list.
-- This isn't needed for normal use of the library.


l = skiplist.new(9)
s = "SweetLOVE"

for i=1,#s do local e = s:sub(i,i) l:insert(e) end

for level = l.maxLevel, 1, -1 do
    local line='Level '..level..": "
    node = l.head

    while node.value ~= _End do
        line=line..node.value..node.width[level].." "
        for i=1,node.width[level]-1 do line=line..'   ' end
        node = node.next[level]
    end
    print(line.."NIL")
end
```

Output:

```
( Y )
( Y, e )
( Y, e, e )
( H, Y, e, e )
```

```
( H, Y, e, e, o )
( H, Y, e, e, o, y )
( H, Y, e, e, e, o, y )
( E, H, Y, e, e, e, o, y )

e
y
nil

1       E
2       H
3       Y
4       e
5       e
6       e
7       o
8       y

nil     value not found: Z
( E, H, e, e, e, o, y )
( E, H, e, e, o, y )
( E, H, e, o, y )
( E, e, o, y )
( E, e, y )
( E, e )
( E )
(  )
e
e
g
1
g
nil     Trying to pop an empty list


Level 3: HEAD4          S6                NIL
Level 2: HEAD1 E3       S1 V2    e2    w1 NIL
Level 1: HEAD1 E1 L1 O1 S1 V1 e1 e1 t1 w1 NIL
```

Skip list:Drawing Order

Skip list:Drawing Order · revision 19074 · last edited 2021-03-14

Z-ordering in LÖVE is tedious because the engine doesn't offer any support for it. You must draw the sprites in order by yourself, making it hard to handle a large number of sprites on different planes.

With the skip list data structure, you can do:

```
zlist = makeSkipList(3) -- 3 is the estimated size of the structure.
                        -- It's indicative, but a good guess improves the performance


Clown = makeSprite (100, 100, 1, "clown.png") -- x, y, z, image
Frogs = makeSprite (100, 110, 1, "frogs.png") -- on the same plane as the Clown, but drawn on top because it's been created after.
Tools = makeSprite (90,  120, 0, "tools.png") -- under the clown


zlist:insert( Frogs )
zlist:insert( Tools )
zlist:insert( Clown ) -- You can insert them in any order

for _, sprite in zlist:iter() do
    -- draw the sprite in Z order :-)
    print( sprite.x, sprite.y, sprite.z, sprite.image )
end
```

Output:

```
90      120     0       tools.png
100     100     1       clown.png
100     110     1       frogs.png
```

If you want to change the position of an item on the Z axis, you remove it and insert it back.

```
Tools.z = 2
zlist:delete(Tools)
zlist:insert(Tools) -- now Tools is on top of the pile

for _,sprite in zlist:iter() do
    print(sprite.x, sprite.y, sprite.z, sprite.image) -- draw(sprite)
end
```

Here's the output:

```
100     100     1       clown.png
100     110     1       frogs.png
90      120     2       tools.png
```

Toy implementation of makesprite:

```
do
local count = 0
function counter()
    count = count + 1
    return count
end
end


do
local lt = function(a,b)
    if a.z < b.z then return true
    elseif a.z==b.z and a.id < b.id then return true
    else return false
    end
end
```

```
local mt={
    __lt=lt,
    __le=lt
}
function makeSprite (x,y,z,image)
    local s={x = x, y = y, z = z, image=image, id = counter()}
-- the id field is necessary because, otherwise, items on the same plane would cause troubles.
    return setmetatable(s,mt)
end

end
```

Skull Basher

Skull Basher · revision 29607 · last edited 2025-04-28

This is a game about running, dodging and shooting stuff. Blast your way through different areas and defeat the evil wizard to win.



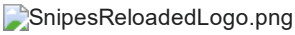
Features

- 9 different areas
- Amazing music by Dusty Wyver
- Local multiplayer
- Hiscore for speedrunners
- Bonuses and secrets
- Ultrawide monitor support
- Hard, but fair retro action

Play Skull Basher (https://2dengine.com/game/skullbasher?ref=love2d.org)

Snipes Reloaded

Snipes Reloaded · revision 3786 · last edited 2021-03-14



Snipes Reloaded is a modern rendition of the 1983 game Snipes (http://en.w ikipedia.org/w iki/Snipes) created by SuperSet Software. The object of the game is to control your creature by moving it through a maze to destroy snipes and the hives that spawn them.

Download

See http://chippedgames.blogspot.com/p/snipes-reloaded.html (http://chippedgames.blogspot.com/p/snipes-reloaded.html) to download and play.

socket

socket · revision 31667 · last edited 2026-07-22

Available since LÖVE 0.5.0

This module is not supported in earlier versions.

Implements a luasocket (https://lunarmodules.github.io/luasocket/) module for TCP/UDP networking. The luasocket module is bundled with the LÖVE binary, but in order to use it, you need to require the module like this:

```
local socket = require("socket")
```

Notes

- Prior to 11.0, LÖVE doesn't fully support non-blocking TCP connections on Windows.
- Starting with LÖVE 11.0, `require("socket")` no longer creates a global variable.
- When using blocking operations (network connect/read/write, or socket.sleep), the whole LÖVE main loop will be blocked. This is usually a bad idea. Try to use only non-blocking operations if possible, or use them in a thread.

Reference Manual

For detailed usage, see the reference manual (https://lunarmodules.github.io/luasocket/reference.html) (backup link (https://web.archive.org/web/20200510190501/http://w3.impa.br/~diego/software/luasocket/reference.html)).

See Also

- love
- Tutorial:Networking with UDP
- lua-enet
- grease (https://github.com/bartbes/love-misc-libs/blob/master/grease/docs.md)

Other Languages

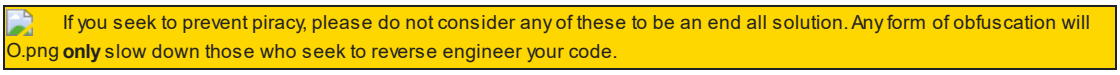
Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

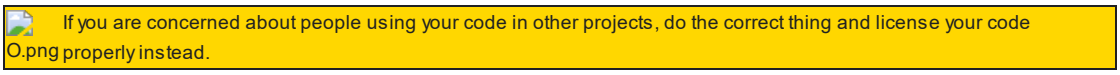
More info

Source Obfuscation

Source Obfuscation · revision 26713 · last edited 2023-07-05

There are three general ways to prevent users from straightforwardly accessing source code found within LÖVE distributed games.





1) Concatinate the .love file with the love binary.

Please see Game_Distribution for advice on how to do this per platform.

To circumvent this, simply treat the concatinated executable as a zip file.

2) Compile Lua source to bytecode.

One can use the a Lua compiler to encode the Lua source code into compiled Lua source code.

Most built versions of LÖVE 0.8.0+ use LuaJIT by default. In the case that they do, you can **destructively** compile a script like so:

```
luajit -b main.lua main.lua
```

To circumvent this, one could use a LuaJIT decompiler such as luajit-decomp (https://github.com/bobsayshilol/luajit-decomp).

Note: while this bytecode will work very well across platforms, it will not work with built versions of LÖVE that do not have LuaJIT. Also note that LÖVE currently uses LuaJIT 2.0 and may switch to LuaJIT 2.1 in the future, which is not compatible. To complicate things even more, there's also LuaJIT 2.1 with GC64 used on ARM64 and some x64 architecture, which has incompatible bytecode with LuaJIT 2.1 (let alone 2.0)

If your built version of LÖVE does not use LuaJIT, you can **destructively** compile a script like so:

```
luac -o main.lua main.lua
```

To circumvent this, one could use a Lua decompiler such as LuaDec (https://github.com/viruscamp/luadec).

Note: this bytecode will only work on your platform (Windows, OSX, Linux) and on that specific architecture (x86, x86_64, ARM).

Here is a simple script that would create a .love for you that is entirely compiled with LuaJIT.

```
#!/bin/sh
# usage: ./bytecode-love.sh <source directory>
WD=`pwd` # determine working directory
TEMP=`mktemp -d` # create a temporary directory
cp -Rv $1/* $TEMP # copy your source code to temp
cd $TEMP # move to temp
for file in $(find . -iname "*.lua") ; do # for each lua file recursively
 luajit -b ${file} ${file} # compile the code with luajit onto itself
done
zip -r $WD/$1-linux-`arch`.love * # zip the result up
rm -rf $TEMP # cleanup
```

3) Use a tool designed for obfusication of Lua source code.

There are many tools available for Lua source obfusication.

http://lua-users.org/wiki/LuaTools ↗

While these tools may obfuscate the code, a veteran programmer will be able to reverse engeineer the code similar to those mentioned earlier.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Splash Screens

Splash Screens · revision 18513 · last edited 2023-07-05

This is a gallery of images suitable for use as splash screens when your game first boots up.

1920x1080p

- 

  A splash screen with a vignette. .XCF project file (http://www.mediafire.com/file/0c05r0q4ai2bvkc/Splash2ColourseparatedBase.xcf).

- 

  A splash screen with no vignette.

- 

  A splash screen that has been desaturated and recoloured a bit.

Animated Splash Screens

Look up [1] (https://github.com/love2d-community/splashes) for animated splash screens!

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

string

string · revision 7936 · last edited 2025-09-30

From the Lua 5.1 reference manual §2.2 (http://www.lua.org/manual/5.1/manual.html#2.2):

String represents arrays of characters. Lua is 8-bit clean: strings can contain any 8-bit character, including embedded zeros ('\0') (see §2.1 (http://www.lua.org/manual/5.1/manual.html#2.1)).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

String exploding

String exploding · revision 19079 · last edited 2021-03-14

This function adds the ability to explode strings in Lua.

```lua
function string.explode(str, div)
    assert(type(str) == "string" and type(div) == "string", "invalid arguments")
    local o = {}
    while true do
        local pos1,pos2 = str:find(div)
        if not pos1 then
            o[#o+1] = str
            break
        end
        o[#o+1],str = str:sub(1,pos1-1),str:sub(pos2+1)
    end
    return o
end
```

Have an example:

```lua
tbl = string.explode("foo bar", " ")
print(tbl[1]) --> foo
-- since we added explode to the string table, we can also do this:
str = "foo bar"
tbl2 = str:explode(" ")
print(tbl2[2]) --> bar
-- to restore the original string, we can use Lua's table.concat:
original = table.concat(tbl, " ")
print(original) --> foo bar
```

String extensions

String extensions · revision 19080 · last edited 2021-03-14

This code modifies the metatable of the string type to add some useful functionality.

```
local meta = getmetatable("") -- get the string metatable

meta.__add = function(a,b) -- the + operator
    return a..b
end

meta.__sub = function(a,b) -- the - operator
    return a:gsub(b,"")
end

meta.__mul = function(a,b) -- the * operator
    return a:rep(b)
end

-- if you have string.explode (check out the String exploding snippet) you can also add this:
meta.__div = function(a,b) -- the / operator
    return a:explode(b)
end

meta.__index = function(a,b) -- if you attempt to do string[id]
    if type(b) ~= "number" then
        return string[b]
    end
    return a:sub(b,b)
end
```

Once you've applied this code, you'll be able to do this:

```
print("foo" + "bar") --> foobar
print("foo the bar" - " the ") --> foobar
print("repetition is repetetive " * 3) --> repetition is repetetive repetition is repetetive repetition is repetetive
tbl = "foo bar" / " "
print(tbl[1]) --> foo
mystring = "abc"
print(mystring[2]) --> b
```

strong

strong · revision 24446 · last edited 2023-07-05

Strong is a pure Lua library that adds many functions to Lua's string library, and a few operators to strings themselves. It's based largely off Ruby's String class (http://www.ruby-doc.org/core/classes/String.html).

Check out the README (https://github.com/BlackBulletIV/strong#readme) for more, and up-to-date, information.

GitHub repository (https://github.com/BlackBulletIV/strong)

Original Thread (http://love2d.org/forums/viewtopic.php?f=5&t=2893)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

subclass

subclass · revision 31664 · last edited 2026-07-22

A simple (yet another) Lua class (OOP) library.

Feature list

- (single) Inheritance (`subclass`)
- Constructor call downwards the hierarchy (from parent to child)
- Table-oriented constructor (the only parameter of a constructor is a table)
- RTTI (`isA`, `isSubclassOf`)

Links

- Github page (https://github.com/LRDPRDX/lua-class)
- Documentation (https://lrdprdx.github.io/lua-class)

Example

```lua
describe('Tutorial', function()
    local Class = require('subclass')
    -- Create a new class
    local Drawable = Class:subclass('Drawable')

    -- Constructor. Called every time a new instance is created.
    -- self is the instance itself. No need to return anything from the constructor.
    function Drawable:new()
        self.color  = self.color or { 0, 0, 0 }
        self.weight = self.weight or 1
    end

    -- Static fields. Can be overridden by a child class
    Drawable.nDim = 2
    Drawable.name = 'drawable'

    -- Method. Can be overridden by a child class.
    -- (note: the dot syntax to explicitly express that the self parameter isn't used)
    function Drawable.size(_)
        return nil
    end

    -- Polyline is a Drawable
    local Polyline = Drawable:subclass('Polyline')

    function Polyline:new()
        self.points = self.points or {}
    end

    -- Override static field
    Polyline.name = 'polyline'

    -- Override method
    function Polyline:size()
        if #self.points == 0 then
            return nil
        end

        local minX, minY, maxX, maxY = nil, nil, nil, nil
        for _, p in ipairs(self.points) do
            minX = minX and math.min(p.x, minX) or p.x
            minY = minY and math.min(p.y, minY) or p.y
            maxX = maxX and math.max(p.x, maxX) or p.x
            maxY = maxY and math.max(p.y, maxY) or p.y
        end

        return { x = maxX - minX,
```

```
                y = maxY - minY }
    end

    function Polyline:nPoints()
        return #self.points
    end

    local d = Drawable {
        color = { 1, 1, 1 }
    }

    local p = Polyline {
        weight = 2,
        points = {
            { x = 0, y = 0 },
            { x = 0, y = 1 },
            { x = 1, y = 1 },
            { x = 1, y = 0 },
        }
    }

    it('subclass of', function()
        assert.is.equal(Drawable:isSubclassOf(Class)    , true)
        assert.is.equal(Polyline:isSubclassOf(Class)    , true)
        assert.is.equal(Polyline:isSubclassOf(Drawable) , true)
    end)

    it('instance of', function()
        assert.is.equal(d:isA(Class)    , true)
        assert.is.equal(d:isA(Drawable) , true)
        assert.is.equal(p:isA(Class)    , true)
        assert.is.equal(p:isA(Drawable) , true)
        assert.is.equal(p:isA(Polyline) , true)
    end)

    it('fields', function()
        assert.is.equal(d.name       , Drawable.name )
        assert.is.equal(d.nDim       , Drawable.nDim )
        assert.is.equal(p.super.name , Drawable.name ) -- access the superclass
        assert.is.same(d.color       , { 1, 1, 1 } )
        assert.is.equal(d.weight     , 1 )

        assert.is.equal(p.name   , Polyline.name )
        assert.is.equal(p.nDim   , Drawable.nDim )
        assert.is.same(p.color   , { 0, 0, 0 } )
        assert.is.equal(p.weight , 2 )
    end)

    it('methods', function()
        assert.is.equal(d:size()    , nil)
        assert.is.same(p:size()     , { x = 1, y = 1 })
        assert.is.equal(p:nPoints() , 4)
    end)
end)
```

Sublime Text

Sublime Text · revision 29110 · last edited 2024-02-16

Fast and customizable cross-platform code editor.

Lua Love package



The Lua Love Package (https://packagecontrol.io/packages/Lua%20Love) provides syntax highlighting, snippets, autocompletion and build systems.

Installation

1. Install Package Control (https://packagecontrol.io/installation) and restart Sublime Text
2. Press `Ctrl+Shift+P` to open the **Command palette** and enter **Package Control: Install Package**
3. Wait until another input field appears and install the **Lua Love** package
4. In the **Command palette** enable syntax highlighting with **Set Syntax: LOVE**
5. Now select build system with `Ctrl+Shift+B` to **LuaLove** and then you can use `Ctrl+B` to start LOVE.
6. **Optional**: Set LOVE syntax as default for all .lua files - `View→Syntax→Open all with current extension as...→LOVE`
7. **Optional**: If LOVE is not in your PATH variable, go to `Preferences→Package Settings→Lua Love` and add:

Sublime Text with Lua Love package (https://packagecontrol.io/packages/Lua%20Love) on Linux

```
{
    "build_system.love.cmd": ["absolute/path/to/love/executable", "${project_path:${folder:${file_path}}}"]
}
```

Console output

By default the console in Sublime Text will not display any output, such as `print()` calls, until the LOVE application has been closed.

To make the console output display live add the following code to the top of your `main.lua` file, or inside `conf.lua`:

```
io.stdout:setvbuf('no')
```

Jumping to errors

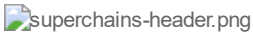
If there was an error during execution, you can use the F4 (next error) and SHIFT-F4 (previous error) to go through the errors that lua encountered.

See also

- Sublime Text Website (https://www.sublimetext.com/)
- Stack Overflow question about console output (https://stackoverflow.com/questions/16338523/live-console-output-in-sublime-text-2)

Super Chains

Super Chains · revision 29605 · last edited 2025-04-28



Super Chains is a unique puzzle experience that will perplex, delight and unwind your mind. Explore a stunning variety of puzzles involving action, strategy and flow. Enjoy the physics-driven gameplay under the soothing soundtrack of L'autre Endroit by Silence. Whether you like a good challenge or prefer an endless, zen-inducing experience then Super Chains has you covered!

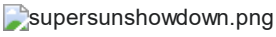
Features

- 20 puzzles focusing on action, strategy or flow
- High scores and achievements
- Soustraction by the artist Silence
- Multiple languages supported
- Palette customization (for the colorblind)
- Infinite replay value

Play Super Chains (https://2dengine.com/game/superchains?ref=love2d.org)

Super Sun Showdown

Super Sun Showdown · revision 12316 · last edited 2021-03-14



A small western style game prototype made at a game jam in Berlin, Germany.

Download: http://g.qubodup.net/supersunshowdown/ ↗

svglover

svglover · revision 26723 · last edited 2023-07-05

svglover is a library to load and display simple SVGs. It works by converting SVGs to LÖVE draw commands.

- Get the source from Github (https://github.com/globalcitizen/svglover)

table

table · revision 7938 · last edited 2025-09-30

From the Lua 5.1 reference manual §2.2 (http://www.lua.org/manual/5.1/manual.html#2.2):

The type table implements associative arrays, that is, arrays that can be indexed not only with numbers, but with any value (except nil). Tables can be heterogeneous; that is, they can contain values of all types (except nil). Tables are the sole data structuring mechanism in Lua; they can be used to represent ordinary arrays, symbol tables, sets, records, graphs, trees, etc. To represent records, Lua uses the field name as an index. The language supports this representation by providing a.name as syntactic sugar for a["name"]. There are several convenient ways to create tables in Lua (see §2.5.7 (http://www.lua.org/manual/5.1/manual.html#2.5.7)).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Ten Second War

Ten Second War · revision 2902 · last edited 2021-03-14

In the game, you are able to control a unit for 10 seconds, during which time the game plays as a top down shooter. After the 10 seconds have finished, time rewinds and you pick a different unit. This time, the first unit will be doing what you did with it earlier as you do something different now. After every unit has been controlled in this way, they will all be acting at the same time, moving as an intelligent army to destroy every enemy on the map within 10 seconds. The variety of unit types are able to work together in many ways, making this a very tactical and skillful game.

- Original Thread (http://love2d.org/forums/viewtopic.php?f=5&t=1480)
- Download Page (http://w w w.moddb.com/games/10-second-w ar/dow nloads/10-second-w ar-v032)



The King and the Crown

The King and the Crown · revision 2886 · last edited 2021-03-14

The King And The Crown is one implemntation of the Pellet Quest System. The game is actually only a graphic demo.



Browse code source here (http://code.google.com/p/the-king-and-the-crown/source/browse/#svn/trunk/src).

Tile Studio

Tile Studio · revision 2457 · last edited 2021-03-14

Tile Studio is a complete development utility for graphics of tile-based games. The application contains a bitmap editor for creating tiles and sprites and a map editor for designing level maps.

Tile Studio was originally designed to be used with the Game Library for the language Clean (a functional language, developed by the University of Nijmegen). But Tile Studio can be used together with ANY programming language, since the output is completely programmable! You can program Tile Studio to output your maps, animation sequences, bitmaps, palettes, etc. in any format and include the output directly into your source code!

- Web Site (http://tilestudio.sourceforge.net/)

tile-collider

tile-collider · revision 19227 · last edited 2023-07-05

This is a module for handling collision with tile maps. It's very simple to use. Set up callbacks and create a handler object, which is used to resolve collision with a tile map.

https://github.com/markandgo/tile-collision ↗

tiny-ecs

tiny-ecs · revision 24481 · last edited 2023-07-05

Entity Component System for lua.

Tiny-ecs is an Entity Component System for lua that's simple, flexible, and useful. Because of lua's tabular nature, Entity Component Systems are a natural choice for simulating large and complex systems. In game programming, ECS is a great way to simplify the game loop. For more explanation on Entity Component Systems, here is some basic info (http://en.wikipedia.org/wiki/Entity_component_system).

Tiny-ecs also works well with objected oriented programming in lua because Systems and Entities do not use metatables. This means you can subclass your Systems and Entities, and use existing lua class frameworks with tiny-ecs, no problem.

Example:

```lua
local tiny = require("tiny")

local talkingSystem = tiny.processingSystem()
talkingSystem.filter = tiny.requireAll("name", "mass", "phrase")
function talkingSystem:process(e, dt)
    e.mass = e.mass + dt * 3
    print(e.name .. ", who weighs " .. e.mass .. " pounds, says, \"" .. e.phrase .. "\"")
end

local joe = {
    name = "Joe",
    phrase = "I'm a plumber.",
    mass = 150,
    hairColor = "brown"
}

local world = tiny.world(talkingSystem, joe)

for i = 1, 20 do
    world:update(1)
end
```

Copy/Paste Latest Source (https://raw.githubusercontent.com/bakpakin/tiny-ecs/master/tiny.lua)

- Source (https://github.com/bakpakin/tiny-ecs)
- Demo Source (https://github.com/bakpakin/tiny-ecs/tree/demo-commandokibbles)
- API (https://bakpakin.github.io/tiny-ecs/doc/)
- Forum Thread (https://love2d.org/forums/viewtopic.php?f=5&t=79937)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Totems Sound

Totems Sound · revision 16983 · last edited 2021-03-14

Totem's Sound

Enjoy the historical journey of Johan Adrian Jacobson to Canada's Wild West in 1881, featuring great outdoor activities such as canoe races, collecting clams, not getting eaten by a giant grizzly and of course fish oil binge drinking. Get in contact with the local tribes that are struggling to preserve their ways of life and cultures in a rapidly changing world. Made by gold extra and Causa Creations in 2014 for the Ethnological Museum of Dahlem/Berlin.

get the game at http://causacreations.itch.io/totems-sound ↗

trAInsported

trAInsported · revision 12324 · last edited 2021-03-14



AIs battling each other in trAInsported

trAInsported takes place in the near future: Your task is to program your company's trains. The trains must automatically transport passengers spawning on the map. Once satisfied, they can be put up against the AIs of other players and will battle in live, online matches.

- Official website (http://trAInsportedgame.no-ip.org)
- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=15166)
- Download page (http://trainsportedgame.no-ip.org/download.php)

Features

- In-game tutorial teaches a lot about the basics in Lua.
- Upload your AI and watch it battle other player's AIs
- Host your own server or play on the main one
- Create your own challenges

Languages

- English
- German
- Spanish
- Finnish
- Russian
- Turkish

Thanks to the translators!

Transform (Library)

Transform (Library) · revision 24536 · last edited 2023-07-05

Transform is a Lua library designed to make it easier handling local/world coordinates, along with a customizable rotation.

It's been highly inspired on Unity's Transform class (https://docs.unity3d.com/ScriptReference/Transform.html).

Link to the GitHub repository:

https://github.com/matiasah/lua-transform ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TsT Module definition

TsT Module definition · revision 8203 · last edited 2021-03-14

How to define a LÖVE module ?

Exactly like a LUA module.

How to define a LUA module ?

There is different way to do. In lua 5.1, a module() function exists : Please do not use it. It will be deprecated in next version. See this discuss for details. (http://lua-users.org/w iki/LuaModuleFunctionCritiqued)

simple module definition

A "private"(?) module (not affecting the global environment)

in file "mymod.lua" :

```
local M = {}

local function blah()

end

local function bar()

end

M.foo = blah
M.bar = bar

return M
```

sample of use in main file :

```
local mymod = require("mymod")
mymod.blah()
mymod.bar()
```

or with global module

```
mymod = require("mymod")
mymod.blah()
mymod.bar()
```

variant 2 in file "mymod.lua" :

```
local M = {}

local function blah()

end

local function bar()

end

M.foo = blah
M.bar = bar

mymod = M -- create the global "mymod" variable
```

sample of use in main file :

```
require("mymod")
mymod.blah()
mymod.bar()
```

more complicated module definition

```
local M = {} -- module namespace
local func = {}

local function blah()

end

local function bar()

end

func.foo = blah
func.bar = bar

M.func = func

M.foo = function(...) return func.foo(...) end
M.bar = function(...) return func.bar(...) end

return M
```

TODO: take sample from luajail/lovejail project

TsT way to learn to Beginners

TsT way to learn to Beginners · revision 6515 · last edited 2023-07-05

On this page

- Step 1 : Install LÖVE
- Step 2 : Install LUA
- Step 3 : explanation
- Step 4 : "algorythmic" tutorial
- Step 5 : lua tuto
- Step 6 : love tuto
- Idea
- More advanced
- Other Languages

Done for a beginner on Windows.

Step 1 : Install LÖVE

1. Download LÖVE
   - http://love2d.org/
   - -> LÖVE 0.7.2 for Windows – Installer 32-bit
2. Install LÖVE : love-0.7.2-win-x86.exe
3. Launch LÖVE (from start menu or from desktop icon)
4. See the beautifull animation
5. Close LÖVE

Step 2 : Install LUA

1. Download LUA
   - http://code.google.com/p/luaforwindows/ ↗
   - Maybe require to install the vcredist_86.exe
2. Install LUA : Lua_V5.1.4-40.exe
3. Launch the "lua command line"
4. A text window appears with a ">" on begin of the line
5. Type :
   - =1+1
6. It prints 2
7. Close LUA with typing the keys ctrl+c

Step 3 : explanation

LUA What is it ?

It's a programmation language

What he can do ?

Some math, counting, manage data, tables, lists, etc. Read files, write files ... or up to(FIXME) print text in his small window.

But that's all !

The guys who made LOVE

Ils ont "rien" inventé (enfin c'est pour simplifier, faut pas le prendre au 1ere degree)

Ils ont assembler des trucs existant fait par d'autres toute la partie graphique, dessiner à l'écran, ca vient d'un autre projet, le son pareil, etc.

meme si on met tout ca ensemble il manque toujours quelque chose pour piloter tout ca. ils ont choisi le LUA pour cela.

LOVE sait compter, gerer des donnees (parce ke cest le lua qui sait le faire) le seul truc qu'il sait faire en plus

c'est des trucs pour dessiner, ou jouer du son qui la sont lié aux autres morceaux dont je parlais avant.

et quand je dis qu'ils ont rien inventé, le fait de relier tout ces trucs ensemble et que ca soit le plus facile a utiliser possible, ca c'est du boulot ;)

c'est l'interet de LOVE.

Pour commencer on va t'apprendre les bases du lua : Compter, faire un tableau, afficher des trucs inutiles

Step 4 : "algorythmic" tutorial

-> sdz ?

Step 5 : lua tuto

...

Step 6 : love tuto

...

Idea

- Editor (NotePad++ @windows, geany @linux)

More advanced

TsT Module definition

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tuesday Convention

Tuesday Convention · revision 7784 · last edited 2021-03-14

The Tuesday Convention establishes the following names for the following people:

- Developer - Developer of LÖVE, makes the engine
- Lover - Uses LÖVE (to make games)
- User - Uses whatever the lovers make

The Scrolls

The ancient scrolls cover the Tuesday Convention, the translation follows:

> At the Tuesday Convention it was said,
> what names certain people had.
> Developers are those who make love,
> and lovers are those who make stuff,
> users are then the people who play.
> This terminology, was set on that day.

Turres Monacorum

Turres Monacorum · revision 13513 · last edited 2021-03-14

Turres Monacorum is a science fiction tower defense 2d game with pre-rendered 3d graphics and a lot of postprocessing shaders.

Current release 0.7 (https://github.com/nczempin/Turres-Monacorum/releases/tag/v0.7)



tween

tween · revision 19228 · last edited 2023-07-05

Small sets of functions for performing tweening in Lua. Based on jQuery's animate function (http://api.jquery.com/animate/) by Enrique García Cota aka kikito (https://github.com/kikito).

Download (https://github.com/kikito/tween.lua/blob/master/tween.lua)

```lua
local tween = require 'tween'

-- increase the volume of music from 0 to 5 in 10 seconds
local music = { volume = 0, path = "path/to/file.mp3" }
tween(10, music, { volume = 5 })

-- make some text fall from the top of the screen, bouncing on y=300, in 4 seconds
local label = { x=200, y=0, text = "hello" }
tween(4, label, { y=300 }, 'outBounce')

-- fade background to black in 2 seconds, and then print a message on the terminal
local backgroundColor = {255,255,255}
tween(2, backgroundColor, {0,0,0}, 'linear', print, "hello!")
```

Links

- Github (https://github.com/kikito/tw een.lua)
- Readme (https://github.com/kikito/tw een.lua/blob/master/README.textile)
- tween.lua (https://github.com/kikito/tw een.lua/blob/master/tw een.lua)

TÖVE

TÖVE · revision 26722 · last edited 2023-07-05

TÖVE brings basic vector graphics to LÖVE. It supports:

- Loading SVGs
- Constructing vector graphics on the fly
- Animating vector graphics
- Gradients
- Different Renderers

Example

```lua
local tove = require "tove"

-- load graphics from svg file. TÖVE also supports
-- constructing vector graphics on the fly.

svgData = love.filesystem.read("assets/rabbit.svg")
graphics = tove.newGraphics(svgData, 200)

-- for rendering, choose among three renderers:
-- "texture" will render into a bitmap
-- "mesh" will tesselate into a mesh
-- "gpux" will use a shader implementation

graphics:setDisplay("mesh", 200)

function love.draw()
    -- render svg at mouse position.

    x, y = love.mouse.getPosition()
    graphics:draw(x, y)
end
```

Links

- Source (https://github.com/poke1024/tove2d)
- Forum Thread (https://love2d.org/forums/viewtopic.php?f=5&t=85269)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

uLove

uLove · revision 19048 · last edited 2023-07-05



This page has no more real relevance to Löve.

~~uLove is only a proposal right now, so nothing here should be considered final. You are free (and encouraged!) to add your thoughts (http://love2d.org/forums/viewtopic.php?f=4&t=1623)~~

uLove (also known as µLÖVE) is a standard subset of functionality intended to reduce Love's binary and memory footprint, and capability requirements. it is specifically intended for targetting devices and contexts that have limited capabilities or permissions, such as portable gaming devices (Dingoo, WIZ, etc), mobile devices (Mobile phones, PDAs, Nanonotes, etc), or constrained contexts (web plugins).

Standard

While any port of love may provide features beyond those defined in uLove, it not generally permitted to remove features.

- uLove archive files are *explicitly* Zip files. (Rather than "nominally", as in LÖVE).
- uLove *must* support image fonts, it specifically *may omit* vector fonts (such as TTF).
- uLove *must* provide a default font, with a size appropriate to the target device, in the same instances that Love would provide a default font. However, this font is *permitted* to not be, or be named after, "Vera Sans".
- uLove *must* support Ogg Vorbis (http://en.w ikipedia.org/w iki/Vorbis) Audio, it *may* omit all other audio formats.
- uLove *must* support PNG (http://en.w ikipedia.org/w iki/Portable_Netw ork_Graphics), and Uncompressed Targa (TGA) (http://en.w ikipedia.org/w iki/Truevi sion_TGA) image formats. It *may omit* all other image formats.
- uLove *may* omit love.mouse, love.keyboard, and love.joystick. Though a uLove host will likely provide at least one of them, none of them are required.
- uLove *may* omit love.physics.
- uLove *may* omit LuaSocket.

Implications

In addition to the above constraints, Lovers must also recognise that:

- Many devices running uLove have limited memory, often 32MB *or less*.
- Many devices targeted by uLove have considerably less processing power than standard systems.
- Many devices targeted by uLove lack the graphics acceleration features used by LOVE. In particular this means that many traditionally slow operations that are painless in love, are actually slow again...
    - Alpha transparency other than exactly 50% (and probably transparency in general)
    - Rotation not a multiple of 90°
    - non-POW2 image scaling (and probably scaling in general)
- Many devices running uLove have low resolutions, in the vein of 320x240 *or less*.
- many uLove targeted devices will likely rely on hardware quirks and assumptions to keep performance up. in particular, you must assume that non-power-of-two images will be slow, or fail outright.
- uLove is targeted at many disparate devices, uLove games should use functions such as those in love.joystick and love.graphics.checkMode to determine the device's capabilities, and adjust themselves accordingly.
- Many devices targeted by uLove have limited controls, usually one d-pad and four face buttons (A, B, Start, Select, or equivalent) is all that can be relied on.
- Many devices running uLove entirely lack mouse support, those that do present mice have touch surfaces. bare in mind that:
    - Touch surfaces usually lack precision.
    - Touch surfaces do not usually allow the pointer to be moved without also generating either a click or mousedown event.
    - Touch surfaces are absolute, rather than relative pointing devices. (that is, the mouse is always moved to a specific location, never by a delta).

Groundwork

These are notes of things that will need to be charged added to the default Love source bundle to accommidate uLove.

- Either the existing OpenGL backend will need to be converted to OpenGL/ES, or a new OpenGL/ES back will need to be developed.
- Defines/Compile targets will need to be defined to remove the heavier dependencies from Love (optionally replacing them with lighter dependancies where appropriate)
- A new SDL-based backend will need to be created for systems that lack OpenGL support.

See also

- uLove Compliance conf.lua

uLove Compliance conf.lua

uLove Compliance conf.lua · revision 19089 · last edited 2021-03-14


This page has no more real relevance to Löve.

Introduction

This conf.lua file can be used to test minimal compliance with the proposed uLove standard.

This is by no means a complete workout of your code, and you should refer to the Standard itself to be sure your code is compliant. but this conf is a useful benchmark, if it runs with this, then you are in with a good shot.

uLove conf.lua

```lua
-- conf.lua
-- use this in place of your games normal conf.lua to see how it deals
-- with some common, or likely, limitations of uLove-targeted platforms.
--
-- Bare in mind, this is by no means a complete test, but its a minimal
-- benchmark; if it can't run under this, then you are definitely going to
-- have problems.
function love.conf(t)
    t.title             = "uLove Minimal Compliance Test"
    t.author            = "Textmode (DMB)"
    t.version           = 60
    t.console           = false


    t.screen.width      = 320
    t.screen.height     = 240
    t.screen.fullscreen = false                 -- this is likely to be true
                                                -- on an uLove target, but few modern systems
                                                -- actually support this res ...so, yeah.

    t.screen.vsync      = true                  -- actually could go either way on a uLove
                                                -- platform, but anything to bring your
                                                -- FPS down, lest you forget that most
                                                -- uLove targets are likely to be sub-500MHz
    t.modules.joystick  = true
    t.modules.audio     = true
    t.modules.keyboard  = true                  --  if you actually have a game pad, you
                                                -- should make sure your game can be played
                                                -- *entirely* using that that. and no cheating with
                                                -- "pro" gamepads; we're talking one D-pad, A, B,
                                                -- R, L, Start, and Select. and that's generous.
    t.modules.event     = true
    t.modules.image     = true
    t.modules.graphics  = true
    t.modules.timer     = true
    t.modules.mouse     = false                 -- there are far more portables without
                                                -- mouse-like input, than those with.
    t.modules.sound     = true
    t.modules.physics   = false                 -- aw, you thought uLove platforms were powerful
                                                -- enough to run physics, that's cute ^_^
end
```

utf8

utf8 · revision 28546 · last edited 2025-03-18

Available since LÖVE 0.9.2

This module is not supported in earlier versions.

This library provides basic support for handling UTF-8 encoded strings.

It provides all its functions inside the table returned by `require("utf8")`. This library does not provide any support for Unicode other than handling UTF-8 encoding. Any operation that needs the meaning of a character, such as character classification, is outside its scope.

For detailed usage, see the reference manual (https://www.lua.org/manual/5.3/manual.html#6.5).



The utf8.char function does not work correctly in 0.9.2; However, it is not an issue since 0.10.0

Examples

Print text the user writes, and erase it using the UTF-8 module.

```
local utf8 = require("utf8")

function love.load()
    text = "Type away! -- "

    -- enable key repeat so backspace can be held down to trigger love.keypressed multiple times.
    love.keyboard.setKeyRepeat(true)
end

function love.textinput(t)
    text = text .. t
end

function love.keypressed(key)
    if key == "backspace" then
        -- get the byte offset to the last UTF-8 character in the string.
        local byteoffset = utf8.offset(text, -1)

        if byteoffset then
            -- remove the last UTF-8 character.
            -- string.sub operates on bytes rather than UTF-8 characters, so we couldn't do string.sub(text, 1, -2).
            text = string.sub(text, 1, byteoffset - 1)
        end
    end
end

function love.draw()
    love.graphics.printf(text, 0, 0, love.graphics.getWidth())
end
```

See Also

- love
- love.textinput

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

variable

variable · revision 22570 · last edited 2025-09-30

A variable is a value like val, var, and any other letter or word that is not already assigned to something else.

Example

Changes a variable by 1 each time the left mouse button is clicked

```
function love.mousepressed( x, y, button )
    if button == 1 then -- Versions prior to 0.10.0 use the MouseConstant 'l'
        time = time + 1
    end
end
```

See Also

- love.mousepressed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

vec2

vec2 · revision 28178 · last edited 2023-07-05

A vector with 2 floating-point number components. These can be accessed with `.r/g`, `.x/y`, or `[0/1]`.

```
vec2 example = vec2(0.5, 1.0);

float x = example.x;
float y = example.y;
```

Read more:

- Vectors on the OpenGL Wiki (https://www.khronos.org/opengl/wiki/Data_Type_(GLSL)#Vectors)
- vec2 on The Book of Shaders (https://thebookofshaders.com/glossary/?search=vec2)

See Also

- vec4
- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

vec4

vec4 · revision 27904 · last edited 2023-07-05

A vector with 4 floating-point number components. These can be accessed with `.r/g/b/a`, `.x/y/z/w`, or `[0/1/2/3]`.

```
vec4 example = vec4(0.333, 0.666, 0.999, 1.0);

float r = example.r;
float g = example.g;
float b = example.b;
float a = example.a;
```

Read more:

- Vectors on the OpenGL Wiki (https://www.khronos.org/opengl/wiki/Data_Type_(GLSL)#Vectors)
- vec4 on The Book of Shaders (https://thebookofshaders.com/glossary/?search=vec4)

See Also

- vec2
- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Virtualization Solutions Provider

Virtualization Solutions Provider · revision 17446 · last edited 2021-03-14

Volley Brawl

Volley Brawl · revision 6257 · last edited 2021-03-14

Volley Brawl is an awesome feature rich slime volley game designed to be fun and easy to learn for all ages, developed by Sigmoid and published by Ohso.



Sporting over 10 different characters, intelligent computer players, full local, LAN and online multiplayer and a variety of other gameplay features, it's a bargain at $2.99.

Plus, we'll be releasing content pack updates every month which will bring new user-designed characters from the OMG! Ubuntu! community, new levels, sound effects, achievements and more!

Buy it now in the Ubuntu Software Center! (Other download and purchasing options hopefully coming.)

vudu

vudu · revision 24601 · last edited 2023-07-05

Visual Unified Debug Utility (vudu) is an in-engine debug utility that allows you to...

- Browse and edit variables
- Fast forward, slow down, or freez
- Execute commands
- View console output in-game
- Create development hotkeys
- View physics wireframe, joints, and collisions
- Draw debug graphics

All wrapped up in a neat, clean, easy-to-use UI, that can be added to your project with a single line of code:

```
vudu:initialize()
```

vudu is maintained by DeltaDaedalus, you can get it here (https://github.com/deltadaedalus/vudu)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Warlocks Tower

Warlocks Tower · revision 17655 · last edited 2021-03-14

Hardship! Sorrow! Fates worse than death and all that! Get ready to reach unspeakable heights in this new movement puzzle! Wear the tidy uniform of brave mailman Tim and be the one who delivers a message of truce to the Evil Wizard, which has been sucking the world dry with his dark magic. But the task is not as easy as it sounds! For every floor is a maze of rooms, each of them cursed, draining away the mailman's life with every step he takes!

Will you be able to reach the tower's top floor and restore peace to the entire planet? Or will you perish, in this place forsaken by the very Gods themselves?

Features

- AMAZING 8-bit flavored graphics and sfx/music! Feels like playing your GameBoy™ again!
- 5 DIFFERENT WORLDS, each with its own theme (dungeon, factory, storage, library and laboratory)!
- Over 100 stages of BRAIN-MELTING puzzles! Find keys, pull levers, lower gates, push jars, ride treadmills... and MORE!
- Learn the patterns and behaviours of many DANGEROUS MONSTERS in order to trick dodge them!
- Play with a friend (OR WITH YOURSELF) in the Tag Team stages! Alternate between Tim and the spunky girl Jess!
- JOYSTICK SUPPORT! Oh my!
- Ingame tips in the form of very cheesy, pseudo-limericks.

Screenshots

- WTTitleScreen.gif
- Warehouse.gif
- DarkLevel.gif
- Teleport.gif
- Treadmill.gif

Warp Run

Warp Run · revision 15437 · last edited 2021-03-14

Run all around a futuristic maze collecting things that look nothing like you.



Trailer (http://www.youtube.com/watch?v=cPWd81DXqVY)

Downloads on Bitbucket (https://bitbucket.org/AaronWizard/warp-run/downloads)

Fourm thread (http://love2d.org/forums/viewtopic.php?f=5&t=9352)

yaoui

yaoui · revision 25221 · last edited 2023-07-05

yaoui is a UI kit for LÖVE. If you need a fast way to build half-decent looking UI without having to worry about much then this module is for you.

Links

- Source code (https://github.com/adonaac/yaoui)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

zip

zip · revision 3729 · last edited 2021-03-14

The ZIP file format is a data compression and archive format. A ZIP file contains one or more files that have been compressed to reduce file size, or stored as-is. .love files are simply renamed .zip files.

Zip files can be read by the following programs:

- 7-Zip
- ALZip
- Archive Utility
- Ark
- Bitser
- bsdtar/libarchive
- File Roller
- Filzip
- FreeArc
- iArchiver
- Info-ZIP
- lZArc
- KGB Archiver
- PeaZip
- PKZIP
- PowerArchiver
- StuffIt
- The Unarchiver
- TUGZip
- WinAce
- WinRAR
- WinZip
- XAD
- Xarchiver
- ZipGenius

ziplib

ziplib · revision 21114 · last edited 2023-07-05

ZIP is a zipping library for löve.
More information about this library can be found here.
https://github.com/matiasah/zip/wiki ↗

The code repository is here.
https://github.com/matiasah/zip ↗